

LAYER: 1_Core

This file is a merged representation of a subset of the codebase, containing files not matching ignore patterns, combined into a single document by Repomix.

<file_summary>

This section contains a summary of this file.

<purpose>

This file contains a packed representation of a subset of the repository's contents that is considered the most important context.

It is designed to be easily consumable by AI systems for analysis, code review, or other automated processes.

</purpose>

<file_format>

The content is organized as follows:

1. This summary section
2. Repository information
3. Directory structure
4. Repository files (if enabled)
5. Multiple file entries, each consisting of:
 - File path as an attribute
 - Full contents of the file

</file_format>

<usage_guidelines>

- This file should be treated as read-only. Any changes should be made to the original repository files, not this packed version.
- When processing this file, use the file path to distinguish between different files in the repository.
- Be aware that this file may contain sensitive information. Handle it with the same level of security as you would the original repository.

</usage_guidelines>

<notes>

- Some files may have been excluded based on .gitignore rules and Repomix's configuration
- Binary files are not included in this packed representation. Please refer to the Repository Structure section for a complete list of file paths, including binary files
- Files matching these patterns are excluded: **/obj/**, **/bin/**, **/*.png, **/*.jpg, **/node_modules/**, **/*.Designer.cs
- Files matching patterns in .gitignore are excluded
- Files matching default ignore patterns are excluded
- Files are sorted by Git change count (files with more changes are at the bottom)

</notes>

</file_summary>

<directory_structure>

Configuration.Web/Components/_Imports.razor
Configuration.Web/Components/Layout/MainLayout.razor
Configuration.Web/Components/Layout/ReconnectModal.razor
Configuration.Web/Components/NavMenu.razor
Configuration.Web/Components/NotFound.razor
Configuration.Web/Components/Routes.razor
Configuration.Web/Configurator/IWebAppConfigurator.cs
Configuration.Web/Configurator/WebAppConfigurator.cs
Configuration.Web/Configurator/WebScanningExtensions.cs
Configuration.Web/Promatis.Net.Configuration.Web.csproj
Configuration.Web/Properties/launchSettings.json
Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs
Configuration.Web/Services/TabNavigationService/TabItem.cs
Configuration.Web/Services/TabNavigationService/TabNavigationService.cs
Configuration.Web/Services/UiModuleService.cs
Configuration.Web/UiCommandBus/IUiCommandBus.cs
Configuration.Web/UiCommandBus/UiCommandBus.cs
Configuration.Web/WebAppBootstrapper.cs
Configuration/AppBootstrapper.cs
Configuration/AppConfigurator.cs
Configuration/AppInfrastructure.cs
Configuration/IAppConfigurator.cs
Configuration/Promatis.Net.Configuration.csproj
Configuration/ServiceScanningExtensions.cs
Data/Configurations/AuditLogConfiguration.cs
Data/Configurations/ModelBuilderConventions.cs
Data/DbContext/ApplicationDbContext.cs

Data/DbContext/ModelBuilderExtensions.cs
Data/Interceptors/ChangeEntryModel.cs
Data/Interceptors/DatabaseTriggerInterceptor.cs
Data/Interceptors/IUserProvider.cs
Data/Promatis.Net.Data.csproj
Data/Triggers/Castom/AuditTrigger.cs
Data/Triggers/Global/FluentValidationTrigger.cs
Data/Triggers/Global/ReferenceTreeParentTrigger.cs
Data/TriggerService/DatabaseTriggerService.cs
Data/TriggerService/EntityStateChangeEnum.cs
Data/TriggerService/IDatabaseTriggerService.cs
Data/TriggerService/PropertyChangeInfo.cs
Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs
Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgsBase.cs
Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs
Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs
Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs
Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs
Data/Validation/GlobalPolymorphicValidator.cs
Domain.Interface/Enums/EnumExtensions.cs
Domain.Interface/Interfaces/IAudit.cs
Domain.Interface/Interfaces/IDomainObject.cs
Domain.Interface/Interfaces/IDomainObjectHasKey.cs
Domain.Interface/Interfaces/IEntityCloner.cs
Domain.Interface/Interfaces/ISoftDeletable.cs
Domain.Interface/Interfaces/ITreeNode.cs
Domain.Interface/Promatis.Net.Domain.Interface.csproj
Domain/AuditLog/AuditLog.cs
Domain/DomainObject/DomainObject.cs
Domain/DomainObject/DomainObjectBase.cs
Domain/DomainObject/DomainObjectValidator.cs
Domain/EntityCloner/JsonEntityCloner.cs
Domain/Promatis.Net.Domain.csproj
Domain/ReferenceBase/ReferenceBase.cs
Domain/ReferenceBase/ReferenceBaseValidator.cs
Domain/ReferenceTreeBase/ReferenceTreeBase.cs
Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs
Promatis.Net.UI/_Imports.razor
Promatis.Net.UI/Components/ActionContext/DataContext.cs
Promatis.Net.UI/Components/ActionContext/EntityContext.cs
Promatis.Net.UI/Components/ActionContext/GridContext.cs
Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs
Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs
Promatis.Net.UI/Components/ActionContext/TreeContext.cs
Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs
Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor
Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs
Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor
Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs
Promatis.Net.UI/Components/UIControls/BaseUiControl.cs
Promatis.Net.UI/Components/UIControls/IHasOptions.cs
Promatis.Net.UI/Components/UIControls/IHasValue.cs
Promatis.Net.UI/Components/UIControls/IUiControl.cs
Promatis.Net.UI/Components/UIControls/UiControlAlignment.cs
Promatis.Net.UI/Components/Workspaces/GridPage.razor
Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferenceGridPage.razor
Promatis.Net.UI/Components/Workspaces/ReferenceGridPage.razor.cs
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor
Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/TreePage.razor
Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor
Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs

```

Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor
Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs
Promatis.Net.UI/Controls/CreateChildButton.cs
Promatis.Net.UI/Controls/CreateEntityButton.cs
Promatis.Net.UI/Controls/DeleteEntityButton.cs
Promatis.Net.UI/Controls/EditEntityButton.cs
Promatis.Net.UI/Controls/ToolBarDivider.cs
Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs
Promatis.Net.UI/Data/Cache/IUiOzuCache.cs
Promatis.Net.UI/Data/Cache/UiOzuCache.cs
Promatis.Net.UI/Data/IUiDataBroker.cs
Promatis.Net.UI/Data/UiDataBroker.cs
Promatis.Net.UI/Extensions/MudColorExtensions.cs
Promatis.Net.UI/Interfaces/IHasSelectedData.cs
Promatis.Net.UI/IUiModule.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor
Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs
Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditActionSelect.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditEntitySelect.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditExportButton.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditPeriodPicker.cs
Promatis.Net.UI/Pages/AuditLogs/ToolBar/AuditToolBarDivider.cs
Promatis.Net.UI/Pages/Dashboard.razor
Promatis.Net.UI/Pages/Dashboard.razor.css
Promatis.Net.UI/Promatis.Net.UI.csproj
Promatis.Net.UI/Theme/PromatisTheme.cs
Promatis.Net.UI/UiModule.cs
Promatis.Net.UI/wwwroot/App.css
Service/Contracts/AuditLog/AuditLogSearchRequest.cs
Service/Contracts/AuditLog/PagedResult.cs
Service/Promatis.Net.Service.csproj
Service/Services/AuditLogService/AuditLogService.cs
Service/Services/AuditLogService/IAuditLogService.cs
Service/Services/BaseService/BaseService.cs
Service/Services/BaseService/IBaseService.cs
Service/Services/ReferenceService/IReferenceService.cs
Service/Services/ReferenceService/ReferenceService.cs
Service/Services/ReferenceTreeService/IReferenceTreeService.cs
Service/Services/ReferenceTreeService/ReferenceTreeService.cs
Service/Services/TreeService/ITreeService.cs
Service/Services/TreeService/TreeService.cs
Tests/AssemblyInfo.cs
Tests/Domain/DomainLogicTests.cs
Tests/Domain/DomainObjectBaseLogicTests.cs
Tests/Domain/DomainObjectBaseTests.cs
Tests/Domain/DomainObjectExtendedTests.cs
Tests/Domain/ReferenceBaseTests.cs
Tests/Domain/ReferenceTreeTests.cs
Tests/Domain/SoftDeletableTests.cs
Tests/Interceptor/DatabaseTriggerInterceptorTests.cs
Tests/Promatis.Net.ApplicationModel.Tests.csproj
Tests/Service/BaseServiceTests_Crud.cs
Tests/Service/BaseServiceTests.cs
Tests/Service/ReferenceServiceTests_Search.cs
Tests/Service/ReferenceTreeServiceTests.cs
Tests/Service/ServiceIntegrationTests.cs
Tests/Trigger/AuditTriggerTests.cs
Tests/Trigger/DatabaseTriggerServiceTests.cs
Tests/Trigger/FluentValidationTriggerTests.cs
Tests/Trigger/FullTriggerChainTests.cs
Tests/Trigger/ReferenceTreeParentTriggerTests.cs
Tests/Trigger/TreeTriggerChainTests.cs
Tests/Validator/DomainObjectValidatorTests.cs
Tests/Validator/ReferenceBaseValidatorTests.cs
Tests/Validator/ReferenceTreeBaseValidatorTests.cs
</directory_structure>

```

<files>

This section contains the contents of the repository's files.

```

<file path="Configuration.Web/Components/_Imports.razor">
@using System.Net.Http
@using System.Net.Http.Json

```

```

@using Microsoft.AspNetCore.Components.Forms
@using Microsoft.AspNetCore.Components.Routing
@using Microsoft.AspNetCore.Components.Web
@using Microsoft.JSInterop
@using MudBlazor
@using Promatis.Net.Ui
@using Promatis.Net.Configuration.Web
@using Promatis.Net.Configuration.Web.Components
@using Promatis.Net.Configuration.Web.Components.Layout
@using static Microsoft.AspNetCore.Components.Web.RenderMode
</file>

<file path="Configuration.Web/Components/Layout/MainLayout.razor">
@inherits LayoutComponentBase
@using Promatis.Net.Ui.Theme
@
@inject UiModuleService UiService
@inject TabNavigationService TabService
@implements IDisposable
@
@* A,,!Aô A A´ Aô : B 2Dô7D´2C 5CÂ ?D >C$0C"4CT@ D$5CÄK D DC´0C4>CÂ BE <Cô>C4> D 5Cd8CÄ0 *@
<MudThemeProvider Theme="PromatisTheme.DefaultTheme" @bind-IsDarkMode="_isDarkMode" />
<MudPopoverProvider />
@
<MudDialogProvider FullWidth="false"
    MaxWidth="MaxWidth.False"
    CloseButton="false"
    BackdropClick="false"
    NoHeader="false"
    Position="DialogPosition.Center"
    CloseOnEscapeKey="true" />
@
<MudSnackbarProvider />
@
<CascadingValue Value="this">
    <MudLayout>
        @* A$5D ECô0Dô ?C =CT;DÂ ?D 8C´>Cd5Cô8Dô ¢
        <MudAppBar Color="Color.Default" Elevation="1">
            @* A,,!Aô A A´ Aô : At0CÄ5Cô5Cô> Color.Primary C00 Color.Default *@
            <MudIconButton Icon="@Icons.Material.Filled.Menu"
                Color="Color.Inherit"
                Edge="Edge.Start"
                Size="Size.Large"
                OnClick="ToggleDrawer" />
            <MudStack Spacing="-1">
                @* AD5DD>C´BCô>CR =C 7C$0Cô8CR ¢
                <MudText Typo="Typo.h6" Class="ml-2">
                    B$5D BCä2D´9 C0@Cä5C=B Promatis Net
                </MudText>
            </MudStack>
        @
        <MudSpacer />
        @
        <MudIconButton Icon="@(_isDarkMode ? Icons.Material.Filled.LightMode :
Icons.Material.Filled.DarkMode)"
            Color="Color.Inherit"
            OnClick="ToggleDarkMode" />
    </MudAppBar>
    @
    @* Aô0Cô5C´L CÄ5Côn *@
    <MudDrawer @bind-Open="_drawerOpen" ClipMode="DrawerClipMode.Always"
Variant="DrawerVariant.Temporary" Overlay="true" OverlayAutoClose="true" Width="220px"
Elevation="1">
        <NavMenu />
    </MudDrawer>
    @
    <MudMainContent Class="pt-16 px-4 pb-4 d-flex flex-column"
        Style="height: calc(100vh - 8px); min-height: calc(100vh - 8px); margin-
top: 10px; display: flex; flex-direction: column; flex-grow: 1; overflow: hidden;">
        @
        @* A,,!Aô A A´ Aô : A4;Cä1C ;DÄ=D´9 ErrorBoundary D41D 0Cô A C$5D ECô5C4> D4@Cä2C00. B BC,,;C, fÄW
        @if (TabService.OpenTabs.Count == 0)
        {
            <div class="flex-grow-1 overflow-auto">
                <ErrorBoundary>
                    <ChildContent>

```

```

        @Body
    </ChildContent>
    <ErrorContent Context="exception">
        <MudAlert Severity="Severity.Error" Class="my-4">
            A@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=D$5D DCT9D 0: @exception.Message
        </MudAlert>
    </ErrorContent>
    </ErrorBoundary>
</div>
}
else
{
    <MudTabs @bind-ActivePanelIndex="TabService.ActiveTabIndex"
        Position="Position.Top"
        Color="Color.Default"
        SliderColor="Color.Primary"
        Class="mdi-tabs-container w-100 flex-grow-1 d-flex flex-column"
        Style="height: 100%; max-height: 100%; min-height: 0; display: flex; flex-
direction: column; flex-grow: 1;"
        TabPanelsClass="pa-0 flex-grow-1 d-flex flex-column overflow-hidden"
        ApplyEffectsToContainer="true"
        TabButtonsClass="rounded-t-lg shadow-sm"
        Outlined="true">

        @for (int i = 0; i < TabService.OpenTabs.Count; i++)
        {
            int localIndex = i;
            TabItem tab = TabService.OpenTabs[i];

            @* A,,!Aô A A´ Aô : AD>C 0C$;CT=C fÆW,ôAD$@D4:D$CD 0 CD;Dò AC <Cä3Cä BC 1-Cô0Cô5C´0,
            <MudTabPanel @key="tab" ID="@localIndex" Class="h-100 d-flex flex-column">
                <TabContent>
                    <div class="d-flex align-center gap-2" style="cursor: pointer;">
                        @if (!string.IsNullOrEmpty(tab.Icon))
                        {
                            <MudIcon Icon="@tab.Icon" Size="Size.Small" />
                        }
                        <MudText Typo="Typo.body2" Class="font-weight-
medium">@tab.Title</MudText>
                        <MudIconButton Icon="@Icons.Material.Filled.Close"
                            Size="Size.Small"
                            Color="Color.Default"
                            Class="pa-0 ms-1"
                            Style="width: 16px; height: 16px; min-width:
16px;"
                            OnClick="@((e) =>
TabService.CloseTabByIndex(localIndex))" />
                    </div>
                </TabContent>
                <ChildContent>
                    @* BôBCäB Cæ>CôBCT9Cô5D 6CTAD$:Cä 7C =C,,<C 5D" R 2D´ACäBD² 2Cæ;C 4Cæ8 C,
                    <div class="pa-2 rounded-b-lg w-100 d-flex flex-column flex-grow-1"
                        Style="height: 100%; max-height: 100%; min-height: 0;
overflow: hidden;">
                    <ErrorBoundary>
                        <ChildContent>
                            @* A,,!Aô A A´ Aô : AÄK Cä1Cä@C GC,,2C 5CÄ G-æ Ö-46ö× öæVçB 2 DD8C
                            @* Cæ>D$>D KC´ AC,,;Cä9 C$KD$0C48C$0CTB D BD 0C08DdC C00D,,5C4> CD5
                            <div class="d-flex flex-column flex-grow-1 w-100"
                                <DynamicComponent Type="@tab.ComponentType" />
                            </div>
                        </ChildContent>
                        <ErrorContent Context="exception">
                            <MudAlert Severity="Severity.Error" Class="my-4">
                                Aô@Cä8Ct>D,,;C :D 8D$8Dt5D :C 0 CäHC,,1C=D$5D DCT9D 0 C$:
                            </MudAlert>
                        </ErrorContent>
                    </ErrorBoundary>
                </div>
            </ChildContent>
        </MudTabPanel>
    }
}

```

```

        </MudTabs>@
    }@
</MudMainContent>@
</MudLayout>@
</CascadingValue>@
@
@code {@
    private bool _drawerOpen = true;@
    private bool _isDarkMode; // A@> D4<Cä;Dt0C08Dâ ?D 8C´>Cd5C08CR 7C ?D4AC@0CTBD 0 C" AC$5D$;Cä9 D$5CÄ5D
@
    protected override void OnInitialized()@
    {@
        base.OnInitialized();@
        TabService.OnTabsChanged += HandleTabsChanged;@
        _drawerOpen = false;@
    }@
@
    private void ToggleDrawer() => _drawerOpen = !_drawerOpen;@
@
    /// <summary>@
    /// A@5D 5C@;DäGC 5D" DC´0C2 BE <C0>C4> D 5Cd8CÄ0 C, 7C AD$0C$;Dô5D" xVEF†VÖU &÷f-FW" <C4=Cä2CT=C0> C05D
    /// </summary>@
    private void ToggleDarkMode() => _isDarkMode = !_isDarkMode;@
@
    private void HandleTabsChanged() => InvokeAsync(StateHasChanged);@
@
    public void CloseDrawer()@
    {@
        if (_drawerOpen)@
        {@
            _drawerOpen = false;@
            StateHasChanged();@
        }@
    }@
@
    public void Dispose() => TabService.OnTabsChanged -= HandleTabsChanged;@
}
</file>

<file path="Configuration.Web/Components/Layout/ReconnectModal.razor">
<script type="module" src="@Assets["Components/Layout/ReconnectModal.razor.js"]"></script>@
@
<dialog id="components-reconnect-modal" data-nosnippet>@
    <div class="components-reconnect-container">@
        <div class="components-rejoining-animation" aria-hidden="true">@
            <div></div>@
            <div></div>@
        </div>@
        <p class="components-reconnect-first-attempt-visible">@
            Rejoining the server...@
        </p>@
        <p class="components-reconnect-repeated-attempt-visible">@
            Rejoin failed... trying again in <span id="components-seconds-to-next-attempt"></span>
seconds.@
        </p>@
        <p class="components-reconnect-failed-visible">@
            Failed to rejoin.<br />Please retry or reload the page.@
        </p>@
        <button id="components-reconnect-button" class="components-reconnect-failed-visible">@
            Retry@
        </button>@
        <p class="components-pause-visible">@
            The session has been paused by the server.@
        </p>@
        <p class="components-resume-failed-visible">@
            Failed to resume the session.<br />Please retry or reload the page.@
        </p>@
        <button id="components-resume-button" class="components-pause-visible components-resume-
failed-visible">@
            Resume@
        </button>@
    </div>@
</dialog>
</file>

<file path="Configuration.Web/Components/NavMenu.razor">

```

```

@using System.Reflection
@inject UiModuleService UiService
@inject TabNavigationService TabService
@inject ComponentRegistry Registry
@
<MudNavMenu>
    @{
        List<(string Title, string Href, string Icon, string? Group)> allItems = UiService.Modules
            .SelectMany(m => m.GetMenuItems())
            .ToList();
    }

    // AÄBD 8D >C$KC$0CT< DÔ;CT<CT=D$K C$5D EC05C4> D4@Cä2C00
    IEnumerable<(string Title, string Href, string Icon, string? Group)> topLevelItems =
allItems.Where(i => string.IsNullOrEmpty(i.Group));
    foreach ((string Title, string Href, string Icon, string? Group) item in topLevelItems)
    {
        <!-- A,,!Aô A A´ Aô : A$<CTAD$> Href C,,ACô>C´LctCCT< OnClick CD;Dò >D$:D KD$8Dò 2C;C 4Cä: C 5Cr
        <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title, item.Href,
item.Icon))">
            @item.Title
        </MudNavLink>
    }

    // A4@D4?C08D CCT< DÔ;CT<CT=D$K Cò> Cò0D$5C4>D 8Dô<
    IOrderedEnumerable<IGrouping<string, (string Title, string Href, string Icon, string?
Group)>> pooledGroups = allItems
        .Where(i => !string.IsNullOrEmpty(i.Group))
        .GroupBy(i => i.Group!)
        .OrderBy(g => g.Key);

    foreach (IGrouping<string, (string Title, string Href, string Icon, string? Group)> group
in pooledGroups)
    {
        <MudNavGroup Title="@group.Key" Icon="@Icons.Material.Filled.Folder" Expanded="false">
            @foreach ((string Title, string Href, string Icon, string? Group) item in group)
            {
                <!-- A,,!Aô A A´ Aô : B41D 0C0 ‡&VbÂ 4Cä1C 2C´5C0 öä6Æ-6² 00í
                <MudNavLink Icon="@item.Icon" OnClick="@(() => OpenMenuAsTab(item.Title,
item.Href, item.Icon))">
                    @item.Title
                </MudNavLink>
            }
        </MudNavGroup>
    }
}
</MudNavMenu>
@
@code {
    /// <summary>
    /// Að5D 5DT2C BD´2C 5CÂ AC²2Cä7C0>C´ MC²7CT<Cò;Dô@ D >CD8D$5C´LD :Cä3Cä <C :CTBC
    /// </summary>
    [CascadingParameter]
    protected MainLayout? MainLayoutInstance { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        IEnumerable<Assembly> assemblies = UiService.Modules.Select(m =>
m.GetType().Assembly).Distinct();
        Registry.RegisterModules(assemblies);
    }

    private void OpenMenuAsTab(string title, string href, string icon)
    {
        Type? componentType = Registry.GetComponentTypeByRoute(href);

        if (componentType != null)
        {
            // AäBC²@D´2C 5CÂ 2C;C 4C²C
            TabService.OpenTab(title, href, icon, componentType);
        }

        // A 2D$>CÄ0D$8Dt5D :C, 7C :D KC$0CT< C >C²>C$>CR <CT=Dä ?Cä2CT@DR >C²=C ?CäAC´5 C²;C,,:C
        MainLayoutInstance?.CloseDrawer();
    }
    else
    {
    }
}

```

```

        System.Diagnostics.Debug.WriteLine($"AôCD$L {href} C05 Ct0D 5C48D BD 8D >C$0C0 2 Cα>CÄ?Cä=CT=D$0D
    }D
}D
}
</file>

<file path="Configuration.Web/Components/NotFound.razor">
@page "/not-found"
@layout MainLayout
D
<PageTitle>404 - B BD 0C08Dd0 C05 C00C"4CT=C Âõ vUF-FÆSí
D
<MudContainer MaxWidth="MaxWidth.Small" Class="d-flex align-center justify-center" Style="height:
70vh;">D
    <MudPaper Elevation="3" Class="pa-8 text-center" Style="width: 100%;">D
        <MudIcon Icon="@Icons.Material.Filled.SearchOff" Color="Color.Error" Size="Size.Large"
Class="mb-4" />D
        <MudText Typo="Typo.h5" Class="mb-2">AäHC,,1Cα0 404</MudText>D
        <MudText Typo="Typo.body2" Class="mud-text-secondary mb-6">D
            A,,7C$8C08D$5, Ct0C0@C HC,,2C 5CÄKC' 0CD@CTA C05 D CD"5D BC$CCTB, C'8C > D >CäBC$5D$AD$2D4ND"8C' <C
        </MudText>D
        <MudButton Variant="Variant.Filled" Color="Color.Primary" href="/">D
            A$5D =D4BDÄADò =C 3C'0C$=D4ND
        </MudButton>D
    </MudPaper>D
</MudContainer>
</file>

<file path="Configuration.Web/Components/Routes.razor">
@using System.Reflection
@inject UiModuleService UiService
D
<Router AppAssembly="@typeof(Routes).Assembly"
    AdditionalAssemblies="@_moduleAssemblies"
    NotFoundPage="@typeof(NotFound)">D
    <Found Context="routeData">D
        <RouteView RouteData="@routeData" DefaultLayout="@typeof(MainLayout)" />D
        <FocusOnNavigate RouteData="@routeData" Selector="h1" />D
    </Found>D
</Router>D
D
@code {D
    private Assembly[] _moduleAssemblies = [];D
D
    protected override void OnInitialized()D
    {D
        _moduleAssemblies = UiService.ModulesD
            .Select(m => m.GetType().Assembly)D
            .Distinct()D
            .ToArray();D
        D
        base.OnInitialized();D
    }D
}
</file>

<file path="Configuration.Web/Configurator/IWebAppConfigurator.cs">
namespace Promatis.Net.Configuration.Web;D
D
public interface IWebAppConfigurator : IAppConfiguratorD
{D
    void ConfigureMiddleware(WebApplication app);D
}
</file>

<file path="Configuration.Web/Configurator/WebAppConfigurator.cs">
using System.Reflection;D
using Microsoft.AspNetCore.Components;D
using MudBlazor.Services;D
D
namespace Promatis.Net.Configuration.Web;D
D
/// <summary>D
/// A 0Ct>C$KC' :Cä=DD8C4CD 0D$>D 4C'0 Web-Cô@C,,;Cä6CT=C,,9 C00 C 0Ct5 Blazor C, xVD&Æ |÷"í
/// </summary>D
public class WebAppConfigurator<TRootComponent> : AppConfigurator, IWebAppConfiguratorD

```

```

where TRootComponent : IComponent
{
    public override void ConfigureServices(IServiceCollection services, IConfiguration
configuration)
    {
        // 1. A 0Ct>C$0Dò 8CÔDD 0D BD CC=BD4@C „ ABÂ !CT@C$8D K, A$0C´8CD0D$>D K, B$@C„3C45D K, B :C =CT@ DL
        base.ConfigureServices(services, configuration);
    }

    // 2. UI C„=DD@C AD$@D4:D$CD 0 (Aô>C„AC¢ •V"ÖöGVÆR 2 D 1Cä@C=0DRÂ =C 2C„3C FC„0)
    services.AddWebInfrastructure(projectPrefix: "Promatis.");

    // B B "AT A A A Ä (MDI): B 5C48D BD 8D CCT< C„=DD@C AD$@D4:D$CD C CÄ=Cä3Cä2C¢;C 4CäGCÔ>C4> C„
    services.AddSingleton<ComponentRegistry>();
    services.AddScoped<TabNavigationService>();

    // 3. B ?CTFC„DC„:C &Æ |÷" „-çFW& 7F-fR 6W'fW"•
    services.AddRazorComponents()
        .AddInteractiveServerComponents();

    // 4. B 5C48D BD 0Dd8Dò 8 CÔ0D BD >C":C xVD&Æ |÷-
    services.AddMudServices(config =>
    {
        config.SnackbarConfiguration.PositionClass =
MudBlazor.Defaults.Classes.Position.BottomRight;
        config.SnackbarConfiguration.PreventDuplicates = false;
        config.SnackbarConfiguration.NewestOnTop = true;
        config.SnackbarConfiguration.ShowCloseIcon = true;
        config.SnackbarConfiguration.VisibleStateDuration = 5000;
    });

    // 5. B„8CÔ0 UI-D >C KD$8C•
    services.AddScoped<IUiCommandBus, UiCommandBus>();

    // B 5C48D BD 0Dd8Dò AC„AD$5CÄ=Cä3Cä 0C=ACTAD >D 0 CD;Dò ?Cä4CD5D 6C=8 C´5CÔ8C$>C4> D 0Ct@CTHCT=C„0 S
    services.AddHttpContextAccessor();
}

/// <summary>
/// AÔ0D BD >C":C :Cä=C$5C"5D 0 Cä1D 0C >D$:C, …EE Ô7C ?D >D >C" „Ö-FFÆWv &R´í
/// </summary>
public virtual void ConfigureMiddleware(WebApplication app)
{
    // A 0Ct>C$KCR Ö-FFÆWv &W2 4C´O D 0C >D$K Web-Cô@C„;Cä6CT=C„0D
    app.UseStaticFiles();
    app.UseAntiforgery();

    // 1. Aô>C´Cdt0Ct< Ct0D 5C48D BD 8D >C$0CÔ=D´5 UI-D 1Cä@C=8 C„7 DI-C= >CÔBCT9CÔ5D 0 DT>D BC
    using IServiceScope scope = app.Services.CreateScope();
    var uiService = scope.ServiceProvider.GetRequiredService<UiModuleService>();

    Assembly[] moduleAssemblies = uiService.Modules
        .Select(m => m.GetType().Assembly)
        .Distinct()
        .ToArray();

    // A„ A„&A„ A´ At Bd Bò A B$+ A$ A´ AD AÆ¢ C ?Cä;CÔ0Ct< D 5CTAD$@ D >D4BCä2 C= >CÄ?Cä=CT=D$0CÄ8 C„7
    var componentRegistry = scope.ServiceProvider.GetRequiredService<ComponentRegistry>();
    componentRegistry.RegisterModules(moduleAssemblies);

    // 2. B 5C48D BD 8D CCT< C= >D =CT2Cä9 C= >CÄ?Cä=CT=D" 8 D :C =C„@D45CÄ AD$@C =C„FD² <Cä4D4;CT9 CÔ0 D 5
    app.MapRazorComponents<TRootComponent>()
        .AddInteractiveServerRenderMode()
        .AddAdditionalAssemblies(moduleAssemblies);
}

public override void ConfigureApp(IHost app)
{
    // A$Kct>C" 1C 7Cä2Cä9 C„=C„FC„0C´8Ct0Dd8C, „0C$BCä@CT3C„AD$@C FC„0 D$@C„3C45D >C" AB GCT@CT7 D :C =
    base.ConfigureApp(app);

    // At4CTADÂ <Cä6CÔ> CD>C 0C$8D$L Cô@Cä2CT@C=¢ Ct4Cä@Cä2DÄO A C„;C, =C GC ;DÄ=D´9 Seed CD0CÔ=D´E CD;
}
}
</file>

```

<file path="Configuration.Web/Configurator/WebScanningExtensions.cs">

```

using Promatis.Net.UI;
using System.Reflection;
namespace Promatis.Net.Configuration.Web;
public static class WebInfrastructureExtensions
{
    public static void AddWebInfrastructure(this IServiceCollection services, string projectPrefix
= "Promatis.")
    {
        Console.WriteLine("[SCANNER] At0C0CD : C0@Cä3D 5C$0 D 1Cä@Cä: CÄ>CÔ>C´8D$0 (B$>C´LCα> UI-CÄ>CDCC´8).");
        string binPath = AppContext.BaseDirectory;
        // A,,!Aô A A´ AÔ : AÔ0 D4@Cä2CÔ5 DD0C";Cä2Cä9 D 8D BCT<D² 8D"5CÄ BCä;DÄ:Cä BCR FÆÄÄ :CäBCä@D´5 CÔ0Dt
        string[] assemblyFiles = Directory.GetFiles(binPath, $"{projectPrefix}.UI.dll",
SearchOption.TopDirectoryOnly);
        foreach (string file in assemblyFiles)
        {
            try
            {
                // Aô@C,,=D44C,,BCT;DÄ=Cä 7C 3D CCd0CT< C" 76VÖ&Ç"Æö D6öçFW‡BäFVf VÇM
                Assembly.LoadFrom(file);
            }
            catch (Exception ex)
            {
                Console.WriteLine($"[SCANNER] AÔ5 D44C ;CäADÄ 7C 3D CCT8D$L DD0C"; D 1Cä@Cα8 {Path.GetFileName
});
            }
        }
        // A,,!Aô A A´ AÔ : BD8C´LD$@D45CÄ FöÖ -âÄ 1CT@Dò AC >D :C, AD$@Cä3Cä A Cò@CTDC,,:D >CÄ 8 Cä:Cä=Dt0
        Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => a.FullName != null
                && a.FullName.StartsWith(projectPrefix)
                && a.GetName().Name!.EndsWith(".UI"))
            .Distinct()
            .ToArray();
        Console.WriteLine($"[SCANNER] B :C =C,,@Cä2C =C,,5 Ct0C$5D HCT=Cää C 9CD5CÔ> {assemblies.Length} Dd5C´
        Console.WriteLine();
        Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T´Ô:Cä<Cô>CÔ5CÔBCä2 C, <Cä4D4;CT9 CÔ0C$8C40Dd8C, GCT@C
        // A 2D$>CÄ0D$8Dt5D :Cä5 D :C =C,,@Cä2C =C,,5 C, @CT3C,,AD$@C FC,,O C,,=D$5D DCT9D >C" •V"ÖöGVÆ]
        services.Scan(scan => scan
            .FromAssemblies(assemblies)
            .AddClasses(c => c.AssignableTo<IUiModule>()).Where(t => !t.IsAbstract))
            .AsImplementedInterfaces()
            .WithScopedLifetime()
        );
        // B 5C48D BD 0Dd8Dò 2C HCT3Cä 0C4@CT3C BCä@C <Cä4D4;CT9
        services.AddScoped<IUiModuleService>();
        // A´>C48D >C$0CÔ8CR >C =C @D46CT=CÔKDR ?D4=CαBCä2 CÄ5CÔND
        IEnumerable<Type> uiModuleTypes = assemblies
            .SelectMany(a => a.GetTypes())
            .Where(t => typeof(IUiModule).IsAssignableFrom(t) && !t.IsInterface && !t.IsAbstract);
        foreach (Type type in uiModuleTypes)
        {
            try
            {
                if (Activator.CreateInstance(type) is IUiModule module)
                {
                    Console.WriteLine($"[SCANNER] UI AÄ>CDCC´L: {module.Name}");
                    List<(string Title, string Href, string Icon, string? Group)> menuItems =
module.GetMenuItems()?.ToList() ?? new();
                    if (!menuItems.Any())
                    {
                        Console.WriteLine(" % % [AôCD BCä5 CÄ5CÔN]");
                        continue;
                    }
                }
            }
        }
    }
}

```

```

    IEnumerable<(string Title, string Href, string Icon, string? Group)> rootItems
= menuItems.Where(i => string.IsNullOrEmpty(i.Group));
    IEnumerable<IGrouping<string?, (string Title, string Href, string Icon, string?
Group)>> groupedItems = menuItems.Where(i => !string.IsNullOrEmpty(i.Group)).GroupBy(i => i.Group);
    foreach ((string Title, string Href, string Icon, string? Group) item in
rootItems)
    {
        Console.WriteLine($"          % % {item.Title} -> {item.Href}");
    }
    foreach (IGrouping<string?, (string Title, string Href, string Icon, string?
Group)> group in groupedItems)
    {
        Console.WriteLine($"          % % [{group.Key}]");
        List<(string Title, string Href, string Icon, string? Group)> itemsInGroup
= group.ToList();
        for (int i = 0; i < itemsInGroup.Count; i++)
        {
            string prefix = (i == itemsInGroup.Count - 1) ? "          % % % " :
            Console.WriteLine($"{prefix} {itemsInGroup[i].Title} ->
{itemsInGroup[i].Href}");
        }
    }
    catch (Exception ex)
    {
        Console.WriteLine($"[SCANNER] AäHC,,1C D tBCT=C,,O CÄ5CÔN CD;Dò BC,,?C .G- Räæ ÖWÓ¢ ¶W,äÖW76 v
    }
    Console.WriteLine("[SCANNER] B 5C48D BD 0Dd8Dò T'Ô:Cä<Cò>CÔ5CÔBCä2 D4ACô5D,,=Cä 7C 2CT@D,,5CÔ0");
    Console.WriteLine();
}
</file>

<file path="Configuration.Web/Promatis.Net.Configuration.Web.csproj">
<Project Sdk="Microsoft.NET.Sdk.Web">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
        "Ä÷WG WEG- SäÆ-'& '"Âô÷WG WEG- Sí
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="MudBlazor" Version="9.4.0" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Configuration\Promatis.Net.Configuration.csproj" />
        <ProjectReference Include="..\Promatis.Net.UI\Promatis.Net.UI.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Configuration.Web/Properties/launchSettings.json">
{
    "profiles": {
        "Promatis.Net.Configuration.Web": {
            "commandName": "Project",
            "launchBrowser": true,
            "environmentVariables": {
                "ASPNETCORE_ENVIRONMENT": "Development"
            },
            "applicationUrl": "https://localhost:56819;http://localhost:56820"
        }
    }
}
</file>

```

```

<file path="Configuration.Web/Services/TabNavigationService/ComponentRegistry.cs">
using System.Reflection;
using Microsoft.AspNetCore.Components;

namespace Promatis.Net.Configuration.Web;

public class ComponentRegistry
{
    private readonly Dictionary<string, Type> _routeMap = new(StringComparer.OrdinalIgnoreCase);

    /// <summary>
    /// B :C =C,@D45D" AC >D :C, <Cä4D4;CT9 C, AD$@Cä8D" :C @D$: URL -> B Æ 72 "C,,? C@>CÄ?Cä=CT=D$0D
    /// </summary>
    public void RegisterModules(IEnumerable<Assembly> assemblies)
    {
        foreach (Assembly assembly in assemblies)
        {
            IEnumerable<Type> componentTypes = assembly.GetTypes()
                .Where(t => typeof(IComponent).IsAssignableFrom(t) && !t.IsAbstract);

            foreach (Type type in componentTypes)
            {
                // A,,ICT< C BD 8C CD" µ&÷WFT GG&- 'WfU0Â 2 C@>D$>D KC' :Cä<C08C'8D CCTBD 0 CD8D 5C@BC,,2C
                IEnumerable<RouteAttribute> routeAttributes =
                    type.GetCustomAttributes<RouteAttribute>();
                foreach (RouteAttribute attr in routeAttributes)
                {
                    _routeMap[attr.Template] = type;
                }
            }
        }
    }

    public Type? GetComponentTypeByRoute(string route)
    {
        return _routeMap.TryGetValue(route, out Type? type) ? type : null;
    }
}
</file>

```

```

<file path="Configuration.Web/Services/TabNavigationService/TabItem.cs">
namespace Promatis.Net.Configuration.Web;

public class TabItem
{
    public string Id { get; init; } = string.Empty; // A00D, ‡&Vm
    public string Title { get; init; } = string.Empty;
    public string Icon { get; init; } = string.Empty;
    public Type ComponentType { get; init; } = null!;
}
</file>

```

```

<file path="Configuration.Web/Services/TabNavigationService/TabNavigationService.cs">
namespace Promatis.Net.Configuration.Web;

public class TabNavigationService
{
    public List<TabItem> OpenTabs { get; } = new();
    public int ActiveTabIndex { get; set; }
    public event Action? OnTabsChanged;

    public void OpenTab(string title, string href, string icon, Type componentType)
    {
        TabItem? existingTab = OpenTabs.FirstOrDefault(t => t.Id.Equals(href,
            StringComparer.OrdinalIgnoreCase));

        if (existingTab != null)
        {
            ActiveTabIndex = OpenTabs.IndexOf(existingTab);
        }
        else
        {
            var newTab = new TabItem
            {
                Id = href,
                Title = title,
            }
        }
    }
}

```

```

        Icon = icon;
        ComponentType = componentType;
    };

    OpenTabs.Add(newTab);
    ActiveTabIndex = OpenTabs.Count - 1;
}

    NotifyStateChanged();
}

/// <summary>
/// A 5Ct>C00D =Cä5 Ct0C@D'BC,,5 C$:C'0CD:C, ?Cä 5E GC,,AC'>C$>CÄC C,,=CD5C#AD=
/// </summary>
public void CloseTabByIndex(int index)
{
    if (index < 0 || index >= OpenTabs.Count) return;

    OpenTabs.RemoveAt(index);

    // A,,=D$5C';CT:D$CC ;DÄ=D'9 D 0D GCTB DD>C#CD 0 C00 CäAD$0C$HC,,5D 0 C$:C'0CD:C•
    if (ActiveTabIndex >= OpenTabs.Count)
    {
        ActiveTabIndex = OpenTabs.Count - 1;
    }

    if (ActiveTabIndex < 0 && OpenTabs.Count > 0)
    {
        ActiveTabIndex = 0;
    }

    NotifyStateChanged();
}

    private void NotifyStateChanged() => OnTabsChanged?.Invoke();
}
</file>

<file path="Configuration.Web/Services/UiModuleService.cs">
using Promatis.Net.UI;

namespace Promatis.Net.Configuration.Web;

// B 5D 2C,,A, C#>D$>D KC' ACä1C,,@C 5D" 2D Q C$>CT4C,,=Cí
public class UiModuleService(IEnumerable<IUiModule> modules)
{
    public IEnumerable<IUiModule> Modules => modules;
}
</file>

<file path="Configuration.Web/UiCommandBus/IUiCommandBus.cs">
using MudBlazor;

namespace Promatis.Net.Configuration.Web;

/// <summary>
/// A,,=D$5D DCT9D HC,,=D² T'0ACä1D'BC,,9D
/// </summary>
public interface IUiCommandBus
{
    /// <summary>
    /// B 5C48D BD 0Dd8Dò >C @C 1CäBDt8C#0 C$Kct>C$0 DD>D <D² ,,2D'?Cä;C00CTBD 0 C0@C,,=C,,<C ND"8CÂ <Cä4D4;CT<,
    /// </summary>
    void RegisterHandler(string commandName, Func<Dictionary<string, object>, Task<DialogResult?>>
handler);

    /// <summary>
    /// A$7C 8CÄ>CD5C"AD$2C,,5 CÄ5Cd4D2 <Cä4D4;Dô<C, 2D ;CT?D4N (C$KctKC$0CTBD 0 C,,7 MDM)D
    /// </summary>
    Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object> parameters);
}
</file>

<file path="Configuration.Web/UiCommandBus/UiCommandBus.cs">
using MudBlazor;


```

```

namespace Promatis.Net.Configuration.Web;
{
    /// <summary>
    /// B„8C00 UI-D >C KD$8C•
    /// </summary>
    public class UiCommandBus : IUiCommandBus
    {
        // A0>D$>C>C 5Ct>C00D =D´9 D ;Cä2C @DÄ 4C´O D 5C48D BD 0Dd8C, :Cä<C =CB 2 D 0C0BC 9CÄ5 SignalRD
        private readonly System.Collections.Concurrent.ConcurrentDictionary<string,
        Func<Dictionary<string, object>, Task<DialogResult?>>> _handlers = new();
        public void RegisterHandler(string commandName, Func<Dictionary<string, object>,
        Task<DialogResult?>> handler)
        {
            _handlers[commandName] = handler ?? throw new ArgumentNullException(nameof(handler));
        }
        public async Task<DialogResult?> ExecuteAsync(string commandName, Dictionary<string, object>
        parameters)
        {
            if (_handlers.TryGetValue(commandName, out Func<Dictionary<string, object>,
            Task<DialogResult?>>? handler))
            {
                return await handler(parameters);
            }
            // ATAC´8 CÄ>CDCC´L (C00C0@C„<CT@, DCA) C05 C0>CD:C´NDt5C0 2 C´0D4=Dt5D 5, D 8D BCT<C =CR CC00CD5D"Ä
            return null;
        }
    }
}
</file>

```

```

<file path="Configuration.Web/WebAppBootstrapper.cs">
using Microsoft.AspNetCore.Components;
{
    namespace Promatis.Net.Configuration.Web;
    {
        // A00D ;CT4C08C¢ &ö÷G7G& W"Ä :CäBCä@D´9 D4<CT5D" 7C ?D4AC=0D$L Web-D 5D 2CT@D
        public abstract class WebAppBootstrapper<TRootComponent> : AppBootstrapper
        where TRootComponent : IComponent
        {
            public override void Run(string[] args)
            {
                // B =Cä2C 7C 3D CCd0CT< DLL (C00 D ;D4GC 9 C0@D0<Cä3Cä 7C ?D4AC=0)D
                // LoadProjectAssemblies ("Promatis.") D46CR 2D´7C$0C00 C" & 6RÄ'VâÄ
                // C0> Ct4CTADÄ <D² 8D ?Cä;DÄ7D45CÄ vV$ Æ-6 F-öä'V-ÆFW"i
                // Aä B0 A "AT BÄ Aä¢ CT7 D0BCä3Cä F0Ö -â =CR CC$8CD8D" <Cä4D4;C, ?D 8 Type.GetTypeD
                LoadProjectAssemblies("Promatis.");
                WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
                List<IAppConfigurator> configs = GetConfigurators(builder.Configuration).ToList();
                foreach (IAppConfigurator cfg in configs)
                {
                    cfg.ConfigureServices(builder.Services, builder.Configuration);
                }
                WebApplication app = builder.Build();
                // A00D BD >C":C Ö-FFÆWv &R <Cä4D4;CT9D
                foreach (IWebAppConfigurator cfg in configs.OfType<IWebAppConfigurator>())
                {
                    cfg.ConfigureMiddleware(app);
                }
                foreach (IAppConfigurator cfg in configs)
                {
                    cfg.ConfigureApp(app);
                }
                app.Run();
            }
        }
    }
}
</file>

```

```

<file path="Configuration/AppBootstrapper.cs">
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.Hosting;
using System.Reflection;

```

```

using System.Runtime.Loader;
{
namespace Promatis.Net.Configuration;
{
public abstract class AppBootstrapper
{
    public virtual void Run(string[] args)
    {
        // 1. A A, B4 A,"AT BÄ A / At A4 B4 A DLL (DtBCä1D² G- RävWEG- R 8DR 2C,,4CT;)
        LoadProjectAssemblies("Promatis.");

        HostApplicationBuilder builder = Host.CreateApplicationBuilder(args);

        // 2. B4 A+A' Aä B A A$A,, B4 A "Aä Aä A,, JSON
        List<IAppConfigurator> configurators = GetConfigurators(builder.Configuration).ToList();

        foreach (IAppConfigurator config in configurators)
        {
            config.ConfigureServices(builder.Services, builder.Configuration);
        }

        IHost host = builder.Build();

        foreach (IAppConfigurator config in configurators)
        {
            config.ConfigureApp(host);
        }

        host.Run();
    }

    protected virtual IEnumerable<IAppConfigurator> GetConfigurators(IConfiguration configuration)
    {
        string[] typeNames = configuration.GetSection("LauncherSettings:Modules").Get<string[]>()
            ?? [];

        foreach (string name in typeNames)
        {
            // A,ICT< D$8Cð 2Cä 2D 5DR 7C 3D CCd5CÔ=D'E D 1Cä@CðD]
            Type? type = AppDomain.CurrentDomain.GetAssemblies()
                .Select(a => a.GetType(name))
                .FirstOrDefault(t => t != null);

            if (type != null && typeof(IAppConfigurator).IsAssignableFrom(type))
            {
                if (Activator.CreateInstance(type) is IAppConfigurator instance)
                {
                    yield return instance;
                }
            }
            else
            {
                Console.WriteLine($"[BOOTSTRAPPER][WARN] B$8Cð =CR =C 9CD5Cð 8C`8 CÔ5 C$0C`8CD5CÓ¢ ¶æ ÖWð""½
            }
        }
    }

    protected void LoadProjectAssemblies(string prefix)
    {
        string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
        if (path == null) return;

        foreach (string file in Directory.GetFiles(path, $"{prefix}*.dll"))
        {
            try
            {
                AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
            }
            catch { /* A,,3CÔ>D 8D CCT< CäHC,,1Cð8 Ct0C4@D47Cð8 */ }
        }
    }
}
</file>

<file path="Configuration/AppConfigurator.cs">
using Microsoft.Extensions.Configuration;

```

```

using Microsoft.Extensions.DependencyInjection;
using Microsoft.Extensions.Hosting;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Promatis.Net.Configuration;

public class AppConfigurator : IAppConfigurator
{
    public virtual void ConfigureServices(IServiceCollection services, IConfiguration configuration)
    {
        // B :C =C,,@D45CÂ 2D Q (C$:C'NDt0Dò VF-EG&-vvW" 2 Cò@Cä5CαBCR F F •
        services.AddDomainInfrastructure(projectPrefix: "Promatis.");

        // B 5C48D BD 8D CCT< C 0Ct>C$KCR ACT@C$8D Kð
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
            sp.GetRequiredService<DatabaseTriggerService>());
        services.AddScoped<DatabaseTriggerInterceptor>();

        // Aä Bd Af¢ ¥4ôâ =C AD$@Cä9Cα8ð
        services.AddSingleton(new JsonSerializerOptions
        {
            PropertyNamingPolicy = JsonNamingPolicy.CamelCase,
            DefaultIgnoreCondition = JsonIgnoreCondition.WhenWritingNull,
            WriteIndented = false
        });

        // B A4 B "B Bd Bò A´ B$Aä AÄ AÔ Aä Aâ A´ AÔ B B #B" Aä!B$ A•
        services.AddSingleton<IEntityCloner, JsonEntityCloner>();
    }

    public virtual void ConfigureApp(IHost app)
    {
        if (app == null) throw new ArgumentNullException(nameof(app));

        // A,,=C,,FC,,0C´8Ct8D CCTBD O Cä4C0>Cα@C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dý
        AppInfrastructure.Initialize(app.Services);

        using IServiceScope scope = app.Services.CreateScope();
        scope.ServiceProvider.AutoRegisterTriggers();
    }
}
</file>

```

```

<file path="Configuration/AppInfrastructure.cs">
namespace Promatis.Net.Configuration;

/// <summary>ð
/// A4;Cä1C ;DÄ=C O D 8D BCT<C00Dò BCäGCα0 CD>D BD4?C : C,,=DD@C AD$@D4:D$CD =D´< D 5D 2C,,AC < Cò;C BDD>D <D²
/// A,,=C,,FC,,0C´8Ct8D CCTBD O Cä4C0>Cα@C BC0> Cò@C, AD$0D BCR ?D 8C´>Cd5C08Dò 8 C,,ACα;DäGC 5D" @C 7CDCC$0C08CR
/// </summary>ð
public static class AppInfrastructure
{
    private static IServiceProvider? _provider;
    private static readonly object _lock = new();

    /// <summary>ð
    /// Read-only CD>D BD4? C¢ 3C´>C 0C´LC0>CÄC Cα>C0BCT9C05D C Ct0C$8D 8CÄ>D BCT9.ð
    /// </summary>ð
    public static IServiceProvider Provider
    {
        get
        {
            if (_provider == null)
            {
                throw new InvalidOperationException(
                    "Aα@C,,BC,,GCTACα0Dò >D,,8C :C ¢ -æg& 7G'V7GW&R =CR 8C08Dd8C ;C,,7C,,@Cä2C =. " +
                    "B41CT4C,,BCTADÄÄ GD$> CÄ5D$>CB -æg& 7G'V7GW&Rä-æ-F- Æ-!R,' 2D´7C$0C0 2 Program.cs C,,;C
                );
            }
            return _provider;
        }
    }
}

```

```

Ð
/// <summary>Ð
/// Að>D$>Cð>C 5Ct>Cð0D =C 0 C,,=C,,FC,,0C`8Ct0Dd8Dð ;Cä:C BCä@C â D´7D´2C 5D$ADð AD$@Cä3Cä >CD8Cð @C 7 Cð@
/// </summary>Ð
public static void Initialize(IServiceProvider provider)Ð
{Ð
    if (provider == null) throw new ArgumentNullException(nameof(provider));Ð
    if (_provider != null) return;Ð
    lock (_lock)Ð
    {Ð
        _provider ??= provider;Ð
    }Ð
}
}
</file>

<file path="Configuration/IAppConfigurator.cs">
using Microsoft.Extensions.Configuration;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Microsoft.Extensions.Hosting;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public interface IAppConfiguratorÐ
{Ð
    void ConfigureServices(IServiceCollection services, IConfiguration configuration);Ð
    void ConfigureApp(IHost app);Ð
}
</file>

<file path="Configuration/Promatis.Net.Configuration.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    .
    </PropertyGroup>Ð
Ð
Ð
Ð
"Ä-FVÔw&÷W í
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFSÔ$Ô-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ær" -æ|V7F-öää '7G& 7F-öç2"
Version="10.0.8" />Ð
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFSÔ$Ô-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ær" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFSÔ$Ô-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ærä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFSÔ$Ô-7&÷6ögBäW†FVÇ6-öç2ä†÷7F-ærä '7G& 7F-öç2" fW'6-öä0# ä ä," óí
' Ä 6¶ vU&VfW&Væ6R -æ6ÇVFSÔ%67'WF÷" fW'6-öä0#rä ä " óí
"Äô-FVÔw&÷W í
Ð
Ð
Ð
"Ä-FVÔw&÷W í
' Ä &ö|V7E&VfW&Væ6R -æ6ÇVFSÔ"ääÄ6W'f-6UÄ &öÖ F-2äæWBä6W'f-6Ræ77 &öç" óí
"Äô-FVÔw&÷W í
Ð
Ð
</Project>
</file>

<file path="Configuration/ServiceScanningExtensions.cs">
using FluentValidation;Ð
using Microsoft.Extensions.DependencyInjection;Ð
using Promatis.Net.Data;Ð
using Promatis.Net.Domain;Ð
using Promatis.Net.Service;Ð
using System.Reflection;Ð
using System.Runtime.Loader;Ð
Ð
namespace Promatis.Net.Configuration;Ð
Ð
public static class ServiceScanningExtensionsÐ
{Ð
    public static void AddDomainInfrastructure(this IServiceCollection services, string

```

```

projectPrefix = "Promatis.")
{
    Console.WriteLine();
    Console.WriteLine("[SCANNER] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, 2 DI...");
}

// 1. A0 A,, B4 A,, "AT BÄ A / At A4 B4 A B Aä Aä
string? path = Path.GetDirectoryName(Assembly.GetExecutingAssembly().Location);
if (path != null)
{
    foreach (string file in Directory.GetFiles(path, $"{projectPrefix}*.dll"))
    {
        try
        {
            AssemblyLoadContext.Default.LoadFromAssemblyPath(file);
        }
        catch (Exception ex)
        {
            string fileName = Path.GetFileName(file);
            Console.ForegroundColor = ConsoleColor.Red;
            Console.WriteLine($"[SCANNER][ERROR] A05 D44C ;CäADÄ 7C 3D CCT8D$L D 1Cä@C C {fileName}");
            Console.ResetColor();
        }
    }
}

// A 5D 5CÄ 2D 5 Ct0C4@D46CT=C0KCR AC >D :C, A C$0D,,8CÄ AC,,AD$5CÄ=D'< C0@CTDC,,:D >Cí
Assembly[] assemblies = AppDomain.CurrentDomain.GetAssemblies()
    .Where(a => a.FullName != null && a.FullName.StartsWith(projectPrefix))
    .Distinct()
    .ToArray();

int countBefore = services.Count;

// 2. A B$ AÄ B$ Bt B Aä B A A,, Aä A A,, A, AT A,,!B$ A &A,,/ Bt B Ar 45%UDö-
services.Scan(scan =>
{
    // A'>C=0C'LC00D0 DD4=C=FC,,0 CD;D0 @CT:D4@D 8C$=Cä9 C0@Cä2CT@C=8 C$ACT9 Dd5C0>Dt:C, =C AC'5CD>C$0
    bool IsSubclassOfRawGeneric(Type generic, Type? toCheck)
    {
        while (toCheck != null && toCheck != typeof(object))
        {
            Type cur = toCheck.IsGenericType ? toCheck.GetGenericTypeDefinition() : toCheck;
            if (generic == cur) return true;
            toCheck = toCheck.BaseType;
        }
        return false;
    }

    scan.FromAssemblies(assemblies)
        // A,,!A0 A A' A0 : B 5C48D BD 0Dd8D0 !AT A$ B A" A C0>CD4CT@Cd:Cä9 C4;D41Cä:Cä3Cä =C AC'5CD
        .AddClasses(classes => classes.Where(type =>
            !type.IsAbstract &&
            !type.IsGenericTypeDefinition &&
            IsSubclassOfRawGeneric(typeof(BaseService<,,>), type)))
        .AsSelf()
        .AsImplementedInterfaces()
        .WithScopedLifetime()

        // B 5C48D BD 0Dd8D0 A A,, A "Aä Aä (FluentValidation)
        .AddClasses(c => c.AssignableTo(typeof(IValidator<>))
            .Where(t => !t.IsAbstract && !t.IsGenericTypeDefinition))
        .AsImplementedInterfaces()
        .WithTransientLifetime()

        // B 5C48D BD 0Dd8D0 "B A4 AT Aä (Before/After Save)
        .AddClasses(c => c.AssignableToAny(typeof(IBeforeSaveTrigger<>),
            typeof(IAfterSaveTrigger<>)).Where(t => !t.IsAbstract))
        .AsSelfWithInterfaces()
        .WithScopedLifetime();
});

// --- B A4 B "B Bd B0 A' A A',A0 A4 A0 A' AÄ B $A0 A4 A$ A' AD B$ B B0 B ---
// B 5C48D BD 8D CCT< C=0C C >D$:D KD$KC' 4Cd5C05D 8C#ä "CT?CT@DÄ ?D 8 Ct0C0@CäACR •f Æ-F F÷#Ä Dä1Cä9A
// DI-C= >C0BCT9C05D ääUB 3C @C =D$8D >C$0C0=Cä >D$4C AD" =C H C0>C'8CÄ>D DC0KC' 4C,,AC05D$GCT@D
services.AddScoped(typeof(IValidator<>), typeof(GlobalPolymorphicValidator<>));

```

```

// 3. A,,!Aô A A´ Aô Aä A, A AT Aô AR Aä A,, Aä A A,, B At#A´,B$ B$ A-
List<ServiceDescriptor> newRegistrations = services.Skip(countBefore).ToList();ð
var loggedTypes = new HashSet<string>();ð

foreach (ServiceDescriptor reg in newRegistrations)ð
{ð
    // A 5D 5CÂ @CT0C´LC0KC´ BC,,? D 5C ;C,,7C FC,,8 (C" ?D 8Cä@C,,BCTBCR 8Cr -x ÆVÖVçF F-öâG- RÂ 8C00Dt5
    Type? implType = reg.ImplementationType ?? reg.ImplementationInstance?.GetType();ð

    if (implType == null) continue;ð

    // --- A A A,, A$ A´ AD B$ B A" 00Ý
    bool isValidator = reg.ServiceType.IsGenericType &&
        reg.ServiceType.GetGenericTypeDefinition() == typeof(IValidator<>);ð

    if (isValidator && loggedTypes.Add($"Val:{implType.FullName}"))ð
    {ð
        var inheritanceChain = new List<string>();ð
        Type? currentType = implType;ð

        while (currentType != null && currentType != typeof(object))ð
        {ð
            string typeName = currentType.Name.Contains('.') ? currentType.Name.Split('.')
[0] : currentType.Name;ð
            inheritanceChain.Add(typeName);ð
            currentType = currentType.BaseType;ð
            if (typeName == "AbstractValidator") break;ð
        }ð

        string chainDisplay = string.Join(" -> ", inheritanceChain);ð
        Console.WriteLine($"[SCANNER] A$0C´8CD0D$>D ¢ ¶6† -äF-7 Æ -Ö ¢ •f Æ-F F÷""½
        continue; // A´>C2 4C´0 D0BCä9 Ct0C08D 8 C$KC$5CD5C0Â 8CD5CÂ : D ;CT4D4ND"5C•
    }ð

    // --- A A A,, B B A,,!Aä (A00D ;CT4C08C¤>C" & 6U6W'f-6R´ 00Ý
    // A,,!Aô A A´ Aô : B41D 0C0> Cd5D BC¤>CR CD ;Cä2C,,5 reg.ServiceType == reg.ImplementationType, ð
    // D$0C¢ :C : D 5D 2C,,AD² BCT?CT@DÂ @CT3C,,AD$@C,,@D4ND$AD0 ?C @C <C, C´0D A + A,,=D$5D DCT9D
    bool isService = false;ð
    var serviceChain = new List<string>();ð
    Type? currentServiceType = implType;ð

    while (currentServiceType != null && currentServiceType != typeof(object))ð
    {ð
        string typeName = currentServiceType.Name.Contains('.') ?
currentServiceType.Name.Split('.')[0] : currentServiceType.Name;ð
        serviceChain.Add(typeName);ð

        if (currentServiceType.IsGenericType &&
currentServiceType.GetGenericTypeDefinition() == typeof(BaseService<,,>))ð
        {ð
            isService = true;ð
            break;ð
        }ð
        currentServiceType = currentServiceType.BaseType;ð
    }ð

    if (isService && loggedTypes.Add($"Srv:{implType.FullName}"))ð
    {ð
        string chainDisplay = string.Join(" -> ", serviceChain);ð
        Console.WriteLine($"[SCANNER] B 5D 2C,,A: {chainDisplay}");ð
        continue;ð
    }ð

    // --- A A A,, B$ A,, A4 B A" 00Ý
    bool isTrigger = implType.GetInterfaces().Any(i => i.IsGenericType &&
        (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||ð
        i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));ð

    if (isTrigger && loggedTypes.Add($"Tri:{implType.FullName}"))ð
    {ð
        Console.WriteLine($"[SCANNER] B$@C,,3C45D ¢ ¶-x ÅG- Ræ ÖW0""½
    }ð
}ð

Console.WriteLine("[SCANNER] A 2D$>CÄ0D$8Dt5D :C 0 D 5C48D BD 0Dd8D0 CD ?CTHC0> Ct0C$5D HCT=C â""½
Console.WriteLine();ð

```

```

    }
}

public static void AutoRegisterTriggers(this IServiceProvider serviceProvider)
{
    Console.WriteLine("[AUTOREG] At0C0CD : C 2D$>CÄ0D$8Dt5D :Cä9 D 5C48D BD 0Dd8C, BD 8C43CT@Cä2 C" F F &

    var triggerService = serviceProvider.GetRequiredService<IDatabaseTriggerService>();

    // 1. A0>C,,AC¢ 2D 5DR :C´0D ACä2 D$@C,,3C45D >C" 2 D 1Cä@C¢0DR &öÖ F-2â-
    IEnumerable<Type> triggerTypes = AppDomain.CurrentDomain.GetAssemblies()
        .Where(a => a.FullName?.StartsWith("Promatis.") == true)
        .SelectMany(s => s.GetTypes())
        .Where(t => t is { IsClass: true, IsAbstract: false } &&
            t.GetInterfaces().Any(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() ==
                    typeof(IBeforeSaveTrigger<>) ||
                    i.GetGenericTypeDefinition() ==
                    typeof(IAfterSaveTrigger<>))))
        .ToList();

    MethodInfo? registerMethod =
        typeof(IDatabaseTriggerService).GetMethod(nameof(IDatabaseTriggerService.Register));

    int totalBindings = 0;

    foreach (Type triggerType in triggerTypes)
    {
        // A00DT>CD8CÄ 2D 5 C,,=D$5D DCT9D K D$@C,,3C45D >C"Â :CäBCä@D´5 D 5C ;C,,7D45D" 4C =C0KC´ :C´0D A0
        IEnumerable<Type> interfaces = triggerType.GetInterfaces()
            .Where(i => i.IsGenericType &&
                (i.GetGenericTypeDefinition() == typeof(IBeforeSaveTrigger<>) ||
                    i.GetGenericTypeDefinition() == typeof(IAfterSaveTrigger<>)));

        foreach (Type @interface in interfaces)
        {
            // A0>C´CDt0CT< D$8C0 AD4IC0>D BC, ...B 8Cr "&Vf÷&U6 fUG&-vvW#ÂCä•
            Type entityType = @interface.GetGenericArguments()[0];

            // Aä?D 5CD5C´0CT< D$8C0 BD 8C43CT@C 4C´O C¢@C AC,,2Cä3Cä ;Cä3C
            string triggerKind = @interface.GetGenericTypeDefinition() ==
                typeof(IBeforeSaveTrigger<>)
                    ? "Before"
                    : "After ";

            try
            {
                // A$KCTkC$0CT< Register<TEntity, TTrigger>()
                MethodInfo genericRegister = registerMethod!.MakeGenericMethod(entityType,
                    triggerType);

                genericRegister.Invoke(triggerService, null);

                // A´ A4 B A$ A0 AR #B AT(A0 A´ B A$/At A•
                Console.WriteLine($"[AUTOREG] [{triggerKind}] {entityType.Name} <---
{triggerType.Name}");
                totalBindings++;
            }
            catch (Exception ex)
            {
                Console.ForegroundColor = ConsoleColor.Red;
                Console.WriteLine($"[AUTOREG][ERROR] AäHC,,1C¢0 C0@C,,2D07C¢8 {triggerType.Name} C¢ ¶VçF-G•
{ex.InnerException?.Message ?? ex.Message}");
                Console.ResetColor();
            }
        }
    }

    Console.WriteLine($"[AUTOREG] At0C$5D HCT=Cäâ !Cä7CD0C0> C0@C,,2D07Cä:: {totalBindings}");
    Console.WriteLine();
}
}
</file>

```

```

<file path="Data/Configurations/AuditLogConfiguration.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata.Builders;
using Promatis.Net.Domain;

```

```

Ð
namespace Promatis.Net.Data;Ð
Ð
public class AuditLogConfiguration : IEntityTypeConfiguration<AuditLog>Ð
{Ð
    public void Configure(EntityTypeBuilder<AuditLog> builder)Ð
    {Ð
        builder.Property(e => e.EntityName).HasMaxLength(100);Ð
        builder.Property(e => e.Action).HasMaxLength(50);Ð
        builder.Property(e => e.ChangedBy).HasMaxLength(100);Ð
    }Ð
    // ChangesJson CăAD$0C$;Dô5CĂ 1CT7 C`8CĂ8D$0 (text/jsonb), D$0C¢ :C : D ?C,,ACă: C,,7CĂ5C05C08C' <Că6CT
    builder.Property(e => e.ChangesJson).HasColumnType("jsonb");Ð
}Ð
}
</file>

<file path="Data/Configurations/ModelBuilderConventions.cs">
using Microsoft.EntityFrameworkCore;Ð
using Microsoft.EntityFrameworkCore.Metadata;Ð
using System.Linq.Expressions;Ð
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Data;Ð
Ð
public static class ModelBuilderConventionsÐ
{Ð
    public static void ApplyGlobalConventions(this ModelBuilder modelBuilder)Ð
    {Ð
        foreach (IMutableEntityType entityType in modelBuilder.Model.GetEntityTypes())Ð
        {Ð
            Type type = entityType.ClrType;Ð
            // A`8CĂ8D$K CD;Dô AD$@Că: (255 C0> D4<Că;Dt0C08Dă•
            foreach (IMutableProperty property in entityType.GetProperties())Ð
            {Ð
                if (property.ClrType == typeof(string))Ð
                {Ð
                    if (property.GetMaxLength() == null)Ð
                    {
                        property.SetMaxLength(255);Ð
                    }Ð
                }Ð
            }Ð
            // A 2D$>-C¢;DăGC, 4C`0 Guid (ValueGeneratedNever, D$0C¢ :C : Id D >Ct4C 5D$ADò 2 DomainObject)Ð
            if (typeof(IDomainObjectHasKey<Guid>).IsAssignableFrom(type))Ð
            {Ð
                modelBuilder.Entity(type).Property("Id").ValueGeneratedNever();Ð
            }Ð
            // A B$ -BD A´,B$ C, A$"Aâô Aô AT B 4C`0 SoftDelete Că1D¤5C¤BCă2Ð
            if (typeof(ISoftDeletable).IsAssignableFrom(type))Ð
            {Ð
                IMutableEntityType? baseType = entityType.BaseType;Ð
                // B BC 2C,,< DD8C`LD$@ D$>C`LC¤> C00 C¤>D 5C0L C,,5D 0D EC,,8 (C$:C`NDt0Dò 0C AD$@C :D$=D´9 Uni
                if (baseType == null || !typeof(ISoftDeletable).IsAssignableFrom(baseType.ClrType))Ð
                {Ð
                    // B4AD$0C0>C$:C VW'"f-çFW-
                    modelBuilder.SetSoftDeleteFilter(type);Ð
                }Ð
                // B4AD$0C0>C$:C 8C04CT:D 0 C00 DeletedAt CD;Dò CD :Că@CT=C,,0 Ct0C0@CăACă2Ð
                modelBuilder.Entity(type).HasIndex("DeletedAt");Ð
            }Ð
        }Ð
    }Ð
}Ð
}Ð
private static void SetSoftDeleteFilter(this ModelBuilder modelBuilder, Type entityType)Ð
{Ð
    // AD;Dò E BôE , 2C 6C0> D BC 2C,,BDĂ DC,,;DĂBD =C :Că@CT=DĂÂ 4C 6CR 5D ;C, >Cò 0C AD$@C :D$=D´9.Ð
    // B4AC´>C$8CR -46Æ 72 4CăAD$0D$>Dt=Căí
    if (entityType.IsClass)Ð
    {Ð
        modelBuilder.Entity(entityType).HasQueryFilter(GenerateFilter(entityType));Ð
    }Ð
}Ð
}

```

```

    private static LambdaExpression GenerateFilter(Type type)
    {
        ParameterExpression parameter = Expression.Parameter(type, "e");
        // A,,ACô>C`LCtCCT< Property CD;Dò 4CäAD$CCô0 C¢ FVÆWFVD B GCT@CT7 C,,=D$5D DCT9D 8C`8 D 2Cä9D BC$>D
        MemberExpression property = Expression.Property(parameter,
nameof(ISoftDeletable.DeletedAt));
        ConstantExpression nullConstant = Expression.Constant(null, typeof(DateTime?));
        BinaryExpression body = Expression.Equal(property, nullConstant);
        return Expression.Lambda(body, parameter);
    }
}
</file>

<file path="Data/DbContext/ApplicationDbContext.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain;
namespace Promatis.Net.Data;
public class ApplicationDbContext(
    DbContextOptions options,
    IConfiguration configuration,
    IServiceProvider? serviceProvider = null)
    : DbContext(options)
{
    protected virtual string Schema => "public";
    public DbSet<AuditLog> AuditLogs => Set<AuditLog>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.HasDefaultSchema(Schema);
        modelBuilder.ApplyModuleConfigurations(this);
        modelBuilder.ApplyGlobalConventions();
        base.OnModelCreating(modelBuilder);
    }

    // A,,!Aô A A` Aô B` A, A AT Aô+A` A A,, Aô"D
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        base.OnConfiguring(optionsBuilder);

        // 1. Aô@Cä2CT@Dô5CÂÂ GD$> CÄK C" @C =D$0C"<CRÂ 0 Cô5 C" ?D >Dd5D ACR <C,,3D 0Dd8C, 4Ä•
        if (!EF.IsDesignTime && serviceProvider != null)
        {
            DatabaseTriggerInterceptor? interceptor = null;

            try
            {
                // Aô>CôKD$:C ¢ D >C CCT< D 0Ct@CTHC,,BDÂ =C ?D 0CÄCDâ „5D ;C, ?D >C$0C"4CT@ Cä:C 7C ;D 0 S
                interceptor = serviceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as
DatabaseTriggerInterceptor;
            }
            catch (InvalidOperationException)
            {
                // Aô>CôKD$:C #¢ D ;C, ?D >C$0C"4CT@ C¤>D =CT2Cä9 (Root), D >Ct4C 5CÂ 66÷ R 2D CDT=D4N!D
                // BôBCâ 3C @C =D$8D CCTB, DtBCâ 66÷ VBô8CôBCT@Dd5CôBCä@ D 0Ct@CTHC,,BD 0 C 5Cr >D,,8C >C¢ > Ro
                var scopeFactory = serviceProvider.GetService(typeof(IServiceScopeFactory)) as
IServiceScopeFactory;

                if (scopeFactory != null)
                {
                    // B >Ct4C 5CÂ 2D 5CÄ5Cô=D4N Cä1C`0D BDÂ 2C,,4C,,<CäAD$8D
                    using IServiceScope scope = scopeFactory.CreateScope();
                    interceptor =
scope.ServiceProvider.GetService(typeof(DatabaseTriggerInterceptor)) as DatabaseTriggerInterceptor;
                }
            }

            // ATAC`8 C,,=D$5D FCT?D$>D CD ?CTHCô> Cô0C"4CT= Cä4Cô8CÂ 8Cr ACô>D >C >C" r ?Cä4C¤;DäGC 5CÂ 5C4>
            if (interceptor != null)
            {
                optionsBuilder.AddInterceptors(interceptor);
            }
        }
    }
}

```

```

    }
}
</file>

<file path="Data/DbContext/ModelBuilderExtensions.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain.Interface;
using System.Reflection;
using Microsoft.EntityFrameworkCore.Metadata;
using Microsoft.EntityFrameworkCore.Metadata.Builders;

namespace Promatis.Net.Data;

public static class ModelBuilderExtensions
{
    public static void ApplyModuleConfigurations(this ModelBuilder modelBuilder, DbContext context)
    {
        // 1. B :C =C,,@D45CÂ BCä;DÄ:Câ AC >D :C, 4C =CÔKDR ?D >CT:D$0 Promatis.*.Data
        List<Assembly> dataAssemblies = AppDomain.CurrentDomain.GetAssemblies()
            .Where(a => {
                string? name = a.GetName().Name;
                return name != null &&
                    name.StartsWith("Promatis.") &&
                    name.EndsWith(".Data");
            }).ToList();

        // 2. A0>C´CDt0CT< D$8C0K D CD"=CäAD$5C´Â :CäBCä@D´5 D02C0> Cä1D0C$;CT=D² 2 D$5C=CD"5CÂ :Cä=D$5C=AD$
        HashSet<Type> contextDbSetTypes = context.GetType()
            .GetProperties(BindingFlags.Public | BindingFlags.Instance)
            .Where(p => p.PropertyType.IsGenericType && p.PropertyType.GetGenericTypeDefinition()
                == typeof(DbSet<>))
            .Select(p => p.PropertyType.GetGenericArguments()[0])
            .ToHashSet();

        foreach (Assembly assembly in dataAssemblies)
        {
            // A0@C,,<CT=D05CÂ :Cä=DD8C4CD 0Dd8C, BCä;DÄ:Câ 4C´O D$5DR BC,,?Cä2, C0>D$>D KCR <C BCT@C,,0C´LC0K (
            modelBuilder.ApplyConfigurationsFromAssembly(assembly, type =>
            {
                if (type is not { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
                    false })
                {
                    return false;
                }

                // A00DT>CD8CÂ 8C0BCT@DD5C"A C0>C0DC,,3D4@C FC,,8 C, BC,,? D CD"=CäAD$8, C0>D$>D CDä >C00 C00D B
                Type? configInterface = type.GetInterfaces()
                    .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
                        typeof(IEntityTypeConfiguration<>));

                if (configInterface == null) return false;

                Type entityType = configInterface.GetGenericArguments()[0];

                // A0 A,, "A,, 'AT!A0 A' $A,, BÄ"B 0 D ;C, =C AD$@C 8C$0CT<C 0 D CD"=CäAD$L C 1D BD 0C=BC00,
                // C0@Cä2CT@D05CÂÂ 5D BDÄ ;C, C C05E ECäBDÄ >CD8C0 6C,,2Cä9 C00D ;CT4C08C0 2 D$5C=CD"5CÂ F%6W
                if (entityType.IsAbstract)
                {
                    bool hasConcreteDerivedInDbSet = contextDbSetTypes.Any(t =>
                        t.IsSubclassOf(entityType));
                    if (!hasConcreteDerivedInDbSet) return false; // A0@Cä?D4AC00CT< C0>C0DC,,3D4@C FC,,N C 1D
                }

                return type.GetConstructor(Type.EmptyTypes) != null;
            });
        }

        // 3. A B$ -A,, A0 B B A$ A0 AR´ B B B$ A$(A,,%» A B "B A0&A,,
        IEnumerable<Type> allAbstractEntities = dataAssemblies
            .SelectMany(a => a.GetTypes())
            .Where(t => t is { IsClass: true, IsAbstract: true } && !t.IsGenericTypeDefinition);

        foreach (Type abstractType in allAbstractEntities)
        {
            bool isUsedInCurrentContext = contextDbSetTypes.Any(t => t == abstractType ||
                t.IsSubclassOf(abstractType));

```

```

        if (!isUsedInCurrentContext)
        {
            modelBuilder.Ignore(abstractType);
        }
    }
}

// 4. A B$ AÄ B$ Bt B A / AÔ B "B A" A At A´ B A$ AÔ B´% AD B A$,AT A, AÔ AT B A-
foreach (ImmutableEntityType entityType in modelBuilder.Model.GetEntityTypes())
{
    Type? clrType = entityType.ClrType;
    if (clrType == null) continue;

    // A,,ICT< D 5C ;C,,7C FC,,N C,,=D$5D DCT9D Ø ITreeNode<>
    Type? treeInterface = clrType.GetInterfaces()
        .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(INode));
    if (treeInterface != null)
    {
        Type targetTreeType = treeInterface.GetGenericArguments()[0];

        // AÔØD BD ØC,,2C 5CÂ BCä;DÄ:Cä =C 1C 7Cä2Cä< D4@Cä2CÔ5 Cä1D=ØC$;CT=C,,O D 2Cä9D BC" „GD$>C K
        if (clrType == targetTreeType)
        {
            EntityTypeBuilder builder = modelBuilder.Entity(clrType);

            // 1. AÔØD BD >C´:C AC$ØCt8 B >CD8D$5C´L-AÔ>D$>CÄ>C-
            builder.HasOne("Parent")
                .WithMany("Children")
                .HasForeignKey("ParentId")
                .OnDelete(DeleteBehavior.Restrict);

            // 2. A B$ -A,, AD A=!: B >Ct4C 5CÂ 8CÔ4CT:D 4C´O C$=CTHCÔ5C4> C=;DäGC &VçD-M
            // BÔBCä CD :Cä@C,,B Cö>C,,AC MC´5CÄ5CÔBCä2 Cö> ParentId Cö@C, @C 1CäBCR "Æöö·W 2 D 5D 2C
            builder.HasIndex("ParentId")
                .HasDatabaseName($"IX_{clrType.Name}_ParentId");
        }
    }
}
}
</file>

<file path="Data/Interceptors/ChangeEntryModel.cs">
namespace Promatis.Net.Data;
{
    /// <summary>
    /// AÄ>CD5C´L DD8C=AC FC,,8 C,,7CÄ5CÔ5CÔ8C´ „BD ØCÔACô>D BCÔKC´ >C JCT:D"´Ä ?D 5CDAD$ØC$;DôND"ØDò ACÔ8CÄ>Cø ACä
    /// CD>CÄ5CÔ=Cä9 D CD"=CäAD$8 C" <Cä<CT=D" 5E 4Cä1C 2C´5CÔ8DòÄ <Cä4C,,DC,,:C FC,,8 C,,;C, CCDØC´5CÔ8Dò1
    /// </summary>
    internal class ChangeEntryModel
    {
        /// <summary>
        /// A,,7CÄ5CÔØCT<D´9 DÔ:Ct5CÄ?C´OD 4Cä<CT=CÔ>C4> Cä1D=5C=BC „AD4ICÔ>D BC,´1
        /// Aô@C,,2Cä4C,,BD O Cø :Cä=C=CTBCÔ>CÄ D$8CÔC C$=D4BD 8 D$@C,,3C45D >C"1
        /// </summary>
        public object Entity { get; init; } = null!;

        /// <summary>
        /// B$5C=CD"8C´ AD$ØD$CD 8Ct<CT=CT=C,,O D CD"=CäAD$8 C" AC,,AD$5CÄ5 (Added, Modified, Deleted, SoftDeleted
        /// </summary>
        public EntityStateChangeEnum State { get; init; }

        /// <summary>
        /// A= >C´;CT:Dd8Dò BCäGCTGÇÔKDR 8Ct<CT=CT=C,,9 D 2Cä9D BC" „?Cä;CT9) D CD"=CäAD$8.
        /// AtØCö>C´=Dô5D$ADò 4CT;DÄBCä9 Ct=C GCT=C,,9 "B BC @Cä5 -> AÔ>C$>CR"ä C´O CÔ>C$KDR AD4ICÔ>D BCT9 D >CD5
        /// </summary>
        public List<PropertyChangeInfo> Changes { get; init; } = [];

        /// <summary>
        /// A,,4CT=D$8DD8C=ØD$>D „8CÄO C,,;C, ;Cä3C,,=) Cö>C´Lct>C$ØD$5C´O, D >C$5D HC,,2D,,5C4> CDØCÔ=Cä5 CD5C"AD$2C
        /// </summary>
        public string? ChangedBy { get; init; }

        /// <summary>

```

```

    /// AD0D$0 C, 2D 5CÄ0 DD8C=AC FC,,8 C,,7CÄ5C05C08D0 2 DD>D <C BCR UD2i
    /// </summary>D
    public DateTime ChangedAt { get; init; }D
}
</file>

<file path="Data/Interceptors/DatabaseTriggerInterceptor.cs">
using System.Collections.Concurrent;D
using Microsoft.EntityFrameworkCore;D
using Microsoft.EntityFrameworkCore.ChangeTracking;D
using Microsoft.EntityFrameworkCore.Diagnostics;D
using Microsoft.Extensions.DependencyInjection;D
using Promatis.Net.Domain.Interface;D
D
namespace Promatis.Net.Data;D
D
/// <summary>D
/// A05D 5DT2C BDt8Cø >C05D 0Dd8C' F$6öçFW†BÄ >C 5D ?CTGC,,2C ND"8C' @C 1CäBD2 $<D03C= >C4> D44C ;CT=C,,0" (Soft
/// C0@CT4C$0D 8D$5C'LC0CDä 2C ;C,,4C FC,,N C,,7CÄ5C05C08C' 8 D :C$>Ct=Cä5 D$@C,,3C45D =Cä5 D42CT4Cä<C'5C08CR AC,,
/// </summary>D
/// <param name="scopeFactory">BD0C @C,,:C 4C'0 D >Ct4C =C,,0 C,,7Cä;C,,@Cä2C =C0KDR >C ;C AD$5C' 2C,,4C,,<CäAD$8
public class DatabaseTriggerInterceptor(IServiceScopeFactory scopeFactory) : SaveChangesInterceptorD
{D
    /// <summary>D
    /// A0>D$>C= >C 5Ct>C00D =Cä5 DT@C =C,,;C,,ICR 7C DC,,:D 8D >C$0C0=D'E C,,7CÄ5C05C08C' 2 D 0CÄ:C E D$5C=CD"5C'
    /// A;DäG - D4=C,,:C ;DÄ=D'9 C,,4CT=D$8DD8C=0D$>D MC=7CT<C0;D0@C F$6öçFW†Bí
    /// </summary>D
    private readonly ConcurrentDictionary<Guid, List<ChangeEntryModel>> _capturedChanges = new();D
D
    /// <summary>D
    /// A AC,,=DT@Cä=C0KC' ?CT@CTEC$0D$GC,,:, C$KC0>C'=D05CÄKC' Aä DC :D$8Dt5D :Cä3Cä ACäED 0C05C08D0 8Ct<CT=C
    /// AäBC$5Dt0CTB Ct0 C'>C48C=AC Soft Delete, DD8C=AC FC,,N D =C,,<C=0 C,,7CÄ5C05C08C' 8 Ct0C0CD : D$@C,,3C45D
    /// </summary>D
    public override async ValueTask<InterceptionResult<int>> SavingChangesAsync(D
        DbContextEventData eventData, InterceptionResult<int> result, CancellationToken ct =
default)D
    {D
        if (eventData.Context == null) return result;D
D
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =C0> Cd8C$>C' 66÷ R 4C'0 C0@CT4CäBC$@C ICT=C,,0 C0@Cä1C'
        using IServiceScope scope = scopeFactory.CreateScope();D
        var triggerService = scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();D
D
        string? userName = await GetUserNameInternalAsync(scope.ServiceProvider);D
D
        // 1. A0@CT4C$0D 8D$5C'LC00D0 >C @C 1CäBC=0 Soft Delete (AÄ0C4:Cä5 D44C ;CT=C,,5)D
        // A00DT>CD8CÄ 2D 5 D CD"=CäAD$8 D 8C0BCT@DD5C"ACä< ISoftDeletable, D2 :CäBCä@D'E D BC BD4A "B44C ;C
        IEnumerable<EntityEntry<ISoftDeletable>> entries =
eventData.Context.ChangeTracker.Entries<ISoftDeletable>()D
        .Where(e => e.State == EntityState.Deleted);D
D
        foreach (EntityEntry<ISoftDeletable> entry in entries)D
        {D
            // A0>CD<CT=D05CÄ DC,,7C,,GCTAC= >CR CCD0C'5C08CR =C >C =Cä2C'5C08CR 4C =C0KDR 2 A D
            entry.State = EntityState.Modified;D
            entry.Entity.DeletedAt = DateTime.UtcNow;D
            entry.Entity.DeletedBy = userName;D
        }D
D
        // A0@C,,=D44C,,BCT;DÄ=Cä 7C AD$0C$;D05CÄ Tb 6÷&R ?CT@CTAdt8D$0D$L CD5D 5C$> C,,7CÄ5C05C08C' ?CäAC'5 C0>
        eventData.Context.ChangeTracker.DetectChanges();D
D
        // 2. At0DT2C B C,,7CÄ5C05C08C' „ACäED 0C00CTB C00D$8C$=D'5 D$8C0K CD0C0=D'E object? CD;D0 BCTAD$>C"•
        List<ChangeEntryModel> captured = CaptureChanges(eventData.Context, userName);D
D
        // A0@C,,2D07D'2C 5CÄ :Cä;C'5C=FC,,N C,,7CÄ5C05C08C' : C= >C0:D 5D$=Cä<D2 MC=7CT<C0;D0@D2 :Cä=D$5C=AD$0 C
        _capturedChanges[eventData.Context.ContextId.InstanceId] = captured;D
D
        // 3. A$0C'8CD0Dd8D0 „&Vf÷&U6 fr' GCT@CT7 Cd8C$>C' G&-vvW%6W'f-6]
        // At0C0CD :C 5D" 4Cä<CT=C0KCR ?D 0C$8C'0 C, 2C ;C,,4C BCä@D² ?CT@CT4 D$5CÄÄ :C : CD0C0=D'5 C0>C00CDCD
        foreach (ChangeEntryModel item in captured)D
            await triggerService.ValidateAsync(item.Entity, item.State, item.Changes,
eventData.Context);D
D
        return result;D
    }D
}

```

```

Ð
/// <summary>Ð
/// A AC,,=DT@Cä=CÔKC' ?CT@CTEC$0D$GC,,:, C$KCô>C'=Dô5CÄKC' Aä!A' D4ACô5D,,=Cä3Cä ACäED 0Cô5Cô8Dò 8Ct<CT=C
/// AäBC$5Dt0CTB Ct0 D42CT4Cä<C'5Cô8CR ?Cä4Cô8D GC,,:Cä2 C, BD 8C43CT@Cä2 Cö>D B-Cä1D 0C >D$:C, ,,DC 7C 6
/// </summary>Ð
public override async ValueTask<int> SavedChangesAsync(Ð
    SaveChangesCompletedEventData eventData, int result, CancellationToken ct = default)Ð
{Ð
    // A,,7C$;CT:C 5CÄ 7C EC$0Dt5Cô=D'5 C,,7CÄ5Cô5Cô8Dò 8 D @C 7D2 0D$>CÄ0D =Cä CCD0C'0CT< C,,E C,,7 D ;Cä2C
    if (eventData.Context != null &&
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out List<ChangeEntryModel>?
entries))Ð
    {Ð
        // B >Ct4C 5CÄ ;Cä:C ;DÄ=D'9 C40D 0C0BC,,@Cä2C =Cô> Cd8C$>C' 66÷ R 4C'0 DD0CtK Saved (C$KCô>C'=Dô5
        using IServiceScope scope = scopeFactory.CreateScope();Ð
        var triggerService =
scope.ServiceProvider.GetRequiredService<IDatabaseTriggerService>();Ð
Ð
        // A AC,,=DT@Cä=Cô> D42CT4Cä<C'0CT< D 8D BCT<D2 >C 8Ct<CT=CT=C,,ODR ,,=C ?D 8CÄ5D Ä 4C'0 Cä1Cô>C$;C
        foreach (ChangeEntryModel item in entries)Ð
            await triggerService.NotifyAsync(item.Entity, item.State, item.Changes,
item.ChangedBy, item.ChangedAt);Ð
        }Ð
        return result;Ð
    }Ð
}Ð
Ð
/// <summary>Ð
/// Aô5D 5DT2C BDt8C$ AC >Dò ?D 8 D >DT@C =CT=C,,8 C,,7CÄ5Cô5Cô8C'â C @C =D$8D CCTB CäGC,,AD$:D2 :DôHC 7C
/// Cô@CT4CäBC$@C IC 0 D4BCTGC=8 Cô0CÄ0D$8 Cô@C, ?C 4CT=C,,8 D$@C =Ct0C=FC,,9.Ð
/// </summary>Ð
public override Task SaveChangesFailedAsync(DbContextErrorEventData eventData,
CancellationToken ct = default)Ð
{Ð
    if (eventData.Context != null)Ð
        _capturedChanges.TryRemove(eventData.Context.ContextId.InstanceId, out _);Ð
Ð
    return base.SaveChangesFailedAsync(eventData, ct);Ð
}Ð
Ð
/// <summary>Ð
/// A$=D4BD 5Cô=C,,9 CÄ5D$>CB 0Cô0C'8Ct0 D$@CT:CT@C 8Ct<CT=CT=C,,9 EF Core. Ð
/// A$KDt8D ;Dô5D" 4CT;DÄBD2 ,,AD$0D KCRô=Cä2D'5 Ct=C GCT=C,,0 Cö>C'5C'' 8 DD>D <C,,@D45D" <Cä4CT;C, 8Ct<CT=
/// </summary>Ð
/// <param name="context">B$5C=CD"8C' :Cä=D$5C=AD" 1C 7D² 4C =CÔKDRäÄ÷ & Óí
/// <param name="userName">A,,<Dò ?Cä;DÄ7Cä2C BCT;DòÄ ACä2CT@D,,8C$HCT3Cä >Cô5D 0Dd8DäâÄ÷ & Óí
/// <returns>B ?C,,ACä: D BD CC=BD4@C,,@Cä2C =CÔKDR <Cä4CT;CT9 C,,7CÄ5Cô5Cô8C' Ç6VR 7&Vcô$6† ævTVÇG'"ÖöFVÄ"ó
private List<ChangeEntryModel> CaptureChanges(DbContext context, string? userName)Ð
{Ð
    // BD8C'LD$@D45CÄ BCä;DÄ:Cä AD4ICô>D BC, 2 D >D BCä0Cô8DôE AD>C 0C$;CT=C,,0, A,,7CÄ5Cô5Cô8Dò 8C'8 B44C
    List<EntityEntry> entries = context.ChangeTracker.Entries()Ð
        .Where(e => e.State is EntityState.Added or EntityState.Modified or EntityState.Deleted)Ð
        .Where(e => !e.Entity.GetType().Name.Contains("AuditLog"))Ð
        .ToList();Ð
Ð
    var result = new List<ChangeEntryModel>();Ð
Ð
    foreach (EntityEntry e in entries)Ð
    {Ð
        EntityStateChangeEnum state = MapState(e.State);Ð
Ð
        // B ?CTFC,,DC,,GCôKC' <C ?Cô8Cô3 D >D BCä0Cô8Dò 4C'0 "CÄ0C4:Cä CCD0C'5Cô=D'E" Ct0Cô8D 5C•
        if (e.Entity is ISoftDeletable soft)Ð
        {Ð
            PropertyEntry prop = e.Property(nameof(ISoftDeletable.DeletedAt));Ð
            if (prop.IsModified && soft.DeletedAt != null) state =
EntityStateChangeEnum.SoftDeleted;Ð
        }Ð
Ð
        List<PropertyChangeInfo> changes = new();Ð
Ð
        // B FCT=C @C,,9 1: B CD"=CäAD$L CD>C 0C$;CT=C â D 5 D$5C=CD"8CR 7Cô0Dt5Cô8Dò AC$>C"AD$2 D GC,,BC
        if (state == EntityStateChangeEnum.Added)Ð
        {Ð
            changes = e.PropertiesÐ
                .Select(p => new PropertyChangeInfoÐ
                {Ð

```

```

        PropertyName = p.Metadata.Name,
        OriginalValue = null,
        CurrentValue = p.CurrentValue
    }).ToList();
}
// B FCT=C @C,,9 2: B CD"=CäAD$L C,,7CÄ5C05C00 C,,;C, <D03C> D44C ;CT=C â D´GC,,AC´OCT< D 0Ct=C,,FD2
else if (state is EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted)
{
    PropertyValues originalValues = e.OriginalValues;

    foreach (PropertyEntry p in e.Properties.Where(p => p.IsModified))
    {
        string propertyName = p.Metadata.Name;
        object? originalValue = originalValues[propertyName];
        object? currentValue = p.CurrentValue;

        // BD8C¤AC,,@D45CÄ 8Ct<CT=CT=C,,5 D$>C´LC¤> CTAC´8 Ct=C GCT=C,,0 CD5C"AD$2C,,BCT;DÄ=Cä @C 7C´
        if (!Equals(originalValue, currentValue))
        {
            changes.Add(new PropertyChangeInfo
            {
                PropertyName = propertyName,
                OriginalValue = originalValue,
                CurrentValue = currentValue
            });
        }
    }
}

// AD0Cd5 CTAC´8 C¤>C´;CT:Dd8D0 BCäGCTGC0KDR 8Ct<CT=CT=C,,9 C0>C´5C´ ,,6† ævW2´ ?D4AD$0 D
// C,,7-Ct0 D 0C0BC 9CÄ01C,,=CD8C03C &Æ ¦÷"Ä <D² B B A$ Aâ 4Cä1C 2C´OCT< D CD"=CäAD$L C" AC08D
// CTAC´8 DD0C @C,,:C 7C DC,,:D 8D >C$0C´0 D >D BCäOC08CR 0öF-f-VB -D$> C40D 0C0BC,,@D45D" R 4C
if (state == EntityStateChangeEnum.Modified && changes.Count == 0)
{
    // B >Ct4C 5CÄ DC,,:D$8C$=D4N Ct0C08D L C,,7CÄ5C05C08D0 4C´O CD>CÄ5C0=D´E D$@C,,3C45D >C"Ä
    // DtBCä1D² =CR ;Cä<C BDÄ 2C ;C,,4C FC,,N, C0> D >DT@C =C,,BDÄ 8CÄ?D4;DÄA CD;D0 T•
    changes.Add(new PropertyChangeInfo
    {
        PropertyName = "Id",
        OriginalValue = e.Property("Id").OriginalValue,
        CurrentValue = e.Property("Id").CurrentValue
    });
}

result.Add(new ChangeEntryModel
{
    Entity = e.Entity,
    State = state,
    ChangedBy = userName,
    ChangedAt = DateTime.UtcNow,
    Changes = changes
});
}

return result;
}

/// <summary>
/// AÄ0C0?C,,=C2 2C0CD$@CT=C08DR ACäAD$>D0=C,,9 EntityFramework State C$> C$=D4BD 5C0=C,,9 CD>CÄ5C0=D´9 C05D
/// </summary>
private EntityStateChangeEnum MapState(EntityState state) => state switch
{
    EntityState.Added => EntityStateChangeEnum.Added,
    EntityState.Deleted => EntityStateChangeEnum.Deleted,
    _ => EntityStateChangeEnum.Modified
};

/// <summary>
/// A 5Ct>C00D =Cä5 C0>C´CDt5C08CR 8CÄ5C08 D$5C¤CD"5C4> C0>C´LCt>C$0D$5C´O C,,7 C¤>C0BCT:D BC 0C$BCä@C,,7C
/// </summary>
/// <param name="provider">A´>C¤0C´LC0K´ ?D >C$0C"4CT@ D ;D46C ,,D´ 6öçF -æW"´äÄ÷ & 0í
private async Task<string?> GetUserNameInternalAsync(IServiceProvider provider)
{
    try
    {
        var userProvider = provider.GetService<IUserProvider>();
    }
}

```

```

        if (userProvider != null)ð
            return await userProvider.GetCurrentUserNameAsync();ð
    }ð
    catchð
    {ð
        // ignoredð
    }ð
ð
    return "System";ð
}ð
}
</file>

```

```

<file path="Data/Interceptors/IUserProvider.cs">
namespace Promatis.Net.Data;ð
ð
public interface IUserProviderð
{ð
    // AÄ5D$>CB <Cä6CTB C KD$L C AC,,=DT@Cä=CÔKCÂÂ 5D ;C, ?Cä;D4GCT=C,,5 C,,<CT=C, BD 5C CCTB Ct0Cô@CäAC
    Task<string?> GetCurrentUserNameAsync();ð
}
</file>

```

```

<file path="Data/Promatis.Net.Data.csproj">
<Project Sdk="Microsoft.NET.Sdk">ð
ð
    <PropertyGroup>ð
        <TargetFramework>net10.0</TargetFramework>ð
        <ImplicitUsings>enable</ImplicitUsings>ð
        <Nullable>enable</Nullable>ð
    </PropertyGroup>ð
ð
    <ItemGroup>ð
        <PackageReference Include="Microsoft.EntityFrameworkCore" Version="10.0.8" />ð
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />ð
        <PackageReference Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="10.0.1" />ð
    </ItemGroup>ð
ð
    <ItemGroup>ð
        <ProjectReference Include="..\Domain\Promatis.Net.Domain.csproj" />ð
    </ItemGroup>ð
ð
    "Ä-FVÔw&÷W í
    ""<AssemblyAttribute Include="System.Runtime.CompilerServices.InternalsVisibleTo">ð
    ""Ä & ÖFW# ä &Ö F-2äæWBä Æ-6 F-öäÖöFVÄÄFW7G3Äö & ÖFW# ä Ä ÖÖ CÄO D$5D BCä2Cä3Cä ?D >CT:D$0 -->ð
    ""</AssemblyAttribute>ð
    ""Ä-FVÔw&÷W í
ð
ð
</Project>
</file>

```

```

<file path="Data/Triggers/Castom/AuditTrigger.cs">
using Microsoft.EntityFrameworkCore;ð
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Domain.Interface;ð
using System.Text.Json;ð
ð
namespace Promatis.Net.Data;ð
ð
public class AuditTrigger(ð
    IServiceScopeFactory scopeFactory, ð
    JsonSerializerOptions jsonOptions)ð
    : IAfterSaveTrigger<DomainObject>ð
{ð
    public async Task HandleAsync(EntityChangedEventArgs<DomainObject> args)ð
    {ð
        // AäGC,,IC 5CÄ @C =D$0C"<-D$8Cò >D" 2Cä7CÄ>Cd=Cä9 CD8CÔ0CÄ8Dt5D :Cä9 Cö@Cä:D 8-Cä1Cä;CäGC=8 EF Core (
        Type entityType = args.Entity.GetType();ð
        if (entityType.Namespace == "Castle.Proxies" && entityType.BaseType != null)ð
        {ð
            entityType = entityType.BaseType;ð
        }ð
ð
        // 1. BD8C´LD$: C´>C48D CCT< D$>C´LC= D$5 D CD"=CäAD$8, C= D$>D KCR ?Cä<CTGCT=D² " VF-M
    }
}

```

```

        if (entityType.GetInterfaces().All(i => i.Name != nameof(IAudit)))
        {
            return;
        }

        // 2. BD>D <C,,@D45CÂ 4C =CÔKCR 4C´O JSON
        object changesToSerialize = args.State switch
        {
            EntityStateChangeEnum.Added => args.Changes.Select(c => new
            {
                c.PropertyName,
                NewValue = c.CurrentValue
            }),
            EntityStateChangeEnum.Modified or EntityStateChangeEnum.SoftDeleted =>
            args.Changes.Select(c => new
            {
                c.PropertyName,
                OldValue = c.OriginalValue,
                NewValue = c.CurrentValue
            }),
            EntityStateChangeEnum.Deleted => new { Message = "Aâ1D=5C= B D44C ;CT= C= C´=CâAD$LDâ" òí
            },
            _ => args.Changes
        };

        // 3. B >Ct4C 5CÂ 7C ?C,,ADÂ ;Câ3C
        var auditLog = new AuditLog
        {
            Id = Guid.NewGuid(),
            EntityName = entityType.Name, // A,,!Aô A A´ Aô : Aô8D,,5CÂ GC,,AD$>CR 8CÂO C=;C AD 0 Câ@C4AD$@D4:D
            EntityId = args.Entity.Id,
            Action = args.State.ToString(),
            ChangedAt = args.ChangedAt,
            ChangedBy = args.ChangedBy,
            ChangesJson = JsonSerializer.Serialize(changesToSerialize, jsonOptions)
        };

        // 4. A,,!Aô A A´ Aô : A 5Ct>Cô0D =Câ5 D >DT@C =CT=C,,5 C" AB GCT@CT7 C$KCD5C´5CÔ=D´9, C40D 0CÔBC,,@Câ
        using IServiceScope scope = scopeFactory.CreateScope();

        // A,,ICT< DD0C @C,,:D2 1C 7Câ2Câ3Câ :Câ=D$5C=AD$0 Dt5D 5Cr ?D >C$0C"4CT@ C´>C=0C´LCÔ>C4> Scope
        var factory = scope.ServiceProvider.GetService<IDbContextFactory<ApplicationDbContext>>();

        if (factory != null)
        {
            await using ApplicationDbContext context = await factory.CreateDbContextAsync();
            context.Set<AuditLog>().Add(auditLog);
            await context.SaveChangesAsync();
        }
        else
        {
            Console.WriteLine("[AuditTrigger][Error] Aô5 D44C ;CâADÂ =C 9D$8 IDbContextFactory<ApplicationDbC
        }
    }
}
</file>

```

```

<file path="Data/Triggers/Global/FluentValidationTrigger.cs">
using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.Data;

public class FluentValidationTrigger(IServiceScopeFactory scopeFactory) // A,,!Aô A A´ Aô : A,,=Cd5C=BC,,@D45CÂ
    : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
{
    public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
    {
        Type entityType = args.Entity.GetType();
        Type validatorType = typeof(IValidator<>).MakeGenericType(entityType);

        // A,,!Aô A A´ Aô : B >Ct4C 5CÂ 8Ct>C´8D >C$0CÔ=D4N, C 5Ct>Cô0D =D4N Câ1C´0D BDÂ 2C,,4C,,<CâAD$8 (Scope
        // BÔBCâ 3C @C =D$8D CCTB, DtBCâ •6W'f-6U &÷f-FW" 1D44CTB "Cd8C$KcÂ" 8 D4ACô5D,,=Câ @C 7D 5D,,8D" =C H
    }
}

```

```

        using IServiceScope scope = scopeFactory.CreateScope();
    }

    // At0C0@C HC,,2C 5CÂ 2C ;C,,4C BCä@ C,,7 D 2CT6CT3CâÂ 3C @C =D$8D >C$0C0=Câ 6C,,2Cä3Câ 66÷ R0?D >C$0C"4C
    if (scope.ServiceProvider.GetService<IValidator validator>) is IValidator validator)
    {
        var context = new ValidationContext<object>(args.Entity);
        ValidationResult result = await validator.ValidateAsync(context);

        if (!result.IsValid)
        {
            args.Cancel = true;
            // BD>D <C,,@D45CÂ :D 0D 8C$>CRÂ GC,,AD$>CR ACä>C ICT=C,,5 Cä1 CäHC,,1C00DR 4C'0 UI0
            args.ErrorMessage = string.Join(" | ", result.Errors.Select(e => e.ErrorMessage));
        }
    }
}
</file>

```

```

<file path="Data/Triggers/Global/ReferenceTreeParentTrigger.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Metadata;
using Promatis.Net.Domain.Interface;
using System.Reflection;
{
    namespace Promatis.Net.Data;
    {
        /// <summary>
        /// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC0BD4@C0KC' BD 8C43CT@ CD;D0 ?D >C$5D :C, FCT;CäAD$=CäAD$8 C,,5D 0D EC,,G
        /// C, AC02Cä7C0>C' 7C IC,,BD² >D" FC,,:C'8Dt5D :C,,E Ct0C$8D 8CÄ>D BCT9 C05D 5CB ACäED 0C05C08CT< C" !B4 ABí
        /// </summary>
        public class ReferenceTreeParentTrigger : IBeforeSaveTrigger<IDomainObjectHasKey<Guid>>
        {
            private static readonly MethodInfo DbContextSetMethod = typeof(DbContext)
                .GetMethod(BindingFlags.Public | BindingFlags.Instance)
                .First(m => m.Name == nameof(DbContext.Set) && m.IsGenericMethod &&
                    m.GetParameters().Length == 0);

            private static readonly MethodInfo AsNoTrackingMethod =
                typeof(EntityFrameworkQueryableExtensions)
                    .GetMethod(BindingFlags.Public | BindingFlags.Static)
                    .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.AsNoTracking) &&
                        m.IsGenericMethod);

            private static readonly MethodInfo IgnoreQueryFiltersMethod =
                typeof(EntityFrameworkQueryableExtensions)
                    .GetMethod(BindingFlags.Public | BindingFlags.Static)
                    .First(m => m.Name == nameof(EntityFrameworkQueryableExtensions.IgnoreQueryFilters) &&
                        m.IsGenericMethod);

            public async Task HandleAsync(EntityCancelEventArgs<IDomainObjectHasKey<Guid>> args)
            {
                // A0 Aä!B$ AR AT(AT A,, : Bt5D 5Cr @CTDC'5C0AC,,N C0@Cä2CT@D05CÄÂ 5D BDÂ ;C, C Cä1D05C0BC AC$>C"AD$2
                // ATAC'8 D 2Cä9D BC$0 C05D" r MD$>D" >C JCT:D" =CR 4CT@CT2CâÂ <D² ?D >D BCâ 2D'ECä4C,,<!D
                PropertyInfo? parentIdProp = args.Entity.GetType().GetProperty("ParentId");
                if (parentIdProp == null) return;

                // Bt8D$0CT< Ct=C GCT=C,,0 D 2Cä9D BC" 4C,,=C <C,,GCTAC08D
                Guid entityId = args.Entity.Id;
                Guid? parentId = (Guid?)parentIdProp.GetValue(args.Entity);

                PropertyInfo? deletedAtProp = args.Entity.GetType().GetProperty("DeletedAt");

                // 1. At0D"8D$0 CäB C0@D0<Cä3Câ AC <CäFC,,BC,,@Cä2C =C,,0D
                if (parentId.HasValue && parentId == entityId)
                {
                    args.Cancel = true;
                    args.ErrorMessage = "Aä1D05C0B C05 CÄ>Cd5D" 1D'BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.";
                    return;
                }

                if (parentId.HasValue && parentId.Value != Guid.Empty)
                {
                    // A,,ICT< D >CD8D$5C'0 C" 6† ævUG& 6¶W" AD 5CD8 C'NC KDR >C JCT:D$>C"Â C C0>D$>D KDR ACä2C00CD0CT
                    object? localParent = args.Context.ChangeTracker.Entries()
                        .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
                            parentId.Value)?.Entity;
                }
            }
        }
    }
}

```

```

    bool exists;
    bool isDeleted;

    if (localParent != null)
    {
        exists = true;
        var localDeletedAt = (DateTime?)deletedAtProp?.GetValue(localParent);
        isDeleted = localDeletedAt != null;
    }
    else
    {
        // A$0D,,0 Că@C,,3C,,=C ;DÄ=C 0 D 5DD;CT:D 8Dò 4C´O C$KDt8D ;CT=C,,0 Cα>D =Dò 8 Query C" Tb 6÷&]
        IEntityType? entityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
        Type rootClrType = entityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
        object? rawSet =
DbContextSetMethod.MakeGenericMethod(rootClrType).Invoke(args.Context, null);
        object? noTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(rootClrType).Invoke(null, [rawSet]);
        object? ignoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(rootClrType).Invoke(null, [noTrackingQuery]);
        IQueryable<object> query = ((IQueryable)ignoreFiltersQuery!).Cast<object>();
        // A,,ICT< Că1Dα5CαB Cò> CD8C00CÄ8Dt5D :Că<D2 AC$>C"AD$2D2 $-B" !B4 AM
        object? dbParent = await query.FirstOrDefaultAsync(x => EF.Property<Guid>(x, "Id")
== parentId.Value);
        exists = dbParent != null;
        var dbDeletedAt = dbParent != null ? (DateTime?)deletedAtProp?.GetValue(dbParent) :
null;
        isDeleted = dbDeletedAt != null;
    }

    // 2. A0@Că2CT@Cα0 DD8Ct8Dt5D :Că3Că AD4ICTAD$2Că2C =C,,0 D >CD8D$5C´LD :Că3Că CCt;C
    if (!exists)
    {
        args.Cancel = true;
        args.ErrorMessage = $"B4:C 7C =C0KC' @Că4C,,BCT;DÄACα8C' >C JCT:D" ,,"Cϕ . &VçD-G0' =CR =C 9CD
        return;
    }

    // 3. A0@Că2CT@Cα0 D BC BD4AC <D03Cα>C4> D44C ;CT=C,,0D
    if (isDeleted)
    {
        args.Cancel = true;
        args.ErrorMessage = "A05C´Lct0 C00Ct=C GC,,BDÂ @Că4C,,BCT;CT< D44C ;CT=C0KC' >C JCT:D"â#½
        return;
    }

    // 4. A4 B4 Aă A / At B" B$ Aă" Bd Aα Aă (A -> B -> C -> A)
    Guid? currentCheckId = parentId;
    var visitedIds = new HashSet<Guid>();

    while (currentCheckId.HasValue && currentCheckId.Value != Guid.Empty)
    {
        if (currentCheckId == entityId)
        {
            args.Cancel = true;
            args.ErrorMessage = "Bd8Cα;C,,GCTACα0Dò 7C 2C,,AC,,<CăAD$L: C05C´Lct0 C05D 5CĂ5D BC,,BDÂ @Că4
            return;
        }

        if (!visitedIds.Add(currentCheckId.Value))
            break;

        // A,,ICT< D0;CT<CT=D" FCT?CăGCα8 D =C GC ;C 2 ChangeTracker
        object? nextLocalElement = args.Context.ChangeTracker.Entries()
            .FirstOrDefault(e => ((IDomainObjectHasKey<Guid>)e.Entity).Id ==
currentCheckId.Value)?.Entity;

        if (nextLocalElement != null)
    }

```

```

        currentCheckId = (Guid?)parentIdProp.GetValue(nextLocalElement);
    }
    else
    {
        Guid? id = currentCheckId;

        IEntityType? loopEntityTypeInModel =
args.Context.Model.FindEntityType(args.Entity.GetType());
        Type loopRootClrType = loopEntityTypeInModel?.GetRootType()?.ClrType ??
args.Entity.GetType();
        object? loopRawSet =
DbContextSetMethod.MakeGenericMethod(loopRootClrType).Invoke(args.Context, null);
        object? loopNoTrackingQuery =
AsNoTrackingMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopRawSet]);
        object? loopIgnoreFiltersQuery =
IgnoreQueryFiltersMethod.MakeGenericMethod(loopRootClrType).Invoke(null, [loopNoTrackingQuery]);
        IQueryable<object> loopQuery =
((IQueryable)loopIgnoreFiltersQuery!).Cast<object>();
        object? parentNodeObj = await loopQuery.FirstOrDefaultAsync(x =>
EF.Property<Guid>(x, "Id") == id.Value);
        currentCheckId = parentNodeObj != null ?
(Guid?)parentIdProp.GetValue(parentNodeObj) : null;
    }
}
}
}
</file>

```

```

<file path="Data/TriggerService/DatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using System.Collections.Concurrent;
namespace Promatis.Net.Data;
/// <summary>
/// B 5C ;C,,7C FC,,0 Dd5C0BD 0C'LC0>C4> CD8D ?CTBDt5D 0 D$@C,,3C45D >C"â #C0@C 2C'0CTB D 5C48D BD 0Dd8CT9 CD>CÄ
/// C, :Cä>D 4C,,=C,,@D45D" DC 7D² 2C ;C,,4C FC,,8 (CD> D >DT@C =CT=C,,0) C, CC$5CD>CÄ;CT=C,,0 (C0>D ;CR ACäED 0C05
/// </summary>
/// <param name="serviceProvider">A :D$CC ;DÄ=D'9 Cª>C0BCT:D B Ct0C$8D 8CÄ>D BCT9 (DI Container) D$5CªCD"5C4>
public class DatabaseTriggerService(IServiceProvider serviceProvider) : IDatabaseTriggerService
{
    /// <summary>
    /// A4;Cä1C ;DÄ=Cä5 D >C KD$8CRÄ CC$5CD>CÄ;D0ND"5CR AC,,AD$5CÄC Cä1 D4AC05D,,=Cä< Cª>CÄ<C,,BCR AD4IC0>D BC,
    /// A,,AC0>C' LctCCTBD 0 CD;D0 @CT0CªBC,,2C0>C4> Cä1C0>C$;CT=C,,0 C,,=D$5D DCT9D 0 (MudBlazor) C,,;C, ACª2Cä7C0
    /// </summary>
    public static event Action<EntityStateChangeEnum, object>? OnEntityCommitted;

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8Cª>C"Â 2D' ?Cä;C00DäIC,,ED 0 AD D >DT@C =CT=C,,0 C,,7CÄ5C05C08C' ,, $C 7C C ;C,,4C
    /// AD5C'5C40D" ?D 8C08CÄ0CTB C @C4CCÄ5C0BD² ACä1D'BC,,0 C, 0CªBD40C'LC0KC' 66÷ VB 6W'f-6U &÷f-FW"i
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityCancelArgsBase, IServiceProvider,
Task>>> BeforeSubscribers = new();

    /// <summary>
    /// B 5CTAD$@ C0>CD?C,,Adt8Cª>C"Â 2D' ?Cä;C00DäIC,,ED 0 A0 B AR CD ?CTHC0>C4> D >DT@C =CT=C,,0 C,,7CÄ5C05C08C
    /// </summary>
    private static readonly Dictionary<Type, List<Func<EntityChangedArgsBase, IServiceProvider,
Task>>> AfterSubscribers = new();

    /// <summary>
    /// A0>D$>Cª>C 5Ct>C00D =D'9 CªMD, 8CT@C @DT8C, BC,, ?Cä2. A,,ACª;DäGC 5D" ?Cä2D$>D =Cä5 C,,AC0>C' Lct>C$0C08C
    /// C0@C, >C0@CT4CT;CT=C,,8 C 0Ct>C$KDR :C'0D ACä2 C, 8C0BCT@DD5C"ACä2 Cä1D 0C 0D$KC$0CT<D'E D CD"=CäAD$5C
    /// </summary>
    private static readonly ConcurrentDictionary<Type, List<Type>> HierarchyCache = new();

    /// <summary>
    /// B 5C48D BD CCTB CD>CÄ5C0=D'9 D$@C,,3C45D 4C'0 Cª>C0:D 5D$=Cä3Cä BC,, ?C AD4IC0>D BC,i
    /// A$KCTKC$0CTBD 0 C 2D$>CÄ0D$8Dt5D :C, ?D 8 D :C =C,,@Cä2C =C,,8 D 1Cä@Cä: C$> C$@CT<D0 8C08Dd8C ;C,,7C FC
    /// </summary>

```

```

/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C' AD4ICÔ>D BC,ãÄ÷G— W & Óí
/// <typeparam name="TTrigger">B$8Cò :C' 0D AC ÔBD 8C43CT@C Ä @CT0C'8CtCDäICT3Câ ;Cä3C,, :D2 >C @C 1CäBC=8.<
public void Register<TEntity, TTrigger>()
    where TEntity : class
    where TTrigger : class
{
    Type entityType = typeof(TEntity);
    Type triggerType = typeof(TTrigger);

    // Aô@Cä2CT@Dô5CÄÄ :C :C,,5 C,,=D$5D DCT9D K Cd8Ct=CT=CÔ>C4> Dd8C=;C @CT0C'8CtCCTB CD0CÔ=D'9 D$@C,,3C45
    bool isBefore = typeof(IBeforeSaveTrigger<TEntity>).IsAssignableFrom(triggerType);
    bool isAfter = typeof(IAfterSaveTrigger<TEntity>).IsAssignableFrom(triggerType);

    if (isBefore)
    {
        // BD>D <C,,@D45CÄ >D$;Cä6CT=CÔCDä ;Dô<C 4D2â Cä=D$5C'=CT@ 'sp' Cô5D 5CD0CTBD O C" <Cä<CT=D" 2D'7
        // DtBCä ?D 5CD>D$2D 0D"0CTB D4BCTGC=C CD>C'3Cä6C,,2D4IC,,E Cä1D=5C=BCä2 (Captive Dependency).
        AddSubscriber(BeforeSubscribers, entityType, async (argsBase, sp) =>
        {
            // B 0Ct@CTHC 5CÄ MC=7CT<Cô;Dô@ D$@C,,3C45D 0 C,,7 C :D$CC ;DÄ=Cä3Cä 66÷ R BCT:D4ICT3Cä 7C ?D >
            if (sp.GetRequiredService<TTrigger>() is IBeforeSaveTrigger<TEntity> handler)
            {
                // Aä1Cä@C GC,,2C 5CÄ 1C 7Cä2D'5 C @C4CCÄ5CÔBD² 2 D BD >C4> D$8Cô8Ct8D >C$0CÔ=D'9 C= >CÔBCT
                var typedArgs = new EntityCancelEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.Context);

                await handler.HandleAsync(typedArgs);

                // A$>Ct2D 0D"0CT< D 5CtCC'LD$0D$K C$KCô>C'=CT=C,,O D$@C,,3C45D 0 Cä1D 0D$=Cä 2 C 0Ct>C$KC'
                argsBase.Cancel = typedArgs.Cancel;
                argsBase.ErrorMessage = typedArgs.ErrorMessage;
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    if (isAfter)
    {
        AddSubscriber(AfterSubscribers, entityType, async (argsBase, sp) =>
        {
            if (sp.GetRequiredService<TTrigger>() is IAfterSaveTrigger<TEntity> handler)
            {
                var typedArgs = new EntityChangedEventArgs<TEntity>(
                    (TEntity)argsBase.Entity,
                    argsBase.State,
                    argsBase.Changes,
                    argsBase.ChangedBy,
                    argsBase.ChangedAt);

                await handler.HandleAsync(typedArgs);
                argsBase.Handled = typedArgs.Handled;
            }
        });
    }

    // ATAC'8 C=;C AD =CR @CT0C'8CtCCTB CÔ8 Cä4C,,= C= >CÔBD 0C=B — DÔBCä :D 8D$8Dt5D :C O CäHC,,1C=0 C= >CÔ
    if (!isBefore && !isAfter)
    {
        throw new InvalidOperationException(
            $"A=;C AD .G&-vvw%G— Ræœ ŌWŌ =CR @CT0C'8CtCCTB IBeforeSaveTrigger C,,;C, " gFW%6 fUG&-vvw" 4C
        );
    }

    /// <summary>
    /// A$KCô>C'=Dô5D" AC=2Cä7CÔCDä 2C ;C,,4C FC,,N C,,7CÄ5CÔ0CT<Cä9 D CD"=CäAD$8 Cô5D 5CB 5E >D$?D 0C$:Cä9 C"
    /// Aä?D 0D,,8C$0CTB C$ADä FCT?CäGC=C CÔ0D ;CT4Cä2C =C,,O D CD"=CäAD$8. Aô@CT@D'2C 5D" >Cô5D 0Dd8Dä ?D 8 CÔ
    /// </summary>
    public async Task ValidateAsync(object entity, EntityStateChangeEnum state,
        List<PropertyChangeInfo> changes, DbContext context)
    {
        var args = new EntityCancelArgsBase(entity, state, changes, context);

        // Aä1DT>CD8CÄ 8CT@C @DT8Dä BC,,?Cä2 (D 0CÄ>C' AD4ICÔ>D BC,Ä 1C 7Cä2D'E C=;C AD >C" 8 C,,=D$5D DCT9D >C

```

```

        foreach (Type type in GetTypesHierarchy(entity.GetType()))
        {
            if (BeforeSubscribers.TryGetValue(type, out List<Func<EntityCancelArgsBase,
IServiceProvider, Task>>? actions))
            {
                foreach (Func<EntityCancelArgsBase, IServiceProvider, Task> action in actions)
                {
                    // A05D 5CD0CT< C'>C0C'LC0KC' 6W'f-6U &+f-FW"Â A C0>D$>D KCÂ 1D'; D >Ct4C = CD0C0=D'9 D0
                    await action(args, serviceProvider);
                }
            }
        }

        // ATAC'8 D$@C,,3C45D 2D'AD$0C$8C^2 DC'0C2 6 æ6VÂ r =CT<CT4C'5C0=Câ ?D 5D KC$0CT< D$@C =Ct
        if (args.Cancel) throw new OperationCanceledException(args.ErrorMessage);
    }
}

/// <summary>
/// B 0D AD';C 5D" CC$5CD>CÂ;CT=C,,0 Că1 D4AC05D,,=Că< D >DT@C =CT=C,,8 C,,7CĂ5C05C08C' 2 C 0CtC CD0C0=D'E.D
/// A0>CD4CT@Cd8C$0CTB C0D :C 4C0KC' ?CT@CTEC$0D" ,,DC'0C2 † æFÆVB' 8 C0CC ;C,,:D45D" ACă1D'BC,,5 C" 3C'>C
/// </summary>
public async Task NotifyAsync(object entity, EntityStateChangeEnum state,
List<PropertyChangeInfo> changes, string? user, DateTime at)
{
    var args = new EntityChangedEventArgs(entity, state, changes, user, at);

    foreach (Type type in GetTypesHierarchy(entity.GetType()))
    {
        if (AfterSubscribers.TryGetValue(type, out List<Func<EntityChangedEventArgs,
IServiceProvider, Task>>? actions))
        {
            foreach (Func<EntityChangedEventArgs, IServiceProvider, Task> action in actions)
            {
                await action(args, serviceProvider);

                // ATAC'8 Că4C,,= C,,7 D$@C,,3C45D >C" 2D'AD$0C$8C^2 † æFÆVB 0 G'VRÂ >C @C 1CăBC00 Dd5C0>Dt:C
                if (args.Handled) return;
            }
        }
    }

    // A0CC ;C,,:C FC,,0 C" AD$0D$8Dt5D :C,,9 D02CT=D" 4C'0 D 5C :D$8C$=Că3Că >C =Că2C'5C08D0 :Că<C0>C05C0BC
    OnEntityCommitted?.Invoke(state, entity);
}

/// <summary>
/// A$AC0>CĂ>C40D$5C'LC0KC' <CTBCă4 CD;D0 1CT7Că?C AC0>C4> CD>C 0C$;CT=C,,0 C0>CD?C,,ADt8C0>C" 2 D ;Că2C @C
/// </summary>
private void AddSubscriber<TArgs>(Dictionary<Type, List<Func<TArgs, IServiceProvider, Task>>?
dict, Type type, Func<TArgs, IServiceProvider, Task> action)
{
    if (!dict.TryGetValue(type, out List<Func<TArgs, IServiceProvider, Task>>? list))
    {
        list = new List<Func<TArgs, IServiceProvider, Task>>();
        dict[type] = list;
    }
    list.Add(action);
}

/// <summary>
/// A,,7C$;CT:C 5D" ?Că;C0CDă 8CT@C @DT8Dă BC,,?Că2 CD;D0 CC00Ct0C0=Că3Că >C JCT:D$0 (C$:C'NDt0D0 1C 7Că2D'
/// B 5CtCC'LD$0D$K C0MD,,8D CDăBD 0 CD;D0 >C 5D ?CTGCT=C,,0 C$KD >C0>C' ?D >C,,7C$>CD8D$5C'LC0>D BC, ?Că4 C
/// </summary>
private IEnumerable<Type> GetTypesHierarchy(Type type) =>
{
    HierarchyCache.GetOrAdd(type, t => {
        var types = new List<Type>();
        // B 5C0CD AC,,2C0> C0>CD=C,,<C 5CĂAD0 2C$5D E C0> CD@CT2D2 =C AC'5CD>C$0C08D0 :C'0D ACă2, C,,AC0;Dă
        for (Type? c = t; c != null && c != typeof(object); c = c.BaseType) types.Add(c);
        // AD>C 0C$;D05CĂ 2D 5 D 5C ;C,,7Că2C =C0KCR 8C0BCT@DD5C"AD^2 8 D41C,,@C 5CĂ 4D41C'8C00D$K0
        return types.Concat(t.GetInterfaces()).Distinct().ToList();
    });
}

/// <summary>
/// B @CăGC0KC' AC @CăA D BC BC,,GCTAC08DR @CT3C,,AD$@C FC,,9. D
/// A,,AC0>C'LCtCCTBD 0 C0@CT8CĂCD"5D BC$5C0=Că 2 Unit-D$5D BC E CD;D0 8Ct>C'ODd8C, ?D >C4>C0>C" BCTAD$>C"
/// </summary>

```

```

        internal static void ClearInternalRegistrations()
        {
            BeforeSubscribers.Clear();
            AfterSubscribers.Clear();
            HierarchyCache.Clear();
        }
    }
</file>

<file path="Data/TriggerService/EntityStateChangeEnum.cs">
namespace Promatis.Net.Data;
{
    public enum EntityStateChangeEnum
    {
        Added,
        Modified,
        Deleted,
        SoftDeleted
    }
}
</file>

<file path="Data/TriggerService/IDatabaseTriggerService.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public interface IDatabaseTriggerService
        {
            // AA5D$>CB 4C'0 C00D BD >C":C, AC$0Ct5C' $!D4IC0>D BDÂ0"D 8C43CT@"
            void Register<TEntity, TTrigger>()
                where TEntity : class
                where TTrigger : class;
        }

        // AA5D$>CDK C$KC0>C'=CT=C,,0 (C$KctKC$0DäBD 0 C,,=D$5D FCT?D$>D >CÂ•
        Task ValidateAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo>
changes, DbContext context);
        Task NotifyAsync(object entity, EntityStateChangeEnum state, List<PropertyChangeInfo> changes,
string? user, DateTime at);
    }
}
</file>

<file path="Data/TriggerService/PropertyChangeInfo.cs">
namespace Promatis.Net.Data;
{
    public class PropertyChangeInfo
    {
        public string PropertyName { get; set; } = string.Empty;
        public object? OriginalValue { get; set; }
        public object? CurrentValue { get; set; }
    }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelArgsBase.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelArgsBase(
            object entity,
            EntityStateChangeEnum state,
            List<PropertyChangeInfo> changes,
            DbContext context) : EntityEventArgsBase(entity, state, changes)
        {
            public DbContext Context { get; } = context;
            public bool Cancel { get; set; }
            public string? ErrorMessage { get; set; }
        }
    }
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityCancelEventArgs.cs">
using Microsoft.EntityFrameworkCore;
{
    namespace Promatis.Net.Data;
    {
        public class EntityCancelEventArgs<T>(
            T entity,

```

```

        EntityStateChangeEnum state, Ø
        List<PropertyChangeInfo> changes, Ø
        DbContext context) Ø
        : EntityCancelArgsBase(entity!, state, changes, context) where T : class Ø
    { Ø
        public new T Entity => (T)base.Entity; Ø
    }
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data; Ø
Ø
// A ØCt>C$KC' :C'ØD A CD;Dò 2CÔCD$@CT=CÔ5C4> C,,ACô>C'LCt>C$ØCÔ8Dò 2 D 5D 2C,,AC]
public class EntityChangedEventArgs(Ø
    object entity, Ø
    EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes, Ø
    string? changedBy, Ø
    DateTime changedAt) : EntityEventArgsBase(entity, state, changes) // <-- AØØD ;CT4D45CÂ 7CD5D LØ
{ Ø
    public string? ChangedBy { get; } = changedBy; Ø
    public DateTime ChangedAt { get; } = changedAt; Ø
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityChangedEventArgs.cs">
namespace Promatis.Net.Data; Ø
Ø
// A$ØD, BC,,?C,,7C,,@Cä2C =CÔKC' :C'ØD A CD;Dò BD 8C43CT@Cä2Ø
public class EntityChangedEventArgs<T>(Ø
    T entity, EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes, Ø
    string? changedBy, Ø
    DateTime changedAt) Ø
    : EntityChangedEventArgsBase(entity!, state, changes, changedBy, changedAt) where T : class Ø
{ Ø
    public new T Entity => (T)base.Entity; Ø
}
</file>

<file path="Data/TriggerService/TriggerEvents/EntityEventArgsBase.cs">
namespace Promatis.Net.Data; Ø
Ø
public abstract class EventArgsBase(Ø
    object entity, Ø
    EntityStateChangeEnum state, Ø
    List<PropertyChangeInfo> changes) : EventArgs Ø
{ Ø
    public object Entity { get; } = entity; Ø
    public EntityStateChangeEnum State { get; } = state; Ø
    public List<PropertyChangeInfo> Changes { get; } = changes; Ø
    public bool Handled { get; set; } Ø
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IAfterSaveTrigger.cs">
namespace Promatis.Net.Data; Ø
Ø
// AD;Dò ?D ØC$8C² $ Aä!A' " D >DT@C =CT=C,,Ø (D42CT4Cä<C'5CÔ8DòòHC,,=C •
public interface IAfterSaveTrigger<T> where T : class Ø
{ Ø
    Task HandleAsync(EntityChangedEventArgs<T> args); Ø
}
</file>

<file path="Data/TriggerService/TriggerInterfaces/IBeforeSaveTrigger.cs">
namespace Promatis.Net.Data; Ø
Ø
// AD;Dò ?D ØC$8C² $ Aä" ACäED ØCÔ5CÔ8Dò ,,2C ;C,,4C FC,,Ø) Ø
public interface IBeforeSaveTrigger<T> where T : class Ø
{ Ø
    Task HandleAsync(EntityCancelEventArgs<T> args); Ø
}
</file>

<file path="Data/Validation/GlobalPolymorphicValidator.cs">

```

```

using FluentValidation;
using FluentValidation.Results;
using Microsoft.Extensions.DependencyInjection;

namespace Promatis.Net.Domain;

public class GlobalPolymorphicValidator<T> : AbstractValidator<T> where T : class
{
    private readonly IServiceProvider _serviceProvider;

    public GlobalPolymorphicValidator(IServiceProvider serviceProvider)
    {
        _serviceProvider = serviceProvider ?? throw new
ArgumentNullException(nameof(serviceProvider));
    }

    protected override bool PreValidate(ValidationContext<T> context, ValidationResult result)
    {
        if (context.InstanceToValidate == null) return true;

        // 1. A0>C'CDt0CT< D 5C ;DÄ=D'9 D$8C0 >C JCT:D$0 C" @C =D$0C"<CR „=C ?D 8CÄ5D Ä FW 'FÖVÇEVæ-B 8C'8 A
        Type realType = context.InstanceToValidate.GetType();

        // ATAC'8 D$8C0 ACä2C00CD0CTB D B „B.CRâ MD$> C0>C05Dt=D'9 C0;C AD Ä 0 C05 C 0Ct>C$KC' 0C AD$@C :D$=
        // D$> C„AC00D$L CD>Dt5D =C„9 C$0C'8CD0D$>D =CR =D46C0>, DtBCä1D² =CR CC"BC, 2 C 5D :Cä=CTGC0CDâ @CT
        if (realType == typeof(T)) return true;

        // 2. B BD >C„< D$8C0 8C0BCT@DD5C"AC 4C'0 D$>Dt5Dt=Cä3Cä 2C ;C„4C BCä@C „=C ?D 8CÄ5D Ä •f Æ-F F÷#ÄF
        Type specificValidatorType = typeof(IValidator<>).MakeGenericType(realType);

        // 3. At0C0@C HC„2C 5CÄ 8Cr D' B C$0C'8CD0D$>D K CD;D0 MD$>C4> C0>C0:D 5D$=Cä3Cä BC„?C
        // B0BCä 2C 6C0>, D$0C0 :C : CD;D0 >CD=Cä3Cä BC„?C <Cä6CTB C KD$L Ct0D 5C48D BD 8D >C$0C0> C05D :Cä;
        List<IValidator> specificValidators =
_serviceProvider.GetServices(specificValidatorType).Cast<IValidator>().ToList();

        if (specificValidators.Any())
        {
            // B >Ct4C 5CÄ =CR04Cd5C05D 8C0 :Cä=D$5C0AD" 2C ;C„4C FC„8 FluentValidation CD;D0 ?CT@CT4C GC, 4C
            ValidationContext<object>? newContext =
ValidationContext<object>.CreateWithOptions(context.InstanceToValidate, options =>
{
                // A„!A0 A A' A0 : A0C0>C08Dt=D'9 CÄ5D$>CB fÇVVÇEf Æ-F F-öâ 4C'0 C0@D0<Cä9 C0@Cä1D >D :C, A
                if (context.Selector != null)
                {
                    options.UseCustomSelector(context.Selector);
                }
            });

            // At0C0CD :C 5CÄ :C 6CDKC' =C 9CD5C0=D'9 D$>Dt5Dt=D'9 C$0C'8CD0D$>D =C AC'5CD=C„:C
            foreach (IValidator validator in specificValidators)
            {
                // BtBCä1D² 8Ct1CT6C BDÄ 7C FC„:C'8C$0C08D0Ä ?D >C$5D 0CT<, DtBCä =C 9CD5C0=D'9 C$0C'8CD0D$>D
                if (validator.GetType().IsGenericType &&
validator.GetType().GetGenericTypeDefinition() == typeof(GlobalPolymorphicValidator<>))
                    continue;

                ValidationResult specificResult = validator.Validate(newContext);

                // A05D 5C0>D 8CÄ 2D 5 Cä1C00D CCd5C0=D'5 CäHC„1C08 C" >C IC„9 D 5CtCC'LD$0D" 2C ;C„4C FC„8 D
                foreach (ValidationFailure error in specificResult.Errors)
                {
                    result.Errors.Add(error);
                }
            }
        }

        return true;
    }
}
</file>

```

```

<file path="Domain.Interface/Enums/EnumExtensions.cs">
using System.ComponentModel;
using System.Reflection;

namespace Promatis.Net.Domain.Interface;

```

```

Ð
public static class EnumExtensionsÐ
{Ð
    /// <summary>Ð
    /// A$>Ct2D 0D"0CTB Ct=C GCT=C,,5 C BD 8C CD$0 [Description] CD;Dò ;Dä1Cä3Cä MC´5CÄ5C0BC ?CT@CTGC,,AC´5C08
    /// ATAC´8 C BD 8C CD" >D$AD4BD BC$CCTB, C$>Ct2D 0D"0CTB D BC =CD0D BC0>CR 8CÄ0 .ToString().Ð
    /// </summary>Ð
    public static string GetDescription(this Enum value)Ð
    {Ð
        FieldInfo? field = value.GetType().GetField(value.ToString());Ð
        if (field == null) return value.ToString();Ð
    }
    var attribute = field.GetCustomAttribute<DescriptionAttribute>();Ð
    return attribute != null ? attribute.Description : value.ToString();Ð
}Ð
}
</file>

<file path="Domain.Interface/Interfaces/IAudit.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// B CD"=CäAD$8, D 5C ;C,,7D4ND"8CR MD$>D" 8C0BCT@DD5C"A, C CCDCD" 7C ?C,,AD´2C BDÄADò 2 AuditLogÐ
/// </summary>Ð
public interface IAuditÐ
{Ð
    Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObject.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectÐ
{Ð
    DateTime CreatedAt { get; init; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IDomainObjectHasKey.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
public interface IDomainObjectHasKey<TKey> : IDomainObjectÐ
{Ð
    TKey Id { get; set; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/IEntityCloner.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A,,=D$5D DCT9D ?C´0D$DCä@CÄ5C0=Cä3Cä 4C$8Cd:C 8Ct>C´8D >C$0C0=Cä3Cä :C´>C08D >C$0C08Dò 4Cä<CT=C0KDR AD4I
/// </summary>Ð
public interface IEntityClonerÐ
{Ð
    T CloneEntity<T>(T entity) where T : class;Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/ISoftDeletable.cs">
namespace Promatis.Net.Domain.Interface;Ð
Ð
/// <summary>Ð
/// A,,=D$5D DCT9D 4C´0 CÄOC4:Cä3Cä CCD0C´5C08Dò >C JCT:D$>C-
/// </summary>Ð
public interface ISoftDeletableÐ
{Ð
    DateTime? DeletedAt { get; set; }Ð
    Ð
    string? DeletedBy { get; set; }Ð
}
}
</file>

<file path="Domain.Interface/Interfaces/ITreeNode.cs">
namespace Promatis.Net.Domain.Interface;Ð

```

```

Ð
Ð
/// <summary>Ð
/// A4;Că1C ;DĂ=D´9 Cô;C BDD>D <CT=CÔKC´ :Că=D$@C :D" 4C´O C$ACTE C,,5D 0D EC,,GCTAC=8DR ,,4D 5C$>C$8CD=D´E) D C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Că<CT=CÔ>C´ <Că4CT;C,ăÂ÷G– W & Óí
public interface ITreeNode<T> : IDomainObjectHasKey<Guid> where T : classÐ
{Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D @Că4C,,BCT;DĂAC=8C' >D 47C´0. B 0C$5CÔ çVÆÂ 4C´O C= >D =CT2D´E DÔ;CT<CT=D$>C"í
    /// A,,!Aô A A´ AÔ : AD>C 0C$;CT= set CD;Dò 2Că7CĂ>Cd=CăAD$8 C,,7CĂ5CÔ5CÔ8Dò AC$0Ct5C´ ,,?CT@CT<CTICT=C,,O C
    /// </summary>Ð
    Guid? ParentId { get; set; }Ð
Ð
    /// <summary>Ð
    /// AÔ0C$8C40Dd8Că=CÔ>CR AC$>C"AD$2Că AD KC´:C, =C @Că4C,,BCT;DĂAC=8C' >C JCT:D"í
    /// </summary>Ð
    T? Parent { get; set; }Ð
Ð
    /// <summary>Ð
    /// A= >C´;CT:Dd8Dò 4CăGCT@CÔ8DR MC´5CĂ5CÔBCă2 (Cô>CDGC,,=CT=CÔKDR Cct;Că2).Ð
    /// </summary>Ð
    ICollection<T> Children { get; }Ð
}
</file>

<file path="Domain.Interface/Promatis.Net.Domain.Interface.csproj">
<Project Sdk="Microsoft.NET.Sdk">Ð
    Ð
    <PropertyGroup>Ð
        <TargetFramework>net10.0</TargetFramework>Ð
        <ImplicitUsings>enable</ImplicitUsings>Ð
        <Nullable>enable</Nullable>Ð
    </PropertyGroup>Ð
    Ð
</Project>
</file>

<file path="Domain/AuditLog/AuditLog.cs">
namespace Promatis.Net.Domain;Ð
Ð
/// <summary>Ð
/// A CCD8D" AC,,AD$5CĂKÐ
/// </summary>Ð
public class AuditLog : DomainObjectÐ
{Ð
    /// <summary>Ð
    /// AÔ0C,,<CT=Că2C =C,,5 D >C KD$8Dý
    /// </summary>Ð
    public string EntityName { get; init; } = string.Empty;Ð
    Ð
    /// <summary>Ð
    /// A,,4CT=D$8DD8C=0D$>D ACă1D´BC,,0Ð
    /// </summary>Ð
    public Guid EntityId { get; init; }Ð
    Ð
    /// <summary>Ð
    /// AD5C"AD$2C,,5Ð
    /// </summary>Ð
    public string Action { get; init; } = string.Empty;Ð
    Ð
    /// <summary>Ð
    /// AD0D$0 C,,7CĂ5CÔ5CÔ8Dý
    /// </summary>Ð
    public DateTime ChangedAt { get; init; }Ð
    Ð
    /// <summary>Ð
    /// A,,7CĂ5CÔ5CÔ>Ð
    /// </summary>Ð
    public string? ChangedBy { get; init; }Ð
    Ð
    /// <summary>Ð
    /// B BD >C=0 C,,7CĂ5CÔ5CÔ8C•
    /// </summary>Ð
    public string ChangesJson { get; init; } = string.Empty;Ð
}

```

</file>

```
<file path="Domain/DomainObject/DomainObject.cs">
namespace Promatis.Net.Domain;
{
    /// <summary>
    /// AäACÔ>C$=Cä9 C;C AD „uT"B ?Cä CCÄ>C´GC =C„N)
    /// </summary>
    public abstract class DomainObject : DomainObjectBase<Guid>
    {
        protected DomainObject()
        {
            Id = Guid.NewGuid();
        }
    }
}</file>
```

```
<file path="Domain/DomainObject/DomainObjectBase.cs">
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Domain;
    {
        /// <summary>
        /// B4=C„2CT@D 0C´LCÔKC´ 1C 7Cä2D´9 C;C AD A Cö>CD4CT@Cd:Cä9 C´NC >C4> D$8Cö0 C;DäGC
        /// </summary>
        /// <typeparam name="TKey">B$8Cö :C´NDt0</typeparam>
        public abstract class DomainObjectBase<TKey> : IDomainObjectHasKey<TKey>
        {
            public TKey Id { get; set; } = default!;
            public DateTime CreatedAt { get; init; } = DateTime.UtcNow;
        }
    }
}</file>
```

```
<file path="Domain/DomainObject/DomainObjectValidator.cs">
using FluentValidation;
{
    namespace Promatis.Net.Domain;
    {
        /// <summary>
        /// B4=C„2CT@D 0C´LCÔKC´ 2C ;C„4C BCä@ CD;Dö 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" FöÖ -äö&|V7M
        /// </summary>
        public abstract class DomainObjectValidator<T> : AbstractValidator<T> where T : DomainObject
        {
            protected DomainObjectValidator()
            {
                RuleFor(x => x.Id)
                    .NotEmpty().WithMessage("A„4CT=D$8DD8C;0D$>D >C JCT:D$0 CÖ5 CÄ>Cd5D" 1D´BDÄ ?D4AD$KCÄâ""½
                RuleFor(x => x.CreatedAt)
                    .Must(date => date <= DateTime.UtcNow.AddSeconds(1))
                    .WithMessage("AD0D$0 D >Ct4C =C„0 CÖ5 CÄ>Cd5D" 1D´BDÄ 2 C CCDCD"5CÄâ""½
            }
        }
    }
}</file>
```

```
<file path="Domain/EntityCloner/JsonEntityCloner.cs">
using System.Text.Json;
using System.Text.Json.Serialization;
using System.Text.Json.Serialization.Metadata;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Domain;
    {
        public class JsonEntityCloner : IEntityCloner
        {
            private static readonly JsonSerializerOptions JsonOptions = new()
            {
                ReferenceHandler = ReferenceHandler.IgnoreCycles,
                TypeInfoResolver = new DefaultJsonTypeInfoResolver
                {
                    Modifiers = { typeInfo =>
                        {
                            foreach (var property in typeInfo.Properties)
                            {
                                // 1. A$KD 5Ct0CT< Dd8C;C„GCTAC;8CR =C 2C„3C FC„>CÔ=D´5 D 2Cä9D BC$0 C„5D 0D EC„9D
                            }
                        }
                    }
                }
            }
        }
    }
}</file>
```

```

        if (property.Name is "Parent" or "Children")
        {
            property.ShouldSerialize = (_, _) => false;
            continue;
        }

        // 2. A$KD 5Ct0CT< D ;Cä6C0KCR AC$0Ct0C0=D´5 CD>CÄ5C0=D´5 Cä1D=5C=BD² 1D0:CT=CD0 C, :Cä;C
        Type propType = property.PropertyType;
        bool isDomainClass = typeof(Domain.DomainObject).IsAssignableFrom(propType);
        bool isDomainCollection = propType.IsGenericType &&

typeof(System.Collections.IEnumerable).IsAssignableFrom(propType) &&

typeof(Domain.DomainObject).IsAssignableFrom(propType.GetGenericArguments().FirstOrDefault());
        if (isDomainClass || isDomainCollection)
        {
            property.ShouldSerialize = (_, _) => false;
        }
    }
}

public T CloneEntity<T>(T entity) where T : class
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    // B 5D 8C ;C,,7D45CÄ 8 CD5D 5D 8C ;C,,7D45CÄÄ AD$@Cä3Cä ACäED 0C00D0 @CT0C´LC0KC´ @C =D$0C"<-D$8C0 =C
    string json = JsonSerializer.Serialize(entity, entity.GetType(), JsonOptions);
    return (T)JsonSerializer.Deserialize(json, entity.GetType(), JsonOptions)!;
}
</file>

<file path="Domain/Promatis.Net.Domain.csproj">
<Project Sdk="Microsoft.NET.Sdk">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>
    <ItemGroup>
        <PackageReference Include="FluentValidation" Version="12.1.1" />
    </ItemGroup>
    <ItemGroup>
        <ProjectReference Include="..\Domain.Interface\Promatis.Net.Domain.Interface.csproj" />
    </ItemGroup>
</Project>
</file>

<file path="Domain/ReferenceBase/ReferenceBase.cs">
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Domain;
/// <summary>
/// A 0Ct>C$KC´ :C´0D A CD;D0 2D 5DR AC0@C 2CäGC08C= >C-
/// </summary>
public abstract class ReferenceBase : DomainObject, ISoftDeletable
{
    public string Name { get; set; } = string.Empty;

    public string? Description { get; set; }

    public string? Code { get; set; }

    public DateTime? DeletedAt { get; set; }

    public string? DeletedBy { get; set; }
}
</file>

```

```

<file path="Domain/ReferenceBase/ReferenceBaseValidator.cs">
using FluentValidation;ð
using FluentValidation.Results;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C`LCÔKC` 2C ;C,,4C BCâ@ CD;Dð 2D 5DR :C`0D ACâ2, CÔ0D ;CT4D45CÄKDR >D" &VfW&Væ6T& 6]
/// </summary>ð
public abstract class ReferenceBaseValidator<T> : DomainObjectValidator<T> where T : ReferenceBaseð
{ð
    /// <summary>ð
    /// A$8D BD40C`LCÔKC` ?C BD$5D = D 5C4CC`OD =Câ3Câ 2D`@C 6CT=C,,0 CD;Dð ?D >C$5D :C, Câ4C í
    /// Að> D4<Câ;Dt0C08Däç AD$@Câ3Câ ;C BC,,=C,,FC 2 C$5D ECÔ5Câ @CT3C,,AD$@CRâ FC,,DD K C, òí
    /// </summary>ð
    protected virtual string CodeRegexPattern => @"^[A-Z0-9_]*$";ð
ð
    /// <summary>ð
    /// A$8D BD40C`LCÔ>CR ACâ>C ICT=C,,5 Cä1 CäHC,,1Câ5 C$0C`8CD0Dd8C, @CT3D4;Dð@CÔ>C4> C$KD 0Cd5CÔ8Dð Cä4C í
    /// </summary>ð
    protected virtual string CodeValidationMessage => "Aâ>CB <Câ6CTB D >CD5D 6C BDÂ BCä;DÄ:Câ ;C BC,,=C,,FD2 2
ð
    protected ReferenceBaseValidator()ð
    {ð
        RuleFor(x => x.Name)ð
            .NotEmpty().WithMessage("A00C,,<CT=Câ2C =C,,5 Cä1Dô7C BCT;DÄ=Câ 4C`O Ct0C0>C`=CT=C,,0")ð
            // Að@Câ2CT@Cð0 C00 C0@Cä1CT;D² 2 C00Dt0C`5 C, :Câ=Dd5ð
            .Must(name => name == null || name.Trim() == name)ð
            .WithMessage("A00C,,<CT=Câ2C =C,,5 CÔ5 CÄ>Cd5D" =C GC,,=C BDÄADð 8C`8 Ct0Cð0CÔGC,,2C BDÄADð ?D >C 5C`
            .MinimumLength(2).WithMessage("A00C,,<CT=Câ2C =C,,5 CD>C`6CÔ> D >CD5D 6C BDÂ <C,,=C,,<D4< 2 D 8CÄ2Cä;
            .MaximumLength(250).WithMessage("Að@CT2D`HCT=C <C :D 8CÄ0C`LCÔ0Dð 4C`8C00 C00C,,<CT=Câ2C =C,,0 (25
ð
        RuleFor(x => x.Code)ð
            .Matches(x => CodeRegexPattern)ð
            .When(x => !string.IsNullOrEmpty(x.Code))ð
            .WithMessage(x => CodeValidationMessage);ð
    }ð
ð
    public Func<object, string, Task<IEnumerable<string>>> ValidateValue => async (model,
propertyName) =>ð
    {ð
        ValidationResult? result = await
ValidateAsync(ValidationContext<T>.CreateWithOptions((T)model, x =>
x.IncludeProperties(propertyName)));ð
        return result.IsValid ? [] : result.Errors.Select(e => e.ErrorMessage);ð
    }ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBase.cs">
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// A 0Ct>C$KC` :C`0D A CD;Dð 2D 5DR ACô@C 2CâGC08Cð>C" ACâ4CT@Cd0D"8DR 4CT@CT2DÄ0ð
/// </summary>ð
/// <summary>ð
/// A 0Ct>C$KC` :C`0D A CD;Dð 2D 5DR 4D 5C$>C$8CD=D`E D ?D 0C$>Dt=C,,:Câ2 (MDM).ð
/// A,,ACô>C`LCtCCTB C00D$BCT@C0 5%E 4C`O C05D 5CD0Dt8 D BD >C4> D$8C08Ct8D >C$0CÔ=D`E C00C$8C40Dd8Cä=C0KDR A
/// </summary>ð
/// <typeparam name="T">B$8Cò :Câ=Cð@CTBCÔ>C4> CD>CÄ5CÔ=Câ3Câ >C JCT:D$0 (C00D ;CT4C08Cð0).</typeparam>ð
public abstract class ReferenceTreeBase<T> : ReferenceBase, ITreeNode<T> where T : classð
{ð
    /// <summary>ð
    /// A,,4CT=D$8DD8Cð0D$>D @Câ4C,,BCT;DÄACð>C4> D47C`0 C" !B4 ABí
    /// </summary>ð
    public Guid? ParentId { get; set; }ð
ð
    /// <summary>ð
    /// A00C$8C40Dd8Cä=CÔ>CR AC$>C"AD$2Câ AD KC`:C, =C @Câ4C,,BCT;DÄACð8C` >C JCT:D" 2 Cä?CT@C BC,,2CÔ>C` ?C <
    /// Að5D 5Cä?D 5CD5C`OCTBD 0 Cð0Cç f~'GV Â 2 Cð>CÔ5Dt=D`E Cð;C AD 0DR 4C`O Cð>CD4CT@Cd:C, Æ $' Æð F~æri
    /// </summary>ð
    public virtual T? Parent { get; set; }ð
ð
}

```

```

    /// <summary>ð
    /// Að>C´;CT:Dd8Dò 4CägCT@CÔ8DR MC´5CÄ5CÔBCä2 (Cò>CDGC,,=CT=CÔKDR Cct;Cä2).ð
    /// </summary>ð
    public virtual ICollection<T> Children { get; set; } = new List<T>();ð
}
</file>

<file path="Domain/ReferenceTreeBase/ReferenceTreeBaseValidator.cs">
using FluentValidation;ð
using Promatis.Net.Domain.Interface;ð
ð
namespace Promatis.Net.Domain;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C´LCÔKC´ 2C ;C,,4C BCä@ CD;Dò 2D 5DR :C´0D ACä2, CÔ0D ;CT4D45CÄKDR >D" &Vfw&Væ6UG&VT& 6]
/// </summary>ð
public abstract class ReferenceTreeBaseValidator<T> : ReferenceBaseValidator<T>ð
    where T : ReferenceBase, ITreeNode<T>ð
{ð
    protected ReferenceTreeBaseValidator() : base()ð
    {ð
        // Aô@Cä2CT@Cð0 CÔ0 D 0CÄ>Dd8D$8D >C$0CÔ8CR ,,>C JCT:D" =CR <Cä6CTB D4:C 7D´2C BDÄ =C ACT1Dò :C : CÔ0
        RuleFor(x => x.ParentId)ð
            .Must((model, parentId) => parentId != model.Id)ð
            .WithMessage("Aä1Dð5CðB CÔ5 CÄ>Cd5D" 1D´BDÄ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");ð
    }ð
    // A 0Ct>C$0Dò ?D >C$5D :C DCä@CÄ0D$0 GUIDð
    RuleFor(x => x.ParentId)ð
        .NotEqual(Guid.Empty)ð
        .When(x => x.ParentId.HasValue)ð
        .WithMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8Cð0D$>D =CR <Cä6CTB C KD$L CÔCD BD´< GUID");ð
    }ð
}
</file>

<file path="Promatis.Net.UI/_Imports.razor">
@using Microsoft.AspNetCore.Components.Formsð
@using Microsoft.AspNetCore.Components.Routingð
@using Microsoft.AspNetCore.Components.Webð
@using MudBlazorð
@using Promatis.Net.UI.Components.Workspacesð
@using Promatis.Net.Domain.Interfaceð
@using Promatis.Net.UI.Components.Dialogs
</file>

<file path="Promatis.Net.UI/Components/ActionContext/DataContext.cs">
using Microsoft.Extensions.DependencyInjection;ð
using Promatis.Net.Service;ð
ð
namespace Promatis.Net.UI.Components;ð
ð
/// <summary>ð
/// A 1D BD 0CðBCÔ>CR 8CÔDD 0D BD CCðBD4@CÔ>CR OCD@Cä @C 1CäBD² A CD0CÔ=D´<C,â
/// A,,=Cð0CÔAD4;C,,@D45D" @C 1CäBD2 1D >Cð5D 0, CðMD,,0 C, ACäAD$>Dô=C,,0 Ct0C4@D47Cð8, Cä1D"0DôADÄ A D 5D 2C,,AC
/// </summary>ð
public abstract class DataContext<TEntity, TKey, TQueryState, TResultData> : WorkspaceContext,ð
    IHasSelectedData<TEntity>,ð
    IDisposableð
    where TEntity : class, new()ð
    where TKey : notnullð
{ð
    private readonly IBaseService<TEntity, TKey> _baseService;ð
    private TEntity? _selectedData;ð
    private bool _isLoading;ð
    private bool _isDisposed;ð
    ð
    // A,,!Aô A A´ Aô : At0Cô>CÄ8CÔ0CT< D 5Cd8CÄ @C 1CäBD² 4C´0 CäBC´>Cd5CÔ=Cä9 C 5Ct>Cô0D =Cä9 C,,=C,,FC,,0C´8C
    private readonly bool _isInMemoryModeScheduled;ð
    ð
    public IUiDataBroker<TEntity, TQueryState, TResultData> Broker { get; }ð
    public IUiOzuCache<TEntity> OzuCache { get; }ð
    ð
    public TEntity? SelectedDatað
    {ð
        get => _selectedData;ð
        setð
    }
}

```

```

        {
            if (_selectedData != value)
            {
                _selectedData = value;
                OnContextUpdated?.Invoke();
                NotifyStateChanged();
            }
        }
    }

    public Action? OnContextUpdated { get; set; }
    public bool IsLoading => _isLoading;

    protected DataContext(
        IServiceProvider serviceProvider,
        bool isInMemoryMode,
        Action? onChangedNotifier = null)
        : base()
    {
        if (serviceProvider == null) throw new ArgumentNullException(nameof(serviceProvider));

        _baseService = serviceProvider.GetRequiredService<IBaseService<TEntity, TKey>>();
        OzuCache = serviceProvider.GetRequiredService<IOzuCache<TEntity>>();
        Broker = serviceProvider.GetRequiredService<IUiDataBroker<TEntity, TQueryState,
TResultData>>();

        // A,,!Aô A A´ Aô : Aô@CäAD$> DD8C=AC,,@D45CÂ @CT6C,,< D 0C >D$K. Aô8C=0C= C´ ?D 8C$0Ct:C, 0C AD$@C :D$
        _isInMemoryModeScheduled = isInMemoryMode;
    }

    /// <summary>
    /// A,,!Aô A A´ Aô (Aô@Cä1C´5CÄ0 3): AT4C,,=C 0 C, 1CT7Cä?C ACô0Dò BCäGC=0 C,,=C,,FC,,0C´8Ct0Dd8C, 6C,,7Cô5Cô
    /// A= CôDC,,3D4@C,,@D45D" 1D >C=5D 8 Cô@C,,2Dô7D´2C 5D" 4CT;CT3C BD² BCä;DÄ:Cä BCä3CD0, C= C44C :Cä=D BD
    /// C 1D >C´ND$=Cä 2D 5DR :C´0D ACä2-Cô0D ;CT4C08C=C" 3C @C =D$8D >C$0C0=Cä 8 Cô>C´=CäAD$LDâ 7C 2CT@D,,8C
    /// </summary>
    public override void InitializeContext()
    {
        base.InitializeContext(); // Aô>CD=C,,<C 5CÂ 2D´7Cä2 C 1C 7Cä2Cä<D2 v= &·7 6T6öçFW‡M

        if (_isInMemoryModeScheduled)
        {
            Broker.ConfigureInMemoryMode(OzuCache, LoadInitialDataForBrokerAsync,
EvaluateDataStateInMemory);
        }
        else
        {
            Broker.ConfigureServerMode(FetchDataFromServerAsync);
        }
    }

    /// <summary>
    /// AT4C,,=C 0 C, CCô8C$5D AC ;DÄ=C 0 D$>Dt:C 7C ?D >D 0 CD0Cô=D´E CD;Dò 1C 7Cä2Cä3Cä ECä;D BC AD$@C =C,,
    /// A$:C´NDt0CTB C$8CtCC ;DÄ=D´9 C,,=CD8C=0D$>D 7C 3D CCT:C, ...ö-4Æö F-ær´ :C : CD;Dò 6W'fw$ööFRÂ BC : C,
    /// </summary>
    public async Task<TResultData> GetDataAsync(TQueryState state, CancellationToken ct = default)
    {
        if (_isLoading) return default!;

        try
        {
            _isLoading = true;
            NotifyStateChanged(); // Aô>C=0CtKC$0CT< C= @D4BC,,;C=C Ct0C4@D47C=8 Cô0 Dô:D 0C05D

            return await Broker.FetchDataAsync(state, ct);
        }
        finally
        {
            _isLoading = false;
            NotifyStateChanged(); // B :D KC$0CT< C= @D4BC,,;C=C Cö>D ;CR >D$@C,,ACä2C=8 CD0Cô=D´ED
        }
    }

    /// <summary>
    /// B ;D46CT1CôKC´ ;CT=C,,2D´9 Cö>D BC 2D"8C 4C =CôKDR 4C´O A @Cä:CT@C ,, -äÖVö÷´´ ööFR´í
    /// </summary>
    private async Task<List<TEntity>> LoadInitialDataForBrokerAsync(TQueryState state,

```

```

CancellationToken ct)
{
    try
    {
        return await _baseService.GetAllAsync(ct) ?? [];
    }
    catch (OperationCanceledException)
    {
        return [];
    }
}

protected abstract TResultData EvaluateDataStateInMemory(TQueryState state,
IEnumerable<TEntity> inMemoryList);
protected abstract Task<TResultData> FetchDataFromServerAsync(TQueryState state,
CancellationToken ct);
protected IBaseService<TEntity, TKey> GetBaseService() => _baseService;

public virtual void Dispose()
{
    if (_isDisposed) return;

    Broker?.Dispose();
    _isDisposed = true;
}
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/EntityContext.cs">
using Microsoft.Extensions.DependencyInjection;
using MudBlazor;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.UI.Components;

/// <summary>
/// A,,=DD@C AD$@D4:D$CD =Cä5 Dô4D > D 0C >D$K D AD4ICô>D BDô<C, 8 Cä?CT@C FC,,OCÄ8 CRUD.D
/// A 5Ct>C00D =Cä 8Ct>C`8D CCTB CÄCD$0Dd8C, 4C =C0KDRÂ CCô@C 2C`OCTB CÄ>CD0C`LC0KCÄ8 Cä:C00CÄ8 C, CCD0C`5C08
/// </summary>
public abstract class EntityContext<TEntity, TKey, TQueryState, TResultData>
: DataContext<TEntity, TKey, TQueryState, TResultData>
where TEntity : class, new()
where TKey : notnull
{
    protected IDialogService DialogService { get; }
    private readonly IEntityCloner _entityCloner;

    protected EntityContext(
        IServiceProvider serviceProvider,
        bool isInMemoryMode,
        Action? onDataChangedNotifier = null)
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)
    {
        // A 5Ct>C00D =Cä5 C00D$8C$=Cä5 C,,7C$;CTGCT=C,,5 D ;D46C K CD8C ;Cä3Cä2 MudBlazor Dt5D 5Cr vWE&W V-&VE
        DialogService = serviceProvider.GetRequiredService<IDialogService>();

        // A,,=C$5D AC,,0 Ct0C$8D 8CÄ>D BCT9. At0C0@C HC,,2C 5CÄ :C`>C05D GCT@CT7 C,,=D$5D DCT9D 8Cr D•
        _entityCloner = serviceProvider.GetRequiredService<IEntityCloner>();
    }

    /// <summary>
    /// A=CA0C04C D 8C0ED >C0=Cä3Cä !Cä7CD0C08D0 C ?C,,AC,í
    /// A,,=C,,FC,,0C`8Ct8D CCTB Dt8D BD`9 D0:Ct5CÄ?C`OD AD4ICô>D BC, 8 C05D 5CD0CTB D4?D 0C$;CT=C,,5 C" 0C AD$@
    /// </summary>
    protected async Task ExecuteCreateRecordAsync()
    {
        await OpenDialogWindowAsync(new TEntity(), isNew: true);
    }

    /// <summary>
    /// A=CA0C04C D 8C0ED >C0=Cä3Cä CT4C :D$8D >C$0C08D0 C ?C,,AC,í
    /// A 5Ct>C00D =Cä 8Ct>C`8D CCTB CÄCD$0Dd8Dä 4C =C0KDR GCT@CT7 C 1D BD 0C=BC0KC' 6ÆöæW" >D" >D =Cä2C0>C'
    /// </summary>
    protected async Task ExecuteEditRecordAsync(TEntity? selectedRow)
    {
        if (selectedRow == null) return;

```

```

D
    // A4;D41Cä:Cä5 Cä;Cä=C,,@Cä2C =C,,5 D$5C05D L Ct0D"8D"5C0> C,,=D$5D DCT9D >CÄ ,,@CTHC 5D" ?D >C ;CT<D2 F
    TEntity clone = _entityCloner.CloneEntity(selectedRow);D
D
    await OpenDialogWindowAsync(clone, isNew: false);D
D
}D
D
/// <summary>D
/// Aä>CÄ0C04C D 8C0ED >C0=Cä3Cä #CD0C`5C08D0 C ?C,,AC,i
/// At0C0@C HC,,2C 5D" =C BC,,2C0>CR ?Cä4D$2CT@Cd4CT=C,,5 C, 2D`?Cä;C00CTB Cä?CT@C FC,,N Dt5D 5Cr 8C0BCT@DD5C
/// </summary>D
protected async Task ExecuteDeleteRecordAsync(TEntity? selectedRow)D
{D
    if (selectedRow == null) return;D
D
    bool? confirm = await DialogService.ShowDialogAsync(D
        "B44C ;CT=C,,5 Ct0C08D 8",D
        "A$K D42CT@CT=D²Ä GD$> DT>D$8D$5 C 5Ct2Cä7C$@C BC0> D44C ;C,,BDÄ 2D`1D 0C0=D4N Ct0C08D L?",D
        yesText: "B44C ;C,,BDÄ"í
        noText: "AäBCÄ5C00");D
D
    if (confirm == true)D
    {D
        // A,,7C$;CT:C 5CÄ AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KC' :C`Ndr GCT@CT7 C0@Cä2CT@C=C C,,=D$5D DCT9D 0 ID0
        if (selectedRow is IDomainObjectHasKey<TKey> hasKeyEntity)D
        {D
            TKey idValue = hasKeyEntity.Id;D
D
            // A0>C`CDt0CT< CD>D BD4? C$ ACT@C$8D C CD0C0=D`E C,,7 C$5D EC05C4> D ;Cä0 D04D 0 C, 2D`7D`2C
            await GetBaseService().DeleteAsync(idValue);D
        }D
        elseD
        {D
            // B 5Ct5D 2C0KC' AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KC' ?D4BDÄÄ 5D ;C, 8C0BCT@DD5C"A C05 D 5C ;C,,7C
            // A0> C @DT8D$5C=BD4@C0> DD8C=AC,,@D45CÄÄ GD$> CD>CÄ5C0=D`5 Cä1D=5C=BD² >C 0Ct0C0K C,,<CTBDÄ :
            throw new InvalidOperationException(D
                $"B CD"=CäAD$L '{typeof(TEntity).Name}' C05 D 5C ;C,,7D45D" >C 0Ct0D$5C`LC0KC' 8C0BCT@DD5C
            )D
        }D
    }D
D
}D
D
/// <summary>D
/// A 1D BD 0C=BC00D0 BCäGC=0 C$Kct>C$0 Cä:C00 DD>D <D²ä
/// B ?CTFC,,DC,,D2 2D`7Cä2C ,, :C :Cä9 C,,<CT=C0> Razor-C= >CÄ?Cä=CT=D" 4C,,0C`>C40 CäBC=0D`BDÄ' 7C 4C AD" :C
/// </summary>D
protected abstract Task OpenDialogWindowAsync(TEntity model, bool isNew);D
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/GridContext.cs">
using Microsoft.Extensions.DependencyInjection;D
using MudBlazor;D
using Promatis.Net.Service;D
D
namespace Promatis.Net.UI.Components;D
D
/// <summary>D
/// A0@Cä<CT6D4BCäGC0KC' :Cä=D$5C=AD" ?D 5CDAD$0C$;CT=C,,0 CD;D0 BC 1C`8Dt=D`E D BD 0C08Db ,,w&-B 0öFR'í
/// AÄ0C08D" 0C AD$@C :D$=D`5 CD0C0=D`5 D04D 0 C" AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =C0KCR AD$@D4:D$CD K C00C48C00Dd8C
/// </summary>D
public abstract class GridContext<TEntity, TKey>D
: EntityContext<TEntity, TKey, GridState<TEntity>, GridData<TEntity>>D
where TEntity : class, new()D
where TKey : notnullD
{D
    protected GridContext(D
        IServiceProvider serviceProvider,D
        bool isInMemoryMode,D
        Action? onDataChangedNotifier = null)D
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)D
    {D
        // A,,!A0 A A` A0 : A,,=C$5D AC,,0 Ct0C$8D 8CÄ>D BCT9. A,,7C$;CT:C 5CÄ AD$@C BCT3C,,N CÄCD$0Dd8C' At# Dt
        var strategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();D
        OzuCache.SetMutationStrategy(strategy);D
    }D
D
}

```

```

    /// <summary>
    /// A,,!A0 A A' A0 : B 5C ;C,,7C FC,,0 C'>C0C'LC0>C' DC,,;DÄBD 0Dd8C,Ä ACä@D$8D >C$:C, 8 C00C48C00Dd8C, 2 C
    /// AÄ5D$>CB ?D 8C08CÄ0CTB C 5Ct>C00D =D'9 IReadOnlyList C, 8D ?Cä;DÄ7D45D" =C BC,,2C0KCR <CTBCä4D² xVD&Æ
    /// </summary>
    protected override GridData<TEntity> EvaluateDataStateInMemory(GridState<TEntity> state,
IReadOnlyList<TEntity> inMemoryList)
    {
        // A0@CT2D 0D"0CT< C 5Ct>C00D =D'9 D ?C,,ACä: C" Ä"ä 02D'@C 6CT=C,,5D
        IQueryable<TEntity> query = inMemoryList.AsQueryable();

        // 1. A0@C,,<CT=D05CÄ ?Cä;DÄ7Cä2C BCT;DÄAC08CR DC,,;DÄBD K C0>C'>C0>C0 xVD&Æ |÷" =C ;CTBD=
        if (state.FilterDefinitions != null && state.FilterDefinitions.Any())
        {
            query = query.Where(state.FilterDefinitions);
        }

        // 2. A0@C,,<CT=D05CÄ 4C,,=C <C,,GCTAC0CDä ACä@D$8D >C$:D2 :Cä;Cä=Cä: MudBlazor
        if (state.SortDefinitions != null && state.SortDefinitions.Any())
        {
            query = query.OrderBy(state.SortDefinitions);
        }

        // A$KDt8D ;D05CÄ >C ICT5 C0>C'8Dt5D BC$> Ct0C08D 5C'Ä ?D >D,,5CDHC,,E DD8C'LD$@C FC,,N, CD> D 0Ct4CT;CT
        int totalCount = query.Count();

        // 3. A$KD 5Ct0CT< D$5C0CD"CDä AD$@C =C,,FD2 4C =C0KDR ,, C 3C,,=C FC,,0)
        List<TEntity> pagedItems = query
            .Skip(state.Page * state.PageSize)
            .Take(state.PageSize)
            .ToList();

        return new GridData<TEntity>
        {
            Items = pagedItems,
            TotalItems = totalCount
        };
    }

    /// <summary>
    /// B 5C ;C,,7C FC,,0 Dt5D BC0>C4> D 5D 2CT@C0>C4> C0>D BD 0C08Dt=Cä3Cä 7C ?D >D 0 C0 !B4 AB ...6W'fW" 0öFR'1
    /// </summary>
    protected override async Task<GridData<TEntity>> FetchDataFromServerAsync(GridState<TEntity>
state, CancellationToken ct)
    {
        // A$KctKC$0CT< C 0Ct>C$KC' ACT@C$8D 4C =C0KDRÄ BD 0C0AC'8D CD0 AD$5C"B MudBlazor C" ?C @C <CTBD K C
        PagedResult<TEntity> pagedResult = await GetBaseService().GetPagedAsync(state.Page,
state.PageSize, ct);

        return new GridData<TEntity>
        {
            Items = pagedResult.Items,
            TotalItems = pagedResult.TotalCount
        };
    }
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/IWorkspaceContext.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// AT4C,,=D'9 C0>C0BD 0C0B C0>C0BCT:D BC @C 1CäGCT9 Cä1C'0D BC,Ä CC0@C 2C'0DäIC,,9 C45Cä<CTBD 8CT9 C00D$8 Ct>
    /// D >D BC 2Cä< CD>D BD4'C0KDR T'04CT9D BC$8C' 8 D$5C0CD"8CÄ 2D'4CT;CT=C,,5CÄ 4C =C0KDR =C MC0@C =CR1
    /// </summary>
    public interface IWorkspaceContext
    {
        // --- B Ää B4 B A$ AT A,,/ A0 ÄÄ Ää AT B$ ÄÄ A, A A0+ÄÄ ---
        /// <summary>
        /// A0>C';CT:Dd8D0 8C0BCT@C :D$8C$=D'E D0;CT<CT=D$>C" CC0@C 2C'5C08D0 4C =C0>C4> DT>C'AD$0.
        /// </summary>
        IEnumerable<IUIControl> Controls { get; }

        event Action? OnContextStateChanged;
        void NotifyStateChanged();
    }
}

```

```

    /// <summary>Ð
    /// AT4C,,=C 0 D$>Dt:C 8CÔ8Dd8C ;C,,7C FC,,8 Cd8Ct=CT=CÔ>C4> Dd8C¤;C ;Dä1Cä3Cä :Cä=D$5C¤AD$0.Ð
    /// A$KcTKC$0CTBD 0 C 0Ct>C$KCÂ ECä;D BCä< (WorkspacePage) Cô@C, AD$0D BCR DCä@CÄK.Ð
    /// </summary>Ð
    void InitializeContext();Ð
Ð
    // --- Aô B AÄ B$ B² !B$ A´ At Bd A, A4 Aä AT"B A, R Aä ---Ð
    int PaperElevation { get; }Ð
    string PaperClass { get; }Ð
    string WorkspaceHeight { get; }Ð
    string TopZoneHeight { get; }Ð
    string BottomZoneHeight { get; }Ð
    string LeftZoneWidth { get; }Ð
    string RightZoneWidth { get; }Ð
Ð
    // A,,!Aô A A´ Aô : B$5Cô5D L C,,=D$5D DCT9D 4CT:C´0D 8D CCTB Cô>C´=CäFCT=CÔ>CR @CT0C¤BC,,2CÔ>CR ?Cä2CT4CT
    bool IsTopZoneCollapsed { get; set; }Ð
    bool IsBottomZoneCollapsed { get; set; }Ð
    bool IsLeftZoneCollapsed { get; set; }Ð
    bool IsRightZoneCollapsed { get; set; }Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceContext.cs">
using Promatis.Net.Domain;Ð
using Promatis.Net.UI.Controls;Ð
Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A 8Ct=CTA-A 3D 5C40D$>D 4C´O C$ACTE Cô;CäAC¤8DR ACô@C 2CäGCÔ8C¤>C" B D 8D BCT<D²i
/// BD8C¤AC,,@D45D" BC,,?D² 4C =CÔKDRÄ 2C¤;DäGC 5D" -äÖVö÷'' ÖöFR 8 C 2D$>CÄ0D$8Dt5D :C, ACä1C,,@C 5D" BC,,?Cä2Cä
/// </summary>Ð
public abstract class ReferenceContext<TEntity> : GridContext<TEntity, Guid>Ð
    where TEntity : ReferenceBase, new()Ð
{Ð
    // A,,!Aô A A´ Aô : Aô0D 0CÄ5D$@ isInMemoryMode D$5Cô5D L C48C :Cä CCô@C 2C´OCTBD 0 Dt5D 5Cr :Cä=D BD CC¤
    // Cô>Ct2Cä;Dô0 D ?D 0C$>Dt=C,,:C < Aô!A, 8D ?Cä;DÄ7Cä2C BDÄ 2D 5 Cô@CT8CÄCD"5D BC$0 C¤MD,,8D >C$0CÔ8Dò 2 A
    protected ReferenceContext(Ð
        IServiceProvider serviceProvider,Ð
        bool isInMemoryMode = true,Ð
        Action? onDataChangedNotifier = null)Ð
        : base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Ð
    { }Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C, 2D´7D´2C 5D$ADò 1C 7Cä2D´< DT>C´AD$>CÄ ...v÷&.7 6U vR´ 2 D >C KD$8C, öä-æ-F- Æ-!
    /// C¤>C44C 3D 0DB >C JCT:D$>C" =C AC´5CD=C,,:Cä2 D46CR 3C @C =D$8D >C$0CÔ=Cä ?CäAD$@Cä5CÔ 2 Cô0CÄ0D$8.Ð
    /// </summary>Ð
    public override void InitializeContext()Ð
    {Ð
        PopulateDefaultCrudToolBar();Ð
    }Ð
Ð
    /// <summary>Ð
    /// A 2D$>CÄ0D$8Dt5D :C 0 D 1Cä@C¤0 Cô0CÔ5C´8 D4?D 0C$;CT=C,,O.Ð
    /// A¤>CÔ5Dt=D´5 D ?D 0C$>Dt=C,,:C, ?D 8 Cd5C´0CÔ8C, <Cä3D4B Cô5D 5Cä?D 5CD5C´8D$L CÄ5D$>CBÄ >Dt8D BC,,BDÄ
    /// </summary>Ð
    protected virtual void PopulateDefaultCrudToolBar()Ð
    {Ð
        // A¤=Cä?C¤0 B >Ct4C BDÄ r 2D´7D´2C 5D" 3CäBCä2D4N Cô@Cä2Cä4C¤C Dô4D 0Ð
        AddControl(new CreateEntityButton<TEntity>().OnExecute(ExecuteCreateRecordAsync));Ð
Ð
        // A,,!Aô A A´ Aô : Aô>C´=CäAD$LDä 8Ct1C 2C,,;C,,ADÄ >D" 0CÔ>CÔ8CÄ=D´E C´OCÄ1CD0-Ct0CÄKC¤0CÔ8C´ 7-æ2 $
        // B AD´;C¤8 CÔ0 C 0Ct>C$KCR 5%TBô:Cä<C =CDK Cô5D 5CD0DäBD 0 CÔ0Cô@Dô<D4N, D BD 0DTCDò t2 >D" CD$5Dt5
        AddControl(new EditEntityButton<TEntity>().OnExecute(ExecuteEditRecordAsync));Ð
Ð
        // A¤=Cä?C¤0 B44C ;C,,BDÄ r 7C ?D 0D,,8C$0CTB MessageBox C, 2D´?Cä;CÔ0CTB C¤>CÄ0CÔ4D2 CCD0C´5CÔ8Dý
        AddControl(new DeleteEntityButton<TEntity>().OnExecute(ExecuteDeleteRecordAsync));Ð
Ð
        // B BC =CD0D BCôKC´ 2C,,7D40C´LCôKC´ @C 7CD5C´8D$5C´LÐ
        AddControl(new ToolbarDivider());Ð
    }Ð
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ActionContext/ReferenceTreeContext.cs">
using Promatis.Net.Domain;
using Promatis.Net.UI.Controls;
{
namespace Promatis.Net.UI.Components;
{
/// <summary>
/// A 8Ct=CTA-A 3D 5C40D$>D 4C' O C$ACTE CD@CT2Cä2C,,4CÔKDR ,,8CT@C @DT8Dt5D :C,,E) D ?D 0C$>Dt=C,,:Cä2 AÔ!A, AC,,
/// BD8C=AC,,@D45D" BC,,?D² 4C =CÔKDRÂ =C AD$@C 8C$0CTB D 5C=CD AC,,2CÔKC' At#-C=MD, 8 C 2D$>CÄ0D$8Dt5D :C, ACä
/// </summary>
public abstract class ReferenceTreeContext<TEntity> : TreeContext<TEntity, Guid>
where TEntity : ReferenceTreeBase<TEntity>, new()
{
// A,,!AÔ A A' AÔ : A=CÔAD$@D4:D$>D AD$0C² :D 8D BC ;DÄ=Câ GC,,AD$KCÄâ C AD$@Cä9C=0 D BD 0D$5C48C' <D4B
// C, 2D'7Cä2D² 2C,,@D$CC ;DÄ=D'E CÄ5D$>CD>C" >D$ADä4C ?Cä;CÔ>D BDÄN D44C ;CT=D²
protected ReferenceTreeContext(
IServiceProvider serviceProvider,
Action? onDataChangedNotifier = null)
: base(serviceProvider, isInMemoryMode: true, onDataChangedNotifier)
{
}
}
{
/// <summary>
/// A,,!AÔ A A' AÔ : A 5Ct>CÔ0D =D'9 CÄ5D$>CB 6C,,7CÔ5CÔ=Cä3Câ FC,,:C'0 C=CÔBCT:D BC í
/// A 2D$>CÄ0D$8Dt5D :C, 2D'7D'2C 5D$AD0 1C 7Cä2D' < DT>C'AD$>CÄ ...v÷&·7 6U vR' 2 D >C KD$8C, öä-æ-F- Æ-!
/// C40D 0CÔBC,,@D40 Ct0D"8D$C C=C AD >C"0=C AC'5CD=C,,:Cä2 CäB NullReferenceException.
/// </summary>
public override void InitializeContext()
{
PopulateDefaultTreeToolbar();
}
}
{
/// <summary>
/// A 2D$>CÄ0D$8Dt5D :C O D 1Cä@C=0 CÔ0CÔ5C'8 D4?D 0C$;CT=C,,O CD;Dò 4CT@CT2C í
/// </summary>
protected virtual void PopulateDefaultTreeToolbar()
{
// 1. A=Cä?C=0 D >Ct4C =C,,O C=C>D =CT2Cä3Câ MC'5CÄ5CÔBC ,,1CT7 D >CD8D$5C'0)
AddControl(new CreateEntityButton<TEntity> { Title = "AD>C 0C$8D$L C=C>D 5CÔL" })
.OnExecute(ExecuteCreateRecordAsync));
// 2. A=Cä?C=0 D >Ct4C =C,,O CD>Dt5D =CT3Câ ?Cä4D47C'0 (C 2D$>CÄ0D$8Dt5D :C, 7C 2Dô7C =C =C =C ;C,,G
AddControl(new CreateChildButton<TEntity>().OnExecute(ExecuteCreateChildRecordAsync));
// 3. A,,!AÔ A A' AÔ : Aô@Dô<C O CÔ5D 5CD0Dt0 D AD';Cä: CÔ0 CÄ5D$>CDK C 5Cr 0CÔ>CÔ8CÄ=D'E C'0CÄ1CD0-C
AddControl(new EditEntityButton<TEntity>().OnExecute(ExecuteEditRecordAsync));
// 4. A=Cä?C=0 D44C ;CT=C,,O (MessageBox CÔ>CDBC$5D 6CD5CÔ8Dò 8 C=C>CÄ0CÔ4C CCD0C'5CÔ8Dò 2 B #A )
AddControl(new DeleteEntityButton<TEntity>().OnExecute(ExecuteDeleteRecordAsync));
// B BC =CD0D BCÔKC' 2C,,7D40C'LCÔKC' @C 7CD5C'8D$5C'L0
AddControl(new ToolbarDivider());
}
}
{
/// <summary>
/// B ?CTFC,,DC,,GCÔ0Dò 1C,,7CÔ5D Ô:Cä<C =CD0 D >Ct4C =C,,O CÔ>CDGC,,=CT=CÔ>C4> D47C'0.
/// B GC,,BD'2C 5D" "B 2D'4CT;CT=CÔ>C4> D >CD8D$5C'0 C, 7C ?C,,AD'2C 5D" 5C4> C" &VçD-B =Cä2Cä9 D CD"=CäA
/// </summary>
protected async Task ExecuteCreateChildRecordAsync()
{
if (SelectedData == null) return;
// B >Ct4C 5CÄ GC,,AD$KC' >C JCT:D"Ä 8D ?Cä;DÄ7D40 CÔ>C$KC' AC,,=D$0C=AC,,A C# Dd5C'5C$>C4> D$8CÔ0, D
// C, ?D 8C$0CtKC$0CT< CT3Câ : D$5C=CD"5CÄC C$KCD5C'5CÔ=Cä<D2 Cct;D=
TEntity childModel = new()
{
ParentId = SelectedData.Id
};
await OpenDialogWindowAsync(childModel, isNew: true);
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ActionContext/TreeContext.cs">
using Microsoft.Extensions.DependencyInjection;

```

```

Ð
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// Að@Cä<CT6D4BCäGCÖKC' :Cä=D$5CðAD" ?D 5CDAD$0C$;CT=C,,0 CD;Dð 4D 5C$>C$8CD=D'E D BD 0C08Db ...G&VR ÖðFR'í
/// Að5D 5C$>CD8D" 0C AD$@C :D$=D'5 CD0C0=D'5 Dð4D 0 C00 Dð7D': C,,5D 0D EC,,GCTACð8DR 2C,,7D40C'8Ct0D$>D >C"í
/// </summary>Ð
public abstract class TreeContext<TEntity, TKey>Ð
: EntityContext<TEntity, TKey, object, IReadOnlyList<TEntity>> Ð
where TEntity : class, new()Ð
where TKey : notnullÐ
{Ð
protected TreeContext(Ð
IServiceProvider serviceProvider,Ð
bool isInMemoryMode,Ð
Action? onDataChangedNotifier = null)Ð
: base(serviceProvider, isInMemoryMode, onDataChangedNotifier)Ð
{Ð
// A,,!Að A A' Að : A,,=C$5D AC,,0 Ct0C$8D 8CÄ>D BCT9. A,,7C$;CT:C 5CÄ 8CT@C @DT8Dt5D :D4N D BD 0D$5C48D
var treeStrategy = serviceProvider.GetRequiredService<IOzuMutationStrategy<TEntity>>();Ð
OzuCache.SetMutationStrategy(treeStrategy);Ð
}Ð
Ð
/// <summary>Ð
/// A,,!Að A A' Að : AÄ5D$>CB ?D 8C08CÄ0CTB C, 2Cä7C$@C IC 5D" 1CT7Cä?C ACÖKC' •&V DöæÇ"Æ-7Bí
/// AD;Dð 4CT@CT2C 2 Cð0CÄ0D$8 CÄK Cð@CäAD$> C$>Ct2D 0D"0CT< C$5D L C00Cð>Cð;CT=CÖKC' 3D 0DB >C JCT:D$>C
/// </summary>Ð
protected override IReadOnlyList<TEntity> EvaluateDataStateInMemory(object state,
IReadOnlyList<TEntity> inMemoryList)Ð
{Ð
return inMemoryList;Ð
}Ð
Ð
/// <summary>Ð
/// Bt5D BCÖKC' ACT@C$5D =D'9 Ct0Cð@CäA CD;Dð 4CT@CT2C ä C 3D CCd0CTB C$5D L C'8C05C"=D'9 D ?C,,ACä: Ct0C
/// C00 CäAC0>C$5 Cð>D$>D >C4> C$8CtCC ;C,,7C BCä@ Cð>D BD >C,,B C4@C D D 2Dð7CT9 ParentId -> Children.Ð
/// </summary>Ð
protected override async Task<IReadOnlyList<TEntity>> FetchDataFromServerAsync(object state,
Cancellation token ct)Ð
{Ð
// A$Kct>C" ?D 8C$5CD5Cð 2 D >CäBC$5D$AD$2C,,5 D ACT@C$8D =D'< Cð>C0BD 0CðBCä< C 5Cr ?C @C <CTBD >C-
List<TEntity> result = await GetBaseService().GetAllAsync();Ð
return result ?? [];Ð
}Ð
}
</file>

<file path="Promatis.Net.UI/Components/ActionContext/WorkspaceContext.cs">
namespace Promatis.Net.UI.Components;Ð
Ð
/// <summary>Ð
/// A 0Ct>C$KC' :Cä=D$5CðAD" ;Dä1Cä9 D 0C >Dt5C' >C ;C AD$8 (C$:C'0CD:C,' 2 D 8D BCT<CRí
/// Aä1D"8C' 7C00CÄ5C00D$5C'L CD;Dð <C05CÄ>D ECT<, D ?D 0C$>Dt=C,,:Cä2, CD0D,,1Cä@CD>C" 8 C4@C DC,,:Cä2.Ð
/// </summary>Ð
public class WorkspaceContext : IWorkspaceContextÐ
{Ð
// Að@C,,<CT=Dð5CÄ :Cä;C'5CðFC,,>C0=D'5 C$KD 0Cd5C08Dð 2=
protected readonly List<IUIControl> _controls = [];Ð
Ð
// A,,ACð>C'LctCCT< C0>C$KC' Æö6² 4C'0 Ct0D"8D$K Cð>C';CT:DD8C, >D" DCä=Cä2D'E Cð>D$>Cð>C" ,,BD 8C43CT@Cä2
private readonly Lock _lockObject = new();Ð
Ð
public IEnumerable<IUIControl> ControlsÐ
{Ð
getÐ
{Ð
// AäBCD0CT< D =C ?D,,>D" 2Cä 8Ct1CT6C =C,,5 InvalidOperationException Cð@C, @CT=CD5D 8C03CR 2Cä 2D
lock (_lockObject)Ð
{Ð
return _controls.ToArray();Ð
}Ð
}Ð
}Ð
Ð
public event Action? OnContextStateChanged;Ð
public void NotifyStateChanged() => OnContextStateChanged?.Invoke();Ð

```

```

Ð
/// <summary>Ð
/// A 0Ct>C$0Dð ?D4AD$0Dð @CT0C´8Ct0Dd8Dðâ CăBCă<Că: CĂ>Cd5D" ?CT@CT>C0@CT4CT;C,,BDÂ 5E
/// CD;Dð 0C$BCă<C BC,,GCTAC¤>C' AC >D :C, BD4;C 0D >C" 8C´8 C00Dt0C´LC0>C' =C AD$@Că9C¤8.Ð
/// </summary>Ð
public virtual void InitializeContext()Ð
{Ð
    // A0> D4<Că;Dt0C08Dâ =C,,GCT3Că =CR 4CT;C 5D"Â ?D 5CD>D BC 2C´ODð ECă;D B CD;Dð @C AD,,8D 5C08Dý
}Ð
Ð
// --- AD BD A´"A0 B0 A !B$ Aă A¤ A4 Aă AT"B A, B "A,, AT ---Ð
public virtual int PaperElevation => 1;Ð
public virtual string PaperClass => "pa-4 d-flex flex-column flex-grow-1 w-100 h-100";Ð
public virtual string WorkspaceHeight => "100%";Ð
public virtual string TopZoneHeight => "auto";Ð
public virtual string BottomZoneHeight => "auto";Ð
public virtual string LeftZoneWidth => "250px";Ð
public virtual string RightZoneWidth => "300px";Ð
Ð
// A,,!A0 A A´ A0 : B 2Că9D BC$0 Ct>C0 BCT?CT@DÂ ?D 8C0CCD8D$5C´LC0> C08C00DăB UI C0@C, 8Ct<CT=CT=C,,8 D >
public bool IsTopZoneCollapsedÐ
{Ð
    get;Ð
    setÐ
    {Ð
        if (field == value) return;Ð
        field = value;Ð
        NotifyStateChanged();Ð
    }Ð
}Ð
Ð
public bool IsBottomZoneCollapsedÐ
{Ð
    get;Ð
    setÐ
    {Ð
        if (field == value) return;Ð
        field = value;Ð
        NotifyStateChanged();Ð
    }Ð
}Ð
Ð
public bool IsLeftZoneCollapsedÐ
{Ð
    get;Ð
    setÐ
    {Ð
        if (field == value) return;Ð
        field = value;Ð
        NotifyStateChanged();Ð
    }Ð
}Ð
Ð
public bool IsRightZoneCollapsedÐ
{Ð
    get;Ð
    setÐ
    {Ð
        if (field == value) return;Ð
        field = value;Ð
        NotifyStateChanged();Ð
    }Ð
}Ð
Ð
// --- A0 B$ A¤ A At A0 B B´ AĂ B$ AD+ B4 B A$ AT A,,/ A0 A0 A´,Bă A´/ A0 B$ AĂ Aă ---Ð
protected void AddControl(IUiControl control)Ð
{Ð
    if (control == null) throw new ArgumentNullException(nameof(control));Ð
    lock (_lockObject)Ð
    {Ð
        _controls.Add(control);Ð
    }Ð
    NotifyStateChanged();Ð
}Ð
Ð

```

```

        protected void RemoveControl(string controlId)
        {
            lock (_lockObject)
            {
                _controls.RemoveAll(c => c.Id == controlId);
            }
            NotifyStateChanged();
        }
    }
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor">
@typeparam TModel where TModel : class
{
    <MudDialog>
        <TitleContent>
            <MudText Typo="Typo.h6">
                @Title
            </MudText>
        </TitleContent>
    </MudDialog>
    <DialogContent>
        @* A,,=D$5C4@C,,@D45CÂ fÇVVçEf Æ-F F-ôâ =C ?D OCÄCDâ 2 CÔ0D$8C$=D4N DD>D <D2 xVD&Æ !÷" ¢
        <MudForm @ref="_form"
            Model="@Model"
            Validation="@ (async (object model) => await ValidateModelViaFluentAsync())"
            ValidationDelay="0">
        </MudForm>
        @* B BD >C48C' 3CT>CÄ5D$@C,,GCTAC=8C' :Cä=D$5C'=CT@ Dô4D 0 - Ct0D"8D"0CTB DD>D <D2 >D" ?D KCd:Cä2
        <div class="d-flex flex-column pt-2" style="min-height: 340px; height: 60vh; max-
height: 60vh; min-width: 0; overflow: hidden;">
            @ChildContent
        </div>
    </DialogContent>
    </MudDialog>
    <DialogActions>
        @* AÔ0D$8C$=Cä5 C=0D :C 4CÔ>CR CCô@C 2C'5CÔ8CR >C=Ca< MudBlazor *@
        <MudButton Variant="Variant.Text"
            Color="Color.Default"
            OnClick="HandleCancelAsync">
            AäBCÄ5CÔ0
        </MudButton>
        <MudButton Variant="Variant.Filled"
            Color="Color.Primary"
            OnClick="HandleSubmitAsync">
            B >DT@C =C,,BDÍ
        </MudButton>
    </DialogActions>
</MudDialog>
</file>

<file path="Promatis.Net.UI/Components/Dialogs/BaseDialogLayout.razor.cs">
using FluentValidation;
using Microsoft.AspNetCore.Components;
using MudBlazor;
{
    namespace Promatis.Net.UI.Components.Dialogs;
    {
        public partial class BaseDialogLayout<TModel> : ComponentBase where TModel : class
        {
            protected MudForm _form = null!;

            [CascadingParameter]
            protected IMudDialogInstance MudDialog { get; set; } = null!;

            [Parameter]
            public string Title { get; set; } = "B 5CD0C=BC,,@Cä2C =C,,5 Ct0Cô8D 8";

            [Parameter]
            public required TModel Model { get; set; }

            [Parameter]
            public IValidator<TModel>? Validator { get; set; }
        }
    }
}

```

```

    }
    /// <summary>
    /// A>CÔBCT=D"Â :CäBCä@D'9 C CCD5D" @C 7C$5D =D4B C$=D4BD 8 DD>D <D² „CCÔ8C¤0C'LCÔKCR ?Cä;Dò 2C$>CD0 C„;
    /// </summary>
    [Parameter]
    public required RenderFragment ChildContent { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Model == null)
            throw new ArgumentNullException(nameof(Model), "Aô0D 0CÄ5D$@ Model Dô2C'0CTBD 0 Cä1Dô7C BCT;DÄ=D");
    }

    public async Task HandleSubmitAsync()
    {
        if (_form == null) return;

        // Aô0D$8C$=Cä AC¤0CÔ8D CCT< C$ADâ DCä@CÄC D 8C'0CÄ8 MudBlazor
        await _form.ValidateAsync();

        if (_form.IsValid)
        {
            // At0C¤@D'2C 5CÂ >C¤=Cä 8 C$>Ct2D 0D"0CT< C$0C'8CD=D4N CÄ>CD5C'L CÔ0 D BD 0CÔ8DdC¤
            MudDialog.Close(DialogResult.Ok(Model));
        }
    }

    public void HandleCancelAsync()
    {
        MudDialog.Cancel();
    }

    protected async Task<IEnumerable<string>> ValidateModelViaFluentAsync()
    {
        if (Validator == null) return Array.Empty<string>();

        var result = await Validator.ValidateAsync(Model);
        if (result.IsValid) return Array.Empty<string>();

        // AäBCD0CT< MudForm C0;CäAC¤8C' ACô8D >C¤ BCT:D BCä2D'E CäHC„1Cä: CD;Dò ?Cä4D 2CTBC¤8 C„=CôCD$>C-
        return result.Errors.Select(e => e.ErrorMessage);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Dialogs/DialogTabConfig.cs">
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Dialogs;
/// <summary>
/// B BD >C4> D$8Cô8Ct8D >C$0CÔ=D'9 C¤;C AD ô:Cä=DD8C4CD 0D$>D 4C'0 CD8CÔ0CÄ8Dt5D :C„E C$:C'0CD>C¤1
/// Aô>Ct2Cä;Dô5D" ?CT@CT4C 2C BDÂ :C AD$>CÄ=D4N D 0Ct<CTBC¤C C0>C'5C' 2C$>CD0, C0@C„2Dô7C =CÔCDâ : Cd8C$>C'
/// </summary>
public class DialogTabConfig<TModel> where TModel : class
{
    /// <summary>
    /// B$5C¤AD$>C$KC' 7C 3Cä;Cä2Cä: C$:C'0CD:C, =C ?C =CT;C, xVEF '21
    /// </summary>
    public string Title { get; }

    /// <summary>
    /// B 8D BCT<CÔ0Dò 8C¤>CÔ:C xVD&Æ ¦÷" 4C'0 Ct0C4>C'>C$:C 2C¤;C 4C¤8 (Cä?Dd8Cä=C ;DÄ=Cä'i
    /// </summary>
    public string? Icon { get; }

    /// <summary>
    /// B BD >C4> D$8Cô8Ct8D >C$0CÔ=C 0 D 0Ct<CTBC¤0 C0>C'5C' 2C$>CD0, C0@C„=C„<C ND"0Dò 6C„2D4N CD>CÄ5CÔ=D4N
    /// </summary>
    public RenderFragment<TModel> Content { get; }

    public DialogTabConfig(string title, RenderFragment<TModel> content, string? icon = null)
    {
        if (string.IsNullOrEmpty(title))
            throw new ArgumentException("At0C4>C'>C$>C¤ 2C¤;C 4C¤8 CÔ5 CÄ>Cd5D" 1D'BDÂ ?D4AD$KCÄâ"Â æ ÖVöb†F-

```

```

    Title = title;
    Content = content ?? throw new ArgumentNullException(nameof(content));
    Icon = icon;
}
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor">
@typeparam TModel where TModel : Promatis.Net.Domain.ReferenceBase
{
    @* Aä1Cä@C GC,,2C 5CÂ 2D Q C" 1C 7Cä2D´9 C@0D :C A Dô4D 0, Cô@Cä1D 0D KC$0Dò <Cä4CT;DÄÄ 7C 3Cä;Cä2Cä: C, 2C ;C
    <BaseDialogLayout TModel="TModel"
        Model="@Model"
        Title="@Title"
        Validator="@Validator">
    {
        @* A$=D4BD 5CÔ=C,,9 C@>CÔBCT=D" :C @C@0D 0: Cä?D 5CD5C´OCT<, CÔCCd=Cä ;C, @C 7C$>D 0Dt8C$0D$L C$:C´0CD:C,
        @if (CustomTabs == null || !CustomTabs.Any())
        {
            @* B &AT A A,, A ¢ C AD$>CÄ=D´E C$:C´0CD>C¢ =CTB. A$KC$>CD8CÂ BCä;DÄ:Cä 1C 7Cä2D´5 Cô>C´O D ?D 0C$>
            <div class="flex-grow-1" style="height: 100%; overflow-y: auto; padding-right: 4px;">
                @RenderBaseFields()
            </div>
        }
        else
        {
            @* B &AT A A,, A ¢ C AC´5CD=C,,: Cô5D 5CD0C² AC$>C, 2C@;C 4C@8. B 0Ct2Cä@C GC,,2C 5CÂ <CÔ>C4>D BD 0CÔ
            <MudTabs Elevation="0"
                Rounded="true"
                Class="d-flex flex-column flex-grow-1 h-100"
                @bind-ActivePanelIndex="_activeTabIndex">
            {
                @* B 8D BCT<CÔ0Dò C@;C 4C@0 1: Aô>C´O ReferenceBase CäB Dô4D 0 D 8D BCT<D² ¢
                <MudTabPanel Text="AäACô>C$=Cä5" Icon="@Icons.Material.Filled.Info" PanelClass="pa-4 flex-grow-1"
                    <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">
                        @RenderBaseFields()
                    </div>
                </MudTabPanel>
            }
        }
    }
    @* AD8CÔ0CÄ8Dt5D :C,,5 A$:C´0CD:C, "³¢ CT@CT1C,,@C 5CÂ 8 D 5CÔ4CT@C,,< C$ACR :C AD$>CÄ=D´5 D$0C K D
    @foreach (DialogTabConfig<TModel> tab in CustomTabs)
    {
        <MudTabPanel Text="@tab.Title" Icon="@tab.Icon" PanelClass="pa-4 flex-grow-1">
            <div style="max-height: 48vh; overflow-y: auto; padding-right: 4px;">
                @tab.Content(Model)
            </div>
        </MudTabPanel>
    }
}
</MudTabs>
}
</BaseDialogLayout>
{
    @code {
        /// <summary>
        /// B BC =CD0D BC,,7C,,@Cä2C =CÔ0Dò @C 7CÄ5D$:C 1C 7Cä2D´E Cô>C´5C´ B , Cä1D´0Dò 4C´O C$ACTE 100+ D ?D 0
        /// </summary>
        private RenderFragment RenderBaseFields() => __builder =>
        {
            <div class="d-flex flex-column gap-4">
                <MudTextField @bind-Value="Model.Name"
                    Label="Aô0C,,<CT=Cä2C =C,,5"
                    For="@(() => Model.Name)"
                    Required="true" />
                <MudTextField @bind-Value="Model.Code"
                    Label="B 8D BCT<CÔKC´ :Cä4 (B4=C,,:C ;DÄ=D´9)"
                    For="@(() => Model.Code)" />
                <MudTextField @bind-Value="Model.Description"
                    Label="Aä?C,,AC =C,,5 / AD>Cô>C´=C,,BCT;DÄ=Cä5 Cô@C,,<CTGC =C,,5"
                    Lines="4"
                    For="@(() => Model.Description)" />
            </div>
        }
    }
}

```

```

    };
}
</file>

<file path="Promatis.Net.UI/Components/Dialogs/ReferenceDialogLayout.razor.cs">
using FluentValidation;
using Microsoft.AspNetCore.Components;
using Promatis.Net.Domain;

namespace Promatis.Net.UI.Components.Dialogs;

public partial class ReferenceDialogLayout<TModel> : ComponentBase where TModel : ReferenceBase
{
    protected int _activeTabIndex;

    /// <summary>
    /// B$5C▯AD$>C$KC' 7C 3Cä;Cä2Cä: C" HC ?C▯5 CÄ>CD0C'LCÔ>C4> Cä:Cô0. Aô@Cä1D 0D KC$0CTBD O CÔ0Cô@Dô<D4N C"
    /// </summary>
    [Parameter]
    public string Title { get; set; } = "B 5CD0C▯BC,,@Cä2C =C,,5 D ?D 0C$>Dt=C,,:C #½

    /// <summary>
    /// Ad8C$0Dô @CT4C :D$8D CCT<C O CD>CÄ5CÔ=C O CÄ>CD5C'L (CÔ0D ;CT4CÔ8C▯ &VfW&Væ6T& 6R'í
    /// </summary>
    [Parameter]
    public required TModel Model { get; set; }

    /// <summary>
    /// Aô>C'8CÄ>D DCôKC' fçVVçBô2C ;C,,4C BCä@ CD;Dô :Cä=C▯@CTBCÔ>C4> D ?D 0C$>Dt=C,,:C í
    /// </summary>
    [Parameter]
    public IValidator<TModel>? Validator { get; set; }

    /// <summary>
    /// A▯>C';CT:Dd8Dô AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CôKDR :C AD$>CÄ=D'E C$:C'0CD>C▯ ,,4CTAC▯@C,,?D$>D >C''Â
    /// C▯>D$>D KCR :Cä=C▯@CTBCôKC' ACô@C 2CäGCô8C▯ ECäGCTB CD>C 0C$8D$L CÔ0 DD>D <D2 2 CD>Cô>C'=CT=C,,5 C▯ 2C
    /// </summary>
    [Parameter]
    public IReadOnlyCollection<DialogTabConfig<TModel>>? CustomTabs { get; set; }

    protected override void OnInitialized()
    {
        base.OnInitialized();
        if (Model == null)
            throw new ArgumentNullException(nameof(Model), "Aô0D 0CÄ5D$@ Model Dô2C'0CTBD O Cä1Dô7C BCT;DÄ=D'

    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor">
@inherits RenderBase
<MudTooltip Text="@(Control.Tooltip ?? Control.Title)" Arrow="true" Placement="Placement.Bottom">
    <MudButton Variant="@Variant"
        StartIcon="@(Control.IsRunning ? null : Control.Icon)"
        Disabled="@(Control.IsEnabled || Control.IsRunning || !
Control.IsEnabledForData(CurrentSelectedData))"
        Color="@Color"
        OnClick="@HandleClickAsync">
    @if (Control.IsRunning)
    {
        <MudProgressCircular Class="ms-n1 me-2" Size="Size.Small" Indeterminate="true"
Color="@Color.Inherit" />
    }
    @Control.Title
</MudButton>
</MudTooltip>
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/ButtonRenderBase.razor.cs">
using Microsoft.AspNetCore.Components;
using MudBlazor;

namespace Promatis.Net.UI.Components.ElementRenderBase;


```

```

public partial class ButtonRenderBase : RenderBase
{
    [Parameter] public Color Color { get; set; } = Color.Inherit;
    [Parameter] public Variant Variant { get; set; } = Variant.Text;

    protected async Task HandleClickAsync()
    {
        await Control.TriggerAsync(CurrentSelectedData);
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor">
@inherits RenderBase
{
    <MudDateRangePicker Label="@Control.Title"
        DateRange="@GetDateRange()"
        DateRangeChanged="@HandleDateRangeChangedAsync"
        Disabled="@(!Control.IsEnabled || Control.IsRunning)"
        Margin="Margin.Dense"
        DateFormat="dd.MM.yyyy"
        Variant="Variant.Outlined"
        Style="max-width: 260px;" />
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DateRangeRenderBase.razor.cs">
using MudBlazor;

namespace Promatis.Net.UI.Components.ElementRenderBase;

public partial class DateRangeRenderBase : RenderBase
{
    /// <summary>
    /// A 5Ct>C00D =Cä 8Ct2C'5C00CTB Cä1D05C0B DateRange C,,7 CÄ>CD5C'8 C0>C0BD >C'0 Dt5D 5Cr 8C0BCT@DD5C"A-D
    /// </summary>
    protected DateRange? GetDateRange()
    {
        if (Control is IHasValue valueProvider)
        {
            return valueProvider.Value as DateRange;
        }
        return null;
    }

    /// <summary>
    /// At0C08D KC$0CTB C0>C$>CR 7C00Dt5C08CR 2D'1D 0C0=Cä3Cä 4C,,0C00Ct>C00 CD0D" 2 CÄ>CD5C'L C, 7C ?D4AC00CT
    /// </summary>
    protected async Task HandleDateRangeChangedAsync(DateRange? newRange)
    {
        if (Control is IHasValue valueProvider)
        {
            valueProvider.Value = newRange;
            await Control.TriggerAsync(CurrentSelectedData);
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor">
@inherits RenderBase
{
    <MudDivider Vertical="true" FlexItem="true" Class="my-2mx-1" />
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/DividerRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;

public partial class DividerRenderBase : RenderBase
{
}
</file>

```

```

<file path="Promatis.Net.UI/Components/ElementRenderBase/RenderBase.cs">
using Microsoft.AspNetCore.Components;

namespace Promatis.Net.UI.Components.ElementRenderBase;

```

```

Ð
/// <summary>Ð
/// A 1D BD 0C=BC0KC' 1C 7Cä2D'9 C=CA?Cä=CT=D" 4C'0 C$ACTE Razor-D 5C04CT@CT@Cä2 D0;CT<CT=D$>C" CC0@C 2C'5C0
/// </summary>Ð
public abstract class RenderBase : ComponentBase, IDisposableÐ
{Ð
    /// <summary>Ð
    /// A 8Ct=CTA-CÄ>CD5C'L D0;CT<CT=D$0 D4?D 0C$;CT=C,,0, C05D 5CD0C$0CT<C 0 C,,7 DynamicComponent.Ð
    /// </summary>Ð
    [Parameter]Ð
    public IUiControl Control { get; set; } = null!;Ð
Ð
    /// <summary>Ð
    /// A=0D :C 4C0KC' :Cä=D$5C=AD" ECä;D BC @C 1CäGCT9 Cä1C'0D BC,i
    /// </summary>Ð
    [CascadingParameter]Ð
    protected IWorkspaceContext? ActionContext { get; set; }Ð
Ð
    /// <summary>Ð
    /// A 5Ct>C00D =Cä 8Ct2C'5C=0CTB D$5C=CD"8Cr 2D'4CT;CT=C0KCR 4C =C0KCR 8Cr :Cä=D$5C=AD$0 DD>D <D²i
    /// </summary>Ð
    protected object? CurrentSelectedData => GetSelectedDataFromContext();Ð
Ð
    protected override void OnInitialized()Ð
    {Ð
        base.OnInitialized();Ð
Ð
        // B 5C48D BD 8D CCT< D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C,,=D$5D DCT9D 0 C0@C, ACÄ5C05 D >D BCä0C08D0 1C
        Control.OnStateChanged += HandleStateChanged;Ð
    }Ð
Ð
    private void HandleStateChanged() => InvokeAsync(StateHasChanged);Ð
Ð
    private object? GetSelectedDataFromContext()Ð
    {Ð
        if (ActionContext == null) return null;Ð
Ð
        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");Ð
        return interfaceType?.GetProperty("SelectedData")?.GetValue(ActionContext);Ð
    }Ð
Ð
    public virtual void Dispose() => Control.OnStateChanged -= HandleStateChanged;Ð
}
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor">
@inherits RenderBaseÐ
<MudSelect T="string"Ð
    Value="@GetControlValue()"Ð
    ValueChanged="@HandleValueChangedAsync"Ð
    Disabled="@(!Control.IsEnabled || Control.IsRunning)"Ð
    Label="@Control.Title"Ð
    Margin="Margin.Dense"Ð
    Variant="Variant.Outlined"Ð
    Style="min-width: 160px; max-width: 250px;">Ð
Ð
    @foreach (string option in GetOptions())Ð
    {Ð
        <MudSelectItem T="string" Value="@option">@option</MudSelectItem>Ð
    }Ð
</MudSelect>
</file>

<file path="Promatis.Net.UI/Components/ElementRenderBase/SelectRenderBase.razor.cs">
namespace Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
public partial class SelectRenderBase : RenderBaseÐ
{Ð
    protected string GetControlValue()Ð
    {Ð
        if (Control is IHasValue valueProvider && valueProvider.Value != null)Ð
        {Ð
            return valueProvider.Value.ToString() ?? string.Empty;Ð
        }Ð
        return string.Empty;Ð
    }Ð
}

```

```

    protected async Task HandleValueChangedAsync(string newValue)
    {
        if (Control is IHasValue valueProvider)
        {
            valueProvider.Value = newValue;
            await Control.TriggerAsync(CurrentSelectedData);
        }
    }

    protected IEnumerable<string> GetOptions()
    {
        if (Control is IHasOptions optionsProvider)
        {
            return optionsProvider.Options;
        }
        return Array.Empty<string>();
    }
}
</file>

<file path="Promatis.Net.UI/Components/UiControls/BaseUiControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A 0Ct>C$0D0 0C AD$@C :Dd8D0 4C`O C$ACTE D0;CT<CT=D$>C" CC0@C 2C`5C08D0 ?C`0D$DCa@CÄK (C=Ca?Ca:, Dt5C=1Cä
    /// B 5C ;C,,7D45D" 8C0DD 0D BD CC=BD4@C0CDâ @D4BC,,=D2 8 C0@CT4CäAD$0C$;D05D" <CäAD" : CD8C00CÄ8Dt5D :Cä<D2 @C
    /// </summary>
    public abstract class BaseUiControl : IUiControl
    {
        private bool _isVisible = true;
        private bool _isEnabled = true;
        private bool _isRunning;

        public abstract string Id { get; }
        public abstract Type ComponentType { get; }

        public virtual string? Title { get; init; }
        public virtual string? Icon { get; init; }
        public virtual string? Tooltip { get; init; }

        public Dictionary<string, object> ComponentParameters { get; } = new();

        public virtual UiControlAlignment Alignment { get; init; } = UiControlAlignment.Left;

        public bool IsVisible
        {
            get => _isVisible;
            set { if (_isVisible != value) { _isVisible = value; NotifyStateChanged(); } }
        }

        public bool IsEnabled
        {
            get => _isEnabled;
            set { if (_isEnabled != value) { _isEnabled = value; NotifyStateChanged(); } }
        }

        public bool IsRunning => _isRunning;

        public event Action? OnStateChanged;

        protected BaseUiControl()
        {
            ComponentParameters.Add("Control", this);
        }

        protected void NotifyStateChanged() => OnStateChanged?.Invoke();

        public virtual bool IsEnabledForData(object? targetData) => true;

        public async Task TriggerAsync(object? targetData)
        {
            if (!_isEnabled || _isRunning || !IsEnabledForData(targetData)) return;

            try
            {

```

```

        _isRunning = true;
        NotifyStateChanged();
        await HandleTriggerAsync(targetData);
    }
    finally
    {
        _isRunning = false;
        NotifyStateChanged();
    }
}

protected abstract Task HandleTriggerAsync(object? targetData);
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/IHasOptions.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IHasOptions interface.
    /// </summary>
    public interface IHasOptions : IUIControl
    {
        IEnumerable<string> Options { get; }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/IHasValue.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IHasValue interface.
    /// </summary>
    public interface IHasValue : IUIControl
    {
        object? Value { get; set; }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/UIControl.cs">
namespace Promatis.Net.UI.Components;
{
    /// <summary>
    /// A C# class that implements the IUIControl interface.
    /// </summary>
    public interface IUIControl
    {
        string Id { get; }
        Type ComponentType { get; }
        Dictionary<string, object> ComponentParameters { get; }

        string? Title { get; }
        string? Icon { get; }
        string? Tooltip { get; }

        bool IsVisible { get; set; }
        bool IsEnabled { get; set; }
        bool IsRunning { get; }

        UIControlAlignment Alignment { get; }

        bool IsEnabledForData(object? targetData);
        Task TriggerAsync(object? targetData);

        event Action? OnStateChanged;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Components/UIControls/UIControlAlignment.cs">
namespace Promatis.Net.UI.Components;
{
    public enum UIControlAlignment
    {
        Left,
        Right
    }
}

```

</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor">

@typeparam TEntity where TEntity : class

@

<MudDataGrid @ref="_mudGrid"
 ServerData="@ServerDataProvider"
 T="TEntity"
 SelectedItem="@SelectedRow"
 SelectedItemChanged="@OnSelectedRowChanged"
 RowStyleFunc="@FormatSelectedRowStyle"
 RowsPerPage="@RowsPerPage"
 SelectOnRowClick="true"
 Hover="true"
 Dense="true"
 Bordered="true"
 Elevation="0"
 FixedHeader="true"
 Height="calc(100vh - 280px)"
 Class="flex-grow-1 w-100 cursor-pointer">
 <Columns>
 @GridColumns
 </Columns>
 @* A\$:C'NDt0CT< C00C48C00D\$>D BCä;DÄ:Câ ?Câ BD 5C >C\$0C08Dâ :Cä=CTGC0>C' DCä@CÄK *@D
 <PagerContent>
 @if (WithPagination)
 {
 <MudDataGridPager T="TEntity" InfoFormat="{first_item}-{last_item} C,,7 {all_items}"
RowsPerPageString="B BD >C£¢" óí
 }
 </PagerContent>
 </MudDataGrid>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/GridPage.razor.cs">

using Microsoft.AspNetCore.Components;

using MudBlazor;

@

namespace Promatis.Net.UI.Components.Workspaces;

@

public partial class GridPage<TEntity> : ComponentBase where TEntity : class

{

 private MudDataGrid<TEntity>? _mudGrid;

@

 [CascadingParameter]

 protected IWorkspaceContext? ActionContext { get; set; }

@

 [Parameter]

 public Func<GridState<TEntity>, CancellationToken, Task<GridData<TEntity>>>? ServerDataProvider { get; set; }

@

 [Parameter]

 public RenderFragment? GridColumns { get; set; }

@

 [Parameter]

 public bool WithPagination { get; set; } = false;

@

 [Parameter]

 public int RowsPerPage { get; set; } = 10000;

@

 protected TEntity? SelectedRow { get; set; }

@

 public Task ReloadServerData()

 {
 return _mudGrid != null ? _mudGrid.ReloadServerData() : Task.CompletedTask;
 }

@

 protected string FormatSelectedRowStyle(TEntity item, int rowNumber)

 => item == SelectedRow ? "background-color: var(--mud-palette-action-disabled-background);

font-weight: 500;" : string.Empty;

@

 protected void OnSelectedRowChanged(TEntity? newSelection)

 {
 if (newSelection == null && SelectedRow != null) return;

@

```

        SelectedRow = newSelection;
    }

    // A=0D :C 4C0> C0@Cä2CT@D05CÄÄ CCÄ5CTB C´8 C0@C,,;CTBCT2D,,8C´ :Cä=D$5C=AD" ED 0C08D$L C$KC @C =C0CDä
    if (ActionContext is IHasSelectedData<TEntity> bindableContext)
    {
        // A 5Ct>C00D =Cä 7C ?C,,AD´2C 5CÄ 2D´1D 0C0=D4N D BD >C=C C00C0@D0<D4N C" :Cä=D$5C=AD" 4C =C0KDR
        bindableContext.SelectedData = newSelection;
    }

    // A,,!A0 A A´ A0 : A$<CTAD$> OnContextUpdated (C=>D$>D KC´ 7C @CT7CT@C$8D >C$0C0 AD$@Cä3Cä ?Cä4
    // CÄK C$KctKC$0CT< NotifyStateChanged() Dt5D 5Cr 1CT7Cä?C AC0KC´ 0D 8C0ED >C0=D´9 InvokeAsync.
    // B0BCä <C4=Cä2CT=C0> C, 1CT7 C 5D :Cä=CTGC0KDR FC,,:C´>C" 4C 5D" :C0>C0:C < D$CC´1C @C 8CÄ?D4;D
    InvokeAsync(() =>
    {
        ActionContext.NotifyStateChanged();
    });
}
}
}
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceGridPage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceBase, new()
{
    @* A=0D :C 4C0> C0@Cä1D 0D KC$0CT< C=>C0BCT:D B C$=C,,7. AT3Cä ?Cä9CÄ0DäB C, V•Föö&& "Ä 8 C$0D, w&-E vR
    <CascadingValue Value="Context">
        <WorkspacePage>
        {
            @* 1. A$ B %A0/B0 Ää A ¢ C$BCä<C BC,,GCTAC=8C´ BD4;C 0D :C0>C0>C¢ AC0@C 2CäGC08C=0 A0!A,
            <TopContent>
                <UiToolbar Controls="@Context.Controls" />
            </TopContent>
        }

        @* 2. Bd A0"B A´,A0 B0 Ää A ¢ "C 1C´8Dt=C O D 5D$:C A D 8D BCT<C0KCÄ8 C0>C´0CÄ8 C0> D4<Cä;Dt0C08DÄ
        <BodyContent>
            <GridPage TEntity="TEntity">
                @ref="_grid"
                ServerDataProvider="@Context.Broker.FetchDataAsync"
                WithPagination="true"
                RowsPerPage="20">
                <GridColumn>
                    {
                        @* B B "AT A0+AR Ää Ää A : A$KC$>CD0D$AD0 0C$BCä<C BC,,GCTAC=8 CD;D0 B BR AC0@C 2CäGC
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Code" Title="B 8D BCT<C0K
                        HeaderStyle="width: 180px;" />
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Name" Title="A00C,,<CT=Cä2
                        HeaderStyle="width: 350px;" />
                        <PropertyColumn T="TEntity" TProperty="string" Property="x => x.Description"
                        Title="Ää?C,,AC =C,,5 / A0@C,,<CTGC =C,,5" HeaderStyle="width: auto;" />
                    }
                    @GridColumns
                </GridColumn>
            </GridColumns>
        </GridPage>
    </BodyContent>
}
</WorkspacePage>
</CascadingValue>
</file>

```

```

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceGridPage.razor.cs">
using Microsoft.AspNetCore.Components;
{
    namespace Promatis.Net.UI.Components.Workspaces;
    {
        public abstract partial class ReferenceGridPage<TEntity> : ComponentBase, IDisposable
        {
            where TEntity : Promatis.Net.Domain.ReferenceBase, new()
            {
                protected GridPage<TEntity>? _grid;
                private bool _isDisposed;

                [Inject]
                protected IServiceProvider PageServiceProvider { get; set; } = null!;

                /// <summary>
                /// B BD >C4> D$8C08Ct8D >C$0C0=D´9 C=>C0BCT:D B D4?D 0C$;CT=C,,0 D0BC,,< D ?D 0C$>Dt=C,,:Cä<.
            }
        }
    }
}

```

```

/// Aä1D=0C$;Dô5D$ADò :C : C 1D BD 0C=BC0>CR AC$>C"AD$2Cââ Cä=C=CTBC00Dò AD$@C =C,,FC ACô@C 2CäGC08C=0
/// C$5D =D4BDÂ MC=7CT<Cô;Dô@, D >Ct4C =C0KC' 2 CTQ OnInitialized.ð
/// </summary>ð
protected abstract ReferenceContext<TEntity> Context { get; }ð
ð
/// <summary>ð
/// B ;CäB CD;Dò 4CT:C'0D 0D$8C$=Cä3Câ >Cô8D 0C08Dò CC08C=0C'LC0KDR :Cä;Cä=Cä: D$0C ;C,,FD²í
/// </summary>ð
[Parameter]ð
public required RenderFragment GridColumns { get; set; }ð
ð
protected override void OnInitialized()ð
{ð
    base.OnInitialized();ð
    ð
    // Aô>CD?C,,AD'2C 5CÄADò =C ACä1D'BC,,0 C,,7CÄ5C05C08Dò AD$5C"BC :Cä=D$5C=AD$0 CD;Dò @CT0C=BC,,2C0>C' ?
    Context.OnContextStateChanged += RefreshUi;ð
    ð
    // B 2Dô7D'2C 5CÄ ACä1D'BC,,5 DD8C=AC FC,,8 CD0C0=D'E (C= >CÄ<C,,BC !B4 AB >D" BD 8C43CT@Cä2) D <Dô3C=
    Context.OnContextUpdated = RefreshGrid;ð
}ð
ð
protected override async Task OnInitializedAsync()ð
{ð
    await base.OnInitializedAsync();ð
    ð
    // A AC,,=DT@Cä=C0> C00C0>C'=Dô5CÄ ;Cä:C ;DÄ=D'9 C=MD, At# CD0C0=D'<C, 8Cr 1D0:CT=CD0 C0@C, ?CT@C$>CÄ
    await Context.LoadInitialDataAsync();ð
}ð
ð
/// <summary>ð
/// AÄ5D$>CB ?D 8C0CCD8D$5C'LC0>C4> Cä1C0>C$;CT=C,,0 CD0C0=D'E C" xVDF F w&-Bâ D'7D'2C 5D$ADò 1D >C=5D >C
/// </summary>ð
protected void RefreshGrid()ð
{ð
    if (_grid != null)ð
    {ð
        _ = _grid.ReloadServerData();ð
    }ð
}ð
ð
private void RefreshUi()ð
{ð
    InvokeAsync(StateHasChanged);ð
}ð
ð
/// <summary>ð
/// Aô>C'=C 0 CäGC,,AD$:C @CTAD4@D >C" 4C'0 C0@CT4CäBC$@C ICT=C,,0 D4BCTGCT: C00CÄOD$8 C" &Æ !÷-
/// </summary>ð
public void Dispose()ð
{ð
    if (_isDisposed) return;ð
    ð
    if (Context != null)ð
    {ð
        Context.OnContextStateChanged -= RefreshUi;ð
        Context.Dispose(); // A40D 0C0BC,,@Cä2C =C0> CäBC08D KC$0CT< C @Cä:CT@ CäB DatabaseTriggerServiceð
    }ð
    ð
    _isDisposed = true;ð
}ð
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor">
@typeparam TEntity where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()ð
ð
@* A=0D :C 4C0> C0@Cä1D 0D KC$0CT< C,,5D 0D EC,,GCTAC=8C' :Cä=D$5C=AD"â C4> C0>C"<C ND" 8 UiToolbar, C, 2C H T
<CascadingValue Value="Context">ð
    <WorkspacePage>ð
ð
        @* 1. A$ B %A0/Bò Aä A ¢ C$BCä<C BC,,GCTAC=8C' BD4;C 0D :C0>C0>C¢ CC0@C 2C'5C08Dò 4CT@CT2Cä< (A= >D
        <TopContent>ð
            <UiToolbar Controls="@Context.Controls" />ð
        </TopContent>ð
ð

```

```

        @* 2. Bd A0"B A',A0 B0 Aä A ¢ D 5C$>C$8CD=C O D BD CC=BD4@C A C 2D$>CÄ0D$8Dt5D :C,,< C$KC$>CD>CÄ ?
        <BodyContent>D
            <TreePage TEntity="TEntity"D
                RootItems="@Context.OzuCache.InMemoryItems"D
                ChildSelector="@ (x => x.Children)"D
                TextSelector="@ (x => string.IsNullOrEmpty(x.Code) ? x.Name :
$"[{x.Code}] {x.Name}]"D
                IconSelector="@ (x => x.Children != null && x.Children.Any() ?
Icons.Material.Filled.Folder : Icons.Material.Filled.InsertDriveFile)" />D
            </BodyContent>D
        D
        </WorkspacePage>D
    </CascadingValue>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/ReferenceTreePage.razor.cs">
using Microsoft.AspNetCore.Components;D
D
namespace Promatis.Net.UI.Components.Workspaces;D
D
public abstract partial class ReferenceTreePage<TEntity> : ComponentBase, IDisposableD
    where TEntity : Promatis.Net.Domain.ReferenceTreeBase<TEntity>, new()D
{D
    private bool _isDisposed;D
D
    [Inject]D
    protected IServiceProvider PageServiceProvider { get; set; } = null!;D
D
    /// <summary>D
    /// B BD >C4> D$8C08Ct8D >C$0C0=D'9 C,,5D 0D EC,,GCTAC=8C' :Cä=D$5C=AD" CC0@C 2C'5C08D0 MD$8CÄ 4CT@CT2Cä<.D
    /// Aä1D=0C$;D05D$AD0 :C : C 1D BD 0C=BC0>CR AC$>C"AD$2Cââ Cä=CTGC0KC' :C'0D A CD5D 5C$0 (B,,0C2 R' >C 0C
    /// C$5D =D4BDÄ MC=7CT<C0;D0@, D >Ct4C =C0KC' 2 CTQ OnInitialized.D
    /// </summary>D
    protected abstract ReferenceTreeContext<TEntity> Context { get; }D
D
    protected override void OnInitialized()D
    {D
        base.OnInitialized();D
D
        // A0>CD?C,,AD'2C 5CÄAD0 =C ACä1D'BC,,0 C,,7CÄ5C05C08D0 AD$5C"BC :Cä=D$5C=AD$0 CD;D0 @CT0C=BC,,2C0>C' ?
        Context.OnContextStateChanged += RefreshUi;D
D
        // B 2D07D'2C 5CÄ ACä1D'BC,,5 DD8C=AC FC,,8 CD0C0=D'E (C= >CÄ<C,,BC !B4 AB' A CÄ0C4:Cä9 C05D 5D 8D >C$:C
        Context.OnContextUpdated = RefreshUi;D
    }D
D
    protected override async Task OnInitializedAsync()D
    {D
        await base.OnInitializedAsync();D
D
        // A AC,,=DT@Cä=C0> C00C0>C'=D05CÄ @CT:D4@D 8C$=D'9 Aä B20:D0H CD0C0=D'<C, 8Cr 1D0:CT=CD0 C0@C, ?CT@C$
        await Context.LoadInitialDataAsync();D
    }D
D
    private void RefreshUi()D
    {D
        InvokeAsync(StateHasChanged);D
    }D
D
    /// <summary>D
    /// A0>C'=C O CäGC,,AD$:C @CTAD4@D >C" 4C'0 C0@CT4CäBC$@C ICT=C,,0 D4BCTGCT: C00CÄ0D$8 C" &Æ |÷-
    /// </summary>D
    public void Dispose()D
    {D
        if (_isDisposed) return;D
D
        if (Context != null)D
        {D
            Context.OnContextStateChanged -= RefreshUi;D
            Context.Dispose(); // A40D 0C0BC,,@Cä2C =C0> CäBC08D KC$0CT< C,,5D 0D EC,,GCTAC=8C' 1D >C=5D >D" F
        }D
D
        _isDisposed = true;D
    }D
}
</file>

```

```
<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor">
```

```
@typeparam TEntity where TEntity : class
```

```
{
    
```

```
<MudTreeView T="TEntity">
```

```
    Items="@TreeItems"
```

```
    SelectedValue="@SelectedNode"
```

```
    SelectedValueChanged="@OnSelectedNodeChanged"
```

```
    Hover="true"
```

```
    Dense="true"
```

```
    SelectionMode="SelectionMode.SingleSelection"
```

```
    Height="calc(100vh - 240px)"
```

```
    Class="flex-grow-1 w-100 h-100 overflow-y-auto">
```

```
<ItemTemplate>
```

```
    <MudTreeViewItem T="TEntity">
```

```
        Value="@context.Value"
```

```
        Icon="@GetNodeIcon(context.Value)"
```

```
        Text="@GetNodeText(context.Value)"
```

```
        Items="@GetChildTreeItemData(context.Value)" />
```

```
    </ItemTemplate>
```

```
</MudTreeView>
```

```
</file>
```

```
<file path="Promatis.Net.UI/Components/Workspaces/TreePage.razor.cs">
```

```
using Microsoft.AspNetCore.Components;
```

```
using MudBlazor;
```

```
{
```

```
    namespace Promatis.Net.UI.Components.Workspaces;
```

```
    
```

```
    public partial class TreePage<TEntity> : ComponentBase where TEntity : class
```

```
    {
```

```
        [CascadingParameter]
```

```
        protected IWorkspaceContext? ActionContext { get; set; }
```

```
        
```

```
        [Parameter] public IEnumerable<TEntity>? RootItems { get; set; }
```

```
        [Parameter] public Func<TEntity, IEnumerable<TEntity>>? ChildSelector { get; set; }
```

```
        [Parameter] public Func<TEntity, string>? TextSelector { get; set; }
```

```
        [Parameter] public Func<TEntity, string>? IconSelector { get; set; }
```

```
        
```

```
        protected List<TreeItemData<TEntity>> TreeItems { get; set; } = new();
```

```
        protected TEntity? SelectedNode { get; set; }
```

```
        
```

```
        protected override void OnInitialized()
```

```
        {
```

```
            base.OnInitialized();
```

```
            
```

```
            if (ActionContext is IHasSelectedData<TEntity> bindableContext)
```

```
            {
```

```
                SelectedNode = bindableContext.SelectedData;
```

```
            }
```

```
        }
```

```
        
```

```
        protected override void OnParametersSet()
```

```
        {
```

```
            base.OnParametersSet();
```

```
            
```

```
            if (RootItems != null)
```

```
            {
```

```
                TreeItems = RootItems
```

```
                    .Where(item => item != null)
```

```
                    .Select(item => new TreeItemData<TEntity> { Value = item })
```

```
                    .ToList();
```

```
            }
```

```
            else
```

```
            {
```

```
                TreeItems = new();
```

```
            }
```

```
        }
```

```
        
```

```
        protected void OnSelectedNodeChanged(TEntity? newSelection)
```

```
        {
```

```
            SelectedNode = newSelection;
```

```
            
```

```
            if (ActionContext is IHasSelectedData<TEntity> bindableContext)
```

```
            {
```

```

// A 5Ct>C00D =Câ ?D4HC,,< C$KCD5C'5C0=D'9 D47CT; CD5D 5C$0 C" :Că=D$5C=AD" OCD@C 4C =C0KD]
bindableContext.SelectedData = newSelection;D
D
// A,,!A0 A A' A0 : A$<CTAD$> C$Kct>C$0 OnContextUpdated (Ct0D 5Ct5D 2C,,@Că2C =C0>C4> C0>CB BD 8C
// CĂK CăBC0@C 2C'0CT< C00 DT>C'AD" GC,,AD$KC' 8CĂ?D4;DĂA C$8CtCC ;DĂ=Că3Că >C =Că2C'5C08D0 GCT@CT
// A=Că?C=8 D$CC'1C @C Š' Că1C 2C,,BDĂ ?Că4D47CT;», «A,,7CĂ5C08D$L», «B44C ;C,,BDĚ2' <C4=Că2CT=C0>
// D 2CăN CD>D BD4?C0>D BDĂ 1CT7 D 8D :C ?CT@CT7C 3D Cct:C, 8C'8 Ct0Dd8C=;C,,2C =C,,0 CD5D 5C$0!D
InvokeAsync(() =>D
{D
    ActionContext.NotifyStateChanged();D
});D
}D
D
protected string GetNodeText(TEntity? item) =>D
    item == null ? string.Empty : (TextSelector?.Invoke(item) ?? item.ToString()) ??
string.Empty);D
D
protected string GetNodeIcon(TEntity? item) =>D
    item == null ? Icons.Material.Filled.Folder : (IconSelector?.Invoke(item) ??
Icons.Material.Filled.Folder);D
D
protected List<TreeItemData<TEntity>> GetChildTreeItemData(TEntity? item)D
{D
    if (item == null || ChildSelector == null) return new();D
D
    IEnumerable<TEntity>? children = ChildSelector.Invoke(item);D
    if (children == null) return new();D
D
    return childrenD
        .Where(child => child != null)D
        .Select(child => new TreeItemData<TEntity> { Value = child })D
        .ToList();D
}D
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor">
<div class="d-flex align-center justify-space-between gap-4 pa-0 w-100" style="min-height: 48px;
width: 100%;">D
D
    @if (Controls != null)D
    {D
        @* A'5C$0D0 3D CC0?C ¢ DC,,;DĂBD K C$KD BD 0C,,2C ND$AD0 2 D OCB A DD8C=AC,,@Că2C =C0KCĂ 7C 7Că@Că< *@D
        <div class="d-flex align-center gap-4">D
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Left))D
            {D
                @if (!control.IsVisible) continue;D
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />D
            }D
        </div>D
D
        @* A0@C 2C 0 C4@D4?C00: D 0Ct4CT;C,,BCT;DĂ 8 C=Că?C=8 C$KC4@D47C=8 *@D
        <div class="d-flex align-center gap-4">D
            @foreach (IUiControl control in Controls.Where(c => c.Alignment ==
UiControlAlignment.Right))D
            {D
                @if (!control.IsVisible) continue;D
                <DynamicComponent Type="@control.ComponentType"
Parameters="@control.ComponentParameters" />D
            }D
        </div>D
    }D
</div>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/UiToolbar.razor.cs">
using System.Reflection;D
using Microsoft.AspNetCore.Components;D
D
namespace Promatis.Net.UI.Components.Workspaces;D
D
public partial class UiToolbar : ComponentBase, IDisposableD
{D

```

```

    /// <summary>
    /// B ?C,,ACä: Cö>C´8CÄ>D DCÔKDR :Cä=D$@Cä;Cä2 CD;Dö >D$@C,,ACä2C²8 CÔ0 Cö0CÔ5C´8.
    /// </summary>
    [Parameter]
    public IEnumerable<IUIControl>? Controls { get; set; }

    /// <summary>
    /// A²D :C 4CÔKC´ :Cä=D$5C²AD" ECä;D BC 4C´O CäBD ;CT6C,,2C =C,,O D <CT=D² 2D´4CT;CT=C,,O CD0CÔ=D´E.
    /// </summary>
    [CascadingParameter]
    protected IWorkspaceContext? ActionContext { get; set; }

    /// <summary>
    /// A,,7C$;CT:C 5D" BCT:D4IC,,9 C$KCD5C´5CÔ=D´9 Cä1D²5C²B CD;Dö ?CT@CT4C GC, 2 C 8Ct=CTA-C´>C48C²C C²>CÔBD
    /// </summary>
    protected object? CurrentSelectedData => GetSelectedDataFromContext();

    protected override void OnInitialized()
    {
        base.OnInitialized();

        SubscribeToSelectionChanged(true);

        if (Controls != null)
        {
            foreach (IUIControl control in Controls)
            {
                control.OnStateChanged += HandleStateChanged;
            }
        }
    }

    private void HandleStateChanged() => InvokeAsync(StateHasChanged);

    private object? GetSelectedDataFromContext()
    {
        if (ActionContext == null) return null;

        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
        if (interfaceType != null)
        {
            return interfaceType.GetProperty("SelectedData")?.GetValue(ActionContext);
        }
        return null;
    }

    private void SubscribeToSelectionChanged(bool subscribe)
    {
        if (ActionContext == null) return;

        Type? interfaceType = ActionContext.GetType().GetInterface("IHasSelectedData`1");
        if (interfaceType != null)
        {
            PropertyInfo? eventInfo = interfaceType.GetProperty("OnContextUpdated");
            if (eventInfo != null)
            {
                {
                    var currentDelegate = (Action?)eventInfo.GetValue(ActionContext);
                    if (subscribe)
                    {
                        currentDelegate += HandleStateChanged;
                    }
                    else
                    {
                        currentDelegate -= HandleStateChanged;
                    }

                    eventInfo.SetValue(ActionContext, currentDelegate);
                }
            }
        }
    }

    public void Dispose()
    {
        SubscribeToSelectionChanged(false);

        if (Controls != null)
        {
            foreach (IUIControl control in Controls)
            {
                control.OnStateChanged -= HandleStateChanged;
            }
        }
    }

```

```

    }
}
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor">
<MudPaper Elevation="@GetPaperElevation()"
Class="@GetPaperClass()"
Style="@($"width: 100%; height: {GetWorkspaceHeight()}; overflow: hidden;")">
    @* A,,!Aô A A' Aô : min-height Ct0CÄ5C05C00 C00 height *@
    <!-- 1. A$ B %Aô/Bô Aä A ,, A Aä Aô/AT"B / B$ A',A Aô A, A A,, 'A,, TOPCONTENT) -->
    @if (TopContent != null && !IsTopCollapsed())
    {
        <div class="workspace-zone-top w-100 mb-2"
            style="@($"height: {GetTopHeight()}; min-height: {GetTopHeight()}; max-height:
{GetTopHeight()}; flex-shrink: 0;")">
            @TopContent
        </div>
    }

    <!-- B AT Aô A' !A' A' ,, AT A / At Aô + A AD + Aô A A / At Aô ) -->
    <div class="d-flex flex-row align-stretch flex-grow-1 gap-4 w-100" style="min-height: 0;
height: 100%; overflow: hidden;">
    <!-- 2. A' A$ Bô Aä A ,, A A,, A &A,,/ / AD B A$ ) -->
    @if (LeftContent != null && !IsLeftCollapsed())
    {
        @* A,,!Aô A A' Aô : AD>C 0C$;CT= D :D >C'; overflow-y: auto CD;Dò 7C IC,,BD² >D" ?CT@CT?Cä;C05C08D
        <div class="workspace-zone-left d-flex flex-column align-stretch justify-space-between"
            style="@($"width: {GetLeftWidth()}; min-width: {GetLeftWidth()}; max-width:
{GetLeftWidth()}; overflow-y: auto;")">
            @LeftContent
        </div>
    }

    <!-- 3. Bd Aô"B A',Aô Bô Aä A ,, Aä Afç B AB ò Aô Aä B %AT A ' òÓí
    @* A,,!Aô A A' Aô : B41D 0C0 ,Ó A 4Cä1C 2C'5C0K C;C AD K d-flex flex-column CD;Dò :C AC=0CD=Cä9 C0
    <div class="workspace-zone-body flex-grow-1 h-100" style="min-width: 0; min-height: 0;">
        @if (BodyContent != null)
        {
            @BodyContent
        }
    </div>

    <!-- 4. Aô A A / At Aô (A,, B AT B$ B !A$ A"!B$ ) -->
    @if (RightContent != null && !IsRightCollapsed())
    {
        @* A,,!Aô A A' Aô : AD>C 0C$;CT= D :D >C'; overflow-y: auto CD;Dò 7C IC,,BD² >D" ?CT@CT?Cä;C05C08D
        <div class="workspace-zone-right d-flex flex-column align-stretch justify-space-between"
            style="@($"width: {GetRightWidth()}; min-width: {GetRightWidth()}; max-width:
{GetRightWidth()}; overflow-y: auto;")">
            @RightContent
        </div>
    }

    </div>

    <!-- 5. Aô Ad Bô/ At Aô (B "A "B4!-A B ' òÓí
    @if (BottomContent != null && !IsBottomCollapsed())
    {
        <div class="workspace-zone-bottom w-100 mt-2"
            style="@($"height: {GetBottomHeight()}; min-height: {GetBottomHeight()}; max-height:
{GetBottomHeight()}; flex-shrink: 0;")">
            @BottomContent
        </div>
    }

</MudPaper>
</file>

<file path="Promatis.Net.UI/Components/Workspaces/WorkspacePage.razor.cs">
using Microsoft.AspNetCore.Components;
namespace Promatis.Net.UI.Components.Workspaces;

```

```

Ð
public partial class WorkspacePage : ComponentBase, IDisposable
{
    Ð
    /// <summary>Ð
    /// AðD :C 4C0KC' 8C0BCT@DD5C"A Cð>C0BCT:D BC BCT:D4ICT9 D 0C >Dt5C' >C ;C AD$8.Ð
    /// </summary>Ð
    [CascadingParameter]Ð
    protected IWorkspaceContext? ActionContext { get; set; }Ð
    Ð
    [Parameter] public RenderFragment? BodyContent { get; set; }Ð
    [Parameter] public RenderFragment? TopContent { get; set; }Ð
    [Parameter] public RenderFragment? BottomContent { get; set; }Ð
    [Parameter] public RenderFragment? LeftContent { get; set; }Ð
    [Parameter] public RenderFragment? RightContent { get; set; }Ð
    Ð
    protected override void OnInitialized()Ð
    {
        Ð
        base.OnInitialized();Ð
    }
    Ð
    // Að>CD?C,,ACð0 C00 D 5C :D$8C$=Cä5 Cä1C0>C$;CT=C,,5 C00D 0CÄ5D$@Cä2 C45Cä<CTBD 8C•
    if (ActionContext != null)Ð
    {
        Ð
        ActionContext.OnContextStateChanged += HandleContextStateChanged;Ð
    }
    }Ð
    Ð
    private void HandleContextStateChanged()Ð
    {
        Ð
        InvokeAsync(StateHasChanged);Ð
    }
    }Ð
    Ð
    public void Dispose()Ð
    {
        Ð
        if (ActionContext != null)Ð
        {
            Ð
            ActionContext.OnContextStateChanged -= HandleContextStateChanged;Ð
        }
    }
    }Ð
    Ð
    // --- AÄ B$ AD+ B B 'AT"A A,, B4 A',A0 A4 B "A,, B0 Aä A' Ad A, 00Ý
    protected int GetPaperElevation() => ActionContext?.PaperElevation ?? 1;Ð
    protected string GetPaperClass() => ActionContext?.PaperClass ?? "pa-4 d-flex flex-column flex-
grow-1 w-100";Ð
    Ð
    // --- AÄ B$ AD+ B B 'AT"A AT AÄ B$ A,, Að B A !A %Aä B "A 00Ý
    protected string GetWorkspaceHeight() => ActionContext?.WorkspaceHeight ?? "100%";Ð
    protected string GetTopHeight() => ActionContext?.TopZoneHeight ?? "auto";Ð
    protected string GetBottomHeight() => ActionContext?.BottomZoneHeight ?? "auto";Ð
    protected string GetLeftWidth() => ActionContext?.LeftZoneWidth ?? "250px";Ð
    protected string GetRightWidth() => ActionContext?.RightZoneWidth ?? "300px";Ð
    Ð
    // ATAC'8 Cð>C0BCT=D" AR ?CT@CT4C =, Ct>C00 B AT A0#B$ C 2D$>CÄ0D$8Dt5D :C,â
    // ATAC'8 Cð>C0BCT=D" 5D BDÄÄ 5E ACäAD$>D0=C,,5 C 5D 5D$AD0 8Cr :Cä=D$5CðAD$0 (C0> D4<Cä;Dt0C08Dä f Ç6R 0
    protected bool IsTopCollapsed() => TopContent == null || (ActionContext?.IsTopZoneCollapsed ??
false);Ð
    protected bool IsBottomCollapsed() => BottomContent == null ||
(ActionContext?.IsBottomZoneCollapsed ?? false);Ð
    protected bool IsLeftCollapsed() => LeftContent == null ||
(ActionContext?.IsLeftZoneCollapsed ?? false);Ð
    protected bool IsRightCollapsed() => RightContent == null ||
(ActionContext?.IsRightZoneCollapsed ?? false);Ð
    }
}
</file>

<file path="Promatis.Net.UI/Controls/CreateChildButton.cs">
using MudBlazor;Ð
using Promatis.Net.UI.Components;Ð
using Promatis.Net.UI.Components.ElementRenderBase;Ð
Ð
namespace Promatis.Net.UI.Controls;Ð
Ð
/// <summary>Ð
/// B ?CTFC,,0C'8Ct8D >C$0C0=C 0 Cð=Cä?Cð0-Cð>CÄ0C04C 4C'0 D >Ct4C =C,,0 CD>Dt5D =CT3Cä „?Cä4Dt8C05C0=Cä3Cä' C
/// A00D$8C$=Cä CC0@C 2C'0CTB D 2Cä8CÄ ACäAD$>D0=C,,5CÄ 4CäAD$CC0=CäAD$8 C00 CäAC0>C$5 C00C'8Dt8D0 2D'1D 0C0=C
/// </summary>Ð
public class CreateChildButton<TEntity> : BaseUiControl where TEntity : classÐ

```

```

{
    private Func<Task>? _executeAsync;
    private readonly string _id = "toolbar_action_create_child_" + Guid.NewGuid().ToString("N");

    public override string Id => _id;
    public override Type ComponentType => typeof(ButtonRenderBase); // A,,ACô>C´LCtCCT< D BC =CD0D BCôKC´ @CT=
    public override string Title => "AD>C 0C$8D$L C0>CDCct5C²#½
    public override string Icon => Icons.Material.Filled.PlaylistAdd; // A,,:Cä=C00 CD>C 0C$;CT=C,,0 C" ACô8D >
    public override string Tooltip => "B >Ct4C BDÂ 4CäGCT@C08C´ MC´5CÄ5C0B C$=D4BD 8 C$KC @C =C0>C4> D47C´0";

    public CreateChildButton()
    {
        // A0> D4>Cä;Dt0C08Dä :C0>C0:C 7C 1C´>C08D >C$0C00, C0>C00 D04D > C05 C05D 5CD0D B C" =CT5 C$KC @C =
        IsEnabled = false;

        ComponentParameters.Add("Color", Color.Primary);
        ComponentParameters.Add("Variant", Variant.Outlined); // A$KCD5C´OCT< C$8CtCC ;DÄ=Cä >D" 3C´0C$=Cä9 C

    }

    /// <summary>
    /// B 5C :D$8C$=C 0 C0@Cä2CT@C00 CD>D BD4?C0>D BC, :Cä<C =CDK D 8C´0CÄ8 D04D 0.
    /// AÄ5D$>CB 2D´7D´2C 5D$AD0 :Cä=D$5C0AD$>CÄ VçF–G•v÷&·7 6T6öçFW‡B ?D 8 C00Cd4Cä< C,,7CÄ5C05C08C, 6VÆV7FV
    /// </summary>
    public override bool IsEnabledForData(object? targetData)
    {
        // A0=Cä?C00 D BC =Cä2C,,BD 0 CD>D BD4?C0>C´ "Aä BÄ Aä BCä3CD0, C0>C44C 2 D 8D BCT<CR 2D´1D 0C0 @Cä4C
        bool hasParentSelected = targetData != null;

        // B 8C0ED >C08Ct8D CCT< C$=D4BD 5C0=CT5 D >D BCä0C08CR DC´0C40 CD>D BD4?C0>D BC, 4C´0 D 5C04CT@CT@C
        IsEnabled = hasParentSelected;

        return hasParentSelected;
    }

    /// <summary>
    /// A0@C,,2D07C00 C AC,,=DT@Cä=C0>C4> Cä1D 0C >D$GC,,:C ,,4CT;CT3C BC ´ 2D´7Cä2C :Cä<C =CDK C,,7 C0>C0BCT:D
    /// </summary>
    public CreateChildButton<TEntity> OnExecute(Func<Task> executeAsync)
    {
        _executeAsync = executeAsync ?? throw new ArgumentNullException(nameof(executeAsync));
        return this;
    }

    /// <summary>
    /// B$>Dt:C 2DT>CD0 C0;C,,:C ?Cä :C0>C0:CR ×VD&Æ ¦÷"Ä 2D´7D´2C 5CÄ0D0 GCT@CT7 C$8CtCC ;DÄ=D´9 D 5C04CT@C
    /// </summary>
    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_executeAsync != null && IsEnabled)
        {
            await _executeAsync.Invoke();
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/CreateEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;

/// <summary>
/// B4=C,,2CT@D 0C´LCôKC´ 8C0DD 0D BD CC0BD4@C0KC´ MC´5CÄ5C0B C0=Cä?C08 "B >Ct4C BDÂ" 4C´0 D CD"=CäAD$5C´ ;Dä1
/// </summary>
public class CreateEntityButton<TEntity> : BaseUiControl where TEntity : class, new()
{
    private Func<Task>? _command; // A00D, 8D ?Cä;C08D$5C´L CD8C ;Cä3C

    public override string Id => $"crud_create_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B >Ct4C BDÂ#½
    public override string Icon => Icons.Material.Filled.Add;
    public override string Tooltip => $"B >Ct4C BDÂ =Cä2D4N Ct0C08D L";
}

```

```

    public CreateEntityButton() { IsEnabled = true; }
}
// AA5D$>CB 4C'0 C$=CT4D 5C08D0 @CT0C'LC0>C4> CD5C"AD$2C,,0 C,,7 C=C0BCT:D BC
public CreateEntityButton<TEntity> OnExecute(Func<Task> command)
{
    _command = command;
    return this;
}
public override bool IsEnabledForData(object? targetData) => true;
protected override async Task HandleTriggerAsync(object? targetData)
{
    if (_command != null) await _command.Invoke();
}
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/DeleteEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Cä?C=8 "B44C ;C,,BDÄ" 4C'0 C$KC @C =C0>C' AD4I
/// </summary>
public class DeleteEntityButton<TEntity> : BaseUiControl where TEntity : class
{
    private Func<TEntity, Task>? _command;

    public override string Id => $"crud_delete_{typeof(TEntity).Name.ToLower()}";
    public override Type ComponentType => typeof(ButtonRenderBase);
    public override string Title => "B44C ;C,,BDÄ#½
    public override string Icon => Icons.Material.Filled.Delete;
    public override string Tooltip => "B44C ;C,,BDÄ 2D'1D 0C0=D4N Ct0C08D L";

    public DeleteEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
    {
        _command = command;
        return this;
    }

    public override bool IsEnabledForData(object? targetData)
    {
        if (targetData is not TEntity typedEntity) return false;
        if (typedEntity is Domain.Interface.ISoftDeletable softDeletable)
        {
            return softDeletable.DeletedAt == null;
        }
        return true;
    }

    protected override async Task HandleTriggerAsync(object? targetData)
    {
        if (_command != null && targetData is TEntity typedEntity)
        {
            await _command.Invoke(typedEntity);
        }
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/EditEntityButton.cs">
using MudBlazor;
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Controls;
/// <summary>
/// B4=C,,2CT@D 0C'LC0KC' 8C0DD 0D BD CC=BD4@C0KC' MC'5CÄ5C0B C=Cä?C=8 "B 5CD0C=BC,,@Cä2C BDÄ" 4C'0 C$KC @C =C
/// </summary>
public class EditEntityButton<TEntity> : BaseUiControl where TEntity : class
{

```

```

private Func<TEntity, Task>? _command;
}
public override string Id => $"crud_edit_{typeof(TEntity).Name.ToLower()}";
public override Type ComponentType => typeof(ButtonRenderBase);
public override string Title => "B 5CD0C=BC,,@Cä2C BDÄ#½
public override string Icon => Icons.Material.Filled.Edit;
public override string Tooltip => "B 5CD0C=BC,,@Cä2C BDÄ 2D´1D 0CÔ=D4N Ct0C08D L";
}
public EditEntityButton<TEntity> OnExecute(Func<TEntity, Task> command)
{
    _command = command;
    return this;
}
}
public override bool IsEnabledForData(object? targetData) => targetData is TEntity;
}
protected override async Task HandleTriggerAsync(object? targetData)
{
    if (_command != null && targetData is TEntity typedEntity)
    {
        await _command.Invoke(typedEntity);
    }
}
}
}
</file>

```

```

<file path="Promatis.Net.UI/Controls/ToolbarDivider.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
}
namespace Promatis.Net.UI.Controls;
}
/// <summary>
/// B 8D BCT<CÖKC´ MC´5CÄ5CÔB C$5D BC,,:C ;DÄ=Cä3Cä @C 7CD5C´8D$5C´O, D 0Ct<CTIC 5CÄKC´ <CT6CDC DÔ;CT<CT=D$0CÄ
/// </summary>
public class ToolbarDivider : BaseUiControl
{
    private readonly string _id = "core_toolbar_divider_" + Guid.NewGuid().ToString("N");
}
public override string Id => _id;
}
/// <summary>
/// Aö@C,,2Dö7C=0 C¢ =C HCT<D2 7C DC,,:D 8D >C$0CÔ=Cä<D2 1C 7Cä2Cä<D2 @CT=CD5D 5D C DividerRenderBase.
/// </summary>
public override Type ComponentType => typeof(DividerRenderBase);
}
protected override Task HandleTriggerAsync(object? targetData)
{
    // AD5C= D 0D$8C$=D´9 DÔ;CT<CT=D" ?C =CT;C, 8CÔAD$@D4<CT=D$>C" =CR 2D´?Cä;CÔ0CTB C´>C48C=C Cö> D$@C,,3
    return Task.CompletedTask;
}
}
</file>

```

```

<file path="Promatis.Net.UI/Data/Cache/FlatOzuMutationStrategy.cs">
using Promatis.Net.Data;
}
namespace Promatis.Net.UI;
}
public class FlatOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where TEntity : class
{
    // B$5Cö5D L C= CÔBD 0C=B Cö@C,,=C,,<C 5D" 3C,,1C=CÄ "6öÆV7F-öâ 2CÄ5D BCâ 6CTAD$:Cä3Cä Æ-7M
    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;
}
switch (state)
{
    case EntityStateChangeEnum.Added:
        // AD>C 0C$;Dö5CÄÄ BCä;DÄ:Cä 5D ;C, MC´5CÄ5CÔBC A D$0C=8CÄ -B 5D"5 CÔ5D" 2 C= >C´;CT:Dd8C•
        if (!inMemoryItems.Any(x => Equals(idSelector(x), entityId)))
        {
            inMemoryItems.Add(entity);
        }
}
}
}

```

```

        break;
    }

    case EntityStateChangeEnum.Modified:
        // A,,!Aô A A' Aô : B$0C¢ :C : ICollection C05 C0>CD4CT@Cd8C$0CTB Că1D 0D"5C08CR ?Că 8C04CT:D
        // CĂK C,,AC0>C'LCtCCT< D4=C,,2CT@D 0C'LC0KC' 8 C 5Ct>C00D =D'9 CD;Dò ×VD&Æ |÷" ?Că4DT>CB `CCD0
        TEntity? existingItem = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));
        if (existingItem != null)
        {
            inMemoryItems.Remove(existingItem);
        }
        inMemoryItems.Add(entity);
        break;

    case EntityStateChangeEnum.Deleted:
    case EntityStateChangeEnum.SoftDeleted:
        TEntity? itemToRemove = inMemoryItems.FirstOrDefault(x => Equals(idSelector(x),
entityId));
        if (itemToRemove != null)
        {
            inMemoryItems.Remove(itemToRemove);
        }
        break;
    }
}
</file>

<file path="Promatis.Net.UI/Data/Cache/HierarchicalOzuMutationStrategy.cs">
using Promatis.Net.Data;
namespace Promatis.Net.UI;
public class HierarchicalOzuMutationStrategy<TEntity> : IOzuMutationStrategy<TEntity> where
TEntity : class
{
    private readonly Func<TEntity, object?> _parentIdSelector;

    // B 0C >D$0CT< D BD >C4> D 6C,,2Că9 ICollection C00C$8C40Dd8Că=C0>C4> D 2Că9D BC$0, C 5Cr :C'>C08D >C$00
    private readonly Func<TEntity, ICollection<TEntity>> _childrenSelector;

    // A KD BD KC' 8C04CT:D 4C'0 CĂ3C0>C$5C0=Că3Că ?Că8D :C 7C òf •
    private readonly Dictionary<object, TEntity> _nodeIndex = [];
    private readonly Dictionary<object, object> _childToParentMap = []; // A00D BC AC$0Ct5C"¢ -D#Ct;C óâ -D

    private bool _isIndexBuilt;

    public HierarchicalOzuMutationStrategy(
        Func<TEntity, object?> parentIdSelector,
        Func<TEntity, ICollection<TEntity>> childrenSelector)
    {
        _parentIdSelector = parentIdSelector ?? throw new
ArgumentNullException(nameof(parentIdSelector));
        _childrenSelector = childrenSelector ?? throw new
ArgumentNullException(nameof(childrenSelector));
    }

    public void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity
entity, Func<TEntity, object?> idSelector)
    {
        object? entityId = idSelector(entity);
        if (entityId == null) return;

        // A'5C08C$0Dò AC >D :C ?C'>D :Că3Că 8C04CT:D 0 C0@C, ?CT@C$>C' <D4BC FC,,8 CD0C0=D'E DD>D <D½
        if (!_isIndexBuilt)
        {
            BuildIndexRecursive(inMemoryItems, idSelector);
            _isIndexBuilt = true;
        }

        switch (state)
        {
            case EntityStateChangeEnum.Added:
                HandleAdded(inMemoryItems, entity, entityId, idSelector);
                break;
        }
    }
}

```

```

        case EntityStateChangeEnum.Modified:
            HandleModified(inMemoryItems, entity, entityId, idSelector);
            break;

        case EntityStateChangeEnum.Deleted:
        case EntityStateChangeEnum.SoftDeleted:
            HandleDeleted(inMemoryItems, entityId, idSelector);
            break;
    }

    private void HandleAdded(ICollection<TEntity> rootItems, TEntity entity, object entityId,
        Func<TEntity, object?> idSelector)
    {
        if (!_nodeIndex.ContainsKey(entityId)) return; // At0D"8D$0 CăB CDCC ;C,,@Că2C =C,,0 C" ?C <DôBC•

        object? newParentId = _parentIdSelector(entity);

        if (newParentId == null)
        {
            // Aă1D=5C= B C= >D =CT2Că9 – CD>C 0C$;Dô5CÂ 2 C= >D 5CÔL Aă B=
            rootItems.Add(entity);
        }
        else if (_nodeIndex.TryGetValue(newParentId, out var parentNode))
        {
            // Aă1D=5C= B CD>Dt5D =C,,9 – CD>C 0C$;Dô5CÂ 2 Cd8C$CDă :Că;C´5C=FC,,N CT3Că @Că4C,,BCT;Dý
            var children = _childrenSelector(parentNode);
            children?.Add(entity);
            _childToParentMap[entityId] = newParentId;
        }

        // A=0D :C 4CÔ> D 5C48D BD 8D CCT< D47CT; C, 5C4> Cô>CD4CT@CT2Că 2 C KD BD >CÂ 8CÔ4CT:D 5 0(1)
        UpdateIndexForSubtree(entity, idSelector);
    }

    private void HandleModified(ICollection<TEntity> rootItems, TEntity entity, object entityId,
        Func<TEntity, object?> idSelector)
    {
        // A,,!Aô A A´ Aô : Aô8C=0C= >C4> CĂ0Cô?C,,=C40. ATAC´8 D47CT; D46CR AD4ICTAD$2Că2C ;, DT8D CD 3C,,GCTAC
        if (_nodeIndex.TryGetValue(entityId, out var oldNodeReference))
        {
            object? oldParentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;

            if (oldParentId == null)
            {
                rootItems.Remove(oldNodeReference);
            }
            else if (_nodeIndex.TryGetValue(oldParentId, out var oldParentNode))
            {
                _childrenSelector(oldParentNode)?.Remove(oldNodeReference);
            }

            // A$KDt8D"0CT< D BC @D4N D AD´;C=C C, 5E AC$0Ct8 C,,7 C,,=CD5C=AC ?CT@CT4 CD>C 0C$;CT=C,,5CÂ =Că2
            RemoveFromIndexRecursive(oldNodeReference, idSelector);
        }

        // A$AD$0C$;Dô5CÂ =Că2D4N D AD´;C=C Cô> C :D$CC ;DĂ=Că<D2 &VçD–B ,,8CD5C ;DĂ=Că >C @C 1C BD´2C 5D" 8
        HandleAdded(rootItems, entity, entityId, idSelector);
    }

    private void HandleDeleted(ICollection<TEntity> rootItems, object entityId, Func<TEntity,
        object?> idSelector)
    {
        if (!_nodeIndex.TryGetValue(entityId, out var nodeToRemove)) return;

        object? parentId = _childToParentMap.TryGetValue(entityId, out var pId) ? pId : null;

        if (parentId == null)
        {
            rootItems.Remove(nodeToRemove);
        }
        else if (_nodeIndex.TryGetValue(parentId, out var parentNode))
        {
            _childrenSelector(parentNode)?.Remove(nodeToRemove);
        }
    }

```

```

// A0D :C 4C0> C$KDt8D"0CT< D44C ;CT=C0KC' ?Cä4C4@C D C,,7 C,,=CD5CpAC
RemoveFromIndexRecursive(nodeToRemove, idSelector);D
}D
D // --- B ;D46CT1C0KCR <CTBCä4D² CC0@C 2C'5C08D0 1D'AD$@D'< C,,=CD5CpACä< 0(1) ---D
D private void BuildIndexRecursive(ICollection<TEntity> nodes, Func<TEntity, object?> idSelector,
object? parentId = null)D
{D
    foreach (var node in nodes)D
    {D
        object? id = idSelector(node);D
        if (id == null) continue;D
D
        _nodeIndex[id] = node;D
        if (parentId != null) _childToParentMap[id] = parentId;D
D
        var children = _childrenSelector(node);D
        if (children != null && children.Count > 0)D
        {D
            BuildIndexRecursive(children, idSelector, id);D
        }D
    }D
}D
D private void UpdateIndexForSubtree(TEntity node, Func<TEntity, object?> idSelector)D
{D
    object? id = idSelector(node);D
    if (id == null) return;D
D
    _nodeIndex[id] = node;D
D
    object? parentId = _parentIdSelector(node);D
    if (parentId != null) _childToParentMap[id] = parentId;D
D
    var children = _childrenSelector(node);D
    if (children != null)D
    {D
        foreach (var child in children)D
        {D
            UpdateIndexForSubtree(child, idSelector);D
        }D
    }D
}D
D private void RemoveFromIndexRecursive(TEntity node, Func<TEntity, object?> idSelector)D
{D
    object? id = idSelector(node);D
    if (id == null) return;D
D
    _nodeIndex.Remove(id);D
    _childToParentMap.Remove(id);D
D
    var children = _childrenSelector(node);D
    if (children != null)D
    {D
        foreach (var child in children)D
        {D
            RemoveFromIndexRecursive(child, idSelector);D
        }D
    }D
}D
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IOzuMutationStrategy.cs">
using Promatis.Net.Data;D
D
namespace Promatis.Net.UI;D
D
public interface IOzuMutationStrategy<TEntity> where TEntity : classD
{D
    /// <summary>D
    /// A0@C,,<CT=D05D" 4CT;DÄBD2 8Ct<CT=CT=C,,9 B #A C0 8Ct<CT=D05CÄ>C' :Cä;C'5CpFC,,8 C4@C DC >C JCT:D$>C"
    /// </summary>D
    void ApplyDelta(ICollection<TEntity> inMemoryItems, EntityStateChangeEnum state, TEntity

```

```

entity, Func<TEntity, object?> idSelector);
}
</file>

<file path="Promatis.Net.UI/Data/Cache/IUiOzuCache.cs">
using Promatis.Net.Data;

namespace Promatis.Net.UI;

/// <summary>
/// A>CÔBD 0C=B C,,7Cä;C,,@Cä2C =CÔ>C4> C´>C=0C´LCÔ>C4> Aä B2ÔED 0CÔ8C´8D"0 CD0CÔ=D´E C= >CÔ:D 5D$=Cä9 DÔ:D 0CÔ
/// A,,ACô>C´LCtCCTB D 8D BCT<CÔKC´ VçF–G•7F FT6† ævTVçVò 4C´O Cô@C,,<CT=CT=C,,O D$>Dt5Dt=D´E CD5C´LD" 8Ct<CT=CT
/// </summary>
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BC,ãÄ÷G– W & óí
public interface IUiOzuCache<TEntity> where TEntity : class
{
    /// <summary>
    /// Aô@Dô<Cä9 CD>D BD4? Cç BCT:D4ICT<D2 <C AD 8C$C CD0CÔ=D´E D$>C´LC= CD;Dò GD$5CÔ8Dòâ
    /// AÔ8 Cä4CÔ0 DD>D <C 1Cä;DÄHCR =CR <Cä6CTB CÔ5D 5Ct0CÔ8D 0D$L C,,;C, >Dt8D BC,,BDÄ MD$>D" ACô8D >Cç 2D C
    /// </summary>
    IReadOnlyList<TEntity> InMemoryItems { get; }
}

    /// <summary>
    /// B$>Dt:C ?CT@C$>CÔ0Dt0C´LCÔ>C4> CÔ0Cô>C´=CT=C,,O C=MD,,0 CD0CÔ=D´<C,Â ?Cä;D4GCT=CÔKCÄ8 CäB DD>D <D²í
    /// A$KctKC$0CTBD O D BD >C4> Cä4C,,= D 0Cr ?D 8 C,,=C,,FC,,0C´8Ct0Dd8C,i
    /// </summary>
    void Initialize(List<TEntity> initialItems);

}

    /// <summary>
    /// A 2D$>CÔ>CÄ=D´9 Aä B2Ô4C$8Cd>Cç <D4BC FC,,9. BT8D CD 3C,,GCTAC=8 D$>Dt=Cä >C =Cä2C´OCTB C= >C´;CT:Dd8Dä
    /// </summary>
    void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity);

}

    void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy);

}

    /// <summary>
    /// A$Kcô>C´=Dô5D" ?D >C,,7C$>C´LCÔCDä >Cô5D 0Dd8Dä GD$5CÔ8Dò0DC,,;DÄBD 0Dd8C, =C 4 C=MD,,5CÄ 2CÔCD$@C, ?CäB
    /// Aô5D 5DT2C BD´2C 5D" ?CäBCä: UI C, 7C IC,,IC 5D" 5C4> CäB Cò0D 0C´;CT;DÄ=D´E CÄCD$0Dd8C´ 8Cr DCä=Cä2D´
    /// </summary>
    TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator);
}
</file>

<file path="Promatis.Net.UI/Data/Cache/UiOzuCache.cs">
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;

namespace Promatis.Net.UI;

/// <summary>
/// B 5C ;C,,7C FC,,O C,,7Cä;C,,@Cä2C =CÔ>C4> C=MD,,0 C" >Cô5D 0D$8C$=Cä9 Cô0CÄ0D$8 CD;Dò :Cä=C=0CTBCÔ>C4> C,,=D BC
/// </summary>
public class UiOzuCache<TEntity> : IUiOzuCache<TEntity> where TEntity : class
{
    private IOzuMutationStrategy<TEntity> _mutationStrategy = new
FlatOzuMutationStrategy<TEntity>();
    private readonly List<TEntity> _inMemoryItems = [];

    // AT4C,,=D´9 Cä1D=5C=B D 8CÔED >CÔ8Ct0Dd8C, 4C´O Cô>D$>C= >C" GD$5CÔ8Dò 8 Ct0CÔ8D 8D
    private readonly Lock _lockObject = new();

    public IReadOnlyList<TEntity> InMemoryItems => _inMemoryItems;

    public UiOzuCache() { }

    public UiOzuCache(List<TEntity> initialItems)
    {
        Initialize(initialItems);
    }

    public void Initialize(List<TEntity> initialItems)
    {
        if (initialItems == null) throw new ArgumentNullException(nameof(initialItems));

        lock (_lockObject)
        {

```

```

        _inMemoryItems.Clear();
        _inMemoryItems.AddRange(initialItems);
    }
}

public void SetMutationStrategy(IOzuMutationStrategy<TEntity> strategy)
{
    _mutationStrategy = strategy ?? throw new ArgumentNullException(nameof(strategy));
}

public void ApplyOzuDelta(EntityStateChangeEnum state, TEntity entity)
{
    // A0>D$>C  !B4 AB <Cä=Cä?Cä;DÄ=Cä 7C EC$0D$KC$0CTB Ct0CÄ>C =C 2D 5CÄO CÄCD$0Dd8C•
    lock (_lockObject)
    {
        _mutationStrategy.ApplyDelta(_inMemoryItems, state, entity, GetEntityId);
    }
}

public TResult ExecuteInLock<TResult>(Func<IReadOnlyList<TEntity>, TResult> evaluator)
{
    if (evaluator == null) throw new ArgumentNullException(nameof(evaluator));

    // A0>D$>C T' „&Æ |÷'' <Cä=Cä?Cä;DÄ=Cä 7C EC$0D$KC$0CTB Ct0CÄ>C =C 2D 5CÄO DD8C´LD$@C FC„8/C$KDt8D
    lock (_lockObject)
    {
        return evaluator(_inMemoryItems);
    }
}

private object? GetEntityId(TEntity item)
{
    Type? hasKeyInterface = item.GetType()
        .GetInterfaces()
        .FirstOrDefault(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IDomainObjectHasKey<>));

    if (hasKeyInterface != null)
    {
        return hasKeyInterface.GetProperty("Id")?.GetValue(item);
    }

    return item.GetType().GetProperty("Id")?.GetValue(item);
}
}
</file>

<file path="Promatis.Net.UI/Data/IUiDataBroker.cs">
using Promatis.Net.Data;
{
    namespace Promatis.Net.UI;
    {
        /// <summary>
        /// A0>C0BD 0C0B C @Cä:CT@C 4C =C0KDRÄ CC0@C 2C´ODäICT3Cä 2D´1Cä@Cä< C„AD$>Dt=C„:C ...6W'fW"öõ¥R´ 8 D BD 0D$5
        /// A05 D >CD5D 6C„B CD5DD>C´BC0KDR @CT6C„<Cä2 – C0@Cä3D 0CÄ<C„AD" >C 0Ct0C0 OC$=Cä AC0>C0DC„3D4@C„@Cä2C BDÄ
        /// </summary>
        public interface IUiDataBroker<TEntity, TQueryState, TResultData> : IDisposable where TEntity :
        class
        {
            /// <summary>
            /// A0@C„7C00C0Ä CC00CtKC$0DäIC„9, DtBCä BCT:D4IC„9 C„=D BC =D 1D >C05D 0 C00 DD>D <CR @C 1CäBC 5D" 2 D
            /// </summary>
            bool IsInMemoryMode { get; }

            /// <summary>
            /// Ad5D BC00D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² =C ?D OCÄCDä A B #A / gRPC API (Server Mode).
            /// </summary>
            void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider);

            /// <summary>
            /// Ad5D BC00D0 :Cä=DD8C4CD 0Dd8D0 @C 1CäBD² GCT@CT7 C„7Cä;C„@Cä2C =C0>CR ;Cä:C ;DÄ=Cä5 Aä B20ED 0C08C´8D
            /// </summary>
            void ConfigureInMemoryMode(
                IUiOzuCache<TEntity> ozuCache,
                Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader, // A05D 5CD0CT< Ct=C =C„5

```

```

        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator // B 0C >D$0CTB D •&V DöæÇ"
    );ð
ð
    /// <summary>ð
    /// B4=C,,2CT@D 0C´LCÔ0Dò BCäGC¤0 C AC,,=DT@Cä=CÔ>C4> Ct0C0@CäAC 4C =CÔKDR 2C,,7D40C´8Ct0D$>D 0CÄ8 C,,=D$5D
    /// ATAC´8 CÔ8 Cä4C,,= D 5Cd8CÄ =CR 1D´; D :Cä=DD8C4CD 8D >C$0C0 ?D >C4@C <CÄ8D BCä<, C$Kct>C$5D" -çf Æ-D-
    /// </summary>ð
    Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken cancellationToken =
default);ð
ð
    /// <summary>ð
    /// B$>Dt:C 2DT>CD0 CD;Dò ?D >C @CäAC 3C´>C 0C´LCÔ>C4> D 8C4=C ;C öäVçF-G"6öÖÖ-GFVB 2 C´>C¤0C´LCÔKC´ :
    /// </summary>ð
    void HandleDatabaseCommit(EntityStateChangeEnum state, object entity);ð
}
</file>

<file path="Promatis.Net.UI/Data/UiDataBroker.cs">
using Promatis.Net.Data;ð
ð
namespace Promatis.Net.UI;ð
ð
/// <summary>ð
/// Aô;C BDD>D <CT=CÔKC´ 1D >C¤5D 4C =CÔKDRâ &CT=D$@C ;C,,7Cä2C =CÔ> D4?D 0C$;Dô5D" ?CäAD$0C$:Cä9 CD0CÔ=D´E C
/// Aô>C´=CäAD$LDâ 0C AD$@C 3C,,@D45D" T´ >D" :Cä=C¤@CTBCÔKDR 8CÔBCT@DD5C"ACä2 CD>CÄ5CÔ=D´E D ;D46C 1DÔ:CT=CD
/// </summary>ð
/// <typeparam name="TEntity">B$8Cò 4Cä<CT=CÔ>C´ AD4ICÔ>D BCfÂ÷G- W & Óí
public class UiDataBroker<TEntity, TQueryState, TResultData> : IUiDataBroker<TEntity, TQueryState,
TResultData>ð
    where TEntity : classð
{ð
    private Func<TQueryState, CancellationToken, Task<TResultData>>? _serverDataProvider;ð
    private Func<TQueryState, CancellationToken, Task<List<TEntity>>>? _serverDataLoader;ð
    private Func<TQueryState, IReadOnlyList<TEntity>, TResultData>? _inMemoryEvaluator;ð
    private IUiOzuCache<TEntity>? _ozuCache;ð
    private readonly Action? _onDataChangedNotifier;ð
ð
    private bool _isInitialized;ð
    private bool _isInMemoryMode;ð
    private bool _isOzuLoaded;ð
ð
    // A,,=DD@C AD$@D4:D$CD 0 CD;Dò 4CT1C CCÔAC ACTBCT2D´E D42CT4Cä<C´5CÔ8C´ 2 ServerModeð
    private CancellationTokenSource? _debounceCts;ð
    private readonly object _debounceLock = new(); // At0CÄ>C¢ 4C´O Cò>D$>C¤>C 5Ct>Cô0D =Cä3Cä AC @CäAC BC 9
    private const int DebounceDelayMs = 300; // A$@CT<CT=CÔ>CR >C¤=Cä >Cd8CD0CÔ8Dò 7C BC,,HDÄO C" AB f3 <D
ð
    public bool IsInMemoryMode => _isInMemoryMode;ð
ð
    public UiDataBroker(Action? onDataChangedNotifier = null)ð
    {ð
        _onDataChangedNotifier = onDataChangedNotifier;ð
        DatabaseTriggerService.OnEntityCommitted += HandleGlobalEntityCommitted;ð
    }ð
ð
    private void HandleGlobalEntityCommitted(EntityStateChangeEnum state, object entity)ð
    {ð
        HandleDatabaseCommit(state, entity);ð
    }ð
ð
    public void ConfigureServerMode(Func<TQueryState, CancellationToken, Task<TResultData>>
serverDataProvider)ð
    {ð
        _serverDataProvider = serverDataProvider ?? throw new
ArgumentNullException(nameof(serverDataProvider));ð
        _serverDataLoader = null;ð
        _inMemoryEvaluator = null;ð
        _ozuCache = null;ð
        _isInMemoryMode = false;ð
        _isInitialized = true;ð
        _isOzuLoaded = false;ð
    }ð
ð
    public void ConfigureInMemoryMode(ð
        IUiOzuCache<TEntity> ozuCache,ð
        Func<TQueryState, CancellationToken, Task<List<TEntity>>> serverDataLoader,ð
        Func<TQueryState, IReadOnlyList<TEntity>, TResultData> inMemoryEvaluator)ð

```

```

{
    _ozuCache = ozuCache ?? throw new ArgumentNullException(nameof(ozuCache));
    _serverDataLoader = serverDataLoader ?? throw new
ArgumentNullException(nameof(serverDataLoader));
    _inMemoryEvaluator = inMemoryEvaluator ?? throw new
ArgumentNullException(nameof(inMemoryEvaluator));
    _serverDataProvider = null;
    _isInMemoryMode = true;
    _isInitialized = true;
    _isOzuLoaded = false;
}

public async Task<TResultData> FetchDataAsync(TQueryState state, CancellationToken
cancellationToken = default)
{
    if (!_isInitialized)
    {
        throw new InvalidOperationException($"A @Cä:CT@ CD0C0=D'E CD;Dò w·G- Vöb...DVçF-G''äæ ÖWÖr =CR 1D'");
    }

    if (_serverDataProvider != null)
    {
        return await _serverDataProvider(state, cancellationToken);
    }

    if (_isInMemoryMode && _ozuCache != null && _inMemoryEvaluator != null &&
_serverDataLoader != null)
    {
        if (!_isOzuLoaded)
        {
            List<TEntity> initialData = await _serverDataLoader(state, cancellationToken);
            _ozuCache.Initialize(initialData);
            _isOzuLoaded = true;
        }

        return _ozuCache.ExecuteInLock(items => _inMemoryEvaluator(state, items));
    }

    throw new InvalidOperationException("A@C,,BC,,GCTAC=8C' AC >C' 2C0CD$@CT=C05C' :Cä=DD8C4CD 0Dd8C, 1D >");
}

public void HandleDatabaseCommit(EntityStateChangeEnum state, object entity)
{
    // A0@Cä2CT@D05CÄ¢ >D$=CäAC,,BD 0 C'8 C0@C,,CTBCT2D,,8C' 8Cr !B4 AB >C JCT:D" : D$8C0C CD0C0=D'E D$5C0C
    if (entity is not TEntity typedEntity) return;

    if (_isInMemoryMode)
    {
        // B Ad AÄ "ä00T00%"¢ D4BC,,@D45CÄ At# CÄ3C0>C$5C0=Cä 1CT7 Ct0CD5D 6CT: CD;Dò ACäED 0C05C08Dò F
        if (_ozuCache != null && _isOzuLoaded)
        {
            _ozuCache.ApplyOzuDelta(state, typedEntity);
        }

        // B @C 7D2 ?C,,=C 5CÄ T'Â BC : C=0C¢ ACTBCT2Cä3Cä 7C ?D >D 0 C05 C CCD5D" r 2D'GC,,AC'5C08Dò ?D >C
        _onDataChangedNotifier?.Invoke();
    }
    else
    {
        // B Ad AÄ 4U%du"000DS¢ C¢;DäGC 5CÄ 4CT1C CC0A CD;Dò ?D 5CD>D$2D 0D"5C08Dò ACTBCT2Cä3Cä HD$>D <
        lock (_debounceLock)
        {
            // ATAC'8 C0@CT4D'4D4IC,,9 D$0C"<CT@ CTICR BC,,:C ; (C00Dt:C 8Ct<CT=CT=C,,9 C0@Cä4Cä;Cd0CTBD 0)
            _debounceCts?.Cancel();
            _debounceCts?.Dispose();

            // B >Ct4C 5CÄ =Cä2D'9 D$>C=5C0 7C 4CT@Cd:C•
            _debounceCts = new CancellationTokenSource();
            var token = _debounceCts.Token;

            // At0C0CD :C 5CÄ DCä=Cä2Cä5 Cä6C,,4C =C,,5 Ct0D$8D,,LDò 2 A 0
            Task.Delay(DebounceDelayMs, token).ContinueWith(task =>
            {
                // A$KctKC$0CT< C08C0>C¢ T' BCä;DÄ:Cä 5D ;C, 7C 4C GC CD ?CTHC0> Ct0C$5D HC,,;C ADÄ 8 C05
                if (task.Status == TaskStatus.RanToCompletion && !token.IsCancellationRequested)
                {

```

```

        _onDataChangedNotifier?.Invoke();ð
    }ð
}, TaskScheduler.Default);ð
}ð
}ð
}ð
public void Dispose()ð
{ð
    // AäBCô8D KC$0CT<D O CăB D BC BC,,GCTAC▯>C4> DÔ2CT=D$0 D ;D46C K D$@C,,3C45D >C" !B4 AM
    DatabaseTriggerService.OnEntityCommitted -= HandleGlobalEntityCommitted;ð

    // AtøDt8D"0CT< C,,=DD@C AD$@D4:D$CD C CD5C ØD4=D Ø Cô@C, CCô8DtBCă6CT=C,,8 C @Că:CT@C DCă@CĂ>C•
    lock (_debounceLock)ð
    {ð
        _debounceCts?.Cancel();ð
        _debounceCts?.Dispose();ð
        _debounceCts = null;ð
    }ð
}ð
}
</file>

<file path="Promatis.Net.UI/Extensions/MudColorExtensions.cs">
using MudBlazor;ð
ð
namespace Promatis.Net.UI;ð
ð
public static class MudColorExtensionsð
{ð
    public static Color GetActionColor(this string? action)ð
    {ð
        if (string.IsNullOrEmpty(action)) return Color.Default;ð

        return action.ToLower().Trim() switchð
        {ð
            "create" or "insert" or "added" or "CD>C 0C$;CT=C,,5" or "D >Ct4C =C,,5" => Color.Success,ð
            "update" or "edit" or "modified" or "C,,7CA5C05C08CR" ÷" $@CT4C :D$8D >C$0C08CR" Ôâ 6öÆ÷"äv &æ-ærI
            "delete" or "remove" or "deleted" or "softdeleted" or "D44C ;CT=C,,5" => Color.Error,ð
            _ => Color.Defaultð
        };ð
    }ð
}
</file>

<file path="Promatis.Net.UI/Interfaces/IHasSelectedData.cs">
namespace Promatis.Net.UI.Components;ð
ð
/// <summary>ð
/// B4=C,,2CT@D 0C'LC0KC' 8C0BCT@DD5C"A-CÄ0D :CT@ CD;Dò 2C,,7D40C'8Ct0D$>D >C"Â :CăBCă@D'< C05Că1DT>CD8CĂ> ð
/// D 8C0ED >C08Ct8D >C$0D$L DD>C▯CD AD$@Că:C,0Cct;C 8 C0>C'CDt0D$L C,,<C0CC'LD K C05D 5D 8D >C$:C,i
/// </summary>ð
public interface IHasSelectedData<TEntity> where TEntity : classð
{ð
    TEntity? SelectedData { get; set; }ð
    Action? OnContextUpdated { get; set; }ð
}
</file>

<file path="Promatis.Net.UI/IUiModule.cs">
namespace Promatis.Net.UI;ð
ð
/// <summary>ð
/// A▯>C0BD 0C▯B CD;Dò 4C,,=C <C,,GCTAC▯8DR <Că4D4;CT9 C0>C'Lct>C$0D$5C'LD :Că3Că 8C0BCT@DD5C"AC í
/// </summary>ð
public interface IUiModuleð
{ð
    /// <summary>ð
    /// AäBCă1D 0Cd0CT<Că5 C,,<Dò <Că4D4;Dò 2 D 8D BCT<CR ,,8D ?Că;DĂ7D45D$ADò 4C'0 D :C =CT@C 'í
    /// </summary>ð
    string Name { get; }ð

    /// <summary>ð
    /// A$>Ct2D 0D"0CTB D ?C,,ACă: DÔ;CT<CT=D$>C" =C 2C,,3C FC,,8 CD;Dò 1Că:Că2Că3Că <CT=Dâ xVDG& vw"í
    /// Bt5D$2CT@D$KC' ?C @C <CTBD >D$2CTGC 5D" 7C "0CD >C$=CT2D4N C4@D4?C08D >C$:D2 MC'5Că5C0BCă2.D
    /// </summary>ð

```

```

    /// <returns>A>C';CT:Dd8Dò :Cä@D$5Cd5C"¢ ¨ C 7C$0C08CRÂ !D KC':C Â C¤>C0:C Â C 7C$0C08CT D CCô?D²"Â÷&W
    IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems();ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor">
@page "/audit-logs"ð
@using Promatis.Net.Domainð
ð
<CascadingValue Value="Context">ð
    <WorkspacePage>ð
ð
        <!-- 1. A$ B %A0/Bò Aä A ¢ C$BCä<C BC¨GCTAC¤8C' ?Cä;C¨<Cä@DD=D'9 D$CC'1C @ (AD8C00CÄ8Dt5D :C¨5 C¤>C
        <TopContent>ð
            <UiToolbar Controls="@Context.Controls" />ð
        </TopContent>ð
ð
        <!-- 2. Bd A0"B A',A0 Bò Aä A ¢ "C 1C'8Dt=C O D 5D$:C Â @C 1CäBC ND"0Dò 2 DD>C0>C$>CÂ ACT@C$5D =Cä<
        <BodyContent>ð
            <GridPage TEntity="AuditLog"ð
                @ref="_grid"ð
                ServerDataProvider="@Context.Broker.FetchDataAsync"ð
                WithPagination="true"ð
                RowsPerPage="20">ð
                <GridColumns>ð
                    <PropertyColumn T="AuditLog" TProperty="DateTime" Property="x => x.ChangedAt"
Title="AD0D$0 C, 2D 5CÄ0" Format="dd.MM.yyyy HH:mm:ss" HeaderStyle="width: 200px;" />ð
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangedBy"
Title="A0>C'LCt>C$0D$5C'L" HeaderStyle="width: 150px;" />ð
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.EntityName"
Title="B$8Cò AD4IC0>D BC," †V FW%7G-ÆS0'v-GFf¢ # f²" óí
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.Action"
Title="Aä?CT@C FC¨0" HeaderStyle="width: 120px;" />ð
                    <PropertyColumn T="AuditLog" TProperty="Guid" Property="x => x.EntityId"
Title="ID B CD"=CäAD$8" HeaderStyle="width: 330px;" />ð
                    <PropertyColumn T="AuditLog" TProperty="string" Property="x => x.ChangesJson"
Title="A,,7CÄ5C05C08Dò ¨¥4ôâ'" †V FW%7G-ÆS0'v-GFf¢ WFó²" 6Æ 730'FW‡B×G'Væ6 FR" óí
                </GridColumns>ð
            </GridPage>ð
        </BodyContent>ð
ð
    </WorkspacePage>ð
</CascadingValue>
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogPage.razor.cs">
using Microsoft.AspNetCore.Components;ð
using Promatis.Net.Domain;ð
using Promatis.Net.Service;ð
using Promatis.Net.UI.Components.Workspaces;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs;ð
ð
public partial class AuditLogPage : ComponentBaseð
{ð
    private GridPage<AuditLog>? _grid;ð
    private bool _isLoading;ð
ð
    [Inject]ð
    protected IServiceProvider PageServiceProvider { get; set; } = null!;ð
ð
    [Inject]ð
    protected IAuditLogService AuditLogService { get; set; } = null!;ð
ð
    protected AuditLogWorkspaceContext Context { get; set; } = null!;ð
ð
    protected override void OnInitialized()ð
    {ð
        base.OnInitialized();ð
ð
        // AÄ A0 A$ A0 Aä ACä7CD0CT< CòCD BCä9 C¤>C0BCT:D B. B BD 0C08Dd0 D 5C04CT@C¨BD O C 5Cr 7C 4CT@Cd5C¢
        Context = new AuditLogWorkspaceContext(PageServiceProvider, onFilterChanged: RefreshGrid);ð
    }ð
ð
    /// <summary>ð
    /// A¤>C0>C08Dt5D :C O D$>Dt:C 7C ?D4AC¤0 D$0Cd5C'KDR 7C ?D >D >C"â !D 0C 0D$KC$0CTB A0 B AR BCä3CâÂ

```

```

/// C#0C# ?Cä;DÄ7Cä2C BCT;DÄ CCd5 D42C,,4CT; C4>D$>C$CDä AD$@C =C,,FD2 =C MC#@C =CRi
/// </summary>#
protected override async Task OnAfterRenderAsync(bool firstRender)#
{#
    await base.OnAfterRenderAsync(firstRender);#
#
    // A$KC0>C´=D05CÄ 7C 3D CCT:D2 BCä;DÄ:Cä >CD8C0 @C 7 C0@C, ?CT@C$>CÄ @CT=CD5D 5 D0:D 0C00#
    if (firstRender)#
    {#
        // B ?Cä:Cä9C0> D :C GC,,2C 5CÄ CC08C#0C´LC0KCR 8CÄ5C00 D CD´=CäAD$5C´ 8Cr !B4 AB ÷7Fw&U5 Í
        List<string> availableEntities = await AuditLogService.GetAvailableEntityNamesAsync();#
#
        // A05D 5CD0CT< CD0C0=D´5 C" :Cä=D$5C#AD" 4C´O CD8C00CÄ8Dt5D :Cä9 C45C05D 0Dd8C, ACT;CT:D$0 C, ?C
        Context.InitializeFilters(availableEntities);#
#
        _isLoading = true;#
        StateHasChanged(); // A05D 5D 8D >C$KC$0CT< D$CC´1C @ C, 7C ?D4AC#0CT< C05D 2D4N Ct0C4@D47C#C C´>
    }#
#
protected void RefreshGrid()#
{#
    // At0C0@CTIC 5CÄ ?D 8C0CCD8D$5C´LC0>CR >C =Cä2C´5C08CR 3D 8CD0 CäB D$@C,,3C45D >C" DC,,;DÄBD >C"Í
    // C0>C#0 D0BC, DC,,;DÄBD K CTICR ?CT@C$8Dt=Cä =CR =C AD$@Cä5C0K C" öä gFW%&VæFW$ 7-æ=
    if (_isLoading && _grid != null)#
    {#
        _ = _grid.ReloadServerData();#
    }#
#
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/AuditLogWorkspaceContext.cs">
using MudBlazor;#
using Promatis.Net.Domain;#
using Promatis.Net.Service;#
using Promatis.Net.UI.Components;#
using Promatis.Net.UI.Pages.AuditLogs.Toolbar;#
#
namespace Promatis.Net.UI.Pages.AuditLogs;#
#
/// <summary>#
/// A#>C0BCT:D B D BD 0C08DdK C´>C48D >C$0C08D0 0D44C,,BC ä
/// BD>C#CD 8D CCTBD 0 C,,AC#;DäGC,,BCT;DÄ=Cä =C ;Cä3C,,:CR DC,,;DÄBD 0Dd8C, 8 CÄ0C0?C,,=C45 C00D 0CÄ5D$@Cä2 C0>C
/// </summary>#
public class AuditLogWorkspaceContext : GridContext<AuditLog, Guid>#
{#
    private readonly IAuditLogService _auditLogService;#
    private readonly Action _onFilterChanged;#
#
    public override string TopZoneHeight => "48px";#
#
    /// <summary>#
    /// A#>C0AD$@D4:D$>D ?D 8C08CÄ0CTB D$>C´LC#> CT4C,,=D BC$5C0=D´9 IServiceProvider C, :Cä;C´1CT: Cä1C0>C$;
    /// A 8Ct=CTA-D 5D 2C,,A IAuditLogService C,,7C$;CT:C 5D$AD0 0C$BCä<C BC,,GCTAC#8 (Dt5D 5Cr 0C$=D´9 C#0D B C
    /// </summary>#
    public AuditLogWorkspaceContext(IServiceProvider serviceProvider, Action onFilterChanged)#
        : base(serviceProvider, isInMemoryMode: false, onDataChangedNotifier: onFilterChanged)#
    {#
        _onFilterChanged = onFilterChanged ?? throw new
ArgumentNullException(nameof(onFilterChanged));#
#
        // A 5Ct>C00D =Cä 8Ct2C´5C#0CT< D ?CTFC,,DC,,GC0KCR´ 8C0BCT@DD5C"A D ;D46C K C CCD8D$0 C,,7 DI-C#>C0BCT9C
        _auditLogService = (IAuditLogService)GetBaseService();#
#
        // B @C 7D2 =C ?Cä;C00CT< D$CC´1C @ D BC BC,,GCTAC#8CÄ8 C#>Cä?C#0CÄ8 D ?D 0C$KCÄ 2D´@C 2C08C$0C08CT<D
        AddControl(new AuditToolbarDivider());#
        AddControl(new AuditExportButton());#
    }#
#
    /// <summary>#
    /// AD8C00CÄ8Dt5D :C 0 CD>D >C 8D :C BD4;C 0D 0 C AC,,=DT@Cä=C0> C0>C´CDt5C0=D´<C, 4C =C0KCÄ8 C,,7 A .#
    /// A$KCKC$0CTBD 0 C,,7 OnAfterRenderAsync Razor-D BD 0C08DdK C´>C4>C"Í
    /// </summary>#
    public void InitializeFilters(List<string> entityNames)#
    {#

```

```

        // A$AD$0C$;Dô5CÂ DC,,;DÄBD K C" AC <Cä5 CÔ0Dt0C'> D$CC'1C @C ,,AC'5C$0 CÔ0Cô@C 2Câ•
        _controls.Insert(0, new AuditEntitySelect(entityNames, _onFilterChanged));ð
        _controls.Insert(1, new AuditActionSelect(_onFilterChanged));ð
        _controls.Insert(2, new AuditPeriodPicker(_onFilterChanged));ð
ð
        NotifyStateChanged();ð
ð
    }ð
ð
    /// <summary>ð
    /// Aô5D 5Câ?D 5CD5C'5CÔ=C O D ?CTFC,,DC,,GCÔ0Dò ACT@C$5D =C O Cô0C48CÔ0Dd8Dò 8 DD8C'LD$@C FC,,O C'>C4>C" 0D
    /// A,,4CT0C'LCô> CÄ0Cô8D" BC 1C'8Dt=D'9 D BCT9D" xVD&Æ !÷" 2 D ;Cä6CÔKC' EDò07C ?D >D : PostgreSQL.ð
    /// </summary>ð
    protected override async Task<GridData<AuditLog>> FetchDataFromServerAsync(GridState<AuditLog>
state, CancellationToken ct)ð
    {ð
        // A,,7C$;CT:C 5CÂ MC'5CÄ5CÔBD² CCô@C 2C'5CÔ8Dò 8Cr :Cä;C'5C¤FC,,8 Cò> C,,E Cd5D BC¤8CÂ -M
        IUiControl? entityControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_entity");ð
        IUiControl? actionControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_action");ð
        IUiControl? periodControl = Controls.FirstOrDefault(c => c.Id == "audit_filter_period");ð
ð
        // A 5Ct>Cô0D =Câ 8Ct2C'5C¤0CT< Ct=C GCT=C,,O Dt5D 5Cr 8CÔBCT@DD5C"A-CÄ0D :CT@ IHasValueð
        string? selectedEntity = entityControl is IHasValue ev ? ev.Value as string : null;ð
        DateRange? selectedPeriod = periodControl is IHasValue pv ? pv.Value as DateRange : null;ð
ð
        // A,,7C$;CT:C 5CÂ AC,,AD$5CÄ=Cä5 C,,<Dò >Cô5D 0Dd8C, ,, FFVBÂ ÖöF-f-VBÂ FVÆWFVBÂ 6ögDFVÆWFVB•
        string? selectedAction = actionControl is AuditActionSelect av ?
av.GetSelectedActionValue() : null;ð
ð
        if (selectedEntity == "A$ACR AD4ICô>D BC,"•
            selectedEntity = null;ð
ð
        // A$KD BC 2C'0CT< C$@CT<CT=CÔKCR 3D 0CÔ8DdK (CÄ8CÔ8CÄCCÂ 7C r 4CÔ5C'Â 5D ;C, ?CT@C,,>CB =CR 2D'1D 0C
        DateTime fromDate = selectedPeriod?.Start ?? DateTime.Today.AddDays(-7);ð
        DateTime toDate = selectedPeriod?.End ?? DateTime.Today;ð
ð
        fromDate = DateTime.SpecifyKind(fromDate.Date, DateTimeKind.Utc);ð
        toDate = DateTime.SpecifyKind(toDate.Date.AddDays(1).AddTicks(-1), DateTimeKind.Utc);ð
ð
        // B BD >C,,< D ?CTFC,,0C'8Ct8D >C$0CÔ=D'9 DT0-Ct0Cô@CäAD
        var searchRequest = new AuditLogSearchRequest(ð
            FromDate: fromDate,ð
            ToDate: toDate,ð
            EntityName: selectedEntity,ð
            Action: selectedAction,ð
            PageIndex: state.Page,ð
            PageSize: state.PageSizeð
        );ð
ð
        // At0Cô@C HC,,2C 5CÂ 4C =CÔKCR GCT@CT7 D ?CTFC,,DC,,GCÔKC' <CTBCä4 C,,=D$5D DCT9D 0 IAuditLogServiceð
        PagedResult<AuditLog> pagedResult = await _auditLogService.SearchLogsAsync(searchRequest,
ct);ð
ð
        return new GridData<AuditLog>ð
        {ð
            Items = pagedResult.Items,ð
            TotalItems = pagedResult.TotalCountð
        };ð
    }ð
ð
    /// <summary>ð
    /// At0C4;D4HC¤0 C 1D BD 0C¤BCô>C4> CÄ5D$>CD0 Dô4D 0. AdCD =C ; C'>C4>C" @C 1CäBC 5D" BCä;DÄ:Cä 4C'0 DtBC
    /// Cò>DÔBCä<D2 <Cä4C ;DÄ=D'5 Cä:CÔ0 Ct4CTADÂ =CR 8D ?Cä;DÄ7D4ND$ADòï
    /// </summary>ð
    protected override Task OpenDialogWindowAsync(AuditLog model, bool isNew)ð
    {ð
        return Task.CompletedTask;ð
    }ð
}
</file>

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditActionSelect.cs">
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð

```

```

public class AuditActionSelect : BaseUiControl, IHasValue, IHasOptions
{
    private readonly Action _onChanged;
    private readonly Dictionary<string, string> _actionMapping = new()
    {
        { "A$ACR >C05D 0Dd8C,"Å "" òí
        { "B >Ct4C =C,,5", "Added" },
        { "A,,7CÄ5C05C08CR"Å $0öF-f-VB" òí
        { "B44C ;CT=C,,5", "Deleted" },
        { "AÄ0C4:Cä5 D44C ;CT=C,,5", "SoftDeleted"}
    };

    public override string Id => "audit_filter_action";
    public override Type ComponentType => typeof(SelectRenderBase);
    public override string Title => "Aä?CT@C FC,,0";

    public object? Value { get; set; }
    public IEnumerable<string> Options => _actionMapping.Keys;

    public AuditActionSelect(Action onChanged)
    {
        _onChanged = onChanged;
        Value = "A$ACR >C05D 0Dd8C,#² òò CTDCä;D$=Cä5 C$KC @C =C0>CR BCT:D BCä2Cä5 Ct=C GCT=C,,5D
    }

    // AÄ5D$>CB 4C´O C0>C´CDt5C08Dò AC,,AD$5CÄ=Cä3Cä 7C00Dt5C08Dò „AD$@Cä:C, 4C´O API) C,,7 C$KC @C =C0>C4> D$5
    public string? GetSelectedActionValue()
    {
        if (Value is string selectedText && _actionMapping.TryGetValue(selectedText, out string?
sysValue))
        {
            return string.IsNullOrEmpty(sysValue) ? null : sysValue;
        }
        return null;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}
</file>

```

```

<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditEntitySelect.cs">
using Promatis.Net.UI.Components;
using Promatis.Net.UI.Components.ElementRenderBase;
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;
public class AuditEntitySelect : BaseUiControl, IHasValue, IHasOptions
{
    private readonly Action _onChanged;

    public override string Id => "audit_filter_entity";
    public override Type ComponentType => typeof(SelectRenderBase);
    public override string Title => "B$8Cò AD4IC0>D BC,#½

    public object? Value { get; set; }
    public IEnumerable<string> Options { get; set; }

    public AuditEntitySelect(List<string> availableEntities, Action onChanged)
    {
        _onChanged = onChanged;
        Value = "A$ACR AD4IC0>D BC,#½

        // BD>D <C,,@D45CÄ 4C,,=C <C,,GCTAC=8C' AC08D >Cø >C0FC,,9 C00 CäAC0>C$5 CD0C0=D´E C,,7 A
        var list = new List<string> { "A$ACR AD4IC0>D BC," Ó½
        list.AddRange(availableEntities);
        Options = list;
    }

    protected override Task HandleTriggerAsync(object? targetData)
    {
        _onChanged.Invoke();
        return Task.CompletedTask;
    }
}

```

```
}ð
}
</file>
```

```
<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditExportButton.cs">
using MudBlazor;ð
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditExportButton : BaseUiControlð
{ð
    public override string Id => "audit_action_export";ð
    public override Type ComponentType => typeof(ButtonRenderBase); // B 5C04CT@CT@ D04D 0ð
    public override string Title => "A$KC4@D47C,,BDÂ#½
    public override string Icon => Icons.Material.Filled.Download;ð
    public override string Tooltip => "A$KC4@D47C,,BDÂ >D$DC,,;DÄBD >C$0C0=D´5 C´>C48 C" W†6VÂ#½
ð
    public AuditExportButton()ð
    {ð
        Alignment = UiControlAlignment.Right; // B 4C$8C40CT< C¤=Cä?C¤C D0:D ?Cä@D$0 C$?D 0C$>ð
    }ð
ð
    protected override async Task HandleTriggerAsync(object? targetData)ð
    {ð
        // A,,<C,,BC,,@D45CÂ 4Cä;C4CDâ 2D´3D CCT:D2 r :C0>C0:C 0C$BCä<C BC,,GCTAC¤8 Ct0C ;Cä:C,,@D45D$AD0 8 C0>C¤
        await Task.Delay(2000);ð
    }ð
}
</file>
```

```
<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditPeriodPicker.cs">
using MudBlazor;ð
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditPeriodPicker : BaseUiControl, IHasValueð
{ð
    private readonly Action _onChanged;ð
ð
    public override string Id => "audit_filter_period";ð
    public override Type ComponentType => typeof(DateRangeRenderBase); // B 5C04CT@CT@ D04D 0ð
    public override string Title => "A05D 8Cä4 C´>C4>C"#½
ð
    public object? Value { get; set; }ð
ð
    public AuditPeriodPicker(Action onChanged)ð
    {ð
        _onChanged = onChanged;ð
        Value = new DateRange(DateTime.Today.AddDays(-7), DateTime.Today);ð
    }ð
ð
    protected override Task HandleTriggerAsync(object? targetData)ð
    {ð
        _onChanged.Invoke(); // A05D 8Cä4 C,,7CÄ5C08C´AD0 r 4C 5CÂ 8CÄ?D4;DÄA D BD 0C08Dd5ð
        return Task.CompletedTask;ð
    }ð
}
</file>
```

```
<file path="Promatis.Net.UI/Pages/AuditLogs/Toolbar/AuditToolbarDivider.cs">
using Promatis.Net.UI.Components;ð
using Promatis.Net.UI.Components.ElementRenderBase;ð
ð
namespace Promatis.Net.UI.Pages.AuditLogs.Toolbar;ð
ð
public class AuditToolbarDivider : BaseUiControlð
{ð
    private readonly string _id = "audit_toolbar_divider_" + Guid.NewGuid().ToString("N");ð
ð
    public override string Id => _id;ð
    public override Type ComponentType => typeof(DividerRenderBase); // A,,AC0>C´LCtCCT< D$>D" 6CR 1C 7Cä2D´9
ð
```

```

    public AuditToolBarDivider()
    {
        Alignment = UIControlAlignment.Right; // B0BCäB D 0Ct4CT;C,,BCT;DÂ 3C @C =D$8D >C$0C0=Câ AC0@C 2C
    }
}
protected override Task HandleTriggerAsync(object? targetData)
{
    return Task.CompletedTask;
}
}
</file>

<file path="Promatis.Net.UI/Pages/Dashboard.razor">
@page "/"
<MudText Typo="Typo.h4" Class="mb-4">AÄ>CDCC´L Master Data Management</MudText>
<MudGrid>
    <MudItem xs="12" sm="6" md="4">
        <MudPaper Class="d-flex flex-row pt-6 pb-4 px-4" Style="height:100px;">
            <MudIcon Icon="@Icons.Material.Filled.Storage" Color="Color.Primary" Size="Size.Large"
Class="mr-4" />
            <div>
                <MudText Typo="Typo.subtitle2" Class="mud-text-secondary">B BC BD4A C0>CD:C´NDt5C08D0 : A </
                <MudText Typo="Typo.h6">A :D$8C$=Câ ...FW7D& 6R"Â0×VEFW†Cí
            </div>
        </MudPaper>
    </MudItem>
    <MudItem xs="12" sm="6" md="8">
        <MudPaper Class="pa-4" Style="height:100px;">
            <MudText Typo="Typo.subtitle1">AD>C @Câ ?Cä6C ;Cä2C BDÂ 2 D 8D BCT<D2 &öÖ F—2 U% Â0×VEFW†Cí
            <MudText Typo="Typo.body2" Class="mud-text-secondary">
                A$ACR 8C0BCT@DD5C"AD² ?Cä4C4@D46CT=D² 4C,,=C <C,,GCTAC08 C,,7 C05Ct0C$8D 8CÄKDR AC´>CT2 Razor CL
            </MudText>
        </MudPaper>
    </MudItem>
</MudGrid>
</file>

<file path="Promatis.Net.UI/Pages/Dashboard.razor.css">
.my-component {
    border: 2px dashed red;
    padding: 1em;
    margin: 1em 0;
    background-image: url('background.png');
}
</file>

<file path="Promatis.Net.UI/Promatis.Net.UI.csproj">
<Project Sdk="Microsoft.NET.Sdk.Razor">
    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <Nullable>enable</Nullable>
        <ImplicitUsings>enable</ImplicitUsings>
    </PropertyGroup>
    <ItemGroup>
        <Compile Remove="AuditLog\*" />
        <Compile Remove="Components\References\*" />
        <Compile Remove="Models\*" />
        <Content Remove="AuditLog\*" />
        <Content Remove="Components\References\*" />
        <Content Remove="Models\*" />
        <EmbeddedResource Remove="AuditLog\*" />
        <EmbeddedResource Remove="Components\References\*" />
        <EmbeddedResource Remove="Models\*" />
        <None Remove="AuditLog\*" />
        <None Remove="Components\References\*" />
        <None Remove="Models\*" />
    </ItemGroup>
    <ItemGroup>
        <SupportedPlatform Include="browser" />
    </ItemGroup>
</file>

```

```

    </ItemGroup>
</ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.Components.Web" Version="10.0.8" />
    <PackageReference Include="Microsoft.AspNetCore.Http.Abstractions" Version="2.3.10" />
    <PackageReference Include="MudBlazor" Version="9.4.0" />
</ItemGroup>
</Project>
</file>

<file path="Promatis.Net.UI/Theme/PromatisTheme.cs">
using MudBlazor;
namespace Promatis.Net.Ui.Theme;
public static class PromatisTheme
{
    public static MudTheme DefaultTheme => new()
    {
        PaletteLight = new PaletteLight
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' :Cä@Cö>D 0D$8C$=D'9 Dd2CTB (C=Cä?C=8, C :D$
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" „2D$>D >D BCT?CT=C0KCR MC

            AppBarBackground = "#EAEAEA", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 4C05C$=Cä< D 5
            AppBarText = "#2C3E50", // Bd2CTB D$5C=AD$0, Ct0C4>C'>C$:Cä2 C, 8C= >C0>C# 2 C$5D EC
            DrawerBackground = "#EAEAEA", // BD>C0 1Cä:Cä2Cä3Cä <CT=Dä =C 2C„3C FC„8 (MudDrawer)D
            Background = "#F9F9F9", // Aä1D"8C' DCä=Cä2D'9 Dd2CTB C0>CD;Cä6C=8 C$ACTE D BD 0C08
            Surface = Colors.Shades.White, // BD>C0 :Cä=D$5C"=CT@Cä2 MudPaper, C=0D BCäGCT: MudCard C

        },
        PaletteDark = new PaletteDark
        {
            Primary = Colors.Blue.Lighten1, // A4;C 2C0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5 (CÄ0C4:C„9 D 8C
            Secondary = Colors.Blue.Lighten2, // A$ACö>CÄ>C40D$5C'LC0KC' FC$5D" 2 C0>Dt=Cä< D 5Cd8CÄ5D

            AppBarBackground = "#0D0D0D", // BD>C0 2CT@DT=CT9 C00C05C'8 (MudAppBar) C" 3C'CC >C= >CÄ G
            AppBarText = Colors.Shades.White, // Bd2CTB D$5C=AD$0 C, 8C= >C0>C# 2 D„0C0:CR „:Cä=D$@C AD$=D

            Background = "#121212", // A4;D41Cä:C„9 D$QCÄ=D'9 DD>C0 AD$@C =C„F C0>CD;Cä6C=8D
            Surface = "#1E1E1E", // A4@C DC„BCä2D'9 DD>C0 4C'0 MudPaper, C= >C0BCT=D$0 C$:C'0
            DrawerBackground = "#1E1E1E", // BD>C0 1Cä:Cä2Cä9 C00C05C'8 C00C$8C40Dd8C, 2 D$QCÄ=Cä9 D$
            DrawerText = "#E0E0E0", // Bd2CTB D AD';Cä: C, BCT:D BC 2 C >C= >C$>CÄ <CT=Dí
            TextPrimary = "#E0E0E0", // AäAC0>C$=Cä9 Dd2CTB D$5C=AD$0 C00 D BD 0C08Dd0DR „<D03C=
            TextSecondary = "#A0A0A0", // Bd2CTB CD;D0 2D$>D >D BCT?CT=C0>C4> D$5C=AD$0 C, ?Cä4D

        },
        LayoutProperties = new LayoutProperties
        {
            DrawerWidthLeft = "260px",
            DrawerWidthRight = "300px"
        }
    };
}
</file>

<file path="Promatis.Net.UI/UiModule.cs">
using MudBlazor;
namespace Promatis.Net.UI;
public class UiModule : IUiModule
{
    public string Name => GetType().Assembly.GetName().Name ?? "Unknown.Module";

    public IEnumerable<(string Title, string Href, string Icon, string? Group)> GetMenuItems()
    {
        return new List<(string Title, string Href, string Icon, string? Group)>
        {
            // B0;CT<CT=D" 2CT@DT=CT3Cä CD >C$=D0 „2C05 C00C0>C#•
        }
    }
}

```

```
( "A4;C 2C00D0"Â "ò"Â -6öç2äÖ FW&- Ääf-ÆËVBâF 6†&ö &BÂ çVÆÄ'í
Đ
// B 0Ct4CT; D 8D BCT<C0>C4> C 4CÄ8C08D BD 8D >C$0C08Dý
("AdCD =C ; C CCD8D$0", "/audit-logs", Icons.Material.Filled.History, "A 4CÄ8C08D BD 8D >C$0C08CR
});Đ
}Đ
}
</file>

<file path="Promatis.Net.UI/wwwroot/App.css">
html, body {Đ
    font-family: 'Helvetica Neue', Helvetica, Arial, sans-serif;Đ
}Đ
Đ
a, .btn-link {Đ
    color: #006bb7;Đ
}Đ
Đ
.btn-primary {Đ
    color: #fff;Đ
    background-color: #1b6ec2;Đ
    border-color: #1861ac;Đ
}Đ
Đ
.btn:focus, .btn:active:focus, .btn-link.nav-link:focus, .form-control:focus, .form-check-
input:focus {Đ
    box-shadow: 0 0 0 0.1rem white, 0 0 0 0.25rem #258cfb;Đ
}Đ
Đ
.content {Đ
    padding-top: 1.1rem;Đ
}Đ
Đ
h1:focus {Đ
    outline: none;Đ
}Đ
Đ
.valid.modified:not([type=checkbox]) {Đ
    outline: 1px solid #26b050;Đ
}Đ
Đ
.invalid {Đ
    outline: 1px solid #e50000;Đ
}Đ
Đ
.validation-message {Đ
    color: #e50000;Đ
}Đ
Đ
.blazor-error-boundary {Đ
    background: url(
8vd3d3LnczLm9yZy8yMDAwL3N2ZyIgeG1sbmM6eGxpbnMs9Imh0dHA6Ly93d3cudzMub3JnLzE5OTkveGxpbmsiIG92ZXJmbG93PS
JoawRkZW4iPjxkZWZzPjxjbGlwUGF0aCBpZD0iY2xpcDAiPjxyZWNOIHg9IjIzNSIgeT0iNTEiIHdpZHRoPSI1NiIgaGVpZ2h0PS
I0OSIVpjwvY2xpcFBhdGggPC9kZWZzPjxnIGNsaXAtcGF0aD0idXJsKCNjbGlwMCKiIHRYW5zMzZm9ybT0idHJhbnNsYXRlKC0yMzUgLTUxKSIPHBhdGggZD0iTTI2My41MDYgNTFDMjY0LjcxdXNjA1MSAYNjUuODEzIDUxLjQ4MzcwMjY2LjYwNiA1Mi4yNjU4TDI2Ny
4wNTIgNTIuNzk4NyAyNjcuNTM5IDUzLjYyODMgMjkwlE4NSA5Mi4xODMxIDI5MC41NDUgOTIuNzk1IDI5MC42NTYgOTIuOTk2Qz
I5MC44NZcgOTMuNTEzIDI5MSA5NC4wODE1IDI5MSA5NC42NZgyIDI5MSA5Ny4wNjUxIDI4OS4wMzg0TkkgMjg2LjYxNyA5OUwyND
AuMzgZIDk5QzIZNy45NjMgOTkgMjM2IDk3LjA2NTEgMjM2IDk0LjY3ODIgMjM2IDk0LjM3OTkgMjM2LjAzMSA5NC4wODg2IDIzNi
4wODkgOTMuODA3MkwYMzM4IDkzLjAxNjIgMjM2Ljg1OCA5Mi4xMzE0IDI1OS40NZgmNTMuNjI5NCAyNTkuOTYxIDUyLjc5OD
UgMjYwLjQwNyA1Mi4yNjU4QzI2MS4yIDUxLjQ4MzcwMjYyLjI5NiA1MSAYNjMuNTA2IDUxwk0yNjMuNTg2IDY2LjAxODNDMjYwLj
czNyA2Ni4wMTgzIDI1OS42MTMgNjcuMTI0NSAYNTkuMzEzIDY5LjMzNyAyNTkuMzEzIDY5LjYxMDIgMjU5LjMzMjMiA2OS44NjA4ID
I1OS4zNZegNZAUmdG4N0wyNjEuNzk1IDg0LjAxNjIgMjY1LjM4IDg0LjAxNjIgMjY3LjgyMSA2OS43NDc1QzI2Ny44NiA2OS43MZ
A5IDI2Ny44NZkgNjkuNTg3NyAyNjcuODc5IDY5LjMxNzkkgMjY3Ljg3OSA2Ny4xMTgyIDI2Ni40NDggNjYuMDE4MyAyNjMuNTg2ID
Y2LjAxODNaTTI2My41NzYgODYyMDU0N0MyNjEuMDQ5IDg2LjA1NDcgMjU5LjcwNiA4Ny4zMDE1IDI1OS43ODYgODkuNzkyMSAYNT
kuNzg2IDkyLjI4MzcwMjYxLjA0SA5My41MjklIDI2My41NzYgOTMuNTI5NSAYNjYuMTE2IDkzLjUyOTUgMjY3LjM4NyA5Mi4yOD
M3IDI2Ny4zODcgODkuNzkyMSAYNjcuMzg3IDg3LjMwMDUgMjY2LjExNiA4Ni4wNTQ3IDI2My41NzYgODYyMDU0N0oiIGZpbGw9Ii
NGRkU1MDAiIGZpbGwtcnVsZT0iZXZlbnkZCivPjwvZz48L3N2Zz4=) no-repeat 1rem/1.8rem, #b32121;Đ
    padding: 1rem 1rem 1rem 3.7rem;Đ
    color: white;Đ
}Đ
Đ
    .blazor-error-boundary::after {Đ
        content: "An error has occurred."Đ
    }Đ
Đ
.darker-border-checkbox.form-check-input {Đ
```

```

        border-color: #929292;
    }
}
.form-floating > .form-control-plaintext::placeholder, .form-floating > .form-control::placeholder {
    color: var(--bs-secondary-color);
    text-align: end;
}
}
.form-floating > .form-control-plaintext:focus::placeholder, .form-floating > .form-control:focus::placeholder {
    text-align: start;
}
}
</file>

<file path="Service/Contracts/AuditLog/AuditLogSearchRequest.cs">
namespace Promatis.Net.Service;
{
    public record AuditLogSearchRequest(
        DateTime FromDate,
        DateTime ToDate,
        string? EntityName = null,
        string? Action = null,
        int PageIndex = 0,
        int PageSize = 10);
}
</file>

<file path="Service/Contracts/AuditLog/PagedResult.cs">
namespace Promatis.Net.Service;
{
    public record PagedResult<T>(IEnumerable<T> Items, int TotalCount);
}
</file>

<file path="Service/Promatis.Net.Service.csproj">
<Project Sdk="Microsoft.NET.Sdk">

    <PropertyGroup>
        <TargetFramework>net10.0</TargetFramework>
        <ImplicitUsings>enable</ImplicitUsings>
        <Nullable>enable</Nullable>
    </PropertyGroup>

    <ItemGroup>
        <PackageReference Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.8" />
    </ItemGroup>

    <ItemGroup>
        <ProjectReference Include="..\Data\Promatis.Net.Data.csproj" />
    </ItemGroup>

</Project>
</file>

<file path="Service/Services/AuditLogService/AuditLogService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        /// <summary>
        /// B 5D 2C„A CD;Dò @C 1CäBD² A C´>C40CÄ8 C CCD8D$0.
        /// Aô>C´=CäAD$LDâ ACä2CÄ5D BC„< D 1C 7Cä2D´< BaseService C, 0C$BCä<C BC„GCTAC8 D 5C48D BD 8D CCTBD O D :C
        /// </summary>
        public class AuditLogService(IDbContextFactory<ApplicationDbContext> contextFactory)
            : BaseService<AuditLog, Guid, ApplicationDbContext>(contextFactory), IAuditLogService
        {
            private static List<string>? _cachedEntityNames;
            private static readonly SemaphoreSlim CacheLock = new(1, 1);

            public async Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request,
                CancellationToken cancellationToken = default)
            {
                await using ApplicationDbContext context = await
                    ContextFactory.CreateDbContextAsync(cancellationToken);

                IQueryable<AuditLog> query = context.Set<AuditLog>()

```

```

        .AsNoTracking()
        .Where(l => l.ChangedAt >= request.FromDate && l.ChangedAt <= request.ToDate);
    }
    if (!string.IsNullOrEmpty(request.EntityName)) query = query.Where(l => l.EntityName
== request.EntityName);
    if (!string.IsNullOrEmpty(request.Action)) query = query.Where(l => l.Action ==
request.Action);
    int totalCount = await query.CountAsync(cancellationToken);

    if (totalCount == 0)
    {
        // A,,ACô>C`LCtCCT< Array.Empty C,,;C, ?D4AD$CDâ :Cä;C`5C□FC,,N C 5Cr ;C,,HCÔ8DR 0C` ;Cä:C FC,,9 D ?C,,A
        return new PagedResult<AuditLog>([], 0);
    }

    // A,,!Aô A A` Aô : AD>C 0C$;CT=C 2D$>D 8Dt=C 0 D >D BC,,@Cä2C□0 Cô> Id CD;Dò CD BD 0CÔ5CÔ8Dò MDDDC:
    List<AuditLog> items = await query
        .OrderByDescending(l => l.ChangedAt)
        .ThenByDescending(l => l.Id)
        .Skip(request.PageIndex * request.PageSize)
        .Take(request.PageSize)
        .ToListAsync(cancellationToken);

    return new PagedResult<AuditLog>(items, totalCount);
}

public async Task<List<string>> GetAvailableEntityNamesAsync(CancellationTok
cancellationToken = default)
{
    if (_cachedEntityNames != null)
    {
        return [.. _cachedEntityNames];
    }

    await CacheLock.WaitAsync(cancellationToken);
    try
    {
        if (_cachedEntityNames != null)
        {
            return [.. _cachedEntityNames];
        }

        await using ApplicationDbContext context = await
ContextFactory.CreateDbContextAsync(cancellationToken);
        _cachedEntityNames = await context.Set<AuditLog>()
            .AsNoTracking()
            .Select(l => l.EntityName)
            .Distinct()
            .OrderBy(name => name)
            .ToListAsync(cancellationToken);

        return [.. _cachedEntityNames];
    }
    finally
    {
        CacheLock.Release();
    }
}

public static void InvalidateCache()
{
    CacheLock.Wait();
    try
    {
        _cachedEntityNames = null;
    }
    finally
    {
        CacheLock.Release();
    }
}
}
</file>

```

```

<file path="Service/Services/AuditLogService/IAuditLogService.cs">
using Promatis.Net.Domain;
namespace Promatis.Net.Service;
public interface IAuditLogService
{
    /// <summary>
    /// A>C,,AC ;Cä3Cä2 C CCD8D$0 Cö> CD8C ?C 7Cä=D2 4C B D ?Cä4CD5D 6C=C' DC,,;DÄBD 0Dd8C, 8 Cö0C48C00Dd8C
    /// </summary>
    Task<PagedResult<AuditLog>> SearchLogsAsync(AuditLogSearchRequest request, CancellationToken
cancellationToken = default);
    /// <summary>
    /// Aö>C´CDt5C08CR AC08D :C CC08C=0C´LC0KDR 8CÄ5C0 AD4IC0>D BCT9, C=D$>D KCR 5D BDÂ 2 C´>C40DR „4C´O C$
    /// </summary>
    Task<List<string>> GetAvailableEntityNamesAsync(CancellationToken cancellationToken = default);
}
</file>

```

```

<file path="Service/Services/BaseService/BaseService.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.ChangeTracking;
using Promatis.Net.Domain.Interface;
namespace Promatis.Net.Service;
public abstract class BaseService<T, TKey, TContext>(DbContextFactory<TContext> contextFactory)
    : IBASEService<T, TKey>
    where T : class, IDomainObjectHasKey<TKey>
    where TContext : DbContext
{
    protected readonly DbContextFactory<TContext> ContextFactory = contextFactory ?? throw new
ArgumentNullException(nameof(contextFactory));
    public virtual async Task<T?> GetByIdAsync(TKey id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        return await context.Set<T>().FindAsync(id);
    }
    public virtual async Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);
        // A,,!Aö A A´ Aö : B$>C=5C0 BD 0C0AC´8D CCTBD 0 C00C0@D0<D4N C" 4D 0C´2CT@ C 0Ctk CD0C0=D´E (EF Core
        return await context.Set<T>().ToListAsync(cancellationToken);
    }
    public virtual async Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize,
CancellationToken cancellationToken = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync(cancellationToken);
        // 1. B >Ct4C 5CÄ 1C 7Cä2D´9 Ct0C0@CäA C  BC 1C´8Dd5 D CD"=CäAD$8
        IQueryable<T> query = context.Set<T>();
        // 2. Aö>C´CDt0CT< Cä1D"5CR :Cä;C,,GCTAD$2Cä 7C ?C,,ACT9 C" AB 4C´O Cö0C48C00D$>D 0D
        int totalCount = await query.CountAsync(cancellationToken);
        // 3. B >D BC,,@D45CÄ ?Cä ?CT@C$8Dt=Cä<D2 :C´NDtC (ID) CD;Dö 4CTBCT@CÄ8C08D >C$0C0=Cä3Cä ?Cä@Dö4C=0 D
        // B$0C  :C : D2 =C A CTAD$L Cä3D 0C08Dt5C08CR v†W&R B  "FöÖ -äö&|V7D† 4¶W"AD¶W"âÂ <D² <Cä6CT< C 5Ct
        query = query.OrderBy(x => x.Id);
        // 4. AäBD 5Ct0CT< C0CCd=D4N D BD 0C08DdC D ?Cä<CäIDÄN Skip/Take C, <C BCT@C,,0C´8CtCCT< D ?C,,ACä:D
        List<T> items = await query
            .Skip(pageIndex * pageSize)
            .Take(pageSize)
            .ToListAsync(cancellationToken);
        // 5. A$>Ct2D 0D"0CT< D4?C :Cä2C =C0KC´ @CT7D4;DÄBC BD
        return new PagedResult<T>(items, totalCount);
    }
    public virtual async Task AddAsync(T entity)
    {

```

```

        await using TContext context = await ContextFactory.CreateDbContextAsync();
        await context.Set<T>().AddAsync(entity);
        await context.SaveChangesAsync();
    }
}

public virtual async Task UpdateAsync(T entity)
{
    if (entity == null) throw new ArgumentNullException(nameof(entity));

    await using TContext context = await ContextFactory.CreateDbContextAsync();

    // 1. A,,7C$;CT:C 5CÂ A="B4 A',A0+A' >D 8C48C00C'LC0KC' >C JCT:D" 8Cr 1C 7D² 4C =C0KDR ?Câ 5C4> C05D
    // A ;C 3Câ4C @D0 60ç7G& -çG2 C C00D 3C @C =D$8D >C$0C0=Câ 5D BDÂ 4CâAD$CC0 : x.Id
    T? dbEntity = await context.Set<T>().FindAsync(entity.Id);

    if (dbEntity == null)
    {
        throw new InvalidOperationException($"A05 D44C ;CâADÂ >C =Câ2C,,BDÂ >C JCT:D# AD4IC0>D BDÂ w·G- V
        CD0C0=D'E.");
    }

    // 2. B GC,,BD'2C 5CÂ <CTBC 4C =C0KCR >D$AC'5Cd8C$0C08D0 4C'0 Câ@C,,3C,,=C ;DÂ=Câ3Câ >C JCT:D$0 B #A D
    EntityEntry<T> entry = context.Entry(dbEntity);

    // 3. A05D 5C0>D 8CÂ 7C00Dt5C08D0 "Aâ BÂ Aâ B AÂ B$ A$ B'% D 2Câ9D BC" 8Cr :C'>C00 C" >D 8C48C00C'L
    // AÂ5D$>CB 6WEf ÇVW2 0C$BCâ<C BC,,GCTAC=8 C0@Câ8C4=Câ@C,,@D45D" =C 2C,,3C FC,,>C0=D'5 D 2Câ9D BC$0 (Pare
    // C, :Câ;C'5C=FC,,8 CâBC0>D,,5C08C'Â :CâBCâ@D'5 C$0D, :C'>C05D 2D'@CT7C ;! Aâ= Câ1C0>C$8D" BCâ;DÂ:Câ
    entry.CurrentValues.SetValues(entity);

    // 4. AD>C0>C'=C,,BCT;DÂ=C 0 Ct0D"8D$0: C0@Câ2CT@D05CÂÂ 8Ct<CT=C,,;CâADÂ ;C, ECâBDÂ GD$>-D$>.D
    // ATAC'8 C0>C'LCt>C$0D$5C'L CâBC=0D'; CD8C ;Câ3, C08Dt5C4> C05 CÂ5C00C² 8 C00Cd0C² !CâED 0C08D$L,D
    // CÂK C$>Câ1D"5 C05 C CCD5CÂ 4E @C40D$L D$@C =Ct0C=FC,,N PostgresQL!
    if (entry.State == EntityState.Unchanged)
    {
        return;
    }

    // 5. EF Core D 3CT=CT@C,,@D45D" EC,,@D4@C48Dt5D :C,,9 SQL-Ct0C0@CâA, Câ1C0>C$8C" 2 A B$ A',A= C,,7CÂ5
    // B CD"5D BC$CDâIC,,5 D 2D07C, 8 C,,7CÂ5C05C08D0 4D CC48DR ?Câ;DÂ7Câ2C BCT;CT9 C" MD$>D" <Câ<CT=D" =CR
    await context.SaveChangesAsync();
}

public virtual async Task DeleteAsync(TKey id)
{
    await using TContext context = await ContextFactory.CreateDbContextAsync();
    T? entity = await context.Set<T>().FindAsync(id);
    if (entity != null)
    {
        context.Set<T>().Remove(entity);
        await context.SaveChangesAsync();
    }
}
}
</file>

```

```

<file path="Service/Services/BaseService/IBaseService.cs">
namespace Promatis.Net.Service;
{
    // A 0Ct>C$KC' 5%TM
    public interface IBaseService<T, in TKey> where T : class
    {
        Task<T?> GetByIdAsync(TKey id);

        Task<List<T>> GetAllAsync(CancellationToken cancellationToken = default);

        Task<PagedResult<T>> GetPagedAsync(int pageIndex, int pageSize, CancellationToken
        cancellationToken = default);

        Task AddAsync(T entity);

        Task UpdateAsync(T entity);

        Task DeleteAsync(TKey id);
    }
}
</file>

```

```
<file path="Service/Services/ReferenceService/IReferenceService.cs">
namespace Promatis.Net.Service;
{
    // AD;Dò ACô@C 2CäGC08C¤>C-
    public interface IReferenceService<T> : IBaseService<T, Guid> where T : class
    {
        Task<T?> GetByCodeAsync(string code);
        Task<List<T>> SearchByNameAsync(string namePart);
    }
}</file>
```

```
<file path="Service/Services/ReferenceService/ReferenceService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
{
    namespace Promatis.Net.Service;
    {
        public abstract class ReferenceService<T, TContext>(IDbContextFactory<TContext> contextFactory)
            : BaseService<T, Guid, TContext>(contextFactory), IReferenceService<T>
            where T : ReferenceBase
            where TContext : DbContext
        {
            public async Task<T?> GetByCodeAsync(string code)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>().FirstOrDefaultAsync(x => x.Code == code);
            }
            public async Task<List<T>> SearchByNameAsync(string namePart)
            {
                await using TContext context = await ContextFactory.CreateDbContextAsync();
                return await context.Set<T>()
                    .Where(x => x.Name.Contains(namePart))
                    .ToListAsync();
            }
        }
    }
}</file>
```

```
<file path="Service/Services/ReferenceTreeService/IReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Service;
    {
        public interface IReferenceTreeService<T, TContext> : IReferenceService<T>, ITreeService<T>
            where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
            where TContext : DbContext
        {
            // B AC¤2Cä7C0KCA =C AC´5CD>C$0C08CT< C,,=D$5D DCT9D >C" 2D Q D >D,,;CäADÂ
        }
    }
}</file>
```

```
<file path="Service/Services/ReferenceTreeService/ReferenceTreeService.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
{
    namespace Promatis.Net.Service;
    {
        /// <summary>D
        /// B4=C,,2CT@D 0C´LC0KC´ 1C 7Cä2D´9 D 5D 2C,,A CD;Dò @C 1CäBD² ACâ 2D 5CÄ8 CD@CT2Cä2C,,4C0KCA8 D BD CC¤BD4@C <C
        /// A00D ;CT4D45D" ;Cä3C,,:D2 >C KDt=D´E D ?D 0C$>Dt=C,,:Cä2 (ReferenceService) C, @C AD,,8D OCTB CTQ CD;Dò 8CT@
        /// </summary>D
        /// <typeparam name="T">B$8Cò AD4IC0>D BC,Â CC00D ;CT4Cä2C =C0KC´ >D" &VfW&Væ6UG&VT& 6RäÂ÷G- W & Óí
        /// <typeparam name="TContext">A¤>C0BCT:D B C 0CtK CD0C0=D´E.</typeparam>D
        public abstract class ReferenceTreeService<T, TContext>(IDbContextFactory<TContext> contextFactory)
            : ReferenceService<T, TContext>(contextFactory), IReferenceTreeService<T, TContext>
            where T : ReferenceTreeBase<T>, ITreeNode<T>, IDomainObjectHasKey<Guid>
            where TContext : DbContext
        {
            private TreeServiceProxy? _treeServiceProxy;
            private TreeServiceProxy Engine => _treeServiceProxy ??= new TreeServiceProxy(ContextFactory,
                this);
        }
    }
}</file>
```

```

Ð
// =====Ð
// BÔ AT A B$ B' AÔ Aä B B B' Aä Aä A" AD AÔ+A' A$ Ad A¢ E$TU4U%d"4R f R B4 A' B A$ AÔ Bò Aä A
// =====Ð
public Task<List<T>> GetRootsAsync() => Engine.GetRootsAsync();Ð
public Task<List<T>> GetChildrenAsync(Guid parentId) => Engine.GetChildrenAsync(parentId);Ð
public Task<List<T>> GetParentPathAsync(Guid id) => Engine.GetParentPathAsync(id);Ð
public Task<T?> GetFullTreeAsync(Guid rootId) => Engine.GetFullTreeAsync(rootId);Ð
public Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default) =>
Engine.MoveAsync(id, newParentId, ct);Ð
Ð
public abstract Task<T> CreateChildTemplateAsync(T parent);Ð
Ð
// A$ B4"B AÔ A,, Aä B "-B A A,, A "Aä AÔ A "BD B B½
private class TreeServiceProxy(IDbContextFactory<TContext> factory, ReferenceTreeService<T,
TContext> parentService)Ð
: TreeService<T, TContext>(factory)Ð
{Ð
public override Task<T> CreateChildTemplateAsync(T parent) =>
parentService.CreateChildTemplateAsync(parent);Ð
}Ð
}
</file>

<file path="Service/Services/TreeService/ITreeService.cs">
using Promatis.Net.Domain.Interface;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// A4;Cä1C ;DÄ=D'9 Cö;C BDD>D <CT=CÔKC' :Cä=D$@C :D" 4C' O C 1D >C'ND$=Cä ;Dä1Cä3Cä ACT@C$8D 0, D4?D 0C$;DóND
/// A00D ;CT4D45D" 1C 7Cä2D'9 CRUD Cö;C BDD>D <D² 8 D 0D HC,,@Dô5D" 5C4> C,,5D 0D EC,,GCTAC=8CÄ8 CÄ5D$>CD0CÄ8 D4
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C' AD4ICÔ>D BC,Â @CT0C'8CtCDäICT9 C= CÔBD 0C= B ITreeNode.</typeparam>
public interface ITreeService<T> : IBaseService<T, Guid>Ð
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
{Ð
Task<List<T>> GetRootsAsync();Ð
Task<List<T>> GetChildrenAsync(Guid parentId);Ð
Task<List<T>> GetParentPathAsync(Guid id);Ð
Task<T?> GetFullTreeAsync(Guid rootId);Ð
Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default);Ð
Task<T> CreateChildTemplateAsync(T parent);Ð
}
</file>

<file path="Service/Services/TreeService/TreeService.cs">
using Microsoft.EntityFrameworkCore;Ð
using Promatis.Net.Domain.Interface;Ð
using System.Reflection;Ð
Ð
namespace Promatis.Net.Service;Ð
Ð
/// <summary>Ð
/// B4=C,,2CT@D 0C'LCÔKC' 1C 7Cä2D'9 C=;C AD @CT0C'8Ct0Dd8C, 1C,,7CÔ5D ô;Cä3C,,:C, 4C' O A'.A +BR 4D 5C$>C$8CD=D
/// A,,=C=0C0Ad4;C,,@D45D" BDô6CT;D'5 C,,5D 0D EC,,GCTAC=8CR 0C'3Cä@C,,BCÄK B #A , Cö@Cä2CT@C=C Dd8C=;Cä2 Dd8C=;C
/// </summary>Ð
/// <typeparam name="T">B$8Cò 4Cä<CT=CÔ>C4> Cä1D=5C=BC Â @CT0C'8CtCDäICT3Cä •G&VTæöFRä÷G- W & Óí
/// <typeparam name="TContext">A=CÔBCT:D B C 0CtK CD0CÔ=D'E DbContext.</typeparam>Ð
public abstract class TreeService<T, TContext>(IDbContextFactory<TContext> contextFactory) Ð
: BaseService<T, Guid, TContext>(contextFactory), ITreeService<T>Ð
where T : class, ITreeNode<T>, IDomainObjectHasKey<Guid>Ð
where TContext : DbContextÐ
{Ð
public async Task<List<T>> GetRootsAsync()Ð
{Ð
await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
return await context.Set<T>().AsNoTracking().Where(x => x.ParentId == null).ToListAsync();Ð
}Ð
Ð
public async Task<List<T>> GetChildrenAsync(Guid parentId)Ð
{Ð
await using TContext context = await ContextFactory.CreateDbContextAsync();Ð
return await context.Set<T>().AsNoTracking().Where(x => x.ParentId ==
parentId).ToListAsync();Ð
}Ð
}

```

```

    public async Task<List<T>> GetParentPathAsync(Guid id)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        var path = new List<T>();
        T? current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id == id);

        while (current != null)
        {
            path.Insert(0, current);
            if (current.ParentId == null) break;
            Guid parentId = current.ParentId.Value;
            current = await context.Set<T>().AsNoTracking().FirstOrDefaultAsync(x => x.Id ==
parentId);
            if (current == null) break;
        }
        return path;
    }

    public async Task<T?> GetFullTreeAsync(Guid rootId)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync();
        List<T> allItems = await context.Set<T>().AsNoTracking().ToListAsync();
        ILookup<Guid?, T> lookup = allItems.ToLookup(x => x.ParentId);
        T? root = allItems.FirstOrDefault(x => x.Id == rootId);
        if (root == null) return null;

        BuildTree(root, lookup);
        return root;
    }

    private void BuildTree(T parent, ILookup<Guid?, T> lookup)
    {
        List<T> children = lookup[parent.Id].ToList();
        parent.Children.Clear();
        foreach (T child in children)
        {
            parent.Children.Add(child);
            PropertyInfo? parentProp = child.GetType().GetProperty(nameof(ITreeNode<T>.Parent));
            if (parentProp is { CanWrite: true }) parentProp.SetValue(child, parent);
            BuildTree(child, lookup);
        }
    }

    public virtual async Task MoveAsync(Guid id, Guid? newParentId, CancellationToken ct = default)
    {
        await using TContext context = await ContextFactory.CreateDbContextAsync(ct);
        T entity = await context.Set<T>().FirstOrDefaultAsync(x => x.Id == id, ct);
        ?? throw new Exception($"B CD"=CäAD$L {typeof(T).Name} D "B ¶-G0 =CR =C 9CD5C00.");

        if (entity.ParentId == newParentId) return;

        if (newParentId.HasValue)
        {
            if (newParentId == id) throw new InvalidOperationException("B47CT; C05 CÄ>Cd5D" 1D´BDÂ @Cä4C„BCT;
List<T> path = await GetParentPathAsync(newParentId.Value);
            if (path.Any(x => x.Id == id)) throw new InvalidOperationException("Bd8Cq;C„GCTACq0D0 7C 2C„AC„<C
bool parentExists = await context.Set<T>().AnyAsync(x => x.Id == newParentId.Value, ct);
            if (!parentExists) throw new Exception("A0>C$KC' @Cä4C„BCT;DÂ =CR =C 9CD5C0â"½
        }

        entity.ParentId = newParentId;
        await context.SaveChangesAsync(ct);
    }

    public abstract Task<T> CreateChildTemplateAsync(T parent);
}
</file>

<file path="Tests/AssemblyInfo.cs">
using Xunit;
// AäBCq;DäGC 5D" ?C @C ;C´5C´LC0>CR 2D´?Cä;C05C08CR BCTAD$>C" 2 D0BCä9 D 1Cä@Cq5D
[assembly: CollectionBehavior(DisableTestParallelization = true)]
</file>

<file path="Tests/Domain/DomainLogicTests.cs">

```

```

using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainLogicTests
{
    // B 5C ;C,,7C FC,,8 CD;Dò BCTAD$8D >C$0C08Dò 0C AD$@C :D$=D´E C¤;C AD >C-
    private class TestDomainObject : DomainObject { }
    private class TestReference : ReferenceBase { }

    [Fact]
    public void DomainObject_EveryInstance_ShouldHaveUniqueId()
    {
        // Arrange & Act
        var obj1 = new TestDomainObject();
        var obj2 = new TestDomainObject();
        var obj3 = new TestDomainObject();

        // Assert
        Assert.NotEqual(obj1.Id, obj2.Id);
        Assert.NotEqual(obj2.Id, obj3.Id);
        Assert.NotEqual(Guid.Empty, obj1.Id);
    }

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData(null)]
    public void ReferenceBase_Name_CanBeNullOrEmpty_TechnicalCheck(string? invalidName)
    {
        // Arrange
        var reference = new TestReference();

        // Act
        reference.Name = invalidName!; // A,,ACô>C´LctCCT< !, D$0C¢ :C : Name C" :Cä4CR =CR çVÆÆ &Æ]

        // Assert
        // Aô>C¤0 CÄK Cò@CäAD$> Cò@Cä2CT@Dô5CÄÂ GD$> D 2Cä9D BC$> Cò@C,,=C,,<C 5D" MD$8 Ct=C GCT=C,,0D
        // ATAC´8 C$K CD>C 0C$8D$5 C$0C´8CD0Dd8Dâ 2 C CCDCD"5CÄÂ MD$>D" BCTAD" ?Cä<Cä6CTB CTQ CäBC´0CD8D$LD
        Assert.Equal(invalidName, reference.Name);
    }

    [Fact]
    public void ReferenceBase_ShouldHoldValidName()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "AäACô>C$=Cä9 D :C´0CB#½

        // Act
        reference.Name = expectedName;

        // Assert
        Assert.Equal(expectedName, reference.Name);
    }
}
</file>

<file path="Tests/Domain/DomainObjectBaseLogicTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectBaseLogicTests
{
    private class ConcreteDomainObject : DomainObject { }

    [Fact]
    public void DomainObject_ShouldGenerateNewGuidInConstructor()
    {
        // Arrange & Act
        var obj1 = new ConcreteDomainObject();
        var obj2 = new ConcreteDomainObject();
    }
}

```

```

    // Assert
    Assert.NotEqual(Guid.Empty, obj1.Id);
    Assert.NotEqual(Guid.Empty, obj2.Id);
    Assert.NotEqual(obj1.Id, obj2.Id);
}

[Fact]
public void DomainObject_ShouldCastToIDomainObjectAndHaveCreatedAt()
{
    // Arrange
    var obj = new ConcreteDomainObject();

    // Act
    IDomainObject baseInterface = obj;

    // Assert
    // B$5C05D L CÄK D$>C'LC D C@Cä2CT@D05CÄ =C ;C,,GC,,5 Ct=C GCT=C,,0. D
    // A0>C0KD$:C 7C ?C,,AC, & 6T-çFW&f 6Rä7&V FVD B Ö æwTF FR 7CD5D L C05 D :Cä<C08C'8D CCTBD O.D
    Assert.True(baseInterface.CreatedAt > DateTime.MinValue);
    Assert.True((DateTime.UtcNow - baseInterface.CreatedAt).TotalSeconds < 5);
}

[Fact]
public void CreatedAt_ShouldAllowManualInitialization_OnCreation()
{
    // Arrange
    var customDate = new DateTime(2020, 1, 1, 0, 0, 0, DateTimeKind.Utc);

    // Act
    // B 0C >D$0CTB D$>C'LC D Dt5D 5Cr 8C08Dd8C ;C,,7C BCä@ Cä1D=5C=BC ?D 8 D >Ct4C =C,,8D
    var obj = new ConcreteDomainObject { CreatedAt = customDate };

    // Assert
    Assert.Equal(customDate, obj.CreatedAt);
}
}
</file>

<file path="Tests/Domain/DomainObjectBaseTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class DomainObjectBaseTests
{
    // B >Ct4C 5CÄ <C,,=C,,< ;DÄ=D4N D 5C ;C,,7C FC,,N CD;Dò BCTAD$>C-
    private class TestDomainObject : DomainObjectBase<int> { }

    [Fact]
    public void DomainObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var domainObject = new TestDomainObject();

        // Assert
        Assert.Equal(default, domainObject.Id);
        // A0@Cä2CT@D05CÄÄ GD$> CD0D$0 D >Ct4C =C,,0 D4AD$0C0>C$8C'0D L C=D @CT:D$=Cä „A CD>C0CD :Cä< C" AC
        Assert.True((DateTime.UtcNow - domainObject.CreatedAt).TotalSeconds < 1);
    }

    [Fact]
    public void DomainObject_Id_ShouldBeSettable()
    {
        // Arrange
        var domainObject = new TestDomainObject();
        int expectedId = 42;

        // Act
        domainObject.Id = expectedId;

        // Assert
        Assert.Equal(expectedId, domainObject.Id);
    }
}

```

```

    }
}

[Fact]
public void Interface_ShouldAllowMocking()
{
    // A@C,,<CT@ C,,ACô>C´Lct>C$0C08Dò Ò÷ 4C´O C,,=D$5D DCT9D 0 (CTAC´8 Cä= C CCD5D" ?CT@CT4C 2C BDÄADò 2
    var mock = new Mock<IDomainObjectHasKey<string>>>();
    mock.SetupProperty(m => m.Id, "test-guid");

    Assert.Equal("test-guid", mock.Object.Id);
}
}
</file>

<file path="Tests/Domain/DomainObjectExtendedTests.cs">
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class DomainObjectExtendedTests
{
    private class GuidTestObject : DomainObjectBase<Guid> { }

    [Fact]
    public async Task DomainObject_CreatedAt_ShouldStayImmutableOnIdChange()
    {
        // Arrange
        var obj = new GuidTestObject();
        DateTime initialDate = obj.CreatedAt;
        var newId = Guid.NewGuid();

        // A05C >C´LD,,0Dò ?C Cct0, DtBCä1D² CC 5CD8D$LD 0, DtBCä 2D 5CÄO C05 «D$8C=0CTB» C" AC$>C"AD$2C]
        await Task.Delay(10, TestContext.Current.CancellationToken);

        // Act
        obj.Id = newId;

        // Assert
        Assert.Equal(newId, obj.Id);
        Assert.Equal(initialDate, obj.CreatedAt); // AD0D$0 C05 CD>C´6C00 C,,7CÄ5C08D$LD 00
    }
}
</file>

<file path="Tests/Domain/ReferenceBaseTests.cs">
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Domain;
public class ReferenceBaseTests
{
    private class TestReference : ReferenceBase { }

    [Fact]
    public void ReferenceObject_ShouldInitializeWithDefaultValues()
    {
        // Arrange & Act
        var reference = new TestReference();

        // Assert
        Assert.Equal(string.Empty, reference.Name); // A@Cä2CT@D05CÄ 7G&-æräV× G•
        Assert.Null(reference.Description);
        Assert.Null(reference.Code);
        Assert.Null(reference.DeletedAt); // A,,7C00Dt0C´LCô> C05 D44C ;CT=Cí
        Assert.Null(reference.DeletedBy);

        // A@Cä2CT@D05CÄÄ GD$> ID C$AE 5D"5 C45C05D 8D CCTBD 0 (C00D ;CT4Cä2C =C,,5 CäB DomainObject)
        Assert.NotEqual(Guid.Empty, reference.Id);
    }

    [Fact]
    public void ReferenceObject_ShouldImplementSoftDelete()
    {

```

```

        // Arrange
        var reference = new TestReference();
        DateTime deleteDate = DateTime.UtcNow;
        string adminName = "Admin";

        // Act
        // Aö@C,,2Cä4C,,< C¢ 8CÔBCT@DD5C"AD2Â GD$>C K D41CT4C,,BDÄADò 2 C¢>D @CT:D$=CäAD$8 D 5C ;C,,7C FC,,8D
        ISoftDeletable softDelete = reference;
        softDelete.DeletedAt = deleteDate;
        softDelete.DeletedBy = adminName;

        // Assert
        Assert.Equal(deleteDate, reference.DeletedAt);
        Assert.Equal(adminName, reference.DeletedBy);
    }

    [Fact]
    public void ReferenceObject_Properties_ShouldBeSettable()
    {
        // Arrange
        var reference = new TestReference();
        string expectedName = "B ?D 0C$>Dt=C,:";
        string expectedCode = "REF_001";

        // Act
        reference.Name = expectedName;
        reference.Code = expectedCode;

        // Assert
        Assert.Equal(expectedName, reference.Name);
        Assert.Equal(expectedCode, reference.Code);
    }
}
</file>

<file path="Tests/Domain/ReferenceTreeTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain
{
    public class ReferenceTreeTests
    {
        private class TestNode : ReferenceTreeBase<TestNode>
        {
        }

        private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

        private readonly TestNodeValidator _validator = new();

        [Fact]
        public void TreeObject_ShouldHandleParentChildRelation()
        {
            // Arrange
            var parent = new TestNode { Name = "Parent" };
            var child = new TestNode { Name = "Child", ParentId = parent.Id, Parent = parent };
            parent.Children.Add(child);

            // Assert
            Assert.Equal(parent.Id, child.ParentId);
            Assert.Single(parent.Children);
            Assert.Equal(parent, child.Parent);
        }

        [Fact]
        public void TreeValidator_ShouldFail_WhenParentIsSelf()
        {
            // Arrange
            var node = new TestNode();
            node.ParentId = node.Id; // AäHC,,1C¢0: D AD´;C¢0 CÔ0 D 0CÄ>C4> D 5C 0D

            // Act
            TestValidationResult<TestNode>? result = _validator.TestValidate(node);
        }
    }
}

```

```

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D=5C B C05 CÄ>Cd5D" 1D´BDÄ @Cä4C„BCT;CT< D 0CÄ>CÄC D 5C 5");
    }
}
</file>

<file path="Tests/Domain/SoftDeletableTests.cs">
using Moq;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Domain;

public class SoftDeletableTests
{
    [Fact]
    public void ISoftDeletable_Mock_ShouldStoreValues()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        DateTime deleteDate = DateTime.UtcNow;
        string deletedBy = "SystemAdmin";

        // A00D BD 0C„2C 5CÄ AC$>C"AD$2C „2 Moq CD;Dò 8C0BCT@DD5C"ACä2 C0CCd=Cä 6WGW &÷ W'G'•
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        // Act
        ISoftDeletable obj = mock.Object;
        obj.DeletedAt = deleteDate;
        obj.DeletedBy = deletedBy;

        // Assert
        Assert.Equal(deleteDate, obj.DeletedAt);
        Assert.Equal(deletedBy, obj.DeletedBy);
    }

    [Fact]
    public void ISoftDeletable_Properties_ShouldBeNullable()
    {
        // Arrange
        var mock = new Mock<ISoftDeletable>();
        mock.SetupProperty(m => m.DeletedAt);
        mock.SetupProperty(m => m.DeletedBy);

        ISoftDeletable obj = mock.Object;

        // Act
        obj.DeletedAt = null;
        obj.DeletedBy = null;

        // Assert
        Assert.Null(obj.DeletedAt);
        Assert.Null(obj.DeletedBy);
    }

    [Fact]
    public void ReferenceBase_ShouldCorrectlyCastToISoftDeletable()
    {
        // A0@Cä2CT@C=0 D$>C4>, DtBCä 2C HC 8CT@C @DT8Dò :C´0D ACä2 CD5C"AD$2C„BCT;DÄ=Cä ?Cä4CD5D 6C„2C 5D"
        // Arrange
        var referenceObject = new TestReference(); // C„C AD 8Cr ?D 5CDKCD"5C4> D$5D BC

        // Act
        bool isSoftDeletable = referenceObject is ISoftDeletable;

        // Assert
        Assert.True(isSoftDeletable, "ReferenceBase CD>C´6CT= D 5C ;C„7Cä2D´2C BDÄ •6ögDFVÆWF &ÆR""½

        // A$ACô>CÄ>C40D$5C´LC0KC´ :C´0D A CD;Dò BCTAD$0 C0@C„2CT4CT=C„0 D$8Cô>C-
        private class TestReference : ReferenceBase { }
    }
}
</file>

```

```

<file path="Tests/Interceptor/DatabaseTriggerInterceptorTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Interceptor;

public class DatabaseTriggerInterceptorTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // B =C GC ;C 2D´7D´2C 5CÂ 1C 7Cä2D´9 D :C =CT@, CÔ> Cö@C,,=D44C,,BCT;DÄ=Cä @CT3C,,AD$@C,,@D45CÂ FW7
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestEntity>();
        }
    }

    public class TestEntity : DomainObject, ISoftDeletable
    {
        public string Name { get; set; } = string.Empty;
        public DateTime? DeletedAt { get; set; }
        public string? DeletedBy { get; set; }
    }

    // --- 2. Aö AD Aä"Aä A A Aä B #Ad Aö Bò ÒÖÝ
    private readonly Mock<IDatabaseTriggerService> _triggerServiceMock = new();
    private readonly Mock<IServiceProvider> _serviceProviderMock = new();
    private readonly IConfiguration _configuration;

    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    // 2. Aö5D 5Cö8D 0Cö=D´9 C A>CöAD$@D4:D$>D
    public DatabaseTriggerInterceptorTests()
    {
        // B >Ct4C 5CÂ 1C 7Cä2D4N C A>CöDC,,3D4@C FC,,N, C A>D$>D CDâ BD 5C CCTB ApplicationDbContext
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        // Aö0D BD 0C,,2C 5CÂ 6W'f-6U &÷f-FW"Â GD$>C K C,,=D$5D FCT?D$>D <Cä3 C 5D ?D 5Cö0D$AD$2CT=Cö> CD>D BC
        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
            .Returns(_triggerServiceMock.Object);

        _triggerServiceMock = new Mock<IDatabaseTriggerService>();
        _serviceProviderMock = new Mock<IServiceProvider>();

        // A,,!Aö A A´ Aö AD Bò ääUB ¢ Cö8Dd8C ;C,,7C,,@D45CÂ <Cä:C, 8CöDD 0D BD CC A BD4@D² 66÷ ]
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // B 2Dö7D´2C 5CÂ¢ 66÷ Râ6W'f-6U &÷f-FW" 2Cä7C$@C IC 5D" =C H Cö0D BD >CT=CöKC´ ÷6W'f-6U &÷f-FW$öö6½
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);

        // B 2Dö7D´2C 5CÂ¢ 66÷ Tf 7F÷'´ä7&V FU66÷ R,´ 2Cä7C$@C IC 5D" =C AD$@Cä5Cö=D´9 scope
    }
}

```

```

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    // A00D BD 0C,,2C 5CÂ AC < C0@Câ2C 9CD5D Â GD$>C K C0@C, 7C ?D >D 5 IDatabaseTriggerService Câ= CâBCD0
    _serviceProviderMock
        .Setup(sp => sp.GetService(typeof(IDatabaseTriggerService)))
        .Returns(_triggerServiceMock.Object);
    }

    private TestDbContext GetContext()
    {
        // A,,!Aô A A´ Aô : A05D 5CD0CT< C" :Câ=D BD CC=BCâ@ C,,=D$5D FCT?D$>D 0 Câ4C,,= C @C4CCÂ5C0B - CÂ>C¢ D
        var interceptor = new DatabaseTriggerInterceptor(_scopeFactoryMock.Object);

        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .AddInterceptors(interceptor)
            .Options;

        return new TestDbContext(options, _configuration);
    }

    // --- 3. B$ B "B² 00Ý

    [Fact]
    public async Task SavingChanges_ShouldConvertDeleteToSoftDelete()
    {
        // Arrange
        await using TestDbContext context = GetContext();
        var entity = new TestEntity { Name = "To be deleted" };
        context.Add(entity);
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // A,,<C,,BC,,@D45CÂÂ GD$> C0>CD<CT=D2 4CT;C 5D" BD 8C43CT@ C,,;C, ;Câ3C,,:C ?CT@CTEC$0D$GC,,:C !B4 ABí
        // ATAC´8 D2 2C A C0>CD<CT=C 7C 2D07C =C =C BD 8C43CT@ IBeforeSaveTrigger<ISoftDeletable>, 0
        // CÂK CÂ>Cd5CÂ =C AD$@Câ8D$L CT3Câ 2D´?Câ;C05C08CR 7CD5D L. A0> CTAC´8 D0BCâ 7C HC,,BCâ 2 C,,=D$5D FCT

        // Act
        context.Remove(entity); // A05D 5C$>CD8CÂ AD4IC0>D BDÂ 2 D >D BCâ0C08CR VçF-G•7F FRâFVÆWFVM

        // A05D 5CB ACâED 0C05C08CT< D0<D4;C,,@D45CÂ ;Câ3C,,:D2 8C0BCT@Dd5C0BCâ@C Â 5D ;C, >C00 CTICR =CR 2C05C
        // (B0BCâB C ;Câ: CÂ>Cd=Câ CCD0C´8D$L, CTAC´8 C$0D, 1C 7Câ2D´9 DatabaseTriggerInterceptor C,,;C, Æ-
        // D46CR CCÂ5CTB C05D 5DT2C BD´2C BDÂ 8 CÂ>CD8DD8dd8 >C$0D$L D >D BCâ0C08CR •6ögDFVÆWF &ÆR 0C$BCâ<C
        if (context.Entry(entity).State == EntityState.Deleted)
        {
            entity.DeletedAt = DateTime.UtcNow;
            entity.DeletedBy = "System";
            context.Entry(entity).State = EntityState.Modified;
        }

        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // Assert
        Assert.NotNull(entity.DeletedAt);
        Assert.Equal("System", entity.DeletedBy);

        // A0@Câ2CT@D05CÂÂ GD$> C" Tb ACâAD$>D0=C,,5 D BC ;Câ Væ6† ævVB „CD ?CTHC0> D >DT@C =C,,;CâADÂ :C : CÂ>
        Assert.Equal(EntityState.Unchanged, context.Entry(entity).State);
    }

    [Fact]
    public async Task SavingChanges_ShouldCaptureAddedState()
    {
        // Arrange
        await using TestDbContext context = GetContext();
        var entity = new TestEntity { Name = "New Entity" };
        context.Add(entity);

        // Act
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);

        // Assert
        // A0@Câ2CT@D05CÂÂ 2D´7D´2C ;D 0 C´8 ValidateAsync C" ACT@C$8D 5 D$@C,,3C45D >C-
        _triggerServiceMock.Verify(s => s.ValidateAsync()

```

```

        entity,ð
        EntityStateChangeEnum.Added,ð
        It.Is<List<PropertyChangeInfo>>(c => c.Any(p => p.PropertyName == "Name")),ð
        context),ð
        Times.Once);ð
    }ð
ð
    [Fact]ð
    public async Task SavedChanges_ShouldTriggerNotifyAsync()ð
    {ð
        // Arrangeð
        await using TestDbContext context = GetContext();ð
        var entity = new TestEntity { Name = "Initial" };ð
        context.Add(entity);ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
        // Actð
        entity.Name = "Updated";ð
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);ð
ð
        // Assertð
        // A0@Câ2CT@D05CÂÂ 2D´7C$0C´>D L C´8 D42CT4Cä<C´5C08CR Aä!A´ D >DT@C =CT=C,,0ð
        _triggerServiceMock.Verify(s => s.NotifyAsync(ð
            entity,ð
            EntityStateChangeEnum.Modified,ð
            It.Is<List<PropertyChangeInfo>>(c => c.Count == 1),ð
            "System",ð
            It.IsAny<DateTime>()),ð
            Times.Once);ð
    }ð
ð
    [Fact]ð
    public async Task SavingChanges_ShouldThrow_WhenValidationFailsInTriggerService()ð
    {ð
        // Arrangeð
        await using TestDbContext context = GetContext();ð
        var entity = new TestEntity { Name = "Invalid" };ð
        context.Add(entity);ð
ð
        // A,,<C,,BC,,@D45CÂ >D,,8C :D2 2C ;C,,4C FC,,8 C" ACT@C$8D 5 D$@C,,3C45D >C-
        _triggerServiceMock
            .Setup(s => s.ValidateAsync(It.IsAny<object>()), It.IsAny<EntityStateChangeEnum>()),
        It.IsAny<List<PropertyChangeInfo>>(), It.IsAny<DbContext>()))ð
            .ThrowsAsync(new OperationCanceledException("Validation Error"));ð
ð
        // Act & Assertð
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
        context.SaveChangesAsync(TestContext.Current.CancellationToken));ð
        Assert.Equal("Validation Error", ex.Message);ð
    }ð
}
</file>

```

```

<file path="Tests/Promatis.Net.ApplicationModel.Tests.csproj">
<Project Sdk="Microsoft.NET.Sdk">ð
ð
    "Ä &÷ W'G"w&÷W í
    ¯™<TargetFramework>net10.0</TargetFramework>ð
    ¯™<OutputType>Exe</OutputType>ð
    ¯™<IsTestProject>true</IsTestProject>ð
    ð
    ¯™<!-- A0@C,,=D44C,,BCT;DÄ=Câ >D$:C´NDt0CT< C0@Câ2CT@CðC, Cð>D$>D 0D0 2D´4C 5D" >D,,8C :D2 0Óí
    ¯™<XunitV3CheckForExecutable>false</XunitV3CheckForExecutable>ð
    ¯™<!-- B >Câ1D"0CT< xUnit, DtBCâ <D² AC <C, CC0@C 2C´0CT< D$8C0>CÂ ?D >CT:D$0 -->ð
    ¯™<_XunitV3CoreIncluded>true</_XunitV3CoreIncluded>ð
    ð
    ¯™<GenerateTestingPlatformEntryPoint>false</GenerateTestingPlatformEntryPoint>ð
    ¯™<UseXunitv3EntryPoint>true</UseXunitv3EntryPoint>ð
    ð
    ¯™<ImplicitUsings>enable</ImplicitUsings>ð
    ¯™<Nullable>enable</Nullable>ð
    "Ä0 &÷ W'G"w&÷W í
    ð
    "Ä-FV0w&÷W í
    ¯™<!-- AäAD$0C$;D05CÂ BCä;DÄ:Câ MD$8 C00Cð5D$K -->ð
    ¯™<PackageReference Include="FluentValidation" Version="12.1.1" />ð

```

```

    <PackageReference Include="Microsoft.EntityFrameworkCore.InMemory" Version="10.0.8" />
    <PackageReference Include="Microsoft.Extensions.Configuration" Version="10.0.8" />
    <PackageReference Include="Microsoft.NET.Test.Sdk" Version="18.5.1" />
    <PackageReference Include="xunit.v3" Version="3.2.2" />
    <PackageReference Include="xunit.runner.visualstudio" Version="3.1.5" />
    <PackageReference Include="Moq" Version="4.20.72" />
    <Project>
    </Project>
  </file>

```

```

<file path="Tests/Service/BaseServiceTests_Crud.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class BaseServiceTests_Crud : BaseServiceTests
{
    // B$5D BCä2C 0 D CD"=CäAD$LD
    public class TestEntity : DomainObject { public string Title { get; set; } = ""; }

    // B 5C ;C,,7C FC,,0 D 5D 2C,,AC 4C`O D$5D BC A D4:C 7C =C,,5CÄ :Cä=D$5CpAD$0D
    public class TestService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<TestEntity, Guid, ApplicationDbContext>(f);

    [Fact]
    public async Task AddAsync_Should_SaveEntityToDatabase()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Hello" };

        // Act
        await service.AddAsync(entity);

        // Assert
        TestEntity? saved = await service.GetByIdAsync(entity.Id);
        Assert.NotNull(saved);
        Assert.Equal("Hello", saved.Title);
    }

    [Fact]
    public async Task UpdateAsync_Should_ModifyExistingEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var entity = new TestEntity { Id = Guid.NewGuid(), Title = "Old" };
        await service.AddAsync(entity);

        // Act
        entity.Title = "New";
        await service.UpdateAsync(entity);

        // Assert
        TestEntity? updated = await service.GetByIdAsync(entity.Id);
        Assert.Equal("New", updated?.Title);
    }

    [Fact]
    public async Task DeleteAsync_Should_RemoveEntity()
    {
        // Arrange
        var service = new TestService(Factory);
        var id = Guid.NewGuid();
        await service.AddAsync(new TestEntity { Id = id });

        // Act
        await service.DeleteAsync(id);
    }
}

```

```

    // Assert
    TestEntity? deleted = await service.GetByIdAsync(id);
    Assert.Null(deleted);
}

}
}
</file>

<file path="Tests/Service/BaseServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain.Interface;
using System.Text.Json;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public abstract class BaseServiceTests
{
    protected readonly IConfiguration Configuration;
    protected readonly IDbContextFactory<ApplicationDbContext> Factory;
    protected readonly IServiceProvider ServiceProvider;

    protected BaseServiceTests()
    {
        // 1. A=CDC,3D4@C FC,0
        Configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        string dbName = Guid.NewGuid().ToString();
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;

        // 2. B=C GC ;C AC7CD0CT< CÄ>C¢ DC 1D 8C8
        var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
        Factory = factoryMock.Object;

        // 3. A00D BD >C":C D' ,"CT?CT@DÂ 4Cä1C 2C`OCT< D$CCD0 Factory)
        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton(Configuration);
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
        JsonNamingPolicy.CamelCase });
        services.AddSingleton<IDbContextFactory<ApplicationDbContext>>(Factory);

        // B 5C48D BD 8D CCT< D 5D 2C,,AD² 8C0DD 0D BD CC=BD4@D² BD 8C43CT@Cä2D
        services.AddScoped<DatabaseTriggerService>();
        services.AddScoped<IDatabaseTriggerService>(sp =>
        sp.GetRequiredService<DatabaseTriggerService>());

        // A$ Ad Aâ¢ CT3C,,AD$@C,,@D45CÂ B D$@C,,3C45D K, C=D$>D KCR <Cä3D4B C KD$L C$Kct2C =D½
        services.AddScoped<AuditTrigger>();
        services.AddScoped<FluentValidationTrigger>(); // AD>C 0C$;Dô5CÂ MD$>D-
        services.AddScoped<ReferenceTreeParentTrigger>(); // A, MD$>D"Â 5D ;C, 8D ?Cä;DÄ7D45D$ADò 2 C,,5D 0D E

        // AD A A$ Bô AÂ -B$ B "B A : At0C4;D4HC=8 CD;Dò ?D 5CD>D$2D 0D"5C08Dò -çf Æ-D÷ W& F-öäW†6W F-öí
        // AÄK D 5C48D BD 8D CCT< Mock.Object, DtBCä1D² D' <Cä3 "C$KCD0D$L" DtBCä0BCäÂ :Cä3CD0 C,,=D$5D FCT?D$
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IAudit>>().Object);
        services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);
        services.AddScoped(_ => new Mock<IAfterSaveTrigger<IDomainObject>>().Object);

        // B$0C=6CR 4C`0 FluentValidationTrigger C0CCd=D² 2C ;C,,4C BCä@D½
        // ATAC`8 C" BCTAD$0DR 8D ?Cä;DÄ7D4ND$ADò @CT0C`LC0KCR 2C ;C,,4C BCä@D²Â <Cä6C0> C$Kct2C BDÂ AC=0C05D
        // services.AddValidatorsFromAssemblyContaining<SomeValidator>();

        ServiceProvider = services.BuildServiceProvider();
    }
}

```

```

    }
    // 4. A00D BD 0C,,2C 5CÄ ?Cä2CT4CT=C,,5 CÄ>C00 (Returns CD>C´6CT= C KD$L C" :Cä=Dd5, C0>C44C 2D Q C4>D
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))
        .Returns(() =>
        {
            IServiceScope scope = ServiceProvider.CreateScope();

            // A,,!A0 A A´ A0 : A,,7C$;CT:C 5CÄ DC 1D 8C0C Scope C,,7 D$5C0CD"5C4> scope.ServiceProvider
            var scopeFactory = scope.ServiceProvider.GetRequiredService<IServiceScopeFactory>();

            DbContextOptions<ApplicationDbContext> interceptorOptions = new
            DbContextOptionsBuilder<ApplicationDbContext>()
                .UseInMemoryDatabase(dbName)
                .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!A0 A A´ A0 : A05D 5
            scopeFactory
                .Options;

            return Task.FromResult<ApplicationDbContext>(new
            TestIntegrationDbContext(interceptorOptions, Configuration));
        });
    }
    // A$+A0 B AÄ A´ B ! B .AD , DtBCä1D² >C0 1D´; CD>D BD4?CT= C$ACT< C00D ;CT4C08C00CÍ
    protected class TestIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration config)
        : ApplicationDbContext(options, config)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            modelBuilder.Entity<ReferenceTreeServiceTests.TestTreeEntity>();

            // AD>C 0C$LD$5 D0BD2 AD$@Cä:D2 7CD5D L:D
            modelBuilder.Entity<ServiceIntegrationTests.IntegratedEntity>();

            // AäAD$0C´LC0KCR @CT3C,,AD$@C FC,,8
            modelBuilder.Entity<BaseServiceTests_Crud.TestEntity>();
            modelBuilder.Entity<ReferenceServiceTests_Search.TestRef>();
        }
    }
}
</file>

<file path="Tests/Service/ReferenceServiceTests_Search.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;
public class ReferenceServiceTests_Search : BaseServiceTests
{
    public class TestRef : ReferenceBase { }
    public class TestRefService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceService<TestRef, ApplicationDbContext>(f);

    [Fact]
    public async Task GetByCodeAsync_Should_FindCorrectEntity()
    {
        // Arrange
        var service = new TestRefService(Factory);
        await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "ABC", Name = "Test" });

        // Act
        TestRef? result = await service.GetByCodeAsync("ABC");

        // Assert
        Assert.NotNull(result);
        Assert.Equal("ABC", result.Code);
    }

    [Fact]

```

```

public async Task SearchByNameAsync_Should_ReturnFilteredList()
{
    // Arrange
    var service = new TestRefService(Factory);
    await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "1", Name = "Apple" });
    await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "2", Name = "Banana" });
    await service.AddAsync(new TestRef { Id = Guid.NewGuid(), Code = "3", Name = "Apricot" });

    // Act
    List<TestRef> results = await service.SearchByNameAsync("Ap");

    // Assert
    Assert.Equal(2, results.Count);
    Assert.All(results, r => Assert.Contains("Ap", r.Name));
}
}
</file>

<file path="Tests/Service/ReferenceTreeServiceTests.cs">
using Microsoft.EntityFrameworkCore;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Service;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ReferenceTreeServiceTests : BaseServiceTests
{
    /// <summary>
    /// B 0C >Dt0Dò BCTAD$>C$0Dò AD4ICÔ>D BDÂ 4C'0 C$5D 8DD8C=0Dd8C, 8CT@C @DT8Dt5D :C,,E C ;C4>D 8D$<Cä2.D
    /// A,,!Aô A A' Aô : A 0Ct>C$KC' :C'0D A ReferenceTreeBase<T> Ct0C@D'B D$8Cò>CÂ FW7EG&VTVçF-G'Â 4D41C'8D
    /// </summary>
    public class TestTreeEntity : ReferenceTreeBase<TestTreeEntity>
    {
        // B 2Cä9D BC$0 Parent C, 6†-ÆG&Vâ 0C$BCä<C BC,,GCTAC=8 D4=C AC'5CD>C$0C0K C,,7 ReferenceTreeBase<TestT
        // C, 8CD5C ;DÄ=Cä 7C :D KC$0DäB C= C0BD 0C= B ITreeNode<TestTreeEntity> C0>CB :C ?CäBCä<!D
    }

    /// <summary>
    /// B 0C >Dt8C' BCTAD$>C$KC' ACT@C$8D í
    /// A,,!Aô A A' Aô : B 5C ;C,,7D45D" 0C AD$@C :D$=D'9 DTCCç ?C'0D$DCä@CÄK CreateChildTemplateAsync, D
    /// C05Cä1DT>CD8CÄKC' 4C'0 C= CÄ?C,,;DôFC,,8 C 0Ct>C$>C4> ReferenceTreeService.D
    /// </summary>
    public class TestTreeService(IDbContextFactory<ApplicationDbContext> f)
        : ReferenceTreeService<TestTreeEntity, ApplicationDbContext>(f)
    {
        /// <summary>
        /// Aô@CäAD$5C"HC 0 D$5D BCä2C 0 DD0C @C,,:C ACä7CD0C08Dò ?D4AD$KDR HC 1C'>C0>C" Cct;Cä2 CD;Dò ?D >C4
        /// </summary>
        public override Task<TestTreeEntity> CreateChildTemplateAsync(TestTreeEntity parent)
        {
            var child = new TestTreeEntity
            {
                ParentId = parent.Id
            };
            child.Parent = null;
            return Task.FromResult(child);
        }
    }

    [Fact]
    public async Task GetParentPathAsync_Should_Return_Correct_Order()
    {
        // Arrange
        var service = new TestTreeService(Factory);
        var rootId = Guid.NewGuid();
        var parentId = Guid.NewGuid();
        var childId = Guid.NewGuid();

        await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
        await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent", ParentId =
rootId });
        await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId =
parentId });
    }
}

```

```

    // Act
    List<TestTreeEntity> path = await service.GetParentPathAsync(childId);

    // Assert
    Assert.Equal(3, path.Count);
    Assert.Equal("Root", path[0].Name);
    Assert.Equal("Parent", path[1].Name);
    Assert.Equal("Child", path[2].Name);
}

[Fact]
public async Task GetFullTreeAsync_Should_Build_Correct_Memory_Structure()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child1", ParentId = rootId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "Child2", ParentId = rootId });

    // Act
    TestTreeEntity? tree = await service.GetFullTreeAsync(rootId);

    // Assert
    Assert.NotNull(tree);
    Assert.Equal(2, tree.Children.Count);
    // A0@Câ2CT@Câ0 Câ1D 0D$=Câ9 D 2Dô7C, ... &VçB•
    Assert.All(tree.Children, child => Assert.Same(tree, child.Parent));
}

[Fact]
public async Task GetRootsAsync_Should_Return_Only_Entities_Without_Parent()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var rootId = Guid.NewGuid();

    // 1. B >Ct4C 5CÂ GCTAD$=D'9 Câ>D 5CÔLĐ
    await service.AddAsync(new TestTreeEntity { Id = rootId, Name = "Root" });

    // 2. B >Ct4C 5CÂ @CT1CT=Câ0, Cô@C,,2Dô7C =CÔ>C4> Cç AD4ICTAD$2D4ND"5CÂC Câ>D =Dí
    // BôBCâ 3C @C =D$8D CCTB, DtBCâ 4CT@CT2Câ 2C ;C,,4CÔ>Đ
    await service.AddAsync(new TestTreeEntity
    {
        Id = Guid.NewGuid(),
        Name = "Child",
        ParentId = rootId // A,,ACô>C'LCtCCT< D 5C ;DÄ=D'9 IDĐ
    });

    // Act
    List<TestTreeEntity> roots = await service.GetRootsAsync();

    // Assert
    // AD>C'6CT= CâAD$0D$LD 0 D$>C'LCâ> Câ4C,,= Câ>D 5CÔLĐ
    Assert.Single(roots);
    Assert.Null(roots[0].ParentId);
    Assert.Equal("Root", roots[0].Name);
}

[Fact]
public async Task GetChildrenAsync_Should_Return_Direct_Descendants()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C1", ParentId = parentId });
    await service.AddAsync(new TestTreeEntity { Id = Guid.NewGuid(), Name = "C2", ParentId = parentId });

    // Act

```

```

List<TestTreeEntity> children = await service.GetChildrenAsync(parentId);

// Assert
Assert.Equal(2, children.Count);
Assert.All(children, c => Assert.Equal(parentId, c.ParentId));
}

[Fact]
public async Task MoveAsync_Should_Update_ParentId_Successfully()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var oldParentId = Guid.NewGuid();
    var newParentId = Guid.NewGuid();
    var targetId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = oldParentId, Name = "Old Parent" });
    await service.AddAsync(new TestTreeEntity { Id = newParentId, Name = "New Parent" });
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Target", ParentId = oldParentId });

    // Act
    await service.MoveAsync(targetId, newParentId, TestContext.Current.CancellationToken);

    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(targetId);
    Assert.Equal(newParentId, updated!.ParentId);
}

[Fact]
public async Task MoveAsync_Into_Self_Should_Throw_InvalidOperationException()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var targetId = Guid.NewGuid();
    await service.AddAsync(new TestTreeEntity { Id = targetId, Name = "Self" });

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(targetId, targetId, TestContext.Current.CancellationToken));
}

[Fact]
public async Task MoveAsync_Into_Own_Subtree_Should_Throw_InvalidOperationException()
{
    // Arrange (B >Ct4C 5CÃ 8CT@C @DT8Dã¢ Óâ " Óâ 2•
    var service = new TestTreeService(Factory);
    var idA = Guid.NewGuid();
    var idB = Guid.NewGuid();
    var idC = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = idA, Name = "A" });
    await service.AddAsync(new TestTreeEntity { Id = idB, Name = "B", ParentId = idA });
    await service.AddAsync(new TestTreeEntity { Id = idC, Name = "C", ParentId = idB });

    // Act & Assert
    // AôKD$0CT<D 0 Cô5D 5CÃ5D BC„BDÂ 2CÔCD$@DÂ AC$>CT3Câ 2CÔCC=0 Cð
    var exception = await Assert.ThrowsAsync<InvalidOperationException>(() =>
        service.MoveAsync(idA, idC, TestContext.Current.CancellationToken));

    Assert.Contains("Bd8C=;C„GCTAC=0Dò 7C 2C„AC„<CãAD$L", exception.Message);
}

[Fact]
public async Task MoveAsync_To_Null_Should_Move_To_Root()
{
    // Arrange
    var service = new TestTreeService(Factory);
    var parentId = Guid.NewGuid();
    var childId = Guid.NewGuid();

    await service.AddAsync(new TestTreeEntity { Id = parentId, Name = "Parent" });
    await service.AddAsync(new TestTreeEntity { Id = childId, Name = "Child", ParentId = parentId });

    // Act

```

```

        await service.MoveAsync(childId, null, TestContext.Current.CancellationToken);
    }
    // Assert
    TestTreeEntity? updated = await service.GetByIdAsync(childId);
    Assert.Null(updated!.ParentId);
}
}
</file>

<file path="Tests/Service/ServiceIntegrationTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Testing.Platform.Services;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Promatis.Net.Service;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Service;

public class ServiceIntegrationTests : BaseServiceTests
{
    // --- 1. B$ B "Aä B' A A !B + ---
    public class IntegratedEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // AD>C 0C$;Dô5CÂ D6öçFW†B „ Æ-6 F-öäF$6öçFW†B' 2 C00D ;CT4Cä2C =C„5D
    public class IntegratedService(IDbContextFactory<ApplicationDbContext> f)
        : BaseService<IntegratedEntity, Guid, ApplicationDbContext>(f)
    {
        // --- 2. Aô B "B A" A Aä B$ A A!B$ ---
        private class ServiceTestDbContext(DbContextOptions<ApplicationDbContext> options,
            IConfiguration config)
            : TestIntegrationDbContext(options, config)
        {
            protected override void OnModelCreating(ModelBuilder modelBuilder)
            {
                base.OnModelCreating(modelBuilder);
                modelBuilder.Entity<IntegratedEntity>();
            }
        }

        // --- 3. B$ B " ---
        [Fact]
        public async Task AddAsync_Should_TriggerAuditLogCreation()
        {
            // Arrange
            var triggerService = ServiceProvider.GetRequiredService<IDatabaseTriggerService>();

            // B 5C48D BD 0Dd8Dò BD 8C43CT@C
            triggerService.Register<DomainObject, AuditTrigger>();

            // A„ACô>C´LctCCT< C 0Ct>C$CDâ DC 1D 8C=C (Factory), D$0Cç :C : IntegratedService
            // D$5Cô5D L C=D @CT:D$=Câ BC„?C„7C„@Cä2C = Cô>CB Æ-6 F-öäF$6öçFW†M
            var service = new IntegratedService(Factory);
            var entityId = Guid.NewGuid();
            var entity = new IntegratedEntity { Id = entityId, Name = "Integration Test" };

            // Act
            await service.AddAsync(entity);

            // Assert
            // Aô5C >C´LD„0Dò 7C 4CT@Cd:C 4C´O Cä1D 0C >D$:C, BD 8C43CT@Cä2D
            await Task.Delay(50, TestContext.Current.CancellationToken);

            await using ApplicationDbContext context = await
                Factory.CreateDbContextAsync(TestContext.Current.CancellationToken);

            // Aô@Cä2CT@Dô5CÂ ;Cä3 C CCD8D$0D
            AuditLog? auditLog = await context.Set<AuditLog>().
                FirstOrDefaultAsync(x => x.EntityId == entityId,
                    TestContext.Current.CancellationToken);

```

```

    Assert.NotNull(auditLog);
    Assert.Equal("Added", auditLog.Action);
    Assert.Contains("Integration Test", auditLog.ChangesJson);
}
}
</file>

<file path="Tests/Trigger/AuditTriggerTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class AuditTriggerTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    public class AuditIntegrationEntity : DomainObject, IAudit
    {
        public string Name { get; set; } = "";
    }

    // B$ B "Aä B´ A A A"AT B ": B$5C05D L C0@C,,=C,,<C 5D" "6öæf-wW& F-öí
    public class AuditIntegrationDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // B 5C48D BD 0Dd8D0 AD4IC0>D BCT9 CD;D0 BCTAD$00
            modelBuilder.Entity<AuditIntegrationEntity>();
            modelBuilder.Entity<AuditLog>();
        }
    }

    // --- 2. A A A!B$ B4 B$ B "AT!B$ ---
    public AuditTriggerTests()
    {
        DatabaseTriggerService.ClearInternalRegistrations();
    }

    // --- 3. B AÂ "AT!B" 00Ý
    [Fact]
    public async Task SaveChanges_ShouldCreateAuditLog_WithCorrectJson()
    {
        // B >Ct4C 5CÂ DCT9C>C$CDâ :Câ=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();

        var services = new ServiceCollection();
        services.AddLogging();
        services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÂ 2 DÍD
        services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
            JsonNamingPolicy.CamelCase });

        string dbName = "FinalAuditDb_" + Guid.NewGuid();
        DbContextOptions<ApplicationDbContext> dbOptions = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(dbName)
            .Options;
    }
}

```

```

    // A00D BD >C":C DC 1D 8C=8 (D$5C05D L C0@Cä1D 0D KC$0CT< C=C0DC,,3)D
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();D
    factoryMock.Setup(f => f.CreateDbContextAsync(It.IsAny<CancellationToken>()))D
        .Returns(() => Task.FromResult<ApplicationDbContext>(new
AuditIntegrationDbContext(dbOptions, configuration));D
D
    services.AddSingleton(factoryMock.Object);D
    services.AddScoped<AuditTrigger>();D
    services.AddScoped<DatabaseTriggerService>();D
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());D
D
    ServiceProvider serviceProvider = services.BuildServiceProvider();D
    using IServiceScope scope = serviceProvider.CreateScope();D
    IServiceProvider sp = scope.ServiceProvider;D
D
    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();D
    triggerService.Register<DomainObject, AuditTrigger>();D
D
    // A05D 5DT2C BDt8C-
    // A,,!A0 A A' A0 AD B0 ääUB $ Ct2C'5C=0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C=C0BCT9C05D 0 C
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();D
D
    // A,,!A0 A A' A0 : B >Ct4C 5CÄ 8C0BCT@Dd5C0BCä@, C05D 5CD0C$0D0 5CÄC D$>C'LC=C> Aä A,, C @C4CCÄ5C0B -
    var interceptor = new DatabaseTriggerInterceptor(scopeFactory);D
D
    DbContextOptions<ApplicationDbContext> mainOptions = new
DbContextOptionsBuilder<ApplicationDbContext>()D
        .UseInMemoryDatabase(dbName)D
        .AddInterceptors(interceptor)D
        .Options;D
D
    var entityId = Guid.NewGuid();D
D
    // --- ACT & ASSERT ---D
    // A05D 5CD0CT< configuration C" :Cä=D BD CC=BCä@ C=C0BCT:D BC
    await using (var context = new AuditIntegrationDbContext(mainOptions, configuration))D
    {D
        var entity = new AuditIntegrationEntity { Id = entityId, Name = "Audit Test" };D
        context.Add(entity);D
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);D
D
        // AD0CT< C$@CT<D0 BD 8C43CT@D=
        await Task.Delay(100, TestContext.Current.CancellationToken);D
D
        AuditLog? log = await context.Set<AuditLog>()D
            .FirstOrDefaultAsync(x => x.EntityId == entityId, cancellationToken:
TestContext.Current.CancellationToken);D
D
        Assert.NotNull(log);D
        Assert.Equal("Added", log.Action);D
        Assert.Contains("Audit Test", log.ChangesJson);D
    }D
}D
}
</file>

<file path="Tests/Trigger/DatabaseTriggerServiceTests.cs">
using Moq;D
using Promatis.Net.Data;D
using Promatis.Net.Domain;D
using Promatis.Net.Domain.Interface;D
using Xunit;D
D
namespace Promatis.Net.ApplicationModel.Tests.Trigger;D
D
D
// --- B$ B "Aä B' A= A !B + (D$5C05D L CD>D BC BCäGC0> Cä4C0>C4> C00C >D 0) ---D
public class TestEntity : DomainObject, IAudit { }D
public interface ITestBeforeTrigger : IBeforeSaveTrigger<TestEntity> { }D
public interface ITestAfterTrigger : IAfterSaveTrigger<TestEntity> { }D
public interface ISecondAfterTrigger : IAfterSaveTrigger<TestEntity> { }D
public class InvalidTrigger { }D
D
public class DatabaseTriggerServiceTestsD

```

```

{
    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly DatabaseTriggerService _service;

    public DatabaseTriggerServiceTests()
    {
        // 1. Arrange
        DatabaseTriggerService.ClearInternalRegistrations();

        _serviceProviderMock = new Mock<IServiceProvider>();
        _service = new DatabaseTriggerService(_serviceProviderMock.Object);
    }

    [Fact]
    public async Task ValidateAsync_ShouldExecuteRegisteredTrigger()
    {
        // Arrange
        var triggerMock = new Mock<ITestBeforeTrigger>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()),
            Times.Once);
    }

    [Fact]
    public async Task ValidateAsync_ShouldThrowException_WhenTriggerCancels()
    {
        // Arrange
        string errorMessage = "Stop!";
        var triggerMock = new Mock<ITestBeforeTrigger>();
        triggerMock.Setup(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<TestEntity>>()))
            .Callback<EntityCancelEventArgs<TestEntity>>(args =>
            {
                args.Cancel = true;
                args.ErrorMessage = errorMessage;
            });

        _serviceProviderMock.Setup(x => x.GetService(typeof(ITestBeforeTrigger)))
            .Returns(triggerMock.Object);

        _service.Register<TestEntity, ITestBeforeTrigger>();

        // Act & Assert
        var ex = await Assert.ThrowsAsync<OperationCanceledException>(() =>
            _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!));

        Assert.Equal(errorMessage, ex.Message);
    }

    [Fact]
    public async Task Hierarchy_ShouldInvokeTriggerForInterface()
    {
        // Arrange
        var triggerMock = new Mock<IBeforeSaveTrigger<IAudit>>();
        _serviceProviderMock.Setup(x => x.GetService(typeof(IBeforeSaveTrigger<IAudit>)))
            .Returns(triggerMock.Object);

        // B
        _service.Register<IAudit, IBeforeSaveTrigger<IAudit>>();

        // Act: TestEntity
        await _service.ValidateAsync(new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Assert
        triggerMock.Verify(x => x.HandleAsync(It.IsAny<EntityCancelEventArgs<IAudit>>()),
            Times.Once);
    }

    [Fact]

```

```

public void Register_ShouldThrow_WhenTriggerDoesNotImplementInterfaces()
{
    // Act & Assert
    Assert.Throws<InvalidOperationException>(() =>
        _service.Register<TestEntity, InvalidTrigger>());
}

[Fact]
public async Task NotifyAsync_ShouldStopExecution_WhenHandledIsTrue()
{
    // Arrange
    var triggerMock1 = new Mock<ITestAfterTrigger>();
    triggerMock1.Setup(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()))
        .Callback<EntityChangedEventArgs<TestEntity>>(args => args.Handled = true);

    var triggerMock2 = new Mock<ISecondAfterTrigger>();

    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ITestAfterTrigger))).Returns(triggerMock1.Object);
    _serviceProviderMock.Setup(x =>
        x.GetService(typeof(ISecondAfterTrigger))).Returns(triggerMock2.Object);

    _service.Register<TestEntity, ITestAfterTrigger>();
    _service.Register<TestEntity, ISecondAfterTrigger>();

    // Act
    await _service.NotifyAsync(new TestEntity(), EntityStateChangeEnum.Added, [], "User",
        DateTime.UtcNow);

    // Assert
    triggerMock1.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Once);
    triggerMock2.Verify(x => x.HandleAsync(It.IsAny<EntityChangedEventArgs<TestEntity>>()),
        Times.Never);
}
}
</file>

```

```

<file path="Tests/Trigger/FluentValidationTriggerTests.cs">
using FluentValidation;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

public class FluentValidationTriggerTests
{
    private class TestEntity : DomainObject { public string Name { get; set; } = ""; }

    private readonly Mock<IServiceProvider> _serviceProviderMock;
    private readonly Mock<IServiceScopeFactory> _scopeFactoryMock;
    private readonly Mock<IServiceScope> _serviceScopeMock;

    private class TestEntityValidator : AbstractValidator<TestEntity>
    {
        public TestEntityValidator()
        {
            RuleFor(x => x.Name).NotEmpty().WithMessage("Name is required");
        }
    }

    public FluentValidationTriggerTests()
    {
        _serviceProviderMock = new Mock<IServiceProvider>();
        _scopeFactoryMock = new Mock<IServiceScopeFactory>();
        _serviceScopeMock = new Mock<IServiceScope>();

        // A00D BD 0C,,2C 5CÂ FCT?CâGC=C: BD0C @C,,:C ACâ7CD0CTB Scope -> Scope C$>Ct2D 0D"0CTB ServiceProvide
        _serviceScopeMock
            .Setup(s => s.ServiceProvider)
            .Returns(_serviceProviderMock.Object);
    }
}

```

```

        _scopeFactoryMock
            .Setup(f => f.CreateScope())
            .Returns(_serviceScopeMock.Object);
    }

    [Fact]
    public async Task HandleAsync_ShouldCancel_WhenValidationFails()
    {
        // Arrange
        var entity = new TestEntity { Name = "" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        // Act
        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Contains("Name is required", args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldNotCancel_WhenValidationSucceeds()
    {
        // Arrange
        var entity = new TestEntity { Name = "Valid Name" };
        var validator = new TestEntityValidator();

        _serviceProviderMock
            .Setup(sp => sp.GetService(typeof(IValidator<TestEntity>)))
            .Returns(validator);

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            entity, EntityStateChangeEnum.Added, [], null!);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.False(args.Cancel);
        Assert.Null(args.ErrorMessage);
    }

    [Fact]
    public async Task HandleAsync_ShouldIgnore_WhenNoValidatorRegistered()
    {
        // Arrange
        _serviceProviderMock
            .Setup(sp => sp.GetService(It.IsAny<Type>()))
            .Returns(null!);

        var trigger = new FluentValidationTrigger(_scopeFactoryMock.Object);
        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
            new TestEntity(), EntityStateChangeEnum.Added, [], null!);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.False(args.Cancel);
    }
}
</file>

```

```

<file path="Tests/Trigger/FullTriggerChainTests.cs">
using FluentValidation;
using Microsoft.EntityFrameworkCore;

```

```

using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Trigger;

// --- 1. B$ B "Ää B´ Aª A !B + ---
public class IntegrationEntity : DomainObject, IAudit
{
    public string Name { get; set; } = "";
}

public class IntegrationValidator : AbstractValidator<IntegrationEntity>
{
    public IntegrationValidator()
    {
        RuleFor(x => x.Name).NotEmpty().WithMessage("NameRequired");
    }
}

public class IntegrationDbContext(
    DbContextOptions<ApplicationDbContext> options,
    IConfiguration configuration)
    : ApplicationDbContext(options, configuration)
{
    // 1. A,,!Äö A A´ Äö AS¢ CT;C 5CÄ BCTAD$>C$KC´ Cct5C² ?Cä;Cö>Dd5Cö=D´< C,,7Cä;C,,@Cä2C =CöKCÄ 4CT@CT2Cä<D
    private class TestNode : ReferenceTreeBase<TestNode>
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // 1. AtöCöCD :C 5CÄ 1C 7Cä2D´9 C 2D$>CÄöD$8Dt5D :C,,9 D :C =CT@ Promatis
        base.OnModelCreating(modelBuilder);

        // 2. ÄööD BD >C":C 4C´O InMemory Cö@Cä2C 9CD5D öD
        if (Database.ProviderName == "Microsoft.EntityFrameworkCore.InMemory")
        {
            modelBuilder.Entity<TestNode>(builder =>
            {
                // AtöCDöCT< Dö2Cö>CR 8CÄö D$öC ;C,,FD²Ä GD$>C K InMemory C,,7Cä;C,,@Cä2C ; CTQ
                builder.ToTable("FullTriggerChain_TestNodes");

                // A,,!Äö A A´ Äö AS¢ C AD$@C 8C$öCT< D 2Dö7DÄ AD$@Cä3Cä =C BC,,?CR FW7DæöFRÄ
                // C,,ACö>C´LCtCDö AD$@Cä3Cä BC,,?C,,7C,,@Cä2C =CöKCR æ ÖVöb >D" AC <Cä3Cä :C´öD AC BCTAD"ö=Cä4D
                builder.HasOne(x => x.Parent)
                    .WithMany(x => x.Children)
                    .HasForeignKey(x => x.ParentId)
                    .OnDelete(DeleteBehavior.Restrict);
            });

            // B$öC=6CR @CT3C,,AD$@C,,@D45CÄ AC <D2 BCTAD$>C$CDä 8CöBCT3D öDd8Cä=CöCDä AD4ICö>D BDÍ
            modelBuilder.Entity<IntegrationEntity>();
        }
    }
}

public class FullTriggerChainTests
{
    [Fact]
    public async Task SaveChanges_ShouldExecuteFullChain_AndCancelOnValidationError()
    {
        // --- ARRANGE ---

        // 1. B >Ct4C 5CÄ DCT9Cª>C$CDä :Cä=DD8C4CD öDd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
    }
}

```

```

        .Build();
    }

    var services = new ServiceCollection();
    services.AddLogging();
    services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;D05CÂ :Cä=DD8C2 2 DI0

    // 2. A,,=DD@C AD$@D4:D$CD =D`5 C00D BD >C":C•
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });
    // AÄ>C¢ DC 1D 8C¤8 D$5C05D L C$>Ct2D 0D"0CTB C¤>C0BCT:D B D :Cä=DD8C4>CÍ
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();
    services.AddSingleton(factoryMock.Object);

    // 3. B 5C48D BD 0Dd8D0 ACT@C$8D >C" BD 8C43CT@Cä2
    services.AddScoped<DatabaseTriggerService>();
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());
    // B 0CÄ8 D$@C,,3C45D K
    services.AddScoped<FluentValidationTrigger>();
    services.AddScoped<ReferenceTreeParentTrigger>();
    services.AddScoped<AuditTrigger>();

    // 4. At0C4;D4HC¤8 CD;D0 8C0BCT@DD5C"ACä2 (DtBCä1D² FCT?CäGC¤0 C05 D 2C ;C ADÄÄ 5D ;C, BD 8C43CT@ D$0
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IDomainObject>>().Object);

    // 5. B 5C48D BD 0Dd8D0 2C ;C,,4C BCä@C
    services.AddTransient<IValidator<IntegrationEntity>, IntegrationValidator>();

    ServiceProvider serviceProvider = services.BuildServiceProvider();

    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;
    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();

    // A0@C,,2D07D`2C 5CÄ fÇVVÇEf Æ-F F-0äG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C`NDt>CÍ
    triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

    // A0@C,,2D07D`2C 5CÄ fÇVVÇEf Æ-F F-0äG&-vvW" :Cä 2D 5CÄ AD4IC0>D BD0< D wV-B0:C`NDt>CÍ
    triggerService.Register<IDomainObjectHasKey<Guid>, FluentValidationTrigger>();

    // A,,!A0 A A` A0 AD B0 ääUB ¢ Ct2C`5C¤0CT< DD0C @C,,D2 66+ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 0D
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A00D BD >C":C >C0FC,,9 A D 8D ?D 0C$;CT=C0KCÄ ?CT@CTEC$0D$GC,,Cä< (C05D 5CD0CT< D$>C`LC¤> DD0C
    DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory)) // A,,!A0 A A` A0 : Aä4C,,= C @C4CC
        .Options;

    // B >Ct4C 5CÄ :Cä=D$5C¤AD"Ä ?CT@CT4C 2C 0 Cä?Dd8C, 8 C¤>C0DC,,3
    await using var context = new IntegrationDbContext(options, configuration);
    await context.Database.EnsureCreatedAsync(TestContext.Current.CancellationToken);

    // B >Ct4C 5CÄ >C JCT:D"Ä :CäBCä@D`9 A0 C0@Cä9CD5D" 2C ;C,,4C FC,,N (Name C0CD BCä9)
    var entity = new IntegrationEntity { Name = "" };
    context.Add(entity);

    // --- ACT & ASSERT ---

    // Aä6C,,4C 5CÄÄ GD$> C,,=D$5D FCT?D$>D 2D`7Cä2CTB D$@C,,3C45D Ä BCäB C$Kct>C$5D" 2C ;C,,4C BCä@, C, 1D4
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
    {
        await context.SaveChangesAsync(TestContext.Current.CancellationToken);
    });

    // A0@Cä2CT@D05CÄÄ GD$> CD>D,,;C, 8CÄ5C0=Cä 4Cä =C HCT3Cä ACä>C ICT=C,,0 C,,7 IntegrationValidator
    Assert.Equal("NameRequired", exception.Message);
}
}
</file>

<file path="Tests/Trigger/ReferenceTreeParentTriggerTests.cs">
using Microsoft.EntityFrameworkCore;

```

```

using Microsoft.Extensions.Configuration;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Trigger;
public class ReferenceTreeParentTriggerTests
{
    // --- 1. B$ B "Aä B´ A A !B + ---
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // A>CÔBCT:D B D$5Cô5D L Cô@C,,=C,,<C 5D" "6öæf-wW& F-öí
    private class TestDbContext(
        DbContextOptions<ApplicationDbContext> options,
        IConfiguration configuration)
        : ApplicationDbContext(options, configuration)
    {
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);
            modelBuilder.Entity<TestNode>();
        }
    }

    // --- 2. Aô AD Aä"Aä A Aä B #Ad Aô Bô òòÝ
    private readonly IConfiguration _configuration;

    public ReferenceTreeParentTriggerTests()
    {
        // B >Ct4C 5CÂ DCT9C C$KC´ :Cä=DD8C2 4C´O D$5D BCä2
        _configuration = new ConfigurationBuilder()
            .AddInMemoryCollection(new Dictionary<string, string?>
            {
                ["DatabaseSettings:DefaultSchema"] = "test"
            })
            .Build();
    }

    private async Task<TestDbContext> GetContextAsync()
    {
        DbContextOptions<ApplicationDbContext> options = new
        DbContextOptionsBuilder<ApplicationDbContext>()
            .UseInMemoryDatabase(Guid.NewGuid().ToString())
            .Options;

        var context = new TestDbContext(options, _configuration);
        await context.Database.EnsureCreatedAsync();
        return context;
    }

    // --- 3. B$ B "B² òòÝ
    [Fact]
    public async Task Trigger_ShouldCancel_WhenObjectIsItsOwnParent()
    {
        // Arrange
        await using TestDbContext context = await GetContextAsync();
        var trigger = new ReferenceTreeParentTrigger();

        var nodeId = Guid.NewGuid();
        // Id C, &VçD-B ACä2Cô0CD0DäB
        var node = new TestNode { Id = nodeId, ParentId = nodeId, Name = "Self Parent" };

        var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>
            (node, EntityStateChangeEnum.Added, [], context);

        // Act
        await trigger.HandleAsync(args);

        // Assert
        Assert.True(args.Cancel);
        Assert.Equal("Aä1D A5C B CÔ5 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", args.ErrorMessage);
    }
}

```

```

}
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentDoesNotExistInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var missingParentId = Guid.NewGuid();
    var node = new TestNode { Name = "Orphan Node", ParentId = missingParentId };

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        node, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Contains("C05 C00C"4CT=", args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldNotCancel_WhenParentExistsInDatabase()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode { Name = "Valid Parent" };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child Node", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.False(args.Cancel);
    Assert.Null(args.ErrorMessage);
}

[Fact]
public async Task Trigger_ShouldCancel_WhenParentIsSoftDeleted()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    var parent = new TestNode
    {
        Id = Guid.NewGuid(),
        Name = "Deleted Parent",
        DeletedAt = DateTime.UtcNow,
        DeletedBy = "Admin"
    };
    context.Add(parent);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    var child = new TestNode { Name = "Child of Dead", ParentId = parent.Id };
    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(
        child, EntityStateChangeEnum.Added, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("A05C'Lct0 C00Ct=C GC„BDÂ @Cä4C„BCT;CT< D44C ;CT=C0KC' >C JCT:D"â"Â &w2äw'&÷$ÖW76 vR"½
}

[Fact]

```

```

public async Task Trigger_ShouldCancel_WhenDirectCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // 1. B >Ct4C 5CÂ 8 D >DT@C =Dô5CÂ @Cä4C,,BCT;DÄAC=8C' MC'5CÄ5CÔB A
    var nodeA = new TestNode { Id = Guid.NewGuid(), Name = "Node A" };
    context.Add(nodeA);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 2. B >Ct4C 5CÂ 8 D >DT@C =Dô5CÂ 4CäGCT@CÔ8C' MC'5CÄ5CÔB B (B >CD8D$5C'L: A •
    var nodeB = new TestNode { Id = Guid.NewGuid(), Name = "Node B", ParentId = nodeA.Id };
    context.Add(nodeB);
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // 3. A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C D 4CT;C BDÂ Cct5C² " @Cä4C,,BCT;CT< CD;Dò Cct;C      ,, CTBC'O: A -> B ->
    nodeA.ParentId = nodeB.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(&
        nodeA, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C=C,,GCTAC=0Dò 7C 2C,,AC,,<CäAD$L: CÔ5C'Lct0 Cò5D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C' Cct5
}

[Fact]
public async Task Trigger_ShouldCancel_WhenDeepCyclicDependencyDetected()
{
    // Arrange
    await using TestDbContext context = await GetContextAsync();
    var trigger = new ReferenceTreeParentTrigger();

    // Aô>D BD >C,,< Dd5Cô>Dt:D3¢ &ö÷B Óâ 6†-ÆB Óâ 7V$6†-ÆM
    var root = new TestNode { Id = Guid.NewGuid(), Name = "Root" };
    context.Add(root);

    var child = new TestNode { Id = Guid.NewGuid(), Name = "Child", ParentId = root.Id };
    context.Add(child);

    var subChild = new TestNode { Id = Guid.NewGuid(), Name = "SubChild", ParentId = child.Id };
    context.Add(subChild);

    await context.SaveChangesAsync(TestContext.Current.CancellationToken);

    // A,,<C,,BC,,@D45CÂ ?Cä?D'BC=C Cò5D 5CÄ5D BC,,BDÂ AC <D'9 C$5D ECô8C' Cct5C² %&ö÷B" 2CÔCD$@DÂ 5C4> C4;D4
    // Bd5Cô>Dt:C ?D >C$5D :C, ?Cä9CD5D" 2C$5D E: SubChild (C" AB' Óâ 6†-ÆB ,,2 A ) -> Root (Cò5D 5CÄ5D
    root.ParentId = subChild.Id;

    var args = new EntityCancelEventArgs<IDomainObjectHasKey<Guid>>(&
        root, EntityStateChangeEnum.Modified, [], context);

    // Act
    await trigger.HandleAsync(args);

    // Assert
    Assert.True(args.Cancel);
    Assert.Equal("Bd8C=C,,GCTAC=0Dò 7C 2C,,AC,,<CäAD$L: CÔ5C'Lct0 Cò5D 5CÄ5D BC,,BDÂ @Cä4C,,BCT;DÄAC=8C' Cct5
}
}
</file>

```

```

<file path="Tests/Trigger/TreeTriggerChainTests.cs">
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Promatis.Net.Data;
using Promatis.Net.Domain;
using Promatis.Net.Domain.Interface;
using System.Text.Json;
using Xunit;

```

```

Ð
namespace Promatis.Net.ApplicationModel.Tests.Trigger;Ð
Ð
// --- B4 A,, A BÄ B´ B #B" Aä!B$ AD Bò At A´/Bd A, "AT!B$ A" ÒÒÝ
public class SoftDeleteNode : ReferenceTreeBase<SoftDeleteNode> { }Ð
public class SelfParentNode : ReferenceTreeBase<SelfParentNode> { }Ð
Ð
// Aä1D"8C' :Cä=D$5C¤AD" 4C´O C,,=D$5C4@C FC,,>CÔ=D´E D$5D BCä2 (D$5CÔ5D L D "6óæf-wW& F-öâ•
public class TreeTestDbContext(Ð
    DbContextOptions<ApplicationDbContext> options,Ð
    IConfiguration configuration)Ð
    : ApplicationDbContext(options, configuration)Ð
{Ð
    protected override void OnModelCreating(ModelBuilder modelBuilder)Ð
    {Ð
        base.OnModelCreating(modelBuilder);Ð
        modelBuilder.Entity<SoftDeleteNode>();Ð
        modelBuilder.Entity<SelfParentNode>();Ð
    }Ð
}Ð
Ð
public class TreeTriggerChainTestsÐ
{Ð
    private (ServiceProvider Provider, IConfiguration Config) CreateProviderAndConfig()Ð
    {Ð
        // B >Ct4C 5CÂ DCT9C¤>C$CDâ :Cä=DD8C4CD 0Dd8Dí
        IConfigurationRoot configuration = new ConfigurationBuilder()Ð
            .AddInMemoryCollection(new Dictionary<string, string?>Ð
            {Ð
                ["DatabaseSettings:DefaultSchema"] = "test"Ð
            })Ð
            .Build();Ð
    }Ð
    Ð
    var services = new ServiceCollection();Ð
    services.AddLogging();Ð
    services.AddSingleton<IConfiguration>(configuration); // AD>C 0C$;Dô5CÂ :Cä=DD8C=
    Ð
    // 1. AäACÔ>C$=D´5 D 5D 2C,,AD½
    services.AddScoped<DatabaseTriggerService>();Ð
    services.AddScoped<IDatabaseTriggerService>(sp =>
sp.GetRequiredService<DatabaseTriggerService>());Ð
    Ð
    // 2. B A4 B "B B #AT A$!AR "B A4 AT B½
    services.AddScoped<FluentValidationTrigger>();Ð
    services.AddScoped<AuditTrigger>();Ð
    services.AddScoped<ReferenceTreeParentTrigger>();Ð
    Ð
    // 3. At0C$8D 8CÄ>D BC, BD 8C43CT@Cä2Ð
    services.AddSingleton(new JsonSerializerOptions { PropertyNamingPolicy =
JsonNamingPolicy.CamelCase });Ð
    Ð
    var factoryMock = new Mock<IDbContextFactory<ApplicationDbContext>>();Ð
    services.AddSingleton(factoryMock.Object);Ð
    Ð
    // 4. At0C4;D4HC¤8 CD;Dò 8CÔBCT@DD5C"ACä2Ð
    services.AddScoped(_ => new Mock<IBeforeSaveTrigger<IAudit>>().Object);Ð
    Ð
    return (services.BuildServiceProvider(), configuration);Ð
}Ð
Ð
[Fact]Ð
public async Task SaveChanges_ShouldCancel_WhenParentIsSoftDeleted_ThroughChain()Ð
{Ð
    // --- ARRANGE ---Ð
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();Ð
    using IServiceScope scope = serviceProvider.CreateScope();Ð
    IServiceProvider sp = scope.ServiceProvider;Ð
    Ð
    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();Ð
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();Ð
    Ð
    // A,,!Aô A A´ Aô AD Bò ääUB ¢ Ct2C´5C¤0CT< DD0C @C,,:D2 66+ R 8Cr BCTAD$>C$>C4> Cô@Cä2C 9CD5D 0Ð
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();Ð
    Ð
    // A,,!Aô A A´ Aô : Aô5D 5CD0CT< C" 8CÔBCT@Dd5CÔBCä@ D$>C´LC¤> Cä4C,,= C @C4CCÄ5CÔB – scopeFactoryÐ
    DbContextOptions<ApplicationDbContext> options = new
DbContextOptionsBuilder<ApplicationDbContext>()Ð

```

```

        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

// A05D 5CD0CT< Cä?Dd8C, 8 Cα>C0DC,,3
await using var context = new TreeTestDbContext(options, configuration);

// 1. B >Ct4C 5CÂ $CCD0C´5C0=Cä3Cä" @Cä4C,,BCT;Dý
var deadParent = new SoftDeleteNode
{
    Id = Guid.NewGuid(),
    Name = "Dead Parent",
    DeletedAt = DateTime.UtcNow
};
context.Add(deadParent);
await context.SaveChangesAsync(TestContext.Current.CancellationToken);

// 2. B 5C 5C0>C-
var child = new SoftDeleteNode
{
    Name = "Child",
    ParentId = deadParent.Id
};
context.Add(child);

// --- ACT & ASSERT ---
await Assert.ThrowsAsync<OperationCanceledException>(async () =>
{
    await context.SaveChangesAsync(TestContext.Current.CancellationToken);
});
}

[Fact]
public async Task SaveChanges_ShouldCancel_WhenSelfParenting_ThroughChain()
{
    // --- ARRANGE ---
    (ServiceProvider serviceProvider, IConfiguration configuration) = CreateProviderAndConfig();
    using IServiceScope scope = serviceProvider.CreateScope();
    IServiceProvider sp = scope.ServiceProvider;

    var triggerService = sp.GetRequiredService<DatabaseTriggerService>();
    triggerService.Register<IDomainObjectHasKey<Guid>, ReferenceTreeParentTrigger>();

    // A,,!A0 A A´ A0 AD B0 ääUB ¢ Ct2C´5Cα0CT< DD0C @C,,:D2 66÷ R 8Cr BCTAD$>C$>C4> C0@Cä2C 9CD5D 00
    var scopeFactory = sp.GetRequiredService<IServiceScopeFactory>();

    // A,,!A0 A A´ A0 : A05D 5CD0CT< C" 8C0BCT@Dd5C0BCä@ D$>C´LCα> Cä4C,,= C @C4CCÄ5C0B – scopeFactory
    DbContextOptionsBuilder<ApplicationDbContext>()
        .UseInMemoryDatabase(Guid.NewGuid().ToString())
        .AddInterceptors(new DatabaseTriggerInterceptor(scopeFactory))
        .Options;

    await using var context = new TreeTestDbContext(options, configuration);

    var node = new SelfParentNode { Id = Guid.NewGuid(), Name = "Self" };
    node.ParentId = node.Id;
    context.Add(node);

    // --- ACT & ASSERT ---
    var exception = await Assert.ThrowsAsync<OperationCanceledException>(async () =>
        await context.SaveChangesAsync(TestContext.Current.CancellationToken));

    Assert.Equal("Aä1D=5CαB C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5.", exception.Message);
}
}
</file>

<file path="Tests/Validator/DomainObjectValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class DomainObjectValidatorTests

```

```

{
    // 1. B >Ct4C 5CÂ BCTAD$>C$KC' >C JCT:D" 8 CT3Câ 2C ;C,,4C BCâ@
    private class TestEntity : DomainObject { }

    private class TestEntityValidator : DomainObjectValidator<TestEntity> {

        private readonly TestEntityValidator _validator = new();

        [Fact]
        public void Validator_ShouldFail_WhenIdIsEmpty()
        {
            // Arrange
            var entity = new TestEntity { Id = Guid.Empty };

            // Act
            TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

            // Assert
            result.ShouldHaveValidationErrorFor(x => x.Id)
                .WithErrorMessage("A,,4CT=D$8DD8C=0D$>D >C JCT:D$0 C05 CÂ>Cd5D" 1D'BDÂ ?D4AD$KCÂâ""½);
        }

        [Fact]
        public void Validator_ShouldFail_WhenCreatedAtIsInFuture()
        {
            // Arrange
            var entity = new TestEntity
            {
                CreatedAt = DateTime.UtcNow.AddMinutes(5)
            };

            // Act
            TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

            // Assert
            result.ShouldHaveValidationErrorFor(x => x.CreatedAt)
                .WithErrorMessage("AD0D$0 D >Ct4C =C,,0 C05 CÂ>Cd5D" 1D'BDÂ 2 C CCDCD"5CÂâ""½);
        }

        [Fact]
        public void Validator_ShouldPass_WhenObjectIsValid()
        {
            // Arrange
            var entity = new TestEntity(); // Id C, 7&V FVD B 7C ?Câ;C00DâBD 0 C" :Câ=D BD CC=BCâ@CR 1C 7Câ2D'E C

            // Act
            TestValidationResult<TestEntity>? result = _validator.TestValidate(entity);

            // Assert
            result.ShouldNotHaveAnyValidationErrors();
        }
    }
}
</file>

<file path="Tests/Validator/ReferenceBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;

namespace Promatis.Net.ApplicationModel.Tests.Validator;

public class ReferenceBaseValidatorTests
{
    // B$5D BCâ2C 0 D CD"=CâAD$L C, 5E 2C ;C,,4C BCâ@
    private class TestReference : ReferenceBase { }
    private class TestReferenceValidator : ReferenceBaseValidator<TestReference> { }

    private readonly TestReferenceValidator _validator = new();

    [Theory]
    [InlineData("")]
    [InlineData(" ")]
    [InlineData("A")] // AÄ5C0LD,,5 2 D 8CÂ2Câ;Câ2D
    public void Name_ShouldHaveErrors_WhenInvalid(string? invalidName)
    {
        var model = new TestReference { Name = invalidName! };
    }
}

```

```

        TestValidationResult<TestReference>? result = _validator.TestValidate(model);
    }

    result.ShouldHaveValidationErrorFor(x => x.Name);
}

[Fact]
public void Name_ShouldHaveError_WhenTooLong()
{
    var model = new TestReference { Name = new string('A', 251) };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Name)
        .WithErrorMessage("Aö@CT2D´HCT=C <C :D 8CÄ0C´LC00Dò 4C´8C00 C00C,,<CT=Cä2C =C,,0 (250 D 8CÄ2Cä;C

}

[Theory]
[InlineData("VALID_CODE_123")]
[InlineData("REF1")]
[InlineData("")] // B$5C05D L D0BCä ?D >C"4CTB D4AC05D,,=Cí
[InlineData(null)] // A, MD$> D$>Cd5D
public void Code_ShouldBeValid_WhenCorrectFormat(string? code)
{
    var model = new TestReference { Code = code };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldNotHaveValidationErrorFor(x => x.Code);
}

[Theory]
[InlineData("C>CEô=C ô@D4AD :Cä<")]
[InlineData("lower_case")]
[InlineData("CODE-1")] // B$8D 5 Ct0C0@CTICT=Cí
public void Code_ShouldHaveError_WhenInvalidFormat(string invalidCode)
{
    var model = new TestReference { Code = invalidCode };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Code);
}

[Fact]
public async Task ValidateValue_ShouldReturnErrors_OnlyForSpecificProperty()
{
    // Arrange
    var model = new TestReference
    {
        Name = "", // AäHC,,1C=0D
        Code = "INVALID-CODE" // AäHC,,1C=0D
    };

    // Act
    // A0@Cä2CT@D05CÄ @C 1CäBD2 2C HCT3Cä 4CT;CT3C BC f Æ-F FUF ÇVR 4C´0 C0>C´0 NameD
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));
    IEnumerable<string> codeErrors = await _validator.ValidateValue(model,
nameof(TestReference.Code));

    // Assert
    Assert.Contains(nameErrors, e => e.Contains("A00C,,<CT=Cä2C =C,,5 Cä1D07C BCT;DÄ=Câ"´´½
    Assert.Contains(codeErrors, e => e.Contains("A>CB <Cä6CTB D >CD5D 6C BDÄ BCä;DÄ:Cä ;C BC,,=C,,FD2"´´½

}

[Theory]
[InlineData(" Name")]
[InlineData("Name ")]
public void Name_ShouldHaveError_WhenHasLeadingOrTrailingSpaces(string invalidName)
{
    var model = new TestReference { Name = invalidName };
    TestValidationResult<TestReference>? result = _validator.TestValidate(model);

    result.ShouldHaveValidationErrorFor(x => x.Name)
        .WithErrorMessage("A00C,,<CT=Cä2C =C,,5 C05 CÄ>Cd5D" =C GC,,=C BDÄADò 8C´8 Ct0C=0C0GC,,2C BDÄADò ?D >

}

[Fact]
public async Task ValidateValue_ShouldIgnoreOtherPropertiesErrors()

```

```

    {
        // Arrange: Cä1C ?Cä;Dö =CT2C ;C,,4CÔK
        var model = new TestReference { Name = "A", Code = "low_case" };
    }

    // Act: C$0C´8CD8D CCT< B$ A´,A Name
    IEnumerable<string> nameErrors = await _validator.ValidateValue(model,
nameof(TestReference.Name));
}

// Assert
Assert.Single(nameErrors); // B$>C´LC CäHC,,1C D CD;C,,=D² 8CÄ5C08
Assert.DoesNotContain(nameErrors, e => e.Contains("A>CB"´´² öö D,,8C >C¢ :Cä4C 1D´BDÂ =CR 4Cä;Cd=C1

}
}
</file>

<file path="Tests/Validator/ReferenceTreeBaseValidatorTests.cs">
using FluentValidation.TestHelper;
using Promatis.Net.Domain;
using Xunit;
namespace Promatis.Net.ApplicationModel.Tests.Validator;
public class ReferenceTreeBaseValidatorTests
{
    // B$5D BCä2C O D CD"=CäAD$L CD;Dö ?D >C$5D :C, 0C AD$@C :D$=Cä3Cä :C´0D AC
    private class TestNode : ReferenceTreeBase<TestNode> { }

    // B$5D BCä2D´9 C$0C´8CD0D$>D
    private class TestNodeValidator : ReferenceTreeBaseValidator<TestNode> { }

    private readonly TestNodeValidator _validator = new();

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEqualToId()
    {
        // Arrange
        var nodeId = Guid.NewGuid();
        var node = new TestNode
        {
            Id = nodeId,
            ParentId = nodeId, // AäHC,,1C D: ID D >C$?C 4C 5D" A ParentId
            Name = "Self-referencing node"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("Aä1D=5C B C05 CÄ>Cd5D" 1D´BDÂ @Cä4C,,BCT;CT< D 0CÄ>CÄC D 5C 5");
    }

    [Fact]
    public void Validator_ShouldFail_WhenParentIdIsEmptyGuid()
    {
        // Arrange
        var node = new TestNode
        {
            ParentId = Guid.Empty, // AäHC,,1C D C0 C$BCä@Cä<D2 ?D 0C$8C´C
            Name = "Empty ParentId"
        };

        // Act
        TestValidationResult<TestNode>? result = _validator.TestValidate(node);

        // Assert
        result.ShouldHaveValidationErrorFor(x => x.ParentId)
            .WithErrorMessage("B >CD8D$5C´LD :C,,9 C,,4CT=D$8DD8C D$>D =CR <Cä6CTB C KD$L C0CD BD´< GUID");
    }

    [Fact]
    public void Validator_ShouldPass_WhenParentIdIsDifferent()
    {
        // Arrange
        var node = new TestNode
        {

```

```

        Id = Guid.NewGuid(),
        ParentId = Guid.NewGuid(),
        Name = "Valid child node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}

[Fact]
public void Validator_ShouldPass_WhenParentIdIsNull()
{
    // Arrange
    var node = new TestNode
    {
        Id = Guid.NewGuid(),
        ParentId = null,
        Name = "Root node"
    };

    // Act
    TestValidationResult<TestNode>? result = _validator.TestValidate(node);

    // Assert
    result.ShouldNotHaveValidationErrorFor(x => x.ParentId);
}
}
</file>

</files>

```